



HPE For HuaPro Emulation User Guide

V24.09.00 November 2024



XEPIK Confidential Information

Table of Contents

Table of Contents	2
Legal Notice	14
1 About This Document	15
1.1 Target Audience	15
1.2 Document Overview	15
1.3 Conventions Used	15
1.4 List of Abbreviations	17
2 HuaPro System	21
2.1 Overview of HuaPro System	21
2.2 HPE Software Features	21
2.3 HuaPro P1 Hardware Features	23
2.4 HuaPro P2E Hardware Features	24
2.5 HPE Feature Compatibility with Hardware	25
2.6 Working Modes of HuaPro	26
3 Quick Start	28
3.1 Setting Up Your Environment	28
3.2 Overview of Emulation Flow	30
3.2.1 Synthesizing the Design for Emulation	30
3.2.2 Compiling the Design with vCom for Emulation	31
3.2.3 Setting Timing Constraints for Emulation	31
3.2.4 Running PnR for Emulation	32
3.2.5 Running the Design for Emulation	32
3.2.6 Debugging the Design for Emulation	33
3.3 Example for Emulation on HuaPro P1	33
4 Synthesizing the Design	38
4.1 vSyn Flow	38
4.2 Starting vSyn	39
4.3 List of vSyn Command Options	39
4.4 Taking a Design to HuaPro	43
4.5 Adding the Synthesis Inputs	44
4.5.1 Loading the Source Files	44
4.5.2 Analyzing Source Files into Libraries	46
4.5.3 Accessing the Include Files	47
4.5.4 Accessing the Verilog Libraries	48
4.6 Specifying the Top-level Module	49
4.7 Defining Text Macros	49
4.8 Adding Synthesis Attributes	51
4.8.1 Supported Attributes	51
4.8.2 Adding Synthesis Attributes in RTL Source Code	54
4.8.3 JSON File Format	54
4.8.4 Adding Synthesis Attributes from a JSON File	55

4.9 Setting the Level of RTL Signal Preservation	59
4.10 Specifying the Compilation Unit Mode	60
4.11 Writing an STB	61
4.11.1 Value Initialization	63
4.11.2 Event Control	63
4.11.3 Cross Module Reference	68
4.12 Using the Stop Control System Task	70
4.13 Generating an XNDB Netlist for Full Vision	71
4.14 Support for Encryption	71
4.14.1 Encryption Envelopes Version 1	71
4.14.2 Encryption Envelopes Version 2	72
4.14.3 Synthesizing and Compiling Encrypted IP Files	74
4.14.4 Example for Encryption Support	75
4.15 Using X-EPIC IP	86
4.16 Using vSyn in Two-step Flow	86
4.17 Enabling Incremental Synthesis	87
4.18 Specifying High-capacity Memories in vSyn	90
4.19 Analyzing the Synthesis Results	90
4.19.1 Checking the Synthesis Log File	90
4.19.2 Checking the Synthesis Area Report	91
4.20 Running Formal Equivalence Check for vSyn Synthesis	92
4.21 Synthesizing the Design Using the Third-Party Tool	97
4.21.1 Using the Third-Party Synthesizer	97
4.21.2 Reasons to Use vEdit	98
4.21.3 Using vEdit	99
4.21.4 vEdit Output Files	101
4.21.5 Limitations of vEdit	102
4.21.6 Classifying Memories	102
5 Compiling the Design	104
5.1 vCom Flow	104
5.2 Starting vCom	106
5.3 List of vCom Tcl Commands	106
5.4 Customizing Your Compilation Script File	109
5.5 Guidelines for Specifying Names on Command Lines	110
5.6 Adding Compilation Inputs	111
5.6.1 Reading the Design	112
5.6.2 Loading the Design	112
5.7 Hardware Description File	113
5.7.1 General Guidelines for HDF	113
5.7.2 VERSION in HDF	114
5.7.3 DEVICE in HDF	114
5.7.4 BOARD in HDF	115
5.7.5 FPGA in HDF	116

5.7.6 CABLE in HDF	117
5.7.7 CARD in HDF	119
5.8 Describing the Hardware Configuration	121
5.9 Setting Timing Constraints	121
5.9.1 Clocks in Emulation	121
5.9.2 Defining the User Clocks	123
5.9.3 Defining the Primary Clocks	123
5.9.4 Excluding the Ports from the Clock Region	124
5.9.5 Setting False Paths	124
5.9.6 Setting Maximum Delay	125
5.10 Terminal Assignment	126
5.10.1 Cable Interfaces on HuaPro P1	126
5.10.2 Cable Interfaces on HuaPro P2E	127
5.10.3 Connecting Hardware to Target System	128
5.10.4 Defining the Cable Connections	129
5.10.5 Assigning Terminals	129
5.10.6 Defining a Delay Bin	130
5.10.7 Inserting a Delay Bin	130
5.10.8 Synchronizing Output Signals	131
5.10.9 Keeping Unassigned Terminals	131
5.10.10 Setting Global Nets	132
5.10.11 Examples for Interacting with External Targets	132
5.11 Netlist Macros	133
5.11.1 Comparison of the Use Flows of Netlist Macros	133
5.11.2 Preparing Files for Netlist Macros	134
5.11.3 Using Netlist Macros in the Simplified Flow	137
5.11.4 Using Netlist Macros in the Standard Flow	139
5.11.5 Binding Netlist Macro Ports to AXI4 Interface	148
5.12 Inserting Instrumentation Logic for Debugging	148
5.12.1 Adding Nets for Signal Access	149
5.12.2 Adding Memories for Memory Access	149
5.12.3 Specifying High-capacity Memories in vCom	150
5.12.4 Inserting Logic for Waveform Generation	150
5.12.5 Adding Nets for ATT Control	152
5.13 Compiling with Optimization	152
5.13.1 Breaking Loops	152
5.13.2 Improving Timing	154
5.13.3 Optimizing the Netlist	156
5.13.4 Handling Driverless Logic	158
5.13.5 Stubbing Modules or Instances	159
5.13.6 Remapping LUTs	159
5.13.7 Setting Memory Optimization Strategies	164
5.13.8 Optimizing Control Sets	165

5.13.9 Shortening FD-related Timing Paths for Hold-time Fix	166
5.14 Performing Design Edit and Design Generation	166
5.15 Checking the Design Area Reports	168
5.16 Running Formal Equivalence Check for Compilation	169
5.17 Running FPGA PnR	172
5.17.1 Estimating CPU and Memory Requirements for Running PnR	173
5.17.2 Relocating PHC Pblocks	173
5.17.3 Running PnR with Pre-defined Strategies	182
5.17.4 Launching Parallel PnR Runs in a Cluster	193
5.18 Checking Timing Analysis Results	195
5.18.1 Checking the PnR Summary Report	197
5.18.2 Checking Timing Analysis Results	199
6 Partitioning the Design	202
6.1 Use Flow for Single-FPGA and Multi-FPGA Designs	202
6.1.1 Single-FPGA Flow	202
6.1.2 Multi-FPGA Flow	204
6.2 Partitioning a Design Automatically or Manually	206
6.3 Setting the Partitioning Strategy	207
6.4 Setting Constraints for Manual Partitioning	207
6.4.1 Setting the FPGA Resource Utilization	208
6.4.2 Defining Indivisibility Rules for Specific Modules	209
6.4.3 Defining Partition Groups	209
6.4.4 Assigning the Placement of Partition Group	212
6.4.5 Combining User-defined Indivisibility Rules with Placement Constraints	213
6.4.6 Setting Area Thresholds for Partitioning	216
6.4.7 Example for Manually Partitioning a Design	217
6.5 Setting the PHC Connection Mode	218
6.6 Using Auto Cabling	220
6.6.1 Dumping the Optimized Interconnection Configuration	220
6.6.2 Adding or Removing User-defined Interconnection Constraints	221
6.6.3 Displaying User-defined Interconnection Constraints	222
6.6.4 Reconstructing Interconnection	222
6.7 Checking Partition Results	222
6.7.1 Checking the Compilation Log File	222
6.7.2 Checking the Partition Area Report	223
6.8 Checking Routing Results	224
6.9 Saving and Restoring Partitioning Results	225
7 Running the Design on Hardware	227
7.1 vDbg Flow	227
7.2 Starting the Runtime Debugger vDbg	228
7.3 List of vDbg Tcl Commands	229
7.4 Customizing Your Run Script File	231
7.5 Opening and Closing Design	232

7.6 Downloading the Design to the Hardware	233
7.6.1 Setting the User Mode	234
7.6.2 Setting the Frequencies of Global Clocks	237
7.6.3 Setting the Frequencies of GT Clocks	238
7.6.4 Connecting vDbg to the Hardware	240
7.6.5 Setting the PHC Voltages	244
7.6.6 Programming the FPGAs	246
7.7 Running the Design for Emulation	246
7.7.1 Starting the Clock	247
7.7.2 Stopping the Clock	248
7.7.3 Getting Current Cycle Number or Time	248
7.7.4 Setting the Divider Count Number	249
7.7.5 Setting the Default Time Unit	250
8 Accessing Signal and Memory Values	251
8.1 Signal Name and File Text Formats	251
8.1.1 Supported Signal Formats	251
8.1.2 Memory File Formats	251
8.2 Accessing Signal Values	255
8.2.1 Forcing and Releasing the Signal Values	255
8.2.2 Depositing the Signal Values	256
8.2.3 Getting the Signal Values	256
8.2.4 Controlling Signal Values by Group	257
8.3 Accessing Design Memories	264
8.3.1 Writing User Data into DUT Memory	264
8.3.2 Reading Data from DUT Memory	265
8.3.3 Listing the DUT Memories	265
8.3.4 Accessing High-capacity Memories	265
8.4 Dumping Memories without Extra Logic Inserted at Compile Time	266
8.5 Using DDR4 Memory Backdoor	266
8.5.1 Writing Data into DDR4 Memory	268
8.5.2 Reading Data from DDR4 Memory	269
8.5.3 Examples for Using DDR4 Memory Backdoor	269
9 Debugging Designs with Probes	274
9.1 Ways of Waveform Generation	274
9.2 Selecting the Probe Mode	275
9.3 Generating the Waveform for Physical Probes	276
9.3.1 Configuring the DDR4 Daughter Board	279
9.3.2 Getting the Maximum Depth Cycle	279
9.3.3 Setting the Trace Size	280
9.3.4 Setting Up the Environment for Generating CWDB Format Waveforms	281
9.3.5 Example for Tracing Physical Probes	282
9.4 Generating the Waveform for Streaming Probes	285
9.4.1 Configuring the Probe HAC Daughter Board	287

9.4.2 Example for Tracing Streaming Probes	289
9.5 Generating the Waveform for GRP Probes	292
9.6 Managing Probes	295
9.6.1 Managing Physical Probes	295
9.6.2 Managing Streaming Probes	296
9.6.3 Managing GRP Probes	298
9.7 Displaying XEDB Waveform Files	299
10 Debugging Designs with Triggers	301
10.1 Stopping Clocks with Triggers	301
10.2 Setting Post-trigger Samples	301
10.3 Examples for Controlling Clocks and Tracing Stop	301
10.4 Controlling the Run with ATT	302
10.4.1 Use Flow of ATT	303
10.4.2 Setting Up the Environment for ATT	304
10.4.3 Adding ATT Nets at Compile Time	304
10.4.4 Preparing ATT Trigger Modules	305
10.4.5 Creating and Enabling ATT Trigger Instances	305
10.4.6 Managing Trigger Instances	306
10.4.7 Executing Callbacks of vDbg Debugging Scripts	308
10.4.8 Limitations of ATT	309
10.5 FSM Language Syntax	309
10.5.1 Defining Verilog FSM Modules	310
10.5.2 Port Declaration	310
10.5.3 State Definition	311
10.5.4 Counter Definition	311
10.5.5 FSM Body Definition	311
10.5.6 Case Statements	312
10.5.7 If-then-else Statements	313
10.5.8 Simple Action Statements	313
10.5.9 Supported FSM Expressions and Operators	314
10.5.10 Lexical Conventions in FSM	315
10.5.11 Examples for FSM Programs	316
10.6 Viewing the Waveform and Adjusting the Capture Controls	318
11 Managing Hardware Resources	319
11.1 Introduction to VRM	319
11.2 Starting vrmctl	320
11.3 vrmctl Commands	321
11.4 VRM Host Configuration	322
11.4.1 Adding a VRM Host	323
11.4.2 Setting the Default VRM Host	324
11.4.3 Showing the VRM Host List	324
11.4.4 Removing a VRM Host	325
11.4.5 Setting the Default VRM Host for All Users	325

11.5 VRM Host Connection	326
11.5.1 Connecting to VRM	326
11.5.2 Disconnecting from VRM	326
11.6 System Information Query	326
11.6.1 Querying System Resource Status	327
11.6.2 Dumping the System HDF	328
11.6.3 Viewing System Status	329
11.7 Runtime Session Management	330
11.7.1 Querying Session Information	331
11.7.2 Removing a Session	331
11.8 Resource Locking	332
11.8.1 Locking Hardware Resources	332
11.8.2 Showing Locked Hardware Resources	334
11.8.3 Unlocking Hardware Resources	334
11.8.4 Design Relocation	335
12 Debug Station GUI	337
12.1 Starting the Runtime Debugger vDbg GUI	337
12.2 GUI Overview	337
12.3 Running and Debugging a Design	339
12.3.1 Downloading the Design to the Hardware from the GUI	339
12.3.2 Running the Design on the Hardware from the GUI	341
12.3.3 Forcing Signals from the GUI	342
12.3.4 Depositing Signals from the GUI	344
12.3.5 Setting up Triggers from the GUI	344
12.3.6 Releasing Signals from the GUI	346
12.3.7 Forcing and Depositing Signals through a File	347
12.3.8 Checking System Status from the GUI	349
12.3.9 Entering Object Names	349
12.3.10 Getting Signal Values from the GUI	350
12.3.11 Generating Waveforms from the GUI	351
12.3.12 Adding Probes through a File	356
12.3.13 Viewing XEDB Waveforms from the GUI	357
12.3.14 Writing and Reading Memories from the GUI	358
12.4 Navigating the Graphical User Interface	360
12.4.1 Undocking and Re-docking the Panes	360
12.4.2 Adjusting the GUI Layout	361
12.4.3 Hiding and Unhiding a Pane	362
13 HPE Command Reference	364
13.1 vsyn	364
13.1.1 -append-log	364
13.1.2 -area-report <file>	364
13.1.3 -attribute <file>	365
13.1.4 -basic-gate-only	365

13.1.5 -bit-enable-for-vcom	365
13.1.6 -block-ram <n>	365
13.1.7 -c -parse-only	365
13.1.8 -D -define <macro>[=<value>]	365
13.1.9 -database <dir>	365
13.1.10 -defparam <name>=<value>	366
13.1.11 -dsp <n>	366
13.1.12 -dup-free-map	366
13.1.13 -error-limit <num>	366
13.1.14 -f -file <filename>	366
13.1.15 -F <filename>	366
13.1.16 -formality <file>	366
13.1.17 -h -help	367
13.1.18 -hcm <N>	367
13.1.19 -help-hidden	367
13.1.20 -I -incdir <dir>	367
13.1.21 -ignore-redefinition	367
13.1.22 -l -log <file>	367
13.1.23 -libext <ext>	367
13.1.24 -lut-ram <n>	368
13.1.25 -nolog	368
13.1.26 -o -netlist <file>	368
13.1.27 -q -quiet	368
13.1.28 -routability	368
13.1.29 -separate-unit	368
13.1.30 -std <year>	368
13.1.31 -support-two-clocks-for-bram	368
13.1.32 -t -top <name>	369
13.1.33 -u	369
13.1.34 -undefine-synthesis-macro	369
13.1.35 -undefined-module-wrapper <file>	369
13.1.36 -undriven-nets <value>	369
13.1.37 -v <libfile>	369
13.1.38 -version	370
13.1.39 -visibility <value>	370
13.1.40 -work <lib>	371
13.1.41 -xndb_dbg <file>	371
13.1.42 -xpm-script <file>	371
13.1.43 -y <libdir>	371
13.2 vcom	372
13.2.1 annotate_xdc_clock	372
13.2.2 bind_ports	373
13.2.3 break_all_loops	373

13.2.4 cable_connection	374
13.2.5 clock_localization	374
13.2.6 clock_port	375
13.2.7 clock_recognition	375
13.2.8 const_prop_option	375
13.2.9 control_set_optimization	376
13.2.10 create_clock	376
13.2.11 create_generated_nmclock	376
13.2.12 delay_cell	377
13.2.13 delay_bin	377
13.2.14 design_edit	378
13.2.15 design_edit_option	378
13.2.16 design_generation	379
13.2.17 design_load	379
13.2.18 design_read	381
13.2.19 direct_phc	381
13.2.20 display	382
13.2.21 dump_clock_region_ports	382
13.2.22 dynamic_trigger	382
13.2.23 elim_common_logic	382
13.2.24 emulator_spec	382
13.2.25 emulator_util	383
13.2.26 fc_load	385
13.2.27 fc_save	385
13.2.28 global_net	385
13.2.29 indivisible_group	386
13.2.30 indivisible_module	386
13.2.31 indivisible_util	386
13.2.32 instance_stub	386
13.2.33 interconn_constraint	387
13.2.34 interconn_dump	388
13.2.35 lite_phc	388
13.2.36 local_fpga_hold_fix	388
13.2.37 logic_replication	389
13.2.38 memory_access	389
13.2.39 memory_options	390
13.2.40 module_stub	390
13.2.41 netlistmacro	390
13.2.42 netlistmacro_param	392
13.2.43 nmport	393
13.2.44 par_assign	393
13.2.45 par_group	394
13.2.46 par_opt_priority	394

13.2.47 partition_option	394
13.2.48 prepartition_option	395
13.2.49 primary_clock	395
13.2.50 read_net	395
13.2.51 remap_optimization	396
13.2.52 routing_localization	397
13.2.53 run_to	397
13.2.54 set_false_path	397
13.2.55 set_max_delay	398
13.2.56 set_phc_speed	398
13.2.57 streaming_trace_net	399
13.2.58 synchronous_reset	399
13.2.59 terminal_assign	399
13.2.60 terminal_delay	401
13.2.61 terminal_osync	402
13.2.62 terminal_keep	404
13.2.63 trace_net	405
13.2.64 trigger_net	405
13.2.65 trigger_net_group	406
13.2.66 write_net	406
13.3 vdbg	406
13.3.1 after_blocking	407
13.3.2 deposit	407
13.3.3 design	408
13.3.4 download	408
13.3.5 dump_mem	408
13.3.6 exit	409
13.3.7 force	409
13.3.8 get_clock	410
13.3.9 get_div	410
13.3.10 get_max_depth_cycles	410
13.3.11 get_phc_vol	411
13.3.12 get_time	411
13.3.13 get_value	411
13.3.14 help	412
13.3.15 hw_server	412
13.3.16 memory	413
13.3.17 set_clock	418
13.3.18 set_div	419
13.3.19 set_phc_vol	419
13.3.20 set_post_trigger_samples	420
13.3.21 set_time_unit	420
13.3.22 set_trace_size	420

13.3.23 set_vrm	421
13.3.24 signal_group	422
13.3.25 stop	422
13.3.26 release	422
13.3.27 run	423
13.3.28 tracedb	424
13.3.29 trace_signals	425
13.3.30 trigger	427
13.4 vrmctl	429
13.4.1 add_vrm_host	429
13.4.2 connect	429
13.4.3 disconnect	429
13.4.4 dump_hdf	430
13.4.5 lock_resource	430
13.4.6 rm_session	431
13.4.7 rm_vrm_host	431
13.4.8 set_default_vrm_host	432
13.4.9 set_default_vrm_host_for_all_user	432
13.4.10 show_locked_resource	432
13.4.11 show_system	432
13.4.12 show_session	433
13.4.13 show_vrm_hosts	433
13.4.14 unlock_resource	433
14 Using the RLM License	434
14.1 License Components and Files	434
14.2 Starting the License Server	436
14.3 Maintaining the License Server	437
14.4 Ports Used in the RLM Licensing System	438
14.5 Troubleshooting Licensing Problems	439
14.5.1 RLM Port Occupation	439
14.5.2 Web Server Port Occupation	440
14.5.3 RLM Lock File Problems	440
14.5.4 Cannot Connect to Server	440
14.5.5 Firewall Issues	441
14.5.6 Cannot Recognize ISV Name	441
14.5.7 No ISV Servers	442
14.5.8 Cannot Find rlm	442
14.5.9 Hostname Translation Failure	442
14.5.10 UDP Port Occupation	443
Appendix A - File Templates and Examples	444
HDF	444
Compilation Script File	446
Run Script File	449

Appendix B - Interface Signals for DDR4 Memory Backdoor

451

Legal Notice

Copyright and Trademark Notice

Copyright © 2024 XEPIC Corporation Limited All Rights Reserved.

This document contains proprietary and confidential information of XEPIC Corporation Limited (“XEPIC”). Without prior written permission of XEPIC, no part of this document may be extracted, copied, transmitted, or distributed in any form or by any means. Uploading this document, in whole or in part, to the generative AI tools is prohibited.

XEPIC, X-EPIC, X-EPIC logo, and certain X-EPIC product names are trademarks or registered trademarks of XEPIC or its subsidiaries. Other trademarks, logos, and product names in this document are trademarked and copyrighted properties of their respective owners or licensors.

Associated third-party software notices may be found in the product-specific Release Notes.

Disclaimer

This document is provided “AS IS” and is subject to change without prior notice. All or any part of the products, services, or features described in this document are intended to represent no more than a general illustration of the products and do not constitute a warranty or representation by XEPIC that the products will conform with the same. XEPIC makes no representations or warranties of any kind (express, implied, or statutory), including any warranties of merchantability, fitness for a particular purpose, title, or non-infringement.

XEPIC Corporation Limited

Website: www.x-epic.com

1 About This Document

This chapter provides an overview of the document, describes the conventions used for the syntax and lists the changes of each update. This chapter includes the following topics:

- [Target Audience](#)
- [Document Overview](#)
- [Conventions Used](#)
- [List of Abbreviations](#)

1.1 Target Audience

The document is targeted for HuaPro system users who are knowledgeable in the following areas:

- Verilog and SystemVerilog languages
- Digital logic designs
- Emulation
- Linux operating system.

1.2 Document Overview

This document describes the processes for synthesizing and compiling design and running the design in the HuaPro system.

Unless where noted, information that is provided in this document applies to the product(s) shown as follows:

- **Hardware:** HuaPro P1 and HuaPro P2E
- **Software:** HPE 24.09.00

For detailed information about the hardware of HuaPro system, refer to *HuaPro P1 Reference Manual* and *HuaPro P2E Reference Manual*.

1.3 Conventions Used

The following table lists the conventions in this document.

Convention	Description
<i>Italic text</i>	Indicates titles of books and figure captions, file paths, and file names.

Convention	Description
Bold text	Indicates titles of chapters, sections, and subtitles.
Blue text	Indicates hyperlinks.
Monospace type	Indicates program and command examples.
Square brackets ([])	Indicates the optional argument or arguments of a command. You have the flexibility to either specify or not specify the optional argument or arguments when executing the command. Do not type the square brackets.
Angle brackets (< >)	Indicates a variable name of the command that you must replace with a valid value or with the name of a file, a path, or an object. Do not type the angle brackets.
Curly braces ({ })	Indicates a set of command parameters. The set might consist of a variable number of parameters or a combination of parameters. Although the missing braces might not lead to an error report, for the best result, type the curly braces as the delimiter.
Separator bar ()	Separates the command choice in a multiple-choice parameter. Do not type the separator bar.
Ellipsis (...)	Indicates that you can enter more than one parameter for one option. Usually, parameters that are followed by an ellipsis are enclosed in braces. Do not type the ellipsis.
Percent (%)	Indicates the generic prompt character which is different in various systems. Examples of commands use the percent sign (%) in this document. However, in examples, some command output that is not related to execution might be omitted. You can obtain the actual output of commands on the screen during operation.

The following table lists the definitions of labels that you can find in this document.

Label	Description
 TIP	Indicates hints, tips, or guidance.
 NOTE	Indicates important restrictions or advice that users might overlook.

Label	Description
 IMPORTANT	Indicates the possibility of damage to a program, device, system, or data that users must pay attention to.
 CAUTION	Indicates a potentially hazardous situation which, if not avoided, might result in personal injury or equipment damage.
 WARNING	Indicates a hazardous situation which, if not avoided, could result in serious personal injury or equipment damage.
 DANGER	Indicates an extremely hazardous situation which, if not avoided, will result in serious personal injury, or even loss of life.

1.4 List of Abbreviations

The following table lists and describes the abbreviations used in this document.

Table 1-1: Abbreviation List

Abbreviation	Definition
ASIC	Application Specific Integrated Circuit
ATT	Accuracy Time Trigger
BRAM	Block Random Access Memory
BNF	Backus-Naur Form
CARRY	Carry chain
CLB	Configurable Logic Block
CPU	Central Processing Unit
CWDB	Common Wave Database
CWI	Common Wave Interface
DCP	Design Checkpoint
DFT	Design For Test
DLUTM	Dual-port LUT Memory
DRAM	Distributed Random Access Memory
DSP	Digital Signal Processing
DUT	Design Under Test
EDIF	Electronic Design Interchange Format

Abbreviation	Definition
FD	D Flip-Flop
FPGA	Field Programmable Gate Array
FSM	Finite State Machine
GC	Global Clock
GPIO	General Purpose Input Output
GRP	Global Register Probe
GT	Gigabyte Transceiver
GUI	Graphic User Interface
HAC	Host Access/Adapter Card
HCM	High-Capacity Memory
HDF	Hardware Description File
HDL	Hardware Description Language
HPE	Hua Prototyping and Emulation
IEEE	Institute of Electrical and Electronic Engineers
IO	Input/Output
IOB	Input Output Block
IP	Intellectual Property
ISV	Independent Software Vendor
ISA	Internal Signal Analyzer
JSON	JavaScript Object Notation
LD	Data Latch
LSF	Load Sharing Facility
LUT	Look-up Table
LUTM	LUTs used as distributed RAM
MAC	Media Access Control
MCP	Multi-cycle Path
MFCU	Multi-file Compilation Unit
MIC	Multi-interface Prototyping Card
MIG	Memory Interface Generator

Abbreviation	Definition
MPAR	Multi-Place-And-Route
OS	Operating System
PC	Personal Computer
PCIe	Peripheral Component Interconnect express
PHC	Prototyping High-speed Connector
PHY	Physical Layer
PnR	Placement and Routing
RAM	Random Access Memory
RLM®	Reprise License Manager
RTL	Register Transfer Level
SerDes	Serializer/Deserializer
SFCU	Single-file Compilation Unit
SGE	Sun Grid Engine
SLR	Super Logic Region
Slurm	Simple Linux Utility For Resource Management
SLUTM	Single-port LUT Memory
SoC	System on Chip
STA	Static Timing Analysis
SSTA	System-level Static Timing Analysis
STB	Synthesizable Testbench
TDM	Time-division Multiplexing
UDP	User Datagram Protocol
URAM	Ultra Random Access Memory
URTA	Universal Runtime Access Library
VCD	Value Change Dump
vCom	Versatile Compiler
vDbg	Versatile Debugger
vEdit	Versatile Editor
VRM	Versatile Resource Manager

Abbreviation	Definition
vSyn	Versatile Synthesizer
XDC	Xilinx® Design Constraints
XEDB	X-EPIC Event Database
XMR	Cross Module Reference
XNDB	X-EPIC Netlist Database
XPM	Xilinx Personality Module

2 HuaPro System

This chapter describes the system flow and topology, and lists features of software and hardware in the following topics:

- [Overview of HuaPro System](#)
- [HPE Software Features](#)
- [HuaPro P1 Hardware Features](#)
- [HuaPro P2E Hardware Features](#)
- [HPE Feature Compatibility with Hardware](#)
- [Working Modes of HuaPro](#)

2.1 Overview of HuaPro System

X-EPIC HuaPro system includes the HPE software, the HuaPro hardware and firmware, and daughter boards. The software suite name is HPE, where H stands for Hua, P stands for prototyping, and E stands for emulation. The whole system, broadly defined, also includes the host PCs or servers that are used for system configuration and control.

HuaPro system provides a fast on-board verification and ease-of-use debugging environment compared to the traditional solutions. The system can help shorten the cycle of software development, hardware and software integration, and system validation for individual IP and subsystems. The following figure shows the HuaPro system overview.

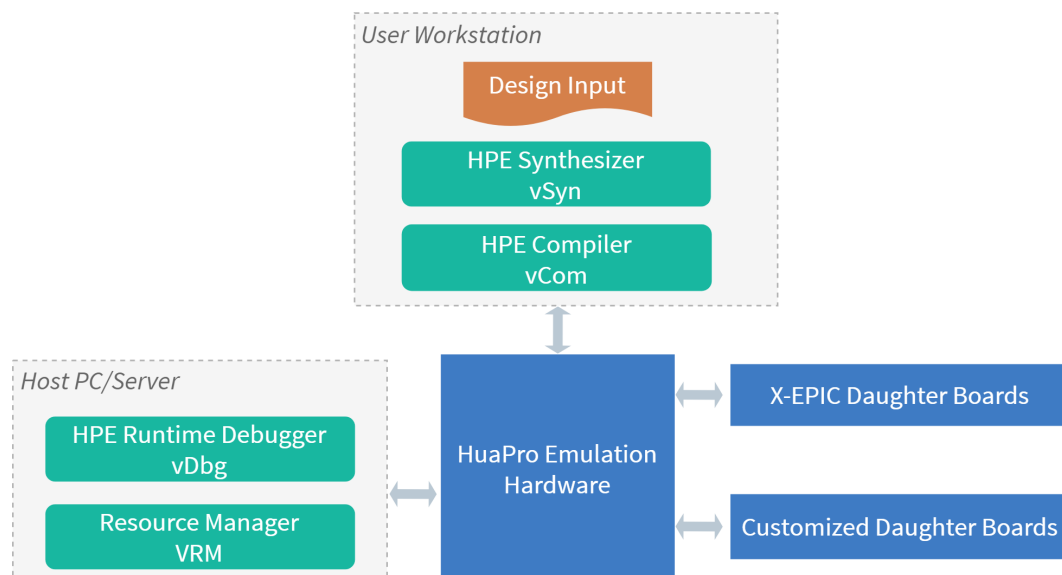


Figure 2-1: HuaPro System Overview

2.2 HPE Software Features

HPE compiler in combination with X-EPIC hardware platforms are used to validate ASIC and SoC designs.

The HPE software offers the following built-in features:

- The synthesizer, vSyn, supports a fully-automated logic-synthesis flow for FPGAs.
 - Verilog and SystemVerilog RTL synthesis
 - Logic optimization
 - FPGA technology mapping including LUT, RAM, and DSP
 - IP protection and license compilation
- The compiler, vCom, performs netlist optimization and modification.
 - Emulation and prototyping working modes
 - Automatic multi-FPGA partitioning and placement
 - Automatic inter-FPGA routing with TDM
 - Automatic clock synthesis for unlimited number of design clocks and for gated-clock timing
 - Automatic configuration and launch of FPGA PnR
 - Native FPGA IP support (custom partition and clocking flow)
 - System-level STA to help estimate the runtime performance
 - Timing, performance, and area optimization techniques
 - PnR strategies and manual optimization techniques
 - Interactive mode and batch mode to run Tcl scripts at compile time
- The on-board debugger, vDbg, starts up the hardware and controls the running and debugging of the design.
 - Fast FPGA bitstream download
 - Signal access: get, force, release, and deposit
 - Signal read and memory dumping without compile-time preset
 - Run and stop control with timers and triggers
 - Memory content reading and writing during runtime
 - DDR4 memory backdoor
 - Physical probes to capture waveforms of signals in cycles with no performance penalty
 - Streaming probes to capture waveforms of selected signals with unlimited depths
 - GRP probes to capture waveforms of all registers with no compile-time presettings
 - VCD, XEDB, and CWDB waveform generation
 - XEDB waveform generation in offline debugging
 - Trigger support with trigger state machine language
 - GUI for debugging and running
 - Interactive mode and batch mode to run Tcl scripts at runtime
- The resource manager, VRM, allocates and manages hardware device resources.
 - Real-time status monitoring
 - System HDF dumping

- Resource locking and relocation
- Multi-user support
- The waveform debugging tool, xWave, provides advanced debugging functions in an intuitive and user-friendly GUI.
- The URTASRV service provides the message validation and communication between the host PC/server and the FPGAs.
- HPE supports the third-party synthesizer, Synplify, and PnR tool, Vivado.

2.3 HuaPro P1 Hardware Features

HuaPro P1 is the first generation of the HuaPro hardware.



Figure 2-2: A Single HuaPro P1 Device



Figure 2-3: Stacked HuaPro P1 Devices

The main features of HuaPro P1 are listed as follows:

- Two FPGAs per single HuaPro P1 device
- Approximate 26 million gate count per FPGA
- Up to 300 MHz design clock for single-FPGA design and 30 MHz design clock for multiple-FPGA design
- 24 PHCs per FPGA for interconnection between two FPGAs within the same device and/or for external connection from FPGA to external devices or daughter boards, and up to 1.4 Gbps per single-end signal
- 8 user clock input for each FPGA and a wide range of clock frequency
- 16 QSFP+ high-speed Serial interfaces for external device connection
- Internal high-bandwidth PCIe Gen3 4-lane connection between FPGA board and system control board
- 64 Gbps infiniband/Mini-SAS HD data channel for virtual- and co-simulation
- Standalone configuration through USB port (USBD) or Ethernet port
- Connection to host PC through USB port (USBH) or Ethernet port
- Real-time monitoring of temperature and voltage
- Cloud system configuration
- Support cascade of HuaPro P1 devices.

For more information about HuaPro P1, refer to *HuaPro P1 Reference Manual*.

2.4 HuaPro P2E Hardware Features

HuaPro P2E is available both in single-device and in cascaded-devices configuration. Up to 8 devices can be cascaded in a rack.



Figure 2-4: A HuaPro P2E Device

The main features of HuaPro P2E are listed as follows:

- Four FPGAs for user per single HuaPro P2E device and approximate 40 million gate count per FPGA.
- Support maximum eight HuaPro P2E devices cascaded in a farm, maximum capacity is 1.2 billion gate every farm.

- 36 PHCs (Prototyping High-speed Connector) per user FPGA for the system interconnection between the four FPGAs within the same device or for the external connection from FPGA to external devices or daughter boards, and up to 1.6 Gb/s per single-ended signal wire on the 34 high-performance PHCs.
- Support 14 transceiver lanes full interconnection from one user FPGA to other three user FPGAs.
- Support 28 lanes high-speed Serial interfaces per FPGA for extensional interconnection or external high-speed Serdes interfaces.
- Up to 300 MHz design clock for single FPGA design and 30 MHz design clock for multiple FPGA design
- 8 user clock inputs for each FPGA and a wide range of clock frequency
- Internal high-bandwidth PCIe Gen3 4-lane connection between FPGA board and system control board
- 64 Gbps Mini-SAS HD data channel for virtual- and co-simulation
- Standalone configuration through USB port (USBD) or Ethernet port
- Connection to host PC through USB port (USBH) or Ethernet port.
- Support global signal view and FPGA-internal memory capture over readback capture.
- On-board control and status collection, real-time monitor on voltage and temperature.
- Support cloud system access and configuration.

For more information about HuaPro P1, refer to *HuaPro P2E Reference Manual*.

2.5 HPE Feature Compatibility with Hardware

The following table shows the compatibility between HPE features and hardware types.

Table 2-1: HPE Feature Compatibility with HuaPro Hardware

Features	Hardware Type
Compilation	
vSyn	P1/P2E
Prototyping mode	P1/P2E
Memory synthesis for URAM	P2E
STB	P1/P2E
IP protection	P1/P2E
Oscillation detection	P1/P2E
Netlist macro enhancement	P1/P2E
New netlist macro	P1/P2E

Features	Hardware Type
Terminal timing	P1/P2E
Terminal assignment	P1/P2E
Direct PHC	P1/P2E
Lite PHC	P2E
Auto cabling	P1/P2E
Compilation with optimization	P1/P2E
PHC Pblock relocation	P1/P2E
Parallel PnR based on cluster	P1/P2E
MPAR	P1/P2E
PnR Summary Report	P1/P2E
Run and Debug	
Single-FPGA runtime relocation	P2E
Clock run and stop	P1/P2E
Signal and memory access	P1/P2E
Memory dumping without compile-time preset	P1/P2E
DDR4 memory backdoor	P2E
Physical probe	P1/P2E
Streaming probe	P1/P2E
GRP probe	P1/P2E
ISA	P2E
ATT CLI and GUI	P2E
Multi-user	P1/P2E

2.6 Working Modes of HuaPro

HuaPro P1 and HuaPro P2E currently supports three working modes: emulation mode, prototyping mode, and base mode. This document covers only the emulation mode.

- **Emulation Mode**

When HuaPro P1/P2E works in the emulation mode, which is also referred to as auto mode and normal mode, the system works as an emulator. You must use the compiler in HPE to compile the design and the debugger in HPE to debug the design.

- The compiler, vCom, compiles the netlist generated by vSyn or the third-party tool and transforms the clocks in the DUT. All user clocks are driven by the root clock.
- The debugger, vDbg, sets the voltages, downloads a bitstream file to each FPGA, and controls the run and stop of clocks. You can use debug technologies to access to signals and memories and visualize waveforms of signals during emulation.

- **Prototyping Mode**

When HuaPro P1/P2E works in the prototyping mode, which is also referred to as assisted mode, the system works as a prototype platform. vCom still supports automatic partitioning but does not perform single-clock domain conversion, so you cannot control the run and stop of clocks at runtime. vDbg still supports certain debug technologies, such as probe tracing and trigger stop. Compared with the emulation mode, HuaPro P1/P2E devices in the prototyping mode can achieve better runtime performance.

- **Base Mode**

In the base mode, HuaPro P1/P2E devices are used as development boards. You need to manually partition the design and configure TDM IP. To set up the hardware, you can install and use X-EPIC BaseMode Setup tool or directly use the serial port console. The BaseMode Setup tool provides a GUI for configuring voltages and clocks, monitoring FPGA status, downloading bitstream files, and so on.

For more information about the prototyping mode, refer to HPE For *HuaPro Prototyping User Guide*. For more information about the base mode, refer to *HuaPro P1 BaseMode Setup User Guide* and *HuaPro P2E BaseMode Setup User Guide*.

3 Quick Start

This chapter describes how to get started with the HuaPro system and provides you with examples to get better understanding of the use flow.

3.1 Setting Up Your Environment

Set the following environment variables before using the HPE software.

- `HPE_HOME`
Specifies the installation path of HPE software.
- `VSYN_HOME`
Specifies the installation path of vSyn. vSyn searches in the directory for input source files and definitions of unknown modules.
- `VCOM_HOME`
Specifies the installation path of vCom. Makefile searches in the directory for PnR scripts and debug IP files.
- `VDBG_HOME`
Specifies the installation path of vDbg. vDbg searches in the directory for configuration files and other runtime input files.
- `XWAVE_HOME`
Specifies the installation path of xWave. HPE package provides a standalone tool, xWave, which you can use to debug waveforms. If you want to use the other versions of xWave, set this environment variable to the path where xWave is installed.
- `RLM_LICENSE`
Specifies the license file path or license server.
 - If the license server runs on the same machine where you run HPE, specify the absolute path of license file.
 - If the license server runs on a different machine from HPE, specify the license specification in the `<port-number>@<host>` format. `<port-number>` is the port number specified in the license file. `<host>` can be the IP address or host name of the license server.

For details, see [Using the RLM License](#).

- `PATH`

Add the `<installation_path>/bin` and the `<installation_path>/xepic_license` directories into your `PATH` environment variable. If the `PATH` environment variable is not set, you can launch the HPE and RLM software using relative path or absolute path.

TIP

You can set the environment variables, `HPE_HOME` , `VSYN_HOME` , `VCOM_HOME` , and `VDBG_HOME` , in one of the following three ways.

- Set the variables in your own startup file and enable the settings before starting HPE.
- Execute `setup.csh` or `setup.sh` that is provided in the installation path before starting HPE. Variables that are set before cannot be overwritten when you use the `setup.csh` or `setup.sh`.
- If you do not set the variables when you start HPE, the variables will be set as specified in the `setup.csh` or `setup.sh` automatically. When you exit HPE, the settings of the variables are cleared.

The following code blocks list the recommended command lines for setting environment variables in Bash and C-Shell. You can replace `<hpe_installation_path>` , `<port-number>` , and `<host>` with your values and directly use the command lines.

Bash:

```
export HPE_HOME=<hpe_installation_path>
export VSYN_HOME=${HPE_HOME}
export VCOM_HOME=${HPE_HOME}
export VDBG_HOME=${HPE_HOME}
export XWAVE_HOME=${HPE_HOME}
export PATH=${HPE_HOME}/bin:${HPE_HOME}/xepic_license:${PATH}
export RLM_LICENSE=<port-number>@<host>
```

C-Shell:

```
setenv HPE_HOME <hpe_installation_path>
setenv VSYN_HOME ${HPE_HOME}
setenv VCOM_HOME ${HPE_HOME}
setenv VDBG_HOME ${HPE_HOME}
setenv XWAVE_HOME ${HPE_HOME}
setenv PATH ${HPE_HOME}/bin:${HPE_HOME}/xepic_license:${PATH}
setenv RLM_LICENSE <port-number>@<host>
```

3.2 Overview of Emulation Flow

The following figure shows the emulation flow.

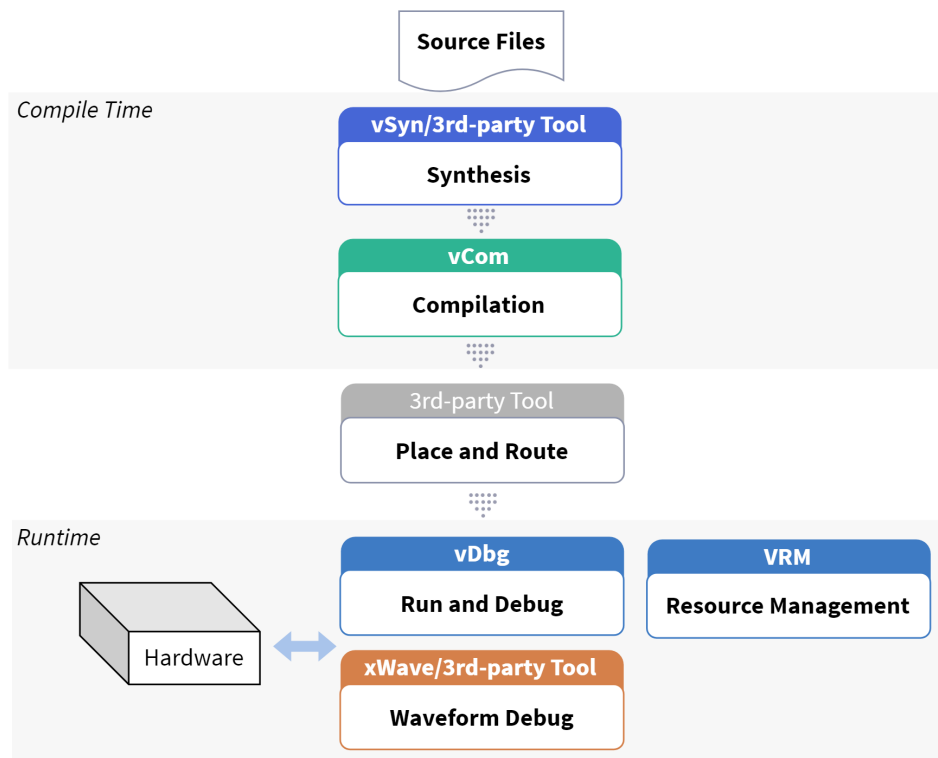


Figure 3-1: Emulation Flow

- [Synthesizing the Design for Emulation](#)
- [Compiling the Design with vCom for Emulation](#)
- [Setting Timing Constraints for Emulation](#)
- [Running PnR for Emulation](#)
- [Running the Design for Emulation](#)
- [Debugging the Design for Emulation](#)

3.2.1 Synthesizing the Design for Emulation

In the emulation mode, you can synthesize a design with any one of the following tools:

- vSyn

The synthesizer, vSyn, analyzes and elaborates the design to build the hierarchy. vSyn performs the front-end synthesis and then generates gate-level netlist in Verilog for the compilation process.

For more information about the use flow, see [Synthesizing the Design](#).

- Third-party synthesizer, Synplify Premier

HPE can compile the design netlist that the third-party synthesizer generates. To solve the signal renaming problem of the synthesizer, you can use vEdit, which is provided in HPE.

For more information about how to use vEdit, see [Synthesizing the Design Using the Third-Party Tool](#).

- VSYN2

Emulation mode supports VSYN2, an experimental feature, which helps speed up the synthesis of a large design. If you want to get a trail, contact XEPIC sales.

 TIP

The analysis and elaboration process of source code is usually known as compilation. However, in this document, unless where noted, the terms of compile, compilation, and compiler refer to the vCom process, and the terms of compile-time and compile time refer to the combined vSyn and vCom process.

3.2.2 Compiling the Design with vCom for Emulation

The compiler in HPE, vCom, reads the gate-level netlist that is generated by the synthesizer. Then vCom compiles the netlist for debugging, partitioning, and optimizing as constrained by the compilation scripts and hardware description. vCom generates a netlist file or netlist files for the design.

You can start vCom for emulation in either of the following two ways:

- Start vCom in the interactive prompt by executing the `vcom` command without any option. Then you can execute the compile-time commands or use the `source` command to execute a compilation script file in the interactive prompt.

```
% vcom  
vcom>
```

- Start vCom in the batch mode by executing the following command.

Syntax:

```
vcom <compilation_script>
```

Where `<compilation_script>` specifies the compilation script file.

For more information about using vCom, see the chapter [Compiling the Design](#).

3.2.3 Setting Timing Constraints for Emulation

In the emulation mode, vCom performs clock conversion, and all clocks are normalized to the root clock. At runtime, you can run and stop user clocks, which are derived from the root clock, to control the running of the design.

This topic focus on the required and optional timing constraints for emulation mode.

Prerequisites

Before setting timing constraints, you can refer to the [Clocks in Emulation](#) topic for better understanding of certain clock terms.

Procedure

In emulation mode, you can set timing constraints as follows:

- Declare the user clocks (see [Defining the User Clocks](#)).
- (Optional) Set another source global clock to be the root clock (default is GC1) (see [Defining the Primary Clocks](#)).
- (Optional) To exclude some ports from the clock region, you can perform the following steps:
 1. Get all ports in the clock region.
 2. Exclude specific port(s) from the clock region.

See [Excluding the Ports from the Clock Region](#) for command usage.

- (Optional) Keep specific paths from timing analysis (see [Setting False Paths](#)).

3.2.4 Running PnR for Emulation

The third-party PnR tool, Vivado, adds physical placement and generates the routed netlists and the bitstream file for each FPGA.

vCom provides a Makefile and scripts for you to quickly start Vivado projects. For more information about how to start PnR, see the chapter [Running FPGA PnR](#).

For a multi-FPGA design, vCom also performs pre-PR and post-PR system-level STA and generates reports. For more information about the reports, see [Checking Timing Analysis Results](#).

3.2.5 Running the Design for Emulation

vDbg is the runtime debugger of the HuaPro system. It connects the hardware and the host server, downloads the bitstream files to the individual FPGAs, and runs a design.

After [Downloading the Design to the Hardware](#), you can start and stop user clocks to control the running of the design. For more information about how to use vDbg to run the design, see

Running the Design on Hardware.

In the emulation mode, you can use the hardware in either the non-VRM mode or the VRM mode. In the VRM mode, the resource manager, VRM, manages hardware resources and makes the board as an efficient allocation unit. For more information about using VRM, see [Managing Hardware Resources](#).

3.2.6 Debugging the Design for Emulation

To help debug a design, vDbg supports the following debugging features in the emulation mode:

- Signal and memory access
 - Signal access: get, force, release, and deposit
 - Memory content reading and writing during runtime
 - DDR4 memory backdoor

For details, see [Accessing Signal and Memory Values](#).

- Waveform generation with probes
 - Physical probes
 - Streaming probes
 - GRP Probes
 - VCD, XEDB, and CWDB waveform types
 - XEDB waveform generation in offline debugging

For details, see [Debugging Designs with Probes](#).

- Stop control with ATT triggers

For details, see [Debugging Designs with Triggers](#).

You can run and debug the design from the Debug Station GUI. In the GUI, you can quickly enter object names by dragging and dropping of the names from Fusion Debug. For more information about using the GUI, see [Debug Station GUI](#).

3.3 Example for Emulation on HuaPro P1

This topic takes an example and guides you through trying out HPE in emulation mode.

Prerequisites

Make sure that you have completed the following steps:

- The license server is started by the system administrator.
- Set environment variables as specified in [Setting Up Your Environment](#). Set the environment variables for the third-party PnR tool and add the path of the executable file

`vivado` to the `PATH` environment variable.

- Make sure that the HuaPro P1 hardware is powered on and ready for use.
- Prepare the source files. You can use your RTL code or copy the source code in the Example Code to your working directory.
- Put all source files in the *design* directory and use the *run* directory for both the compile-time and runtime working directory.

Procedure

Perform the following steps to try out HPE in the non-VRM mode:

1. Execute the `vsyn` command to synthesize the design file, *count.v*, and generate the gate-level netlist, *dut_top.g.v*.

```
% cd run
% vsyn ../design/count.v -t dut_top -o count.g.v
```

TIP

If the *dut_top.g.v* file is not generated successfully in the working directory, check the error messages in the log file *vsyn.log*.

2. Execute the compilation script file, *compile.tcl*, with `vCom` in the same working directory as `vSyn`.

```
% vcom compile.tcl
```

TIP

If the *fpgaCompDir* directory is not generated successfully under the compile-time working directory, check the error messages in the log file, *vcom.log*.

NOTE

Do not move the *fpgaCompDir* directory from its origin location. Otherwise, errors may occur because of the hard-coded locations that are written in the *Makefile* file.

3. Enter the *fpgaCompDir* directory and execute *Makefile* to do the back-end synthesis and the PnR.

TIP

- If the *.bin* file is not generated successfully in the *fpgaCompDir* directory, check the errors in the log files of the PnR tool.
- This PnR step might take tens of minutes to a few hours.

```
% cd fpgaCompDir
% make all
```

4. Execute the run script file *run.tcl* with vDbg in the host PC.

```
% vdbg run.tcl

.....
[HuaPro]Connecting to hardware
[HuaPro]Done
[HuaPro]Downloading the bitstream files to FPGAs ...
[info] 11043 Download successfully! Design is ready to run.
[info] 10735 count_wave opened.
[info] 12831 Create physical probe ph0.
[info] 12831 Create physical probe ph1.
[info] 12831 Create physical probe ph2.
[HuaPro]Add nets in wave ...
[HuaPro]Running the design ...
[info] 12063 Upload waveform successfully.
[HuaPro]Done. Check the waveform file.
```

5. Load the waveform directory, *count_wave.xvcf*, which is generated in the working directory with Fusion Debug or xWave waveform viewer.

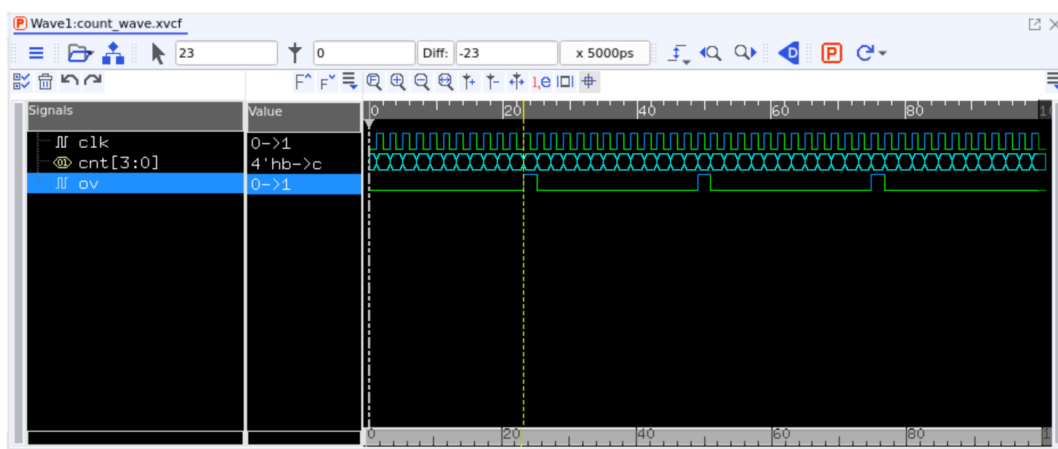


Figure 3-2: Viewing Waveforms

Example Code

The structure of the source files is shown as follows:

- design
 - count.v
 - hw-config.hdf
- run
 - compile.tcl
 - run.tcl

Source RTL code *count.v* is shown as follows:

```
module dut_top(  
    input wire clk,  
    input wire rst,  
    input wire en,  
    output reg [3:0] cnt,  
    output reg ov  
);  
  
parameter CNT_MAX = 'b1100;  
  
always @(posedge clk or posedge rst) begin  
    if (rst) begin  
        cnt <= 'd0;  
    end  
    else if (en) begin  
        if (cnt < CNT_MAX) begin  
            cnt <= cnt + 1'b1;  
        end  
        else begin  
            cnt <= 'd0;  
        end  
    end  
end  
  
always @(*) begin  
    if(cnt == CNT_MAX)  
        ov = 1;  
    else  
        ov = 0;  
end  
  
endmodule
```

Hardware description file *hw-config.hdf* is shown as follows:


```

VERSION += 1.0
DEVICE += P1
BOARD += {"index":0, "type":"TWO_CHIP_BOARD"}
CARD += {"type":"CAPC_16G_V1", "connector":"0.A.1"}

```

Compilation script file *compile.tcl* is shown as follows:

```

design_read -netlist count.g.v
design_load -top dut_top
emulator_spec -add {file ../design/hw-config.hdf}
create_clock -sig_name clk -frequency 100Mhz

trace_net -add {ov}
trace_net -add {cnt}
trace_net -add {clk}

design_edit
design_generation

```

Run script file *run.tcl* is shown as follows:

```

puts "\[HuaPro\]Connecting to hardware"
design .
hw_server localhost
puts "\[HuaPro\]Done"

puts "\[HuaPro\]Downloading the bitstream files to FPGAs ..."
download

puts "\[HuaPro\]Running the design ..."
tracedb -enable
tracedb -open count_wave -xedb -overwrite
trace_signals -add ov
trace_signals -add cnt
trace_signals -add clk
force en 'b1
run 100rclk
tracedb -upload
tracedb -close

puts "\[HuaPro\]Done. Check the waveform file."
exit

```

4 Synthesizing the Design

This chapter introduces how to synthesize a design and generate a gate-level netlist for the compilation process using vSyn or the third-party synthesizer.

You can get an overview of vSyn flow and a summary of command and options in the following topics:

- [vSyn Flow](#)
- [Starting vSyn](#)
- [List of vSyn Command Options](#)

For detailed information about specific vSyn options, see the following topics:

- [Adding the Synthesis Inputs](#)
- [Specifying the Top-level Module](#)
- [Defining Text Macros](#)
- [Adding Synthesis Attributes](#)
- [Setting the Level of RTL Signal Preservation](#)
- [Specifying the Compilation Unit Mode](#)
- [Writing an STB](#)
- [Using the Stop Control System Task](#)
- [Generating an XNDB Netlist for Full Vision](#)
- [Support for Encryption](#)
- [Using X-EPIC IP](#)
- [Using vSyn in Two-step Flow](#)
- [Enabling Incremental Synthesis](#)
- [Analyzing the Synthesis Results](#)

To use the supported third-party synthesizer and vEdit, see the following topics for instructions:

- [Synthesizing the Design Using the Third-Party Tool](#)

4.1 vSyn Flow

vSyn first analyzes the Verilog or SystemVerilog source code to build the hierarchy tree and checks syntax and semantic errors. Then vSyn elaborates the design to generate a netlist. If you encounter errors, you must resolve the errors before you continue to synthesize the design.

vSyn can preserve RTL names and automatically perform the front-end synthesis and FPGA technology mapping. Then vSyn generates gate-level netlist in the Verilog format for the compilation process.

The following figure shows the input and output files for vSyn synthesis.

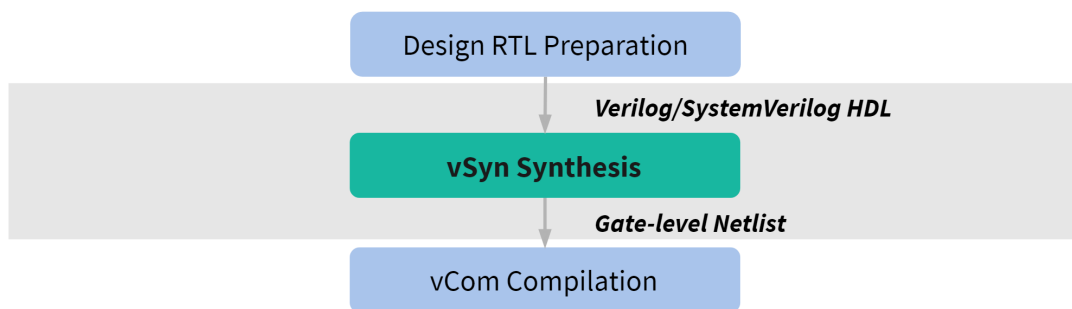


Figure 4-1: vSyn Flow

HPE also supports the third-party synthesizer, Synplify premier. In the Synplify flow, the library is built into the generated netlist. For more information about using Synplify premier, see the section [Synthesizing the Design Using the Third-Party Tool](#).

4.2 Starting vSyn

Use the following command line to start vSyn.

Syntax:

```
vsyn <source-files> [<options>]
```

Where,

- `<source-files>` specifies the source files which can be one or more Verilog or SystemVerilog files.
- `<options>` specifies options that control how vSyn works. For more information, see the table in [List of vSyn Command Options](#).

Two or more files can be separated by space in the command line. Files in a command line are processed in a left-to-right order. You can also specify a file list or file lists to load the source code. Files written in a file list are processed from the first line to the last line. For more information, see [Loading the Source Files](#).

4.3 List of vSyn Command Options

The following table lists all common options of the `vsyn` command. You can print the help information of these common options by executing the `vsyn -h` or `vsyn -help` command line. You can also read the related references for more information.

Table 4-1: A Summary of vSyn Common Options

Option	Description	Reference
<code>-append-log</code>	Appends messages to the log file specified by the <code>-l/-log</code> option. In the elaboration step of two-step synthesis or the incremental synthesis, if this option is not used for an existing log file, the new logs will overwrite the old logs.	-
<code>-area-report <file></code>	Specifies the area report. If not specified, the area report is saved in <code>vsyn_area.rpt</code> by default.	Checking the Synthesis Area Report
<code>-attribute <file.json></code>	Annotates attributes from a JSON file.	Adding Synthesis Attributes from a JSON File
<code>-c/-parse-only</code>	Stops the synthesis after vSyn analyzes input files without elaboration.	Using vSyn in Two-step Flow
<code>-D/-define <macro>[=<value>]</code>	Defines the macros in all source files. If not specified, the value of macro is <code>1</code> by default.	Defining Text Macros
<code>-database <dir></code>	Specifies the directory name of database.	Analyzing Source Files into Libraries
<code>-defparam <name>=<value></code>	Specifies parameter values.	-
<code>-error-limit <num></code>	Limits the number of printed error messages. Setting the number to zero disables the limit.	-
<code>-f/-file <filename></code>	Reads source files and command options from a specified file list. The relative paths in the file list are relative to the working directory where you execute the command <code>vsyn</code> .	Adding the Synthesis Inputs
<code>-F <filename></code>	Reads source files and command options from a specified file list. The relative paths in the file list are relative to the directory where the file list is.	Adding the Synthesis Inputs
<code>-h/-help</code>	Prints the options and displays the descriptions.	-

Option	Description	Reference
<code>-hcm <N></code>	Sets the size threshold for HCM implementation.	Specifying High-capacity Memories in vSyn
<code>-help-hidden</code>	Prints the hidden options and displays the descriptions.	-
<code>-I/-incdir <dir></code>	Searches include files in the specified directories.	Accessing the Include Files
<code>-libext <ext></code>	Searches definitions of unknown module in the library files with the specified extension. This option must be used with the <code>-y</code> option.	Accessing the Verilog Libraries
<code>-l/-log <file></code>	Specifies a file to store logs. The default is <i>vsyn.log</i> .	Checking the Synthesis Log File
<code>-nolog</code>	Synthesizes the design without generating the log file.	-
<code>-o/-netlist <file></code>	Specifies the name of the output netlist file. If this option is not specified, the source file is synthesized without any output netlist.	-
<code>-q/-quiet</code>	Enables the quiet mode to suppress non-essential messages.	-
<code>-separate-unit</code>	Treats each file in the command line as a separate compilation unit.	Specifying the Compilation Unit Mode
<code>-std <year></code>	Specifies the SystemVerilog and Verilog standard to use. Valid values: 1995, 2001, 2005, 2009, 2012, and 2017 (default). Note that this option is under test.	-
<code>-t/-top <name></code>	Specifies the top-level modules of the design. You can specify multiple top-level modules with repetition of <code>-t</code> or <code>-top</code> . If the top-level module is not specified, vSyn analyzes the design and selects the top-level modules by default.	Specifying the Top-level Module

Option	Description	Reference
<code>-u</code>	Converts all identifiers in the source design files and in the generated netlist file to uppercase. You can convert all Verilog and SystemVerilog identifiers to uppercase to make sure that the source description does not rely on the case sensitivity.	-
<code>-v <libfile></code>	Searches definitions of unknown modules in the specified library files.	Accessing the Verilog Libraries
<code>-version</code>	Prints the version information of vSyn.	-
<code>-visibility <value></code>	Specifies the level of RTL signal preservation. Valid values: <code>module</code> , <code>port</code> , and <code>state</code> (default).	Setting the Level of RTL Signal Preservation
<code>-work <lib></code>	Specifies a library to store intermediate files. The default is <code>work</code> .	Analyzing Source Files into Libraries
<code>-y <libdir></code>	Searches definitions of unknown modules in the specified library directories.	Accessing the Verilog Libraries

The following table alphabetically lists hidden options of the `vsyn` command that are not frequently used. Most of the options are currently under test and only for testing purposes. You can print the help information of these hidden options by executing the `vsyn -help-hidden` command line. You can also read the related references for more information.

NOTE

Note that you should not rely on the availability and reliability of options marked with the asterisk (*). If required, contact XEPIC for details of these options.

Table 4-2: A Summary of vSyn Hidden Options

Option	Description
<code>-basic-gate-only *</code>	Uses only Verilog built-in primitives for combinational gates.
<code>-bit-enable-for-vcom</code>	Uses the bit enable model for vCom.
<code>-block-ram <n> *</code>	Specifies the array size threshold for inferring BRAM. BRAM can be used only if the array size is above the threshold. Default value is 256. Valid value of <code><n></code> is any non-negative integer or -1. Setting the value to -1 disables the block RAM synthesis.

Option	Description
<code>-dsp <n> *</code>	Specifies the gate-count threshold for inferring DSP blocks from operators. DSP can be used only if the gate count is above the threshold. Default value is 500. Valid value of <code><n></code> is any non-negative integer or -1. Setting the value to -1 forbids inferring DSP blocks. Higher number means more LUTs.
<code>-dup-free-map</code>	Enables LUT remapping enhancement. You can use this option tentatively to get better optimization results.
<code>-formality <file> *</code>	Generates Formality scripts automatically.
<code>-ignore-redefinition *</code>	Uses the first definition if a module has duplicated definition.
<code>-lut-ram <n> *</code>	Specifies the array size threshold for using LUTM. LUTM can be used only if the array size is above the threshold. Default value is 256. Valid value of <code><n></code> is any non-negative integer or -1. Setting the value to -1 disables the LUTM synthesis.
<code>-routability</code>	Synthesizes the design with the routability optimization enabled.
<code>-support-two-clocks-for-bram</code>	Supports two clocks for synthesis from asynchronous read and write ports to synchronous ports. Note that the functionality of memory is not guaranteed to be correct.
<code>-undefine-synthesis-macro *</code>	Undefines the <code>SYNTHESIS</code> macro.
<code>-undefined-module-wrapper <file> *</code>	Generates the undefined module wrapper file.
<code>-undriven-nets <value> *</code>	Specifies the driver for undriven nets during synthesis. Valid values are <code>0</code> , <code>1</code> , <code>x</code> , and <code>z</code> . If the option is not specified, vSyn leaves the undriven nets dangling.
<code>-xndb_dbg <file></code>	Generates a netlist to preserve all RTL signal names.
<code>-xpm-script <file> *</code>	Generates Vivado synthesis scripts for XPM cells.

4.4 Taking a Design to HuaPro

The starting point of the emulation flow is to write or get the input source files for the synthesizer. The inputs can be Verilog or SystemVerilog source codes.

- The designs that are larger than FPGAs probably need partitioning at compile time.
- The designs that are faster than FPGAs might not be able to run at full SoC speed.

- For the designs that are FPGA-hostile, you need to modify the RTL code and make the designs ready for FPGA synthesis, placement, and routing. Then HPE can map the design efficiently into target resources of FPGAs.

Contact XEPIC supports if you need more guides for the transitions.

4.5 Adding the Synthesis Inputs

vSyn supports synthesizable Verilog and SystemVerilog code in the source files. You can organize the source files, libraries, and synthesis options as instructed below.

4.5.1 Loading the Source Files

vSyn supports many ways of specifying source files and options. You can use one way or combine the ways.

- Directly specify the source files and options in the command line.

Syntax:

```
vsyn <source_files> <options>
```

Where,

- `<source_files>` specifies source files in Verilog or SystemVerilog.
- `<options>` can be any available vSyn options.
- Use the `-f` option to specify a file list or file lists that contain source files and options.
- Use the `-F` option to specify a file list or file lists that contain source files and options.

You can specify the following options in the file list:

- `-f` and `-F`
- `-y` and `-v`
- `+libext+` instead of `-libext`. The syntax is `+libext+<string1>+<string2>+...<stringN>`.
- `+define+` instead of `-D` or `-define`. The syntax is `+define+<macro_name>="<macro_string>"`. For example, `+define+WIDTH=5`. You can omit quotation marks if the string does not contain white spaces.
- `+incdir+` instead of `-I` or `-incdir`. The syntax is `+incdir+<dir1>+<dir2>+...+<dirN>`.

 TIP

Source files can be specified by absolute and relative paths. You can also use environment variables in the file paths.

Example:

Take the following structure of the source files for example. All source files are stored in the `src` directory. The `vsyn` command line is executed in the `run` directory.

```
- run
- src
  - flist.lst
  - top.v
  - dut
    - m.v
    - n.v
```

- The following command line directly specifies the source files:

```
% vsyn ../src/top.v ../src/dut/m.v -o netlist.v
```

- The following command line uses the `-f` option to specify a file list:

```
% cd /home/userA/run
% vsyn -f ../src/flist.lst -o netlist.v
```

While the file list, `flist.lst`, is shown as follows:

```
../src/top.v
../src/dut/m.v
```

- The following command line uses the `-F` option to specify a file list:

```
% export SRC_PATH=/home/userA/src/
% cd /home/userA/run
% vsyn -F ../src/flist.lst -o netlist.v
```

While the file list is shown as follows:

```
top.v
dut/m.v
${SRC_PATH}/dut/n.v
```

4.5.2 Analyzing Source Files into Libraries

By default, vSyn analyzes source files into the *work* library. To use a specified library or multiple libraries, you can use either of the following methods:

- Use the `-work` option on the `vsyn` command line.

```
vsyn -work <lib> <source_files> <options>
```

- Use a library map file. vSyn supports libraries and configurations to configure contents of a design as defined in IEEE Std. 1800-2017.

```
vsyn <lib_map> {<cfg> | -t <lib>.<module_name>} <source_files> <options>
```

Where,

- `<lib_map>` specifies the library map file.
- `<cfg>` specifies the configuration file.
- `-t <lib>.<module_name>` specifies the top module. If you do not specify the configuration file, you must use the `-t` or `-top` option to specify the top module.
- `<source_files>` specifies all source files. You can also specify the source files together with the library map file and configuration file in a file list and specify the file list using the `-f` or `-F` option.
- `<options>` specifies other required vSyn options.

Examples for Using Multiple Libraries

The following examples use a design which instantiates an adder module and a subtraction module in the top module to show how to use multiple libraries. The source files of the design are listed as follows.

- The `add.v` file specifies the definition of the `add` module.

```
module add(sum,co,a,b,c_in);
  output [3:0] sum;
  output co;
  input [3:0] a,b;
  input c_in;
  reg [3:0] sum;
  reg co;

  always @ (a or b or c_in) begin
    {co,sum} = a + b + c_in;
  end

endmodule
```

- The *lib.map* file defines the library mapping.

```
library toplib    top.v;
library sublib    sub.v;
library addlib    add.v;
```

- The *config.sv* file is a Verilog configuration file which specifies the library of the top module and the default library search order. As configured by *config.sv*, the `addlib` library is to be searched.

```
config cfg;

    design toplib.top;
    default liblist addlib sublib;

endconfig
```

- The *filelist* file lists the library map file, the configuration file and the sources files.

```
lib.map
config.sv
top.v
add.v
sub.v
```

Example 1: Uses multiple libraries by means of the `-work` option. As configured with the command lines, the source file, *top.v*, *add.v*, and *sub.v* are analyzed into the library, `toplib`, `addlib`, and `sublib`, respectively.

```
% vsyn top.v -c -work toplib -database libs.vsyn
% vsyn add.v -c -work addlib -database libs.vsyn
% vsyn sub.v -c -work sublib -database libs.vsyn
% vsyn config.sv -database libs.vsyn -o vsyn.g.v
```

Example 2: Uses multiple libraries by specifying a library map file, *lib.map* with the configuration file *config.sv*. As configured by the library map file, the source file *top.v*, *add.v*, and *sub.v* are analyzed into the library `toplib`, `addlib`, and `sublib`, respectively.

```
% vsyn lib.map config.sv top.v add.v sub.v add.v -o vsyn.g.v
```

4.5.3 Accessing the Include Files

You can use the `-I` or `-incdir` option to specify the searching path for ``include` directives.

4.5.4 Accessing the Verilog Libraries

vSyn resolves all modules in the source files. When vSyn cannot find a module definition in the source files, vSyn searches in the Verilog libraries that are associated with source design files. To specify the searching path for the unknown modules, use the `-v`, `-y`, and `-libext` options.

- Specify library files with the `-v` option.

```
vsyn -v <libfile> [<other_options>] <source_files>
```

Each library file can have the definition of multiple modules. If you want to add multiple library files, you can use multiple options `-v` on one command line.

- Specify library directories with the `-y` option.

```
vsyn -y <libdir> [-libext <extension>] [<other_options>] <source_files>
```

The names of the files in the library directories must be the same as the names of the corresponding instantiations in the module description. Each file in the specified library can only have the definition of one module. If you want to add multiple library directories, you can use multiple options `-y` on one command line.

When you add Verilog libraries with the `-y` option, you can specify the extensions of files that vSyn can search in libraries. If you want to specify multiple file extensions, you can use multiple options `-libext` on one command line.

Example

Take the following structure of the source files for example:

```
- top.v
- lib
  - full_adder.v
  - half_adder.v
```

The following command line searches for unknown modules in the specified file:

```
% vsyn top.v -v lib/full_adder.v -o top.g.v
```

The following command line searches for unknown modules in the files with the specified extension `.v` in the specified path:

```
% vsyn top.v -y lib/ -libext .v -o top.g.v
```

4.6 Specifying the Top-level Module

You must use the `-t` or `-top` option to specify the top-level module or modules for synthesis.

NOTE

- If the `-t` or `-top` option is not specified, vSyn uses the top module or multiple top modules which come from the automatic design analysis.

4.7 Defining Text Macros

You can use text macros to conditionally customize the source code that is to be synthesized. You can use the `-D` and `-define` options with the `vsyn` command to define text macros on the command line.

The macro which is defined in the command line is equivalent to that is defined by using the ``define` compiler directive in the source code. The `-D` and `-define` options can be used individually or in combination with the ``define` and ``ifdef` directives.

Perform the following steps to use the text macro during synthesis.

1. Define the text macro in any or combination of the two ways:
 - Use the directive ``define` in the source code.
 - Use the `-D` or `-define` option on the `vsyn` command line to define a macro or change the macro value in the synthesis. A command-line macro overrides a macro of the same name that is defined by the ``define` compiler directive.
2. Use the text macro in the following ways:
 - Use the grave accent character (```) followed by the macro name in the source code. The synthesizer substitutes the text of macro with the defined string.
 - Use the ``ifdef` directive and other conditional directives to include optionally part of the source code during synthesis.

The text macro definitions from high precedence to low precedence are shown as follows. Definition that has a higher precedence overrides the definition that has a lower precedence.

- The ``undef` compiler directive in the source code.
- The `-D` and `-define` options in the command line.
- The ``define` compiler directive in the source code.

Example 1:

The following command line defines the value of macro `M1` as `1` and `M2` as `8` :

```
% vsyn dut.v -o netlist.v -define M1 -define M2=8
```

Example 2:

The following example shows a combination of the `-D` option, ``define`, and ``ifdef` directives. You can see how definition and usage of text macros affect the synthesis result.

In this example, the three macros `WIDTH`, `ADD`, and `M` are used in the RTL code.

- Macro `WIDTH` is defined in the source code and redefined as `5` in the command line, so the output signal `sig` has five bits.
- Macro `ADD` is defined as `x+y` in the command line, so the output signal `add` is constant 5.
- Macro `M` is defined as `1` by default in the command line and undefined in the source code, so the ``else` branch takes effect and the output signal `out` is constant 0.

The source design file `dut.v`, the command line of synthesis, and the generated netlist `dut.g.v` are shown as follows.

The RTL code of `dut.v`:

```
`define WIDTH 3
`undef M

module top(
    output [`WIDTH-1:0] sig,
    output [2:0] add,
    output out
);
    assign sig = 0;

    assign add = `ADD(3,2);

`ifdef M
    assign out = 1;
`else
    assign out = 0;
`endif

endmodule
```

The command line to start synthesis:

```
% vsyn -D WIDTH=5 -D "ADD(x,y)=(x+y)" -D M dut.v -o dut.g.v
```

The netlist `dut.g.v`:

```

module top (output [4:0] sig, output [2:0] add, output out);

    assign sig[4] = 1'b0; // sig[4:0] = 5'b00000;
    assign sig[3] = 1'b0;
    assign sig[2] = 1'b0;
    assign sig[1] = 1'b0;
    assign sig[0] = 1'b0;
    assign out = 1'b0;
    assign add[2] = 1'b1; // add[2:0] = 3'h5;
    assign add[1] = 1'b0;
    assign add[0] = 1'b1;

endmodule

```

4.8 Adding Synthesis Attributes

Attributes can be attached to a design to perform or not perform certain actions when vSyn synthesizes the design. vSyn supports annotating attributes in RTL source code or from a JSON file.

4.8.1 Supported Attributes

vSyn supports the synthesis attributes, `trace_net`, `write_net`, `syn_ramstyle`, `ultra`, and `syn_black_box`.

- **`trace_net` Attribute**

The `trace_net` attribute affects the preservation of nets. `trace_net` is useful only for the `-visibility port` and `-visibility state` options. The `-visibility module` option keeps every RTL net from being optimized.

In the following RTL example, the `trace_net` attribute is set for the wire `t1`.

```

(* trace_net *) wire t1 = !a;
assign b = !t1;

```

In the generated gate-level netlist, wire `t1` is kept and not be optimized away.

```

wire t1;
assign b = a;
not (t1, b);

```

**TIP**

Signals that have the `trace_net` attribute are kept during synthesis, but they do not drive anything. Take care with these signals in the later flow.

- **write_net Attribute**

The `write_net` is for purpose of force and deposit at runtime. This attribute can not only preserve the nets but also keep the driving signals.

In the following JSON example, the `write_net` attribute is set for wire `t1`.

```
{
  m1: {
    t1: {
      "write_net": 1,
    }
  }
}
```

In the generated gate-level netlist, wire `t1` is kept and drives net `b`.

```
wire t1;
not (t1, a);
not (b, t1);
```

- **syn_ramstyle Attribute**

The attribute has two valid values: `block_ram` and `ultra`.

When the memory size is larger than 256 bits, vSyn automatically maps the memory to XPMs. You can specify the resource type with the `syn_ramstyle` attribute.

- `block_ram` gives priority to infer the BRAM primitives.
- `ultra` gives priority to infer the URAM primitives.

Example:

```
(* syn_ramstyle="block_ram" *) reg [215:0] ram [31:0];
```

- **ultra Attribute**

The `ultra` attribute instructs vSyn to give priority to infer the URAM primitives. The function of `ultra` is the same with `syn_ramstyle="ultra"`.

 NOTE

If URAM is used in the netlist, you can only run the design on HuaPro P2E. HuaPro P1 does not support URAM memory synthesis.

You can set this attribute in RTL source code or in a JSON file. The following is an example of JSON file.

```
{
  m1: {
    mem: {
      "ultra": 1,
    }
  }
}
```

- **syn_black_box Attribute**

Defines a black box for synthesis. If a module has the `syn_black_box` attribute, vSyn does not output the netlist of this module.

In the following RTL example, the `syn_black_box` attribute is added to the `m2` module.

```
module test(in1, in2, in3, in4, q);
input in1, in2, in3, in4;
output q;
wire out1, out2;
m1 inst1 (.A(in1), .B(in2), .C(out1));
assign out2 = out1 & in3;
m2 inst2 (.X(out2), .Y(in4), .Z(q));
endmodule

module m1(A, B, C);
input A, B;
output C;
endmodule

(* syn_black_box *) module m2(X, Y, Z);
input X, Y;
output Z;
endmodule
```

After synthesis, vSyn does not output the netlist of module `m2`.

```

module test (input in1, input in2, input in3, input in4, output q);

wire out1;
wire out2;
LUT2 #(.INIT(4'h8)) i5(.I0(out1), .I1(in3), .O(out2));
m1 inst1 (.A(in1), .B(in2), .C(out1));
m2 inst2 (.X(out2), .Y(in4), .Z(q));

endmodule

module m1 (input A, input B, output C);

LUT2 #(.INIT(4'h8)) i5(.I0(A), .I1(B), .O(C));

endmodule

```

- **xpvs_use_xdram Attribute**

Specifies memories which are to be implemented using HCM. If you add this attribute to a memory, vSyn marks it as a user-defined HCM. User-defined HCMs have the highest priority. They are effective even if you use the option `-hcm -1` to disable the automatic HCM generation. Bit-blasted memories cannot be specified as HCMs.

```
(* xpvs_use_xdram *) reg [31:0] mem [0:2**26-1];
```

4.8.2 Adding Synthesis Attributes in RTL Source Code

You can add a synthesis attribute in RTL source code in either of the following ways:

- Use the `(*` and `*)` delimiters.
- Specify the attribute in a comment.

For example, any one of the following code lines controls vSyn to infer a block RAM.

```
(* syn_ramstyle="block_ram" *) reg [215:0] ram [31:0];
```

```
reg [215:0] ram [31:0] /* synthesis syn_ramstyle="block_ram" */;
```

```
reg [215:0] ram [31:0]; // synthesis syn_ramstyle="block_ram"
```

4.8.3 JSON File Format

The advantages of using the JSON file is that you do not need to modify the RTL source code directly. You only need to specify attributes in an external file just like specifying attributes in

attribute name space and then specify the file on the `vsyn` command line.

The synthesis attributes, `trace_net`, `write_net`, `syn_ramstyle`, `ultra`, and `syn_black_box` can be defined in a JSON file in the following format:

```
{
  "module1": {
    "attribute1": "value",
    "object1": {
      "attribute2": "value"
    },
    "generate_block1": {
      "object2": {
        "attribute1": "value",
        "attribute2": "value"
      },
      "object3": {
        "attribute1": "value",
        "attribute2": "value"
      }
    }
  },
  "module2": {
    "object4": {
      "attribute1": "value",
      "attribute2": "value"
    }
  }
}
```

4.8.4 Adding Synthesis Attributes from a JSON File

Perform the following steps to annotate attributes from a JSON file:

1. Specify the scope for the attribute.
 - Attributes that directly follow the module name belong to the whole module scope. The JSON file includes attribute specifications for all modules in the design.
 - Attributes that follow the generate block name belong to the generate block. For named generate blocks in the source code, specify the name in the JSON file. For unnamed generate blocks, use the names that are assigned as `genblk<n>`. If generate regions are not used, you can also use the external names for generate blocks that do not have regions. Refer to IEEE Std 1800-2017 for details about the external names for unnamed generate blocks.

- Attributes that follow the object name belong to the object. You can declare attributes for signals and parameters. vSyn does not support annotating attributes for a declaration or a statement.
2. Declare the attributes in the JSON object format `{"key1":value1, "key2":value2, ...}`.
 3. Specify the JSON file on the `vsyn` command line.

Syntax:

```
vsyn -attribute <file.json> [<other-options>] <source-files>
```

Where,

- `-attribute <file.json>` specifies the JSON file that includes the attribute definition.
- `<other-options>` can be any other available options for `vsyn`.
- `<source-files>` specifies the source files.

Example:

Here shows an example of an RTL source file and a JSON file including attributes for the three modules in the design.

The RTL source file *rtl.v* is shown as follows:

```
module select_a (  
    input a, b, sel,  
    output out  
);  
  
    logic l;  
    assign out = sel ? a : b;  
  
endmodule  
  
module select_b (  
    input a, b, sel,  
    output out  
);  
  
    logic l;  
    assign out = sel ? b : a;  
  
endmodule  
  
module top(  
    input a, b, sel,  
    output out  
);  
  
    parameter SEL_A = 1;  
  
    generate  
        if (SEL_A) begin  
            logic l0;  
            select_a inst_a(a, b, sel, out);  
        end else begin  
            logic l1;  
            select_b inst_b(a, b, sel, out);  
        end  
    endgenerate  
  
endmodule
```

The JSON file *attr.json* is shown as follows:

```
{
  "select_a": {
    "l": {
      "attr_for_logic": 1
    }
  },
  "select_b": {
    "l": {
      "attr_for_logic": 1
    }
  },
  "top": {
    "genblk1": {
      "l0": {
        "attr_if0": 1
      },
      "l1": {
        "attr_if1": 1
      }
    }
  }
}
```

The following command line annotates attributes from the JSON file, *attr.json*, in the source file, *rtl.v*.

```
% vsyn rtl.v -attribute attr.json -o vsyn.g.v
```

The attribute declarations in the JSON file is equivalent to the attribute instances that appear in the Verilog description (shown as follows).

```

module select_a (
    input a, b, sel,
    output out
);

    (* attr_for_logic *) logic l;
    assign out = sel ? a : b;

endmodule

module select_b (
    input a, b, sel,
    output out
);

    (* attr_for_logic *) logic l;
    assign out = sel ? b : a;

endmodule

module top(
    input a, b, sel,
    output out
);

    parameter SEL_A = 1;

    generate
        if (SEL_A) begin
            (* attr_if0 *) logic l0;
            select_a inst_a(a, b, sel, out);
        end else begin
            (* attr_if1 *) logic l1;
            select_b inst_b(a, b, sel, out);
        end
    endgenerate

endmodule

```

4.9 Setting the Level of RTL Signal Preservation

By default, vSyn preserves the sequential logic and optimizes the combinational logic in the RTL code. You can use the `-visibility` to set the signal visibility to preserve all the signals or to perform more optimizations. Valid values of visibility are `module`, `state`, and `port`. By default, the visibility is set to `state`.

- `module` keeps all the signals in the module from being optimized. Set the visibility to

`module` if you want to debug the internal signals at runtime.

- `state` preserves all ports and the sequential logic.
- `port` prohibits visibility of all internal signals and keeps only the ports from being optimized.

4.10 Specifying the Compilation Unit Mode

The synthesizer can work in two modes:

- Single-file Compilation Unit (SFCU) Mode

By default, the SFCU mode is enabled. In this mode, all files in the command line are treated as a single compilation unit. The declarations in each file are accessible within the whole files.

- Multi-file Compilation Unit (MFCU) Mode

In the MFCU mode, each file in the command line is treated as a separate compilation unit. The declarations in each unit scope are accessible only within its corresponding file. You can use the `-separate-unit` option in the `vsyn` command line to enable the MFCU mode.

```
vsyn -separate-unit <source-files> [<other-options>]
```

In the following example, the source files *define.v* and *dut.v* are specified in the command line.

define.v code:

```
`define M
```

dut.v code:

```
module top(
    output out
);

`ifdef M
    assign out = 1;
`else
    assign out = 0;
`endif

endmodule
```

By default, the two files are considered as a single compilation unit, so the output signal `out` is assigned to `1` after the synthesis.


```
% vsyn define.v dut.v -o dut.g.v -q
```

dut.g.v:

```
module top (output out);

    assign out = 1'b1;

endmodule
```

If the `-separate-unit` option is used to enable the MFCU mode, the output signal `out` is assigned to `0` after the synthesis.

```
% vsyn define.v dut.v -o dut.g.v -separate-unit -q
```

dut.g.v:

```
module top (output out);

    assign out = 1'b0;

endmodule
```

4.11 Writing an STB

A testbench is a Verilog module which simulates and analyzes the DUT. A testbench module is usually not synthesizable. vSyn supports synthesizing partial unsynthesizable testbench written in the RTL code. The supported testbench is called *synthesizable testbench* (STB). You can simulate the testbench on the hardware at runtime to check the functionality of the DUT.

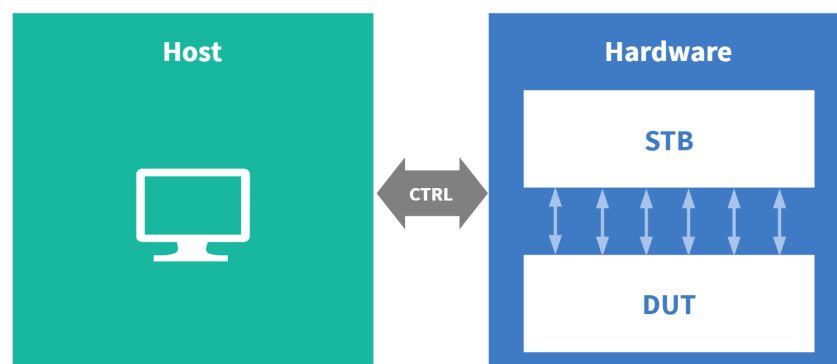


Figure 4-2: Using an STB

General guidelines

The general guidelines for writing an STB are as follows:

- All the constructs must be in initial blocks.
- Multiple event control statements are not allowed in a single initial block. In other words, you must use the same clock signal and the same edge (all posedge or all negedge).
- Each construct has the following two modes. All constructs listed in this section support both modes.
 - Top module with ports. For example,

```
module Tb(clk, d0, d1, d2, q);
.....
```

- Top module without ports. For example,

```
module Tb();
bit d0, d1, d2;
bit clk;
bit q;
.....
```

**TIP**

Use the syntax provided in this topic. Other testbench syntax might not be supported.

Limitations

The limitations for writing an STB are as follows:

- Mixing variable initialization and event control statements in one initial block is not supported currently. For example, in the following construct, the initial value of `q` is ignored after vSyn synthesis.

```
initial begin
  q = -1;
  @(posedge clk) q = d0;
  @(posedge clk) q = d1;
  @(posedge clk) q = d2;
end
```

- You can only declare the loop variable globally. The format `for (int i = 0; ...)` is not supported in the STB.
- The coding style `forever #7 q = ~q` in the testbench is not supported in the STB. The iteration times must be limited.

This topic provides testbench example code and describes behavior for the following syntax:

- Value Initialization
- Event Control
 - Implicit Finite State Machine
 - Repeat with Constant Counts
 - Repeat with Variable Counts
 - While Loop
 - Do-While Loop
 - For Loop
 - Forever Loop
 - Foreach Loop
- Cross Module Reference
 - Downward Cross Module Reference
 - Upward Cross Module Reference
 - Upward and Downward Cross Module Reference

4.11.1 Value Initialization

Two ways to initialize variables:

- Using global declarations, such as `bit a = 1 ;`
- Using the blocking assignment in initial blocks, such as `initial a = 1 ;`

4.11.2 Event Control

4.11.2.1 Implicit Finite State Machine

Implicit FSMs can simulate value transition that is controlled by multiple clock edges during the initial stage of simulation. Here shows an example of the supported implicit FSM construct in the STB and describes how it behaves in the simulation.

Testbench Code Example

```
bit q = 1;
initial begin
  @(posedge clk) q = d0;
  @(posedge clk) q = d1;
  @(posedge clk) q = d2;
end
```

Signals and Variables

- `q` is the output variable.
- `clk` is the system clock used to control value transitions.

- `d0` , `d1` , and `d2` are input signals.

Behavior

In the time slot 0, `q` is set to `1` . After the first rising edge of `clk` , `q` is set to `d0` . After the second rising edge of `clk` , `q` is set to `d1` . After the third rising edge of `clk` , `q` is set to `d2` .

4.11.2.2 Repeat with Constant Counts

The repeat construct with constant counts can simulate repeating the clock edge event for constant times. Here shows an example of the supported repeat construct with constant counts in the STB and describes how it behaves in the simulation.

Testbench Code Example

```
initial q = 1;
initial begin
repeat (7) @(posedge clk);
q = d0;
end
```

Signals and Variables

- `q` is the output variable.
- `clk` is the system clock used to control value transitions.
- `d0` is the input signal.

Behavior

In the time slot 0, `q` is set to `1` . After seven rising edges of `clk` , `q` is set to `d0` .

4.11.2.3 Repeat with Variable Counts

The repeat construct with variable counts can simulate repeating variable times of clock edge event. Here shows an example of the supported repeat construct with variable counts in the STB and describes how it behaves in the simulation.

Testbench Code Example

```
bit q = 1;
reg [3:0] cnt = 4'b0111;

initial begin
repeat (cnt) @(posedge clk);
q = d0;
end
```

Signals and Variables

- `q` is the output variable.
- `cnt` is the repeat count.
- `clk` is the system clock used to control value transitions.
- `d0` is the input signal.

Behavior

In the time slot 0, `q` is set to `1` and `cnt` is set to `7`. After seven rising edges of `clk`, `q` is set to `d0`.

4.11.2.4 While Loop

While loop can repeat clock edge events. The repeat count is specified by a while loop. Here shows an example of the supported while loop construct in the STB and describes how it behaves in the simulation.

Testbench Code Example

```
initial begin
  q = 1;
  cnt = 3'b000;
end

while (cnt < 3'011) begin
  @(posedge clk)
  cnt++;
end

q = d0;
```

Signals and Variables

- `q` is the output variable.
- `cnt` is the control variable used in the while condition.
- `clk` is the system clock used to control value transitions.
- `d0` is the input signal.

Behavior

In the time slot 0, `q` is set to `1` and `cnt` is set to `0`. Then after three rising edges of `clk`, `q` is set to `d0`.

4.11.2.5 Do-While Loop

The do-while loop can repeat clock edge events. The repeat count is specified by a while loop.

The do-while loop is similar to the while loop, except that the do-while loop waits at least one clock event. Here shows an example of the supported do-while loop construct in the STB and describes how it behaves in the simulation.

Testbench Code Example

```
bit q = 1;
reg [3:0] cnt = 3'b000;

initial begin
    do begin
        @(posedge clk);
    end while (cnt < 3'b011);

    @(posedge clk) q = d0;
end
```

Signals and Variables

- `q` is the output variable.
- `cnt` is the control variable used in the while condition.
- `clk` is the system clock used to control value transitions.
- `d0` is the input signal.

Behavior

In the time slot 0, `q` is set to `1` and `cnt` is set to `0`. After three rising edges of `clk`, `q` is set to `d0`.

4.11.2.6 For Loop

The for loop can repeat clock edge events. The repeat count is controlled by the for termination condition. Here shows an example of the supported for loop construct in STB and describes how it behaves in the simulation.

Testbench Code Example

```
int i;
reg q = 1;
initial begin
    for (i = 0; i < 7; i++)
        @(posedge clk);
end
q = d0;
```

Signals and Variables

- `i` is the loop variable.
- `q` is the output variable.
- `clk` is the system clock used to control value transitions.
- `d0` is the input signal.

Behavior

In the time slot 0, `q` is set to `1`. After seven rising edges of `clk`, `q` is set to `d0`.

4.11.2.7 Forever Loop

The forever loop can repeat executing sequential blocks for infinite times. Here shows an example of the supported forever loop construct in the STB and describes how it behaves in the simulation.

Testbench Code Example

```
reg q = 1;
reg [7:0] cnt = 0;
initial begin
    forever begin
        @(posedge clk);
        cnt++;
        if (cnt > 7) begin
            cnt = 0;
            q = ~q;
        end
    end
end
```

Signals and Variables

- `q` is the output variable.
- `cnt` is the variable to control the waveform of the output values.
- `clk` is the system clock used to control value transitions.

Behavior

The initial value of the output `q` is `0`. `q` changes its value every seven rising edges of `clk`.

4.11.2.8 Foreach Loop

The foreach loop is used to specify the iteration over the elements of an array. Here shows an example of the supported foreach loop construct in STB and describes how it behaves in the simulation.

Testbench Code Example

```

logic [7:0] arr [0:2] = '{1,2,3};
reg q= 1;
initial begin
    @(posedge clk);
    foreach(arr[i]) begin
        q += i + arr[i];
    end
end
end

```

Signals and Variables

- `arr` is the array on which the foreach loop operates.
- `q` is the output variable.
- `clk` is the system clock used to control value transitions. Note that the clock event statement `@(posedge clk);` is necessary for the initial value of `q` to be effective.
- `i` is the loop variable.

Behavior

After the rising edge of `clk`, `q` is set to 9 $((0+1)+(1+2)+(2+3))$.

4.11.3 Cross Module Reference

4.11.3.1 Downward Cross Module Reference

Downward Cross Module Reference (XMR) can access identifiers cross modules via downward path references. The downward path references start with instance names. Here shows an example of supported downward XMR construct in the STB and describes how it behaves in the simulation.

Testbench Code Example

```

sub inst();

initial begin
    @(posedge clk);
    inst.t = d0 & d1;
end

assign q = inst.t;

```

Signals and Variables

- `clk` is the system clock used to control value transitions. Note that if you remove the clock event statement, you can only use constant assignments. In other words, if you remove the statement `@(posedge clk);` in the example above, you must replace `d0 &`

`d1` with `1'b1` or other constant.

- `inst.t` is the downward path reference.
- `d0` and `d1` are input signals.
- `q` is the output variable.

Behavior

After the rising edge of `clk`, `q` is set to `d0 & d1`.

4.11.3.2 Upward Cross Module Reference

Upward XMR can access identifiers cross modules via upward path references. The upward path references start with module names. Here shows an example of supported upward XMR construct in the STB and describes how it behaves in the simulation.

Testbench Code Example

```
module tb(input clk, d0, d1, output bit q);
  sub inst(clk);
endmodule

module sub(clk);
  initial begin
    @(posedge clk);
    tb.q = tb.d0 & tb.d1;
  end
endmodule
```

Signals and Variables

- `clk` is the system clock used to control value transitions. Note that you cannot remove the clock event statement even for constant assignments. In other words, you cannot remove the statement `@(posedge clk);` in the example above even if you replace `tb.d0 & tb.d1` with `1'b0` or other constant.
- `d0` and `d1` are input signals.
- `q` is the output variable.
- `tb.q`, `tb.d0`, and `tb.d1` are upward path references.

Behavior

After the rising edge of `clk`, `q` is set to `d0 & d1`.

4.11.3.3 Upward and Downward Cross Module Reference

Upward-and-downward XMR can access identifiers cross modules via upward-and-downward path references. Here shows an example of supported upward-and-downward XMR construct

in the STB and describes how it behaves in the simulation.

Testbench Code Example

```
module tb(input clk, d0, d1, output bit q);
  sub1 inst1(clk, q);
  sub2 inst2(d0, d1);
endmodule

module sub1(clk, output bit q);
  initial begin
    @(posedge clk);
    q = tb.inst2.t1 ^ tb.inst2.t2;
  end
endmodule

module sub2(input d0, d1);
  bit t1, t2;
  assign t1 = d0 & d1;
  assign t2 = d0 | d1;
endmodule
```

Signals and Variables

- `clk` is the system clock used to control value transitions. Note that you cannot remove the clock event statement even for constant assignments.
- `d0` and `d1` are input signals.
- `q`, `t1`, and `t2` are the output variables.
- `tb.inst2.t1` and `tb.inst2.t2` are upward-and-downward path references.

Behavior

After the rising edge of `clk`, `q` is set to $(d0 \& d1) \wedge (d0 \mid d1)$.

4.12 Using the Stop Control System Task

In the vSyn synthesis flow, HPE supports adding `$stop` system tasks in RTL code. `$stop` can suspend the clock at runtime. The syntax for the stop control system task is shown as follows:

Syntax:

```
$stop [<n>];
```

Where `<n>` specifies the type of diagnostic message that is printed. Valid values are as follows:

- `0` prints nothing.
- `1` prints running time and location. If no type is specified, the default value is `1`.

- `2` prints running time, location, memory in use, and CPU usage.

4.13 Generating an XNDB Netlist for Full Vision

To enable Full Vision at runtime, you need to generate an XNDB netlist, additional to the gate-level netlist, at synthesis time and load the netlist at compile time.

Use the `-xndb_dbg <file>` option of the `vsyn` command to generate an XNDB netlist.

Example:

```
vsyn -xndb_dbg fnl.db -o netlist.v -top top dut.v
```



TIP

The synthesis time might increase when you add the `-xndb_dbg` option for netlist generation.

4.14 Support for Encryption

HPE supports the IEEE 1735-2014 version 1 (V1) and version 2 (V2) encryption standard, [Recommended Practice for Encryption and Management of Electronic Design Intellectual Property \(IP\)](https://ieeexplore.ieee.org/document/7274481) (<https://ieeexplore.ieee.org/document/7274481>). The IP protection is supported at all stages of the HPE tool flow.

4.14.1 Encryption Envelopes Version 1

Before encrypting source files, you must create encryption envelopes which contain information that identifies protected envelopes using the ``pragma protect` expressions.

An example of the encryption envelope for version 1 is as follows. `<key_owner>`, `<key_keyname>`, `<key_method>`, and `<XEPIK_Public_Key>` represent the content from XEPIC. `<Module_Definition>` represents partial or the whole definition of the module that to be encrypted.

```

`pragma protect data_method = "aes128-cbc"
`pragma protect encoding = (enctype = "base64")
`pragma protect key_keyowner = <key_owner>
`pragma protect key_method = <key_method>
`pragma protect key_keyname = <key_keyname>
`pragma protect key_public_key
<XEPIC_Public_Key>

`pragma protect begin
<Module_Definition>
`pragma protect end

```

Where,

Pragma Keyword	Description	Type
Mandatory		
key_keyowner	Specifies the owner of the key encryption key.	Required
key_keyname	Specifies the name of the key encryption key.	Required
key_public_key	Specifies the public key for key encryption.	Required
Optional		
key_method	Specifies the key encryption algorithm.	Optional
data_method	Specifies the data encryption algorithm.	Optional
encoding	Specifies the coding scheme for encrypted data.	Optional
begin	Opens a new encryption envelope.	Required
end	Closes an encryption envelope.	Required

TIP

When you want to use an X-EPIC public key, you must ask XEPIC for `key_keyowner` , `key_method` , `key_keyname` , and `key_public_key` .

4.14.2 Encryption Envelopes Version 2

An example of the encryption envelope for version 2 is as follows. `<key_owner>` , `<key_keyname>` , `<key_method>` , and `<XEPIC_Public_Key>` represent the content from XEPIC. `<Module_Definition>` represents partial or the whole definition of the module that to be encrypted.

```

`pragma protect version = 2

`pragma protect begin_commonblock
`pragma protect control error_handling="delegated"
`pragma protect control runtime_visibility="delegated"
`pragma protect end_commonblock

`pragma protect begin_toolblock
`pragma protect rights_digest_method="sha256"
`pragma protect key_keyowner =<key_owner>
`pragma protect key_method =<key_method>
`pragma protect key_keyname =<key_name>
`pragma protect key_public_key
<XEPIC_public_key>
`pragma protect control error_handling="delegated"
`pragma protect end_toolblock=""

`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (encype = "base64")

`pragma protect begin
<Module_Definition>
`pragma protect end

```

Where,

Pragma Keyword	Description	Type
Version		
version	Indicates the version 2 of encryption envelops.	Required
Common block		
begin_commonblock	Begins a common block.	Optional
control	Specifies a common right.	Optional
end_commonblock	Ends a common block.	Optional
Tool block		
begin_toolblock	Begins a tool block that contains the encryption key and the tool-specific rights.	Required
rights_digest_method	Specifies the hash algorithm.	Required
key_keyowner	Specifies the owner of the key encryption key.	Required
key_method	Specifies the key encryption algorithm.	Optional
key_keyname	Specifies the name of the key encryption key.	Required

Pragma Keyword	Description	Type
<code>key_public_key</code>	Specifies the public key for key encryption.	Required
<code>control</code>	Specifies a tool-specified right.	Optional
<code>end_toolblock</code>	Ends a tool block. After encryption, the hash result of the tool block and common block is placed in this pragma.	Required
Data encryption algorithm		
<code>data_method</code>	Specifies the data encryption algorithm.	Optional
Coding scheme		
<code>encoding</code>	Specifies the coding scheme for encrypted data.	Optional
Protected region		
<code>begin</code>	Opens a new encryption envelope.	Required
<code>end</code>	Closes an encryption envelope.	Required

 TIP

When you want to use an X-EPIC public key, you must ask XEPIC for `key_keyowner` , `key_method` , `key_keyname` , and `key_public_key` .

4.14.3 Synthesizing and Compiling Encrypted IP Files

To synthesize an encrypted RTL file, which has decryption envelopes, you can use vSyn in the same way as you synthesize unprotected files. Irrespective of the encryption standard of the original RTL IPs, the output netlist generated by vCom is compliant with the IEEE 1735 V2 encryption standard.

One module in the RTL can be fully encrypted or partially encrypted. A typical usage of partial protection is to encrypt certain ports and all logic of a module so that to expose the ports for debugging or other purposes. Another usage of partial protection, which is not commonly used, is to encrypt a part or parts of module logic. In the gate-level netlist generated by vSyn, all logic in the module is encrypted, except the instances of unencrypted memories and vCom primitives.

 TIP

You can create encryption envelopes at the granularity of the module. And then encrypt Verilog RTL IPs with the GalaxSim simulator or with any other encryption tools that support the V1 or V2 encryption standard. For details about using GalaxSim for IP protection, refer to the “IP Protection” section in *GalaxSim User Guide*.

You can use the X-EPIC public key or the supported third-party key. The basic principle is that use the tool which provides the public encryption key to decrypt and synthesize source files. The following table lists the supported encryption tool and synthesizer for source files with only X-EPIC encryption public key, only third-party encryption public key, or both of them.

Public Encryption Key	Encryption Tool	Synthesizer
X-EPIC	GalaxSim or other encryption tools	vSyn
Third-party tool	GalaxSim or other encryption tools	Third-party tool
X-EPIC and third-party tool	GalaxSim or other encryption tools	vSyn or third-party tool

vCom compiles the gate-level netlist in accordance with the same security level and encryption algorithm as the encrypted RTL file. If a module is partially protected, vCom keeps the top ports of the module, you can access the ports by setting them as read nets or write nets. However, you cannot debug the signals that are encrypted in the module.

4.14.4 Example for Encryption Support

The module in the design can be fully encrypted or partially encrypted as shown in the examples in this topic.

- **Example 1:** The whole module is encrypted by GalaxSim in a version 1 encryption envelope. After vSyn synthesis, no module information is displayed in the netlist.
- **Example 2:** All logic definition and the port definition related to memory `mem_cd` are encrypted by GalaxSim in version 2 encryption envelopes. After vSyn synthesis, ports with no protection are kept. You can set the ports to read nets or write nets.

Example 1

The following example encrypts a design, `add.v`, with GalaxSim and synthesizes the encrypted design with vSyn.

1. Prepare the original RTL and use the protection pragmas to create version 1 encryption envelopes for module `a`.

The *add.v* RTL file is shown as follows:

```
`pragma protect key_keyowner = <key_owner>
`pragma protect key_method = <key_method>
`pragma protect key_keyname = <key_keyname>
`pragma protect key_public_key
<XEPIK_Public_Key>

`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (enctype = "base64")

`pragma protect begin
module a (
    input wire clk,
    input wire a,
    input wire b,
    output reg c
);

    always @(posedge clk) begin
        c <= a + b;
    end

endmodule
`pragma protect end
```

2. Use GalaxSim to encrypt the RTL.

```
% galaxsim -protect add.v -key key_v1.txt

[add.v] encrypted, overwritten to [add.vp].
Fri May 19 11:04:32 2023
```

The output *add.vp* encrypted file is shown as follows:


```

`pragma protect begin_protected
`pragma protect version=1
`pragma protect encrypt_agent="galaxsim"
`pragma protect encrypt_agent_info="https://www.x-epic.com"
`pragma protect key_keyowner = <key_owner>
`pragma protect key_method = <key_method>
`pragma protect key_keyname = <key_keyname>
`pragma protect encoding = (enctype = "base64", line_length = 64, bytes =
256)
`pragma protect key_block
<Public_Encryption_Key>

`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (enctype = "base64", line_length = 64, bytes =
176)
`pragma protect data_block
8nk1lgWBKQeR5H0C2ABVGyKcktJ0esL29lpTJj1uyIvJjrI5dAxmW5RBzdoSiDga
iVtXd+FVIz+xySgS3CXAes6JHEyCGy6sZ6xqff0pd12wSupg5ohjtEKDQqUu0LKk
f++LuE+s/uJXt15dGZUQp4QjL6BGxtweaGtEg3pTBHGXHt80jto1qU15TXGEJahm
0SrbIfXfFXPDxf4Al4Ipn0yrJvpCMC186LGQR0/eBg=
`pragma protect end_protected

```

3. Use vSyn to synthesize the encrypted RTL.

```
% vsyn add.vp -o add.vm
```

The output *add.vm* netlist in version 2 is shown as follows:

```
// VSYN v23.09.00 x86_64 Sept.26 2023

`pragma protect begin_protected
`pragma protect version=2
`pragma protect encrypt_agent="vsyn"
`pragma protect encrypt_agent_info="https://www.x-epic.com"
`pragma protect begin_commonblock
`pragma protect end_commonblock
`pragma protect begin_toolblock
`pragma protect rights_digest_method="sha256"
`pragma protect key_keyowner = <key_owner>
`pragma protect key_method = <key_method>
`pragma protect key_keyname = <key_keyname>
`pragma protect key_block
<Public_Encryption_Key>
`pragma protect end_toolblock="<tool_block>"

`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (enctype = "base64", line_length = 64, bytes =
208)
`pragma protect data_block
8nk1lgWBKQeR5H0C2ABVGSB7A8qKfu+vPyYmZz2p6keadPVQam7p6G96nuTNq/ug
GNwTC7DSUSXK7/DYH4naiuJDwW8Wnml0HDpB2FtVj7yrR3PuFBYYdDEV6Rd8T+ZW
sNE6++No11Bdb+nBVkQUm0B1Exznxd0hJsiKACnBP0X0DqyMQnLBTg/OMjxwEGtH
AI0j3JLZu/ACHjc6xk4R7LFvY5Bzn9GwNtVsJMr8Yx3/PgRPwLfYzH1AU/eJ+vEf
s5UND270DLj6DJk2YKe0fA==
`pragma protect end_protected
```

Example 2

The following example encrypts a design, `mem.v`, with GalaxSim and synthesizes the encrypted design with vSyn.

1. Prepare the original RTL and use the protection pragmas to create version 2 encryption envelopes for the second always statement in module `test`.

The `mem.v` RTL file is shown as follows:

```
module test (
    input  clk,
    input  rstn,
    input          en_a,
    input [7:0]    addr_a,
    input [7:0]    data_a,

    input          en_b,
    input [7:0]    addr_b,
    output reg [7:0] dout_b,
```

```

`pragma protect begin
    input          en_c,
    input  [7:0]   addr_c,
    input  [7:0]   data_c,

    input          en_d,
    input  [7:0]   addr_d,
    output reg [7:0] dout_d
`pragma protect end
);

    logic [7:0] mem_ab [255:0];
    reg  [7:0] mem_ab_vr;
    wire [7:0] mem_ab_vr_err;

`pragma protect begin
    logic [7:0] mem_cd [255:0];
    reg  [7:0] mem_cd_vr;
    wire [7:0] mem_cd_vr_err;
`pragma protect end

`pragma protect begin
    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            mem_ab_vr <= 0;
        end else begin
            if (en_a) begin
                mem_ab[addr_a] <= data_a;
            end else begin
                mem_ab_vr <= data_a;
            end
        end
    end
    assign mem_ab_vr_err = mem_ab_vr + 1;

    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            dout_b <= 0;
        end else begin
            if (en_b) begin
                dout_b <= mem_ab[addr_b];
            end else begin
                dout_b <= 0;
            end
        end
    end

    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            mem_cd_vr <= 0;

```

```

    end else begin
        if (en_c) begin
            mem_cd[addr_c] <= data_c;
        end else begin

            mem_cd_vr <= data_c;
        end
    end
end
assign mem_cd_vr_err = mem_cd_vr + 1;

always @(en_d,addr_d,dout_d) begin
    if (en_d) begin
        dout_d = mem_cd[addr_d];
    end else begin
        dout_d = 0;
    end
end

`pragma protect end

endmodule

```

2. Use GalaxSim to encrypt the RTL.

```

% galaxsim -protect mem.v -key key_v2.txt

[mem.v] encrypted, overwritten to [mem.vp].
Mon Jul 31 13:33:01 2023

```

The key file is shown as follows:

```

`pragma protect version = 2
`pragma protect begin_commonblock
`pragma protect control error_handling="delegated"
`pragma protect control runtime_visibility="delegated"
`pragma protect end_commonblock
`pragma protect begin_toolblock
`pragma protect rights_digest_method="sha256"
`pragma protect key_keyowner = <key_owner>
`pragma protect key_method = <key_method>
`pragma protect key_keyname = <key_keyname>
`pragma protect key_public_key
<Public_Encryption_Key>
`pragma protect end_toolblock=""
`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (enctype = "base64")

```

The output *mem.vp* encrypted file is shown as follows:

```

module test (
  input    clk,
  input    rstn,
  input          en_a,

  input  [7:0]  addr_a,
  input  [7:0]  data_a,
  input          en_b,

  input  [7:0]  addr_b,
  output reg [7:0]  dout_b,

  `pragma protect begin_protected
  `pragma protect version=2
  `pragma protect encrypt_agent="galaxsim"
  `pragma protect encrypt_agent_info="https://www.x-epic.com"
  `pragma protect begin_commonblock
  `pragma protect control_error_handling="delegated"
  `pragma protect control_runtime_visibility="delegated"
  `pragma protect end_commonblock
  `pragma protect begin_toolblock
  `pragma protect rights_digest_method="sha256"
  `pragma protect key_keyowner=<key_owner>
  `pragma protect key_method=<key_method>
  `pragma protect key_keyname=<key_keyname>
  `pragma protect key_block
  <Public_Encryption_Key>
  `pragma protect end_toolblock="<tool_block>"
  `pragma protect data_method = "aes256-cbc"
  `pragma protect encoding = (enctype = "base64", line_length = 64, bytes =
160)
  `pragma protect data_block
Axbr4tmaJRzG3f5g0leu9kG9BV4YE8Iqiiuqmo+88GKW/oJP0Uwcj7QFINVlSu21
IEUAj/ud4A6/Jb9X4wQE2PMQLIcaj99X7yCxcqKYhog0oTNQLgaqIg6TGYcwDfaW
XFiB0ThoehksUpzDQIhbqYsoUD10l4GPqHpyCCKB5mea6al24mnvxmKt04Ttxt0go
rjNbbW/koZ7NfisZ0azkGw==
  `pragma protect end_protected
);

  logic [7:0] mem_ab [255:0];
  reg    [7:0] mem_ab_vr;
  wire   [7:0] mem_ab_vr_err;

  `pragma protect begin_protected
  `pragma protect version=2
  `pragma protect encrypt_agent="galaxsim"
  `pragma protect encrypt_agent_info="https://www.x-epic.com"
  `pragma protect begin_commonblock

```

```

`pragma protect control error_handling="delegated"
`pragma protect control runtime_visibility="delegated"
`pragma protect end_commonblock
`pragma protect begin_toolblock
`pragma protect rights_digest_method="sha256"
`pragma protect key_keyowner=<key_owner>
`pragma protect key_method=<key_method>
`pragma protect key_keyname=<key_keyname>

`pragma protect key_block
<Public_Encryption_Key>
`pragma protect end_toolblock="<tool_block>"

`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (enctype = "base64", line_length = 64, bytes =
112)
`pragma protect data_block
ucgjiLNZ4ufSZo+EahkU+Z9WxPFwWXvtm94WU1XRXlk/zLT41D5bd2R+dLleEu2Y
.....
`pragma protect end_protected

`pragma protect begin_protected
`pragma protect version=2
`pragma protect encrypt_agent="galaxsim"
`pragma protect encrypt_agent_info="https://www.x-epic.com"
`pragma protect begin_commonblock
`pragma protect control error_handling="delegated"
`pragma protect control runtime_visibility="delegated"
`pragma protect end_commonblock
`pragma protect begin_toolblock
`pragma protect rights_digest_method="sha256"
`pragma protect key_keyowner=<key_owner>
`pragma protect key_method=<key_method>
`pragma protect key_keyname=<key_keyname>
`pragma protect key_block
<Public_Encryption_Key>
`pragma protect end_toolblock="<tool_block>"

`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (enctype = "base64", line_length = 64, bytes =
944)
`pragma protect data_block
y9fr0qPwi9B29m26CkhM9wGYbYDhV5y6Pj6v+lEcVsvd+zVKsrhbDDfLA9Zh9zky
OkINrtqPtmESCf/QM8uq2CScUZGbKmCIuyP/V4YwCqCP1dwC2zR+XKm9IympCtBb
.....
`pragma protect end_protected

endmodule

```

3. Use vSyn to synthesize the encrypted RTL.

```
% vsyn mem.vp -o mem.vm
```

The output *mem.vm* netlist is shown as follows:

```
// VSYN v23.09.00 x86_64 Sept.26 2023

module test (input clk, input rstn, input en_a, input [7:0] addr_a,
             input [7:0] data_a, input en_b, input [7:0] addr_b, output [7:0]
dout_b,

`pragma protect begin_protected
`pragma protect version=2
`pragma protect encrypt_agent="vsyn"
`pragma protect encrypt_agent_info="https://www.x-epic.com"
`pragma protect begin_commonblock
`pragma protect control_error_handling="delegated"
`pragma protect control_runtime_visibility="delegated"
`pragma protect end_commonblock
`pragma protect begin_toolblock
`pragma protect rights_digest_method="sha256"
`pragma protect key_keyowner=<key_owner>
`pragma protect key_method=<key_method>
`pragma protect key_keyname=<key_keyname>
`pragma protect key_block
<Public_Encryption_Key>
`pragma protect end_toolblock="<tool_block>"
`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (encype = "base64", line_length = 64, bytes =
128)
`pragma protect data_block
Axbr4tmaJRzG3f5g0leu9q12dL8+Da2J3qIeRttQzZxoinU10d7qqpc1+oMD+QTH
.....
`pragma protect end_protected

);

`pragma protect begin_protected
`pragma protect version=2
`pragma protect encrypt_agent="vsyn"
`pragma protect encrypt_agent_info="https://www.x-epic.com"
`pragma protect begin_commonblock
`pragma protect control_error_handling="delegated"
`pragma protect control_runtime_visibility="delegated"
`pragma protect end_commonblock
`pragma protect begin_toolblock
`pragma protect rights_digest_method="sha256"
`pragma protect key_keyowner=<key_owner>
`pragma protect key_method=<key_method>
```

```

`pragma protect key_keyname=<key_keyname>
`pragma protect key_block
<Public_Encryption_Key>
`pragma protect end_toolblock="<tool_block>"

`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (enctype = "base64", line_length = 64, bytes = 32)
`pragma protect data_block
Axbr4tmaJRzG3f5g0leu9nS/zGMR9c+keeqlb1F0M+U=
`pragma protect end_protected

    wire [7:0] mem_ab_vr;
`pragma protect begin_protected
`pragma protect version=2
`pragma protect encrypt_agent="vsyn"
`pragma protect encrypt_agent_info="https://www.x-epic.com"
`pragma protect begin_commonblock
`pragma protect control_error_handling="delegated"
`pragma protect control_runtime_visibility="delegated"
`pragma protect end_commonblock
`pragma protect begin_toolblock
`pragma protect rights_digest_method="sha256"
`pragma protect key_keyowner=<key_owner>
`pragma protect key_method=<key_method>
`pragma protect key_keyname=<key_keyname>
`pragma protect key_block
<Public_Encryption_Key>
`pragma protect end_toolblock="<tool_block>"

`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (enctype = "base64", line_length = 64, bytes = 48)
`pragma protect data_block
Axbr4tmaJRzG3f5g0leu9jd7w4v7MXNBaVrXjGLZiEYyzG1q3ePcY5ydg3M1Zdc0
`pragma protect end_protected

    \VCOMSDPRAM%8%1%8%8%8 mem_ab (.clka(clk), .clkb(clk), .rstb(n299),
        .ena(1'b1), .enb(1'b1), .addra({addr_a}), .addrb({addr_b}),
        .dina({data_a}),
        .wea({n300}), .doutb({dout_b}));
`pragma protect begin_protected
`pragma protect version=2
`pragma protect encrypt_agent="vsyn"
`pragma protect encrypt_agent_info="https://www.x-epic.com"
`pragma protect begin_commonblock
`pragma protect control_error_handling="delegated"
`pragma protect control_runtime_visibility="delegated"
`pragma protect end_commonblock
`pragma protect begin_toolblock
`pragma protect rights_digest_method="sha256"
`pragma protect key_keyowner=<key_owner>

```



```

`pragma protect key_method=<key_method>
`pragma protect key_keyname=<key_keyname>
`pragma protect key_block
<Public_Encryption_Key>
`pragma protect end_toolblock="<tool_block>"

`pragma protect data_method = "aes256-cbc"
`pragma protect encoding = (encype = "base64", line_length = 64, bytes =
3488)
`pragma protect data_block
Axbr4tmaJRzG3f5g0leu9mqr5V3s4mickS3FeNtwi9VLZNtn6kkEcbTr0I0Cp6cz
26pwukpIpHjHF+67XEsPzM7B3AaQUT+ay/EIISTq9A0mjlbR7EmZTpZwXpzu63hn
.....
`pragma protect end_protected

endmodule

`ifndef VCOM_COMP

module \VCOMSDPRAM%8%1%8%8%8 #(.AWIDTH=8, DWIDTH=8, BYTE_WRITE_WIDTH=8,
    MEMORY_INIT_FILE="none", READ_RESET_VALUE_B="00", READ_LATENCY_B=1)
(input clka,
    input clkb, input rstb, input ena, input enb, input regceb, input [7:0]
addra,
    input [7:0] addrb, input [7:0] dina, input [0:0] wea, output [7:0]
doutb);

    V$SDPRAM #(.AWIDTH(AWIDTH), .DWIDTH(DWIDTH), .WE_WIDTH(1),
.MEMORY_INIT_FILE(MEMORY_INIT_FILE),
    .READ_RESET_VALUE_B(READ_RESET_VALUE_B),
    .READ_LATENCY_B(READ_LATENCY_B)) sdp_ram_8_8_8_1 (.clka(clka),
    .clkb(clkb), .rstb(rstb), .ena(ena), .enb(enb), .regceb(regceb),
    .addra({addra}), .addrb({addrb}), .dina({dina}), .wea({wea}),
    .doutb({doutb}));

endmodule
`endif

`ifndef VCOM_COMP
module \VCOMDPDISTRAM%8%1%8%8%0%8 #(.AWIDTH=8, DWIDTH=8,
    BYTE_WRITE_WIDTH=8, MEMORY_INIT_FILE="none", READ_DATA_WIDTH_A=0,
    READ_DATA_WIDTH_B=8) (input clka, input ena, input enb, input [7:0]
addra,
    input [7:0] addrb, input [7:0] dina, input [0:0] wea, output [7:0]
douta,
    output [7:0] doutb);

    V$LUTRAM #(.AWIDTH(AWIDTH), .DWIDTH(DWIDTH), .WE_WIDTH(1),
.MEMORY_INIT_FILE(MEMORY_INIT_FILE)) dp_dist_rom_8_8 (.clka(clka),
    .addra({addra}), .addrb({addrb}), .dina({dina}), .wea({wea}),
    .douta({douta}),
    .doutb({doutb}));

```

```
endmodule
`endif
```

4.15 Using X-EPIC IP

You can use X-EPIC IP, third-party IP, and user IP in your design. To use X-EPIC IP, you must obtain a valid license for the IP and the IP can only be synthesized with vSyn. Contact your XEPIC Sales Representative for more information about IP licensing.

For more information about integrating third-party and user IP, see [Netlist Macros](#).

4.16 Using vSyn in Two-step Flow

By default, vSyn analyzes the source files and generates the gate-level netlist in one step. You can use vSyn in two-step flow when you want to only check the design for syntax errors or to use multiple libraries by means of the `-work` option.

Currently, to use two-step synthesis, you need to use the `-database` option in both two steps to specify the same database.

Perform the following steps to use vSyn in two-step flow:

1. Analyze the source files.

```
vsyn <source_files> -c|-parse-only -database <dir> <options>
```

Where,

- `<source_files>` specifies all source files. For more information about different ways of specifying the source files, see [Adding the Synthesis Inputs](#). This option is required.
- `-c` or `-parse-only` controls vSyn to stop after analysis. This option is required.
- `-database <dir>` specifies a database to store intermediate files that result from analyzing the source files. This option is required.
- `<options>` specifies other available vSyn options.

2. Elaborate the design.

```
vsyn -database <dir> -o|-netlist <file> <options>
```

Where,

- `-database <dir>` specifies the database that is generated in the analysis step. This option is required.
- `-o <file>` or `-netlist <file>` specifies the name of the output netlist file. If this

option is not specified, no netlist file is generated.

- `<options>` specifies other vSyn options.

Examples for Using vSyn in Two-step Flow

- Analyzes the source files into the default library, *work* and then elaborates the design.

```
% vsyn add.v sub.v top.v -c -database libs.vsyn
% vsyn -t top -c -database libs.vsyn -o vsyn.g.v
```

- Analyzes the source files into the library, *lib*, using the `-work` option and then elaborates the design.

```
% vsyn add.v sub.v top.v -c -work lib -database libs.vsyn
% vsyn -t lib.top -database libs.vsyn -o vsyn.g.v
```

- Analyzes the source files into multiple libraries, *addlib*, *sublib*, and *toplib* using the `-work` options in sequence. Then elaborates the design with the top module specified as *toplib.top*.

```
% vsyn add.v -c -work addlib -database libs.vsyn
% vsyn sub.v -c -work sublib -database libs.vsyn
% vsyn top.v -c -work toplib -database libs.vsyn
% vsyn -t toplib.top -database libs.vsyn -o vsyn.g.v
```

- Analyzes the source files into multiple libraries using a library map file and a configuration file, and elaborates the design.

```
% vsyn lib.map config.sv add.v sub.v top.v -c -database libs.vsyn
% vsyn -database libs.vsyn -o vsyn.g.v
```

- Analyzes the source files into multiple libraries using a library map file and then elaborates the design with the top module specified because no configuration file is loaded.

```
% vsyn lib.map add.v sub.v top.v -c -database libs.vsyn
% vsyn -t toplib.top -database libs.vsyn -o vsyn.g.v
```

4.17 Enabling Incremental Synthesis

During design house RTL coding iterations or platform bring-up iteration, it is common for the source code to change only minimally. In this scenario, recompiling everything from scratch is a waste of time. Incremental synthesis allows you to do selective synthesis and update database incrementally to improve synthesis-time performance.

Incremental synthesis is disabled by default in vSyn. To enable incremental synthesis, add the `-database <dir>` option on the `vsyn` command line. If the specified directory does not exist, vSyn creates the directory and stores intermediate data into it. If the specified directory exists, vSyn restores the data from the existing directory for current synthesis and stores the latest data into it later.

You can enable incremental synthesis in one-step flow or two-step flow. In two-step flow, the `-database` option must be used in both analysis and elaboration steps and specifies the same directory.

If you change the values of the options that have impact on the generation of gate-level netlist, incremental synthesis will be disabled automatically even though the `-database` option is added. For example, the `-dsp <n>` option sets the gate count threshold for inferring DSP. If the value of `<n>` is changed, the count of inferred DSPs might change, which means that the netlist structure might change. Therefore, the design must be recompiled even though the RTL source code is not changed.

The followings list the options of which the value change can cause incremental synthesis to be disabled.

- `-attribute <file.json>`
- `-basic-gate-only`
- `-bit-enable-for-vcom`
- `-block-ram <n>`
- `-D/-define <macro>[=<value>]`
- `-defparam <name>=<value>`
- `-dsp <n>`
- `-ignore-redefinition`
- `-lut-ram <n>`
- `-no-tech-map`
- `-pmode`
- `-routability`
- `-separate-unit`
- `-std`
- `-t/-top`
- `-u`
- `-undriven-nets <value>`
- `-v`
- `-visibility <value>`
- `-y`

Examples

The examples use a design which instantiates an adder module and a subtraction module in

the top module. The following lists all source files of the design, and only the source file *add.v* is modified with a little change.

```
top.v
add.v
sub.v
lib.map
```

The example shows how to use the incremental synthesis in different ways according to the content configurations of a design.

Example #1: When using the default library, *work*, enables incremental synthesis. By adding the `-append-log` option, the new logs are appended to the default log file, *vsyn.log*.

```
% vsyn top.v add.v sub.v -t top -database libs.vsyn -o vsyn.g.v
% sed -i 's/a + b/a * b/' add.v
% vsyn top.v add.v sub.v -t top -database libs.vsyn -o vsyn.g.v -append-log
```

After executing the above two command lines, vSyn prints the following messages indicating that only the `add` module is recompiled.

```
INFO VSYN-12368 top.v:1: incremental elaboration is restoring netlist 'top'
-- add_2.v:1: compiling module 'add'
-- add_2.v:1: finish compiling module 'add'
INFO VSYN-12368 sub.v:1: incremental elaboration is restoring netlist 'sub'
```

Example #2: When using multiple libraries with the `-work` option, enables incremental synthesis.

```
% vsyn top.v -c -work toplib -database libs.vsyn
% vsyn add.v -c -work addlib -database libs.vsyn
% vsyn sub.v -c -work sublib -database libs.vsyn
% vsyn config.sv -o vsyn.g.v -database libs.vsyn
% sed -i 's/a + b/a * b/' add.v
% vsyn add.v -c -work addlib -database libs.vsyn
% vsyn config.sv -o vsyn.g.v -database libs.vsyn
```

Example #3: When using multiple libraries with a library map file, enables incremental synthesis.

```
% vsyn lib.map config.sv top.v add.v sub.v -database libs.vsyn -o vsyn.g.v
% sed -i 's/a + b/a * b/' add.v
% vsyn lib.map config.sv top.v add.v sub.v -database libs.vsyn -o vsyn.g.v
```

4.18 Specifying High-capacity Memories in vSyn

Large capacity memory can hardly be mapped to FPGA embedded memory resources. By default, vSyn marks memories with over 1GB size as HCMs and vCom maps them to external DDR. In the synthesis time, you can mark specific memories or a certain range of memories as HCMs.

- **User-defined HCM**

Specify a specific memory as an HCM by adding the `xpvs_use_xdram` attribute to its declaration statement in RTL. For example:

```
(* xpvs_use_xdram *) reg [31:0] mem [0:2**26-1];
```

- **Automatic Mark Based on Threshold**

Use the `-hcm <N>` option to specify a threshold. The memories whose size exceeds 2^N bits are marked as HCMs. Valid value of the threshold is `-1` or any non-negative integer smaller than 37. The default threshold is 30, which means that the HCM size threshold is 1Gb. Setting the threshold to `-1` disables automatic mark of HCM.

To avoid using up HCM DDR resources, it is recommended to disable automatic mark of HCM and use vSyn attributes to specify the memories that need to be implemented as HCM.

The memory to be marked as HCM has the following limitations:

- The maximum size is 16GB.
- The maximum data width is 2048 bits.
- The maximum address width is 31 bits.
- Total number of memory ports must be less than or equal to 8.

4.19 Analyzing the Synthesis Results

You can check synthesis log files for more information about synthesis results and check the area report for details about area.

4.19.1 Checking the Synthesis Log File

By default, vSyn generates a log file for each synthesis run. The log file contains critical results and information about the synthesis process.

You can find the following information in the log file:

- Software version
- Command line

- Design files
- FPGA area results
- Error, warning, and notes
- Performance summary.

The default log file is *vsyn.log*. You can use the `-l <file>` or `-log <file>` option to specify another log file. You can also omit the log file using the `-nolog` option. In the elaboration step of two-step synthesis or the incremental synthesis, if you do not want the new messages to overwrite the old one, add the `-append-log` option on the `vsyn` command line.

4.19.2 Checking the Synthesis Area Report

To get the FPGA resources that the synthesized design utilized, you can refer to the area report which is saved as *vsyn_area.rpt* by default. If required, you can specify another file name by using the `-area-report <file>` option on the `vsyn` command line.

An example of the area report is shown as follows:

vSyn Area Report Example										
<pre> 1 #----- 2 # Version : XEPIC VSYN v24.06.00.d079 x86_64 May.27 2024 3 # Command : vsyn rams_sp_nc.v -o vsyn.g.v -block-ram -l 4 # Date : Mon May 27 17:36:17 2024 5 #----- </pre>						Tool Version and Command History				
<pre> 7 1. Design Utilization 8 ----- 9 LUT* 1030 10 LUT1 0 0 % 11 LUT2 2 0 % 12 LUT3 8 0 % 13 LUT4 64 5 % 14 LUT5 0 0 % 15 LUT6 993 93 % 16 LUT6_2 0 0 % 17 SOFT_HLUTNM 74 18 MUXF7 0 19 MUXF8 0 20 CARRY 0 21 DSP 0 22 RAM (Estimation*) 23 BRAM18 (16kb) 0 24 URAM288 (288kb) 0 25 DLUTM (64b) 0 26 SLUTM (32b) 256 27 Register 16400 28 Latch 0 29 ----- 30 LUT * : LUT = LUT1 + LUT2 + LUT3 + LUT4 + LUT5 + LUT6 + LUT6_2 - SOFT_HLUTNM / 2 = 1067 - 74 / 2 = 1030 31 Estimation * : RAM number here is estimated, accurate count will be reported by vcom 32 </pre>						Total Resource Count by Type				
<pre> 34 2. Memory Utilization 35 ----- 36 Module BRAM18(16kb) URAM288(288kb) DLUTM(64b) SLUTM(32b) Instance 37 ----- ----- ----- ----- ----- ----- 38 rams_sp_nc 0 0 0 256 1 39 ----- ----- ----- ----- ----- ----- 40 </pre>						Memory Resource Count by Module				
<pre> 42 3. Module Utilization 43 ----- 44 Module LUT SOFT_HLUTNM MUXF7 MUXF8 CARRY DSP RAM Register Ctl-Set / FF Instance 45 ----- ----- ----- ----- ----- ----- ----- ----- ----- ----- ----- 46 rams_sp_nc 1030 74 0 0 0 0 256 16400 0.062500 1 47 ----- ----- ----- ----- ----- ----- ----- ----- ----- ----- ----- 48 </pre>						Total Resource Count by Module				
<pre> 49 [Ctl-Set / FF]: The ratio here is estimation as the netlist hierarchy 50 </pre>										

Figure 4-3: Example of vSyn Area Report

The area report records the vSyn version and command history and shows the statistics of several FPGA resources in following three tables.

- **Total Resource Count by Type**

From the first table, you can get an overview of the FPGA resources used in the design and the percentage of each type of LUT. LUTs includes LUT1, LUT2, LUT3, LUT4, LUT5, LUT6, LUT6_2, and LUTs with SOFT_HLUTM property. RAM includes BRAM18, URAM288, DLUTM (Dual-port LUT RAM), and SLUTM (Single-port LUT RAM). vSyn gives an estimate of the number of RAM resources. The resource count might change after vCom compilation.

The number of LUT resources are calculated with these formulas:

- $N_{LUT6_2} = N_{HLUTNM} / 2$
- $N_{LUT} = N_{LUT1} + N_{LUT2} + N_{LUT3} + N_{LUT4} + N_{LUT5} + N_{LUT6} + N_{LUT6_2} - (N_{SOFT_HLUTNM} / 2)$

- **Memory Resource Count by Module**

From the second table, you can get the usage of memory resources for each module in the design.

- **Total Resource Count by Module**

From the last table, you can get resource usage for each module in the design. The number of LUTs per module is the sum of the number of LUTs for all its instances. vSyn estimates the ratio of control set counts to FF counts based on the netlist hierarchy. A lower ratio indicates the better usage of the control set.

4.20 Running Formal Equivalence Check for vSyn Synthesis

To avoid the functional problems caused by the inequivalence between RTL and vSyn-generated netlist, it is recommended to perform equivalence check using X-EPIC formal verification tools (GalaxFV or GalaxEC) or third-party tools.

The following figure illustrates the work flow of vSyn formal check. HPE provides a python script to invoke GalaxSim, a simulator developed by XEPIC, to split the design into several domains by module and then invoke vSyn to synthesize each case. After that, formal tools check the equivalence for each case. You can check summary results of all cases from the default log file, *validate.log*.

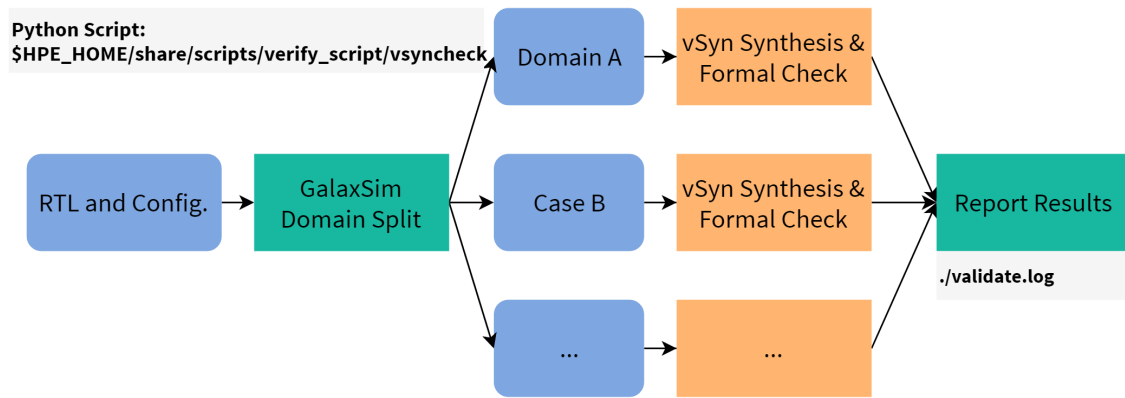


Figure 4-4: Work Flow of vSyn Formal Check

Step 1: Set up the environment for vSyn formal check

Before starting the formal check, make sure that the following tools are installed and required environment variables are configured, such as `GALAXSIM_HOME` and `PATH`.

- GalaxSim 23.08.00 or later version (Required)
- At least one of the following formal verification tools:
 - Formality
 - GalaxFV SEC Application
 - GalaxEC LEC Application
 - Conformal
- VCS (Required)

Step 2: Start vSyn formal check

Use the following command line to start vSyn formal check.

```
<abs_path_of_script> -f <filelist> \
    [-I <dir>] \
    -top <top> \
    -formality|-conformal|-sec|-lec \
    [-vsyn_opt "\<vsyn_options>"] \
    [-time_limit <value>] \
    [-process <num>] \
    [-clean] \
    [-no_double_check] \
    [-no_memory_check] \
    [-log <file>] \
    -h|-help
```

Where,

- `<abs_path_of_script>` specifies the absolute path of the `vsyncheck` script file which is provided in the HPE installation package. For example, `$HPE_HOME/share/scripts/verify_script/vsyncheck`.

- `-f <filelist>` specifies the file list of input source files and command options. Include the options that are used to load synthesis inputs in the file list, such as `-f` , `-F` , `-y` , `-v` , `+libext+` , `+define+` , and `+incdir+` .
- `-I <dir>` specifies directories to search include files.
- `-top <top>` specifies the top-level module of the design.
- `-formality` uses the Formality tool to run the formal check.
- `-conformal` uses the Conformal tool to run the formal check.
- `-sec` uses the SEC Application of GalaxFV to run the formal check.
- `-lec` uses the LEC Application of GalaxEC to run the formal check.
- `-vsyn_opt "\<vsyn_options>"` specifies the vSyn options that are used to configure synthesis. Options loading synthesis inputs must be specified in the file list specified by the `-f` option.

Enclose the vsyn options with double quotation marks, use a backslash before the first option, and separate multiple options with space. For example, `-vsyn_opt "\-bit-enable-for-vcom -routability"` .

To define text macros, use `+define+` in the file list specified by `-f` . Do not specify the `-D` or `-define` option for `-vsyn_opt` .

- `-time_limit <value>` specifies the maximum running time of a single process in seconds. The default value is 3600.
- `-process <num>` specifies the maximum number of process. Default is 1. Valid values range from 1 to 46.
- `-clean` reruns the formal check without repeating domain split.
- `-log <file>` specifies the name of log file. If not specified, the default log file is *validate.log* in the current directory.
- `-h` and `-help` print help messages.

TIP

- The formal check is run incrementally. If the formal check is interrupted, you can restart it directly. The python script detects the completed part and continues to check the remaining.
- If you do not want incremental check, delete the *domain* directory and then restart the formal check.

Command Example:

```
% $HPE_HOME/share/scripts/verify_script/vsyncheck -f ../rtl_src/vsyn_flist -
formality -vsyn_opt "\-bit-enable-for-vcom -routability" -top system -process 20
```

Step 3: View formal check results

Check the summary results in the default log file, *validate.log*, or the log file specified by the `-log` option. The later results are appended to the log file instead of being overwritten.

The following figure shows an example of the log file, *validate.log*. It reports that the equivalence check of case `npu_gtoa_pool_a0d7a8355e0b3dac` failed.

```

1  INFO: start at 2024-06-19 16:03:36.316796
2  INFO: ===== RUNNING =====
3  INFO: case Adder_e36729279ad7e016 passed (Formality Success)
4  INFO: case InvIncr_4e80ebd83253f64d passed (Formality Success)
5  INFO: case Subtraction_785fdd03205ea0e9 passed (Formality Success)
6  INFO: case top_80737c943c00a158 passed (Formality Success)
7  INFO: case npu_types_643d741d555be167 passed (Not a Module)
8  INFO: case npu_gtoa_types_ad74af2cd99fc6dc passed (Not a Module)
9  ERROR: case npu_gtoa_pool_a0d7a8355e0b3dac failed (Formality Failed: incremental)
10 INFO: end at 2024-06-19 16:03:44.091549
11 INFO: elapsed 7.775 seconds
12 INFO: see log file /home/.../validate.log

```

Figure 4-5: Example of vSyn Formal Check Report

To debug failed cases, you can check the logs in the `./domain/domains/<case_name>` directory for detailed information.

Table 4-3: Key Results in Formal Check Log File

Tool	Result	Description
vSyn	Not a Module	GalaxSim might generate non-module files. This result can be ignored.
	Synthesis Timeout	The running time of synthesis exceeds the limit specified by <code>-time_limit</code> .
	Synthesis Failed	The synthesis fails, which possibly results from syntax errors in RTL generated by GalaxSim.
Formality	Formality Success	The equivalence check passes.
	Formality Failed	The equivalence check fails.
	Formality Timeout	The running time of equivalence check exceeds the limit specified by <code>-time_limit</code> .

Tool	Result	Description
SEC	Formality Not Run	The equivalence check is not started because the module is empty or no compare points exist.
	Formality Error	Formality reports error messages.
	Formality Inconclusive	The equivalence check is inconclusive. This result usually occurs for large size modules or complex modules.
	SeqEquCheck Success	The equivalence check passes.
	SeqEquCheck Failed	The equivalence check fails.
	SeqEquCheck Timeout	The running time of equivalence check exceeds the limit specified by <code>- time_limit</code> .
	SeqEquCheck Error	GalaxFV reports error messages.
	SeqEquCheck Inconclusive	The equivalence check is inconclusive. This result usually occurs for large size modules or complex modules.
	GalaxEC-LEC Success	The equivalence check passes.
	GalaxEC-LEC Failed	The equivalence check fails.
LEC	GalaxEC-LEC Timeout	The running time of equivalence check exceeds the limit specified by <code>- time_limit</code> .
	GalaxEC-LEC Not Run	The equivalence check is not started because the module is empty or no compare points exist.
	GalaxEC-LEC Error	GalaxEC reports error messages.
	GalaxEC-LEC Inconclusive	The equivalence check is inconclusive. This result usually occurs for large size modules or complex modules.
	Conformal Success	The equivalence check passes.
Conformal	Conformal Failed	The equivalence check fails.
	Conformal Timeout	The running time of equivalence check exceeds the limit specified by <code>- time_limit</code> .

Tool	Result	Description
	Conformal Error	Conformal reports error messages.
	Conformal Has Abort Points	Some points has been excluded for equivalence check.
VCS	Is Memory and VCS Success	The equivalence check passes.
	Is Memory and VCS Timeout	The running time of equivalence check exceeds the limit specified by <code>- time_limit</code> .
	Is Memory and VCS Error	VCS reports error messages.
	Is Memory and VCS Failed	The equivalence check fails.

4.21 Synthesizing the Design Using the Third-Party Tool

HPE can compile the design netlist that Synplify Premier generates. HPE software provides the self-developed tool, vEdit, to solve the signal renaming problem of the third-party synthesizer.

The comparison between the basic synthesis process and the synthesis process with vEdit is shown in the following figure. If you need to insert certain debugging logic at compile time, vEdit pre-edit can generate necessary constraint and Tcl files to help keep signal names.

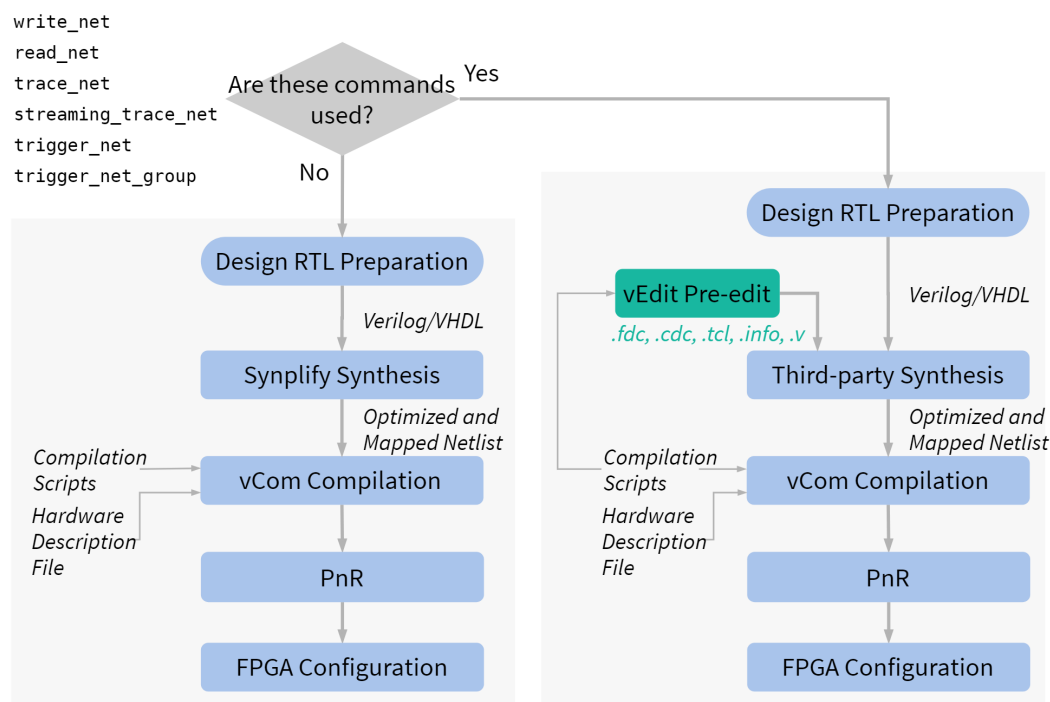


Figure 4-6: Synthesis Process without vEdit (Left) and with vEdit (Right)

4.21.1 Using the Third-Party Synthesizer

Perform the following steps to synthesize a design with X-EPIC vEdit.

1. If the following commands are used for compiling the design, use the vEdit and prepare the synthesis script referring to [Using vEdit](#).

- `write_net`
- `read_net`
- `trace_net`
- `streaming_trace_net`
- `trigger_net`
- `trigger_net_group`

2. Start the synthesizer and then synthesize the source design in one of the following two ways.

- Execute the synthesis script generated by vEdit.

```
synplify_premier -batch synplify_runtime.tcl
```

- Execute your own synthesis script.

```
synplify_premier -batch <user_synthesis_script>
```

3. Check that the netlist file with a `.vm` file extension is generated in the `./rev_1` directory. If the netlist is not generated successfully, refer to the log files of the synthesizer for the error information.

4.21.2 Reasons to Use vEdit

Usually, Synplify Premier optimizes the RTL source code as much as possible. The optimization includes:

- Renaming the flattening and internal nets in the netlist.
- Optimizing away the sourceless logic.
- Optimizing away the loadless logic.

Here is an example to show the signal renaming problem. If you define a bus signal `D[35:0]` in the RTL source code, the signal is separated into two parts in the netlist.

In the RTL code:

```
wire [35:0] D;
```

In the netlist:

```
wire [0:0] D_i; wire [35:1] D
```

If the signal `D` is specified as physical probe at compile time, only a part of the signal `D[35:1]` can be caught in the trace data and the other part of signal `D[0]` is missed.

vEdit is a utility in the HPE tool solving the signal renaming problem. vEdit can ensure that the RTL signals are accessible at runtime. vEdit also provides suggested options and attributes in the output files for synthesis.

If you want to debug the design as indicated in the following table, use vEdit before the synthesis progress. Otherwise, the compile-time commands may not take effect. If you do not use vEdit, you can also ensure the successful use of signals by studying the netlist and then going back to specify correct constraints at compile time.

Table 4-4: Features and Commands that Need vEdit Intervention

Debug Feature	Compile-time Command
Signal backdoor read	<code>read_net</code>
Signal backdoor write	<code>write_net</code>
Physical probe tracing	<code>trace_net</code>
Streaming probe tracing	<code>streaming_trace_net</code>
Trigger control	<code>trigger_net</code> and <code>trigger_net_group</code>

4.21.3 Using vEdit

With vEdit, you do not need to have knowledge of the synthesizer. You only need to follow the steps manually before the synthesis.

Perform the following steps to use vEdit and prepare the synthesis script.

1. Specify the full bus width in your compilation script file. For more information, see [Customizing Your Compilation Script File](#).

In the following example, `sig_c` is a 2-bit signal, `sig_d` is a 1-bit signal, and `sig_g` is a 10-bit signal.

The original compile-time commands are as follows:

```
trace_net -add a_inst.b_inst.sig_c
trace_net -add a_inst.d_inst.sig_d
write_net -add a_inst.e_inst.f_inst.sig_g
```

Change the original commands into:

```
trace_net -add {a_inst.b_inst.sig_c[1:0]}
trace_net -add a_inst.d_inst.sig_d
write_net -add {a_inst.e_inst.f_inst.sig_g[9:0]}
```

2. Execute the compilation script file with vEdit and specify the top module. The vEdit output files are automatically saved in *ve_output_file* in the working directory.

```
python3 $VCOM_HOME/share/gift/vEdit.py <compile_script_file> <top_module>
```

3. Revise the synthesis script in either of the following ways:.

- Specify the top module name and add Verilog files in the vEdit output file, *synplify_runtime.tcl*.

```
#specify your TOP module below
set TOP_MODULE <top_module>

#specify your verilog file below
#add_file -verilog ALL_YOUR_VERILOG_FILES
add_file -verilog <verilog_files>
```

- Specify the path of the vEdit output files and set the options in your own synthesis script.

```
add_files -verilog ve_output_file/vedit_keep.v ve_output_file/vedit_bb.v
add_files -fpga_constraint ve_output_file/vedit.fdc
add_files -compiler_constraint ve_output_file/vedit.cdc
set_option -hdl_define -set "vedit"
set_option -include_path ve_output_file
```

4. Include the vEdit output RTL code in the top module.

The original RTL code of top module is:

```
module top (.....);

.....

sub sub_inst (
    .....
);

.....

endmodule
```


The RTL code including vEdit output RTL code is:

```

module top (.....);

.....

sub sub_inst (

    .....

);

.....

`ifdef vedit
    `include "vedit_inc.v"
`endif
endmodule

```

5. Copy the vEdit output file *vedit_keep.v* to your runtime working directory.

4.21.4 vEdit Output Files

When you execute the compilation script file with vEdit, the output files are automatically saved in the *ve_output_file* directory.

The following table lists the output files with description.

Table 4-5: vEdit Output Files

Files	Description
<i>vedit_inc.v</i>	Contains the RTL code that is generated by vEdit. This file needs to be included in the top module.
<i>vedit_bb.v</i>	Defines a netlist macro module for synthesis. The netlist macro module is instantiated in <i>vedit_inc.v</i> .
<i>vedit_keep.v</i>	Defines a module to keep the signals in the synthesis. The module is instantiated in <i>vedit_inc.v</i> .
<i>synplify_runtime.tcl</i>	You can run Synplify premier by using the file or take the file as a reference to get better run-time performance.
<i>synplify_area.tcl</i>	You can run Synplify premier by using the file or take the file as a reference to get a better area minimized result.

Files	Description
<i>synplify_balance.tcl</i>	You can run Synplify premier by using the file or take the file as a reference to get a balanced result between runtime performance and area.
<i>vedit.cdc</i>	Compiler directives constraint file that contains suggested directives for synthesis.
<i>vedit.fdc</i>	Synthesis design constraint file that contains suggested attributes for synthesis.
<i>vedit_keep.tcl</i>	Lists the design-edit commands that vEdit supports in the compilation script file. This file can be specified in the compilation script file to replace the commands.
<i>vedit_keep.info</i>	Contains the reference information for signal names. The runtime debugger vDbg searches this file in the working directory.

4.21.5 Limitations of vEdit

vEdit has the following limitations:

- vEdit does not take effect if the signal is sourceless.
- vEdit does not support multi-dimension signal or array expressions, for example `wire [3:0] sig_a [3:0] .`
- vEdit does not take effect for instance-based probing, so vEdit leaves the command unchanged. The following example shows an unsupported command.

```
trace_net -depth 0 -add a_inst.b_inst
```

- vEdit does not support signals under SystemVerilog interface expressions because of third-party tool limitations.

4.21.6 Classifying Memories

HPE provides a python script `classify_mem.py` to classify the memories of the design. If you use the memory access function based on the netlist file that is generated by the third-party tool, you must use the script to generate a data mapping file that has `.mi` extension. Otherwise, the runtime debugger cannot access the memory. The script also generates a Tcl file. You must use the listed commands in your compilation script file without any modification.

To classify the memories of the design, use the following command line:

Syntax:

```
python3 ${VCOM_HOME}/share/gift/classify_mem.py <memory> <netlist> <top_module>
```

Where,

- `<memory>` specifies the hierarchical path of memory. The top module name can be omitted.
- `<netlist>` specifies the netlist file that is generated from the third-party synthesizer.
- `<top_module>` specifies the top module name.

Example:

The following example shows the command line to classify the design memory and shows the content of the generated Tcl file.

```
% python3 ${VCOM_HOME}/share/gift/classify_mem.py ram.array rev_1/dut_top.vm  
dut_top  
% cat dut_top.ram.array.tcl  
memory_access -add {dut_top.ram.array_0_0}
```

5 Compiling the Design

At compile time, you must use the compiler vCom to edit and flatten the gate-level netlist to prepare for PnR and runtime debugging. This chapter describes how to start and use the compiler vCom in the following topics:

You can get an overview of vCom flow and a summary of commands in the following topics:

- [vCom Flow](#)
- [Starting vCom](#)
- [List of vCom Tcl Commands](#)
- [Customizing Your Compilation Script File](#)
- [Guidelines for Specifying Names on Command Lines](#)

For detailed information about specific concepts and vCom command usage, see the following topics:

- [Adding Compilation Inputs](#)
- [Hardware Description File](#)
- [Describing the Hardware Configuration](#)
- [Setting Timing Constraints](#)
- [Terminal Assignment](#)
- [Netlist Macros](#)
- [Inserting Instrumentation Logic for Debugging](#)
- [Compiling with Optimization](#)
- [Performing Design Edit and Design Generation](#)

To start PnR and check results, see the following topics for guidance:

- [Running FPGA PnR](#)
- [Checking Timing Analysis Results](#)

5.1 vCom Flow

The following figure shows the input and output files for vCom compilation.

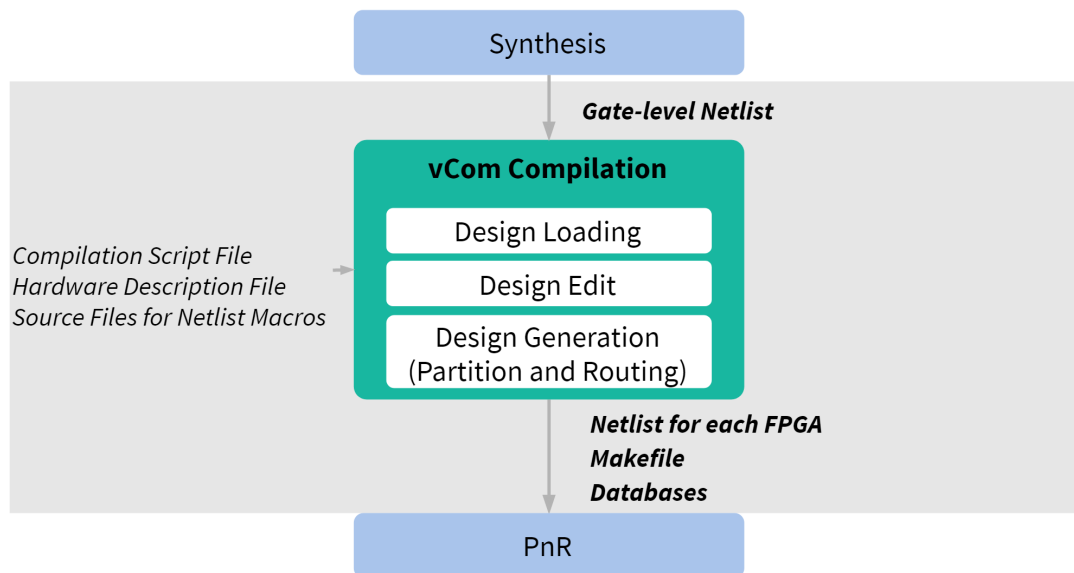


Figure 5-1: vCom Flow

The compilation process has three stages.

- **Design Load**

vCom reads and validates gate-level netlists and hardware description file and then sets up the flat netlist databases.

- **Design Edit**

vCom pre-compiles the netlist for optimization, clock transformation, and multi-drive resolution. vCom also edits the netlist as constrained in the scripts for purposes of debugging, partitioning, and optimizing. You can insert instrument logics for runtime debugging in this stage.

- **Design Generation**

vCom compiles the netlist databases. This stage contains two steps for a multi-FPGA design in a single device or across devices:

- **Design Partition:** When the calculated size of a large design is larger than the system-default resource utilization of a single FPGA, you can partition the design in the automatic mode or manual mode.
- **Design Routing:** Design routing plays an important role in the execution speed of the final design. This step can maximize the system frequency by minimizing the ratio of time division multiplexing and the hop number.

Input files of vCom include the following:

- A gate-level netlist that is generated by vSyn or Synplify.
- vCom compilation script file.
- An HDF.

- Source files for netlist macros if you use netlist macros.

Output files of vCom include the following:

- A netlist file for each FPGA part.
- Makefiles and scripts that help quickly start the back-end synthesis and PnR and easily check results.
- Databases required for runtime debugging.

5.2 Starting vCom

The compiler in HPE, vCom, reads the gate-level netlist that is generated by the synthesizer. Then vCom compiles the netlist for debugging, partitioning, and optimizing as constrained by the compilation scripts and hardware description. vCom generates a netlist file or netlist files for the design.

You can start vCom for emulation in either of the following two ways:

- Start vCom in the interactive prompt by executing the `vcom` command without any option. Then you can execute the compile-time commands or use the `source` command to execute a compilation script file in the interactive prompt.

```
% vcom
vcom>
```

- Start vCom in the batch mode by executing the following command.

Syntax:

```
vcom <compilation_script>
```

Where `<compilation_script>` specifies the compilation script file.

For more information about using vCom, see the chapter [Compiling the Design](#).

5.3 List of vCom Tcl Commands

The following table lists all compile-time commands.

Table 5-1: vCom Tcl Commands

Command	Description
Common Operations	
<code>design_read</code>	Reading the Design

Command	Description
<code>design_load</code>	Loading the Design
<code>emulator_spec</code>	Describing the Hardware Configuration
<code>design_edit</code>	Performing Design Edit and Design Generation
<code>design_generation</code>	Performing Design Edit and Design Generation
<code>display</code>	Performing Design Edit and Design Generation
<code>run_to</code>	Performing Design Edit and Design Generation
Timing Constraints	
<code>create_clock</code>	Defining the User Clocks
<code>primary_clock</code>	Defining the Primary Clocks
<code>dump_clock_region_ports</code>	Excluding the Ports from the Clock Region
<code>clock_port</code>	Excluding the Ports from the Clock Region
<code>set_false_path</code>	Setting False Paths
<code>annotate_xdc_clock</code>	Setting False Paths
<code>set_max_delay</code>	Setting Maximum Delay
Target System Solution	
<code>cable_connection</code>	Defining the Cable Connections
<code>terminal_assign</code>	Assigning Terminals
<code>delay_bin</code>	Defining a Delay Bin
<code>terminal_delay</code>	Inserting a Delay Bin
<code>terminal_osync</code>	Synchronizing Output Signals
<code>terminal_keep</code>	Keeping Unassigned Terminals
<code>global_net</code>	Setting Global Nets
Netlist Macro	
<code>netlistmacro</code>	Defining Netlist Macros
<code>netlistmacro_param</code>	Defining Netlist Macro Parameters
<code>create_generated_nmclock</code>	Adding Timing Constraints for Netlist Macros
<code>nmport</code>	Adding IO Constraints for Netlist Macros
<code>bind_ports</code>	Binding Netlist Macro Ports to AXI4 Interface
Debugging Logic Insertion	

Command	Description
<code>write_net</code>	Adding Nets for Signal Access
<code>read_net</code>	Adding Nets for Signal Access
<code>memory_access</code>	Adding Memories for Memory Access
<code>trace_net</code>	Adding Nets for Physical Probes
<code>streaming_trace_net</code>	Adding Nets for Streaming Probes
<code>trigger_net</code>	Adding Nets for ATT Control
<code>trigger_net_group</code>	Adding Nets for ATT Control
<code>dynamic_trigger</code>	Adding Nets for ATT Control
Optimization	
<code>break_all_loops</code>	Breaking Loops
<code>delay_cell</code>	Breaking Loops
<code>logic_replication</code>	Using Logic Replication
<code>routing_localization</code>	Using Routing Localization
<code>clock_recognition</code>	Using Clock Recognition
<code>clock_localization</code>	Using Clock Localization
<code>elim_common_logic</code>	Common Logic Elimination
<code>synchronous_reset</code>	Handling Asynchronous Resets
<code>design_edit_option</code>	Handling Driverless Logic and Loadless Logic Optimization
<code>const_prop_option</code>	Constant Propagation
<code>memory_options</code>	Setting Memory Optimization Strategies, Specifying High-capacity Memories in vCom
<code>module_stub</code>	Stubbing Modules or Instances
<code>instance_stub</code>	Stubbing Modules or Instances
<code>remap_optimization</code>	Remapping LUTs
<code>control_set_optimization</code>	Optimizing Control Sets
<code>local_fpga_hold_fix</code>	Shortening FD-related Timing Paths for Hold-time Fix
PHC Connection	
<code>lite_phc</code>	Setting the PHC Connection Mode

Command	Description
<code>direct_phc</code>	Setting the PHC Connection Mode
<code>set_phc_speed</code>	Setting the PHC Connection Mode
<code>interconn_dump</code>	Using Auto Cabling
<code>interconn_constraint</code>	Using Auto Cabling
Partitioning	
<code>par_opt_priority</code>	Setting the Partitioning Strategy
<code>emulator_util</code>	Setting the FPGA Resource Utilization
<code>indivisible_module</code>	Defining Indivisibility Rules for Specific Modules
<code>indivisible_group</code>	Defining Indivisibility Rules for Specific Modules
<code>indivisible_util</code>	Setting Area Thresholds for Partitioning
<code>prepartition_option</code>	Setting Area Thresholds for Partitioning
<code>par_group</code>	Defining Partition Groups
<code>par_assign</code>	Assigning the Placement of Partition Group
<code>fc_save</code>	Saving and Restoring Partitioning Results
<code>fc_load</code>	Saving and Restoring Partitioning Results

5.4 Customizing Your Compilation Script File

vCom supports to run in the batch mode. In the batch mode, you can include compile-time commands that you want to run with vCom in a Tcl script file and execute the Tcl script using vCom. The Tcl script is called *the compilation script file* in this document.

Procedure

Perform the following steps to customize your compilation script file for compilation.

TIP

You are recommended to add vCom Tcl commands as described in the steps. Some vCom Tcl commands must be executed in a prescribed sequence. Otherwise, vCom might exit with errors or compilation configuration might be ignored. See [Compilation Script File](#) in Appendix A for a template of the compilation script file.

1. Use a text file editor or other editors to create a new Tcl script file. Save the file with a Tcl extension and type the name, such as *compile.tcl*.

2. (Optional) To use netlist macros, add the `netlistmacro` and `netlistmacro_param` command lines (see [Netlist Macros](#)).
3. Start the script by adding the `design_read` and `design_load` command lines to load a design and specify the top module (see [Adding Compilation Inputs](#)).
4. Add the `emulator_spec` command lines to load the hardware description file (see [Describing the Hardware Configuration](#)).
5. Add the `create_clock` command lines to set the clocks (see [Setting Timing Constraints](#)).
6. (Optional) Add optional commands for compilation configuration.
 - a. To connect the hardware with the off-board target system, add the `cable_connection` and `terminal_assign` command lines (see [Connecting Hardware to Target System](#)).
 - b. If you want to customize the optimization options, add commands, such as `logic_replication` and `break_all_loops`, by referring to [Compiling with Optimization](#).
 - c. If you want to debug the design at runtime, add command lines for debugging. For details, see [Inserting Instrumentation Logic for Debugging](#).
 - d. To manually partition the design, add command lines, such as `emulator_util` and `par_group` by referring to [Setting Constraints for Manual Partitioning](#).
7. Add the `design_edit` command to start the compilation (see [Performing Design Edit and Design Generation](#)).
8. Add the `design_generation` command to generate the output files (see [Performing Design Edit and Design Generation](#)).
9. (Optional) Add other standard Tcl commands, such as `set` and `puts`. Check the Tcl script syntax before you save the compilation script file.

5.5 Guidelines for Specifying Names on Command Lines

A netlist file that vSyn or Synplify generates defines the gate-level logic within each logic block. A flattened netlist file that vCom generates defines primitive instantiation, less redundant logic and more instrumentation logic. The information in the netlist files includes definitions of the elements (nets, instances, cells, and terminals) and their connection.

- **Net:** A wire in the design.
- **Instance:** A module or instance in the design that can be expanded to lower hierarchy levels.

- **Leaf Instance:** A module or instance in the design that cannot be expanded to lower hierarchy levels.
- **Cell:** A low-level logic cell that is an instance of integrated FPGA elements.
 - Register cells
 - Verilog primitives, such as gates and buffers
 - Xilinx FPGA primitives, such as XPMs
- **Port:** An input or output port relative to the current level of hierarchy.
- **Terminal:** A top-level input or output port that can be assigned to hardware interfaces.

On some vCom and vDbg Tcl command lines, you need to specify names of nets, instances, and terminals. Adhere to the following guidelines when you specify the names.

- Make sure that the specified names of elements match the names in the netlist code.

In Synplify flow, the names of elements in RTL code might change after synthesis.

- The elements might be removed for optimization.
- The elements might be renamed.
- The elements might be separated. For example, an internal signal, `a[10:0]`, is separated to the `a[10:1]` signal and the `a_z[0]` signal.

After vCom compilation, the elements might be removed for netlist optimization.

- Specify hierarchical path names of elements. Use periods (.) as separators. The top module name can be omitted in the names.
- Use Verilog-style escaped identifies if the names contain non-alpha numeric characters such as spaces, escaping characters (\), periods (.), square brackets ([]), and so on.

For example,

- To include the `xyz[1].qrs[0]` instance in a partition group, you can specify the instance name as follows.

```
% par_group -instance {gp1 .\xyz[1].qrs[0] }
```

- To trace part bits of a bus signal, `top.inst.a`, you can specify the net name as follows.

```
% trace_net -add {top.inst.a[3:0]}
```

- Use curly braces to present a string as a single argument to a command so that avoid unexpected error reports.

5.6 Adding Compilation Inputs

You can create libraries for different netlist files and specify the library from which the netlist files are loaded for compilation. By default, vCom names the library for netlist files as *work*.

Perform the following steps to add the compilation inputs.

1. Get the netlist files ready for loading and start vCom.
2. Use the `design_read` command to read the netlist files in either of the following ways:
 - Use the default *work* library.
 - Create a new library or specify an existing library.
3. Use the `design_load` command to load the netlists from libraries and specify the top module in either of the following ways:
 - Load a netlist from the default *work* library.
 - Load a netlist from a library or libraries created by the `design_read` command.

5.6.1 Reading the Design

vCom supports the netlist file that is generated by vSyn and the third-party synthesizer. To read netlist files and generate libraries for different netlist files, use the `design_read` command line before you load the design:

```
design_read [-work <library_name>] -netlist {<file_list>}
```

If you do not specify the `-work` option, vCom generates a default library named *work*.

5.6.2 Loading the Design

To load netlist files from libraries for vCom, use the `design_load` command line:

```
design_load [-library {<library_list>}] -top <top_name>
```



NOTE

- If `design_read` has generated the default library, *work*, vCom loads the default library automatically even if you do not specify the `-library` option.
- The libraries to be loaded must be created on the `design_read` command line.
- Netlist macros must be defined and related source files must be loaded before you execute the `design_load` command. For more information about how to define netlist macros, see [Netlist Macros](#).

5.7 Hardware Description File

To distinguish the compilation of different hardware, you need to get or write the HDF to describe the hardware that you use. The HDF includes the following information about physical components. The hardware resources that are defined in the HDF are used according to the compilation constraints and the source design.

- Available board resources
- Available FPGA resources
- Interconnect topology.

```
### Specify HDF version
VERSION += <hdf_version>

### Specify device type
DEVICE += <hardware_type>

### Add board. index and type are required. SN and CLK are optional.
BOARD += {"index": <board_index>, "type": <board_type>, "IP": <ip_number>, "SN":
<SN_number>, "CLKIN0": <clk_board_index>, "CLKIN1": <clk_board_index>, "CLKIN2":
<clk_board_index>}

### Remove FPGA. board and fpga are required.
FPGA -= {"board": <board_index>, "fpga": <fpga_index>}

### Add connection. type and connection are required.
CABLE += {"type": <cable_type>, "connection": <connector_id_pair>}

### Add daughter board. type and connection are required.
CARD += {"type": <card_type>, "connector": <connector_id>, "protocol":
<connection_mode>,"channel": <connection_parameter>, "peer": <card_host_ip>}
```

See [HDF](#) for templates and examples of the HDF.

5.7.1 General Guidelines for HDF

The syntax of each entry line in HDF is as follows:

```
<resource> +=|-<description>
```

Where,

- `<resource>` specifies the physical components. Valid values for the components are `VERSION` , `DEVICE` , `BOARD` , `FPGA` , `CABLE` , and `CARD` .
- `+=|-` describes whether the resource is added or removed from the default.

`+=` means enabling specific resource. Note that `+=` is used for all resources except for FPGA.

`-=` means disabling the specific resource. Note that `-=` is only used for FPGA.

- `<description>` specifies the configuration information with a value or in a format similar to JSON object.

```
{"key1":value1, "key2":value2, ...}
```

- The object begins with a left brace ({) and ends with a right brace (}).
- Use a key-value pair to represent the data. Use a colon (:) after each key and separate the key-and-value pairs with a comma (,).
- A value can be a string in double quotes or a number.

General guidelines for HDFs includes:

- Comment lines can be interspersed between entry lines. Start the comment lines with the double slash (//) or pound sign (#).
- The content in an HDF is case-sensitive. Use the valid values as specified.
- Describe resources from outside to inside and from system itself to target system attached, e.g., VERSION >= DEVICE > BOARD > FPGA > CABLE >= CARD.

5.7.2 VERSION in HDF

Compilation is based on a specified version of HDF. To describe the HDF version, add a line in the following text format:

Syntax:

```
VERSION += <hdf_version>
```

Where `<hdf_version>` specifies the HDF version. Valid values are `1.0` and `2.0`. HDF version 2.0 is compatible with both P1 and P2E. Syntax described in this topic is for HDF version 2.0.

5.7.3 DEVICE in HDF

The HDF can be used for different types of hardware. To describe the device in the HDF, add a line in the following text format:

Syntax:

```
DEVICE += <hardware_type>
```

Where `<hardware_type>` specifies the type of specific device. Valid values are `P1` and `P2E`. A

mixture of P1 and P2E devices in a system is not supported.

5.7.4 BOARD in HDF

If one or more devices are cascaded in the system, you need to number and describe each device, which is called board in HDF. To describe the board in the HDF, add a line or lines in the following text format:

Syntax:

```
BOARD += {"index": <board_index>, "type": <board_type>, "IP": <ip_number>,  
"SN": <SN_number>, "CLKIN0": <clk_board_index>, "CLKIN1": <clk_board_index>,  
"CLKIN2": <clk_board_index>}
```

Where,

- **"index": <board_index>**

Specifies the board index. It is a *required* parameter. Valid value starts from zero and must be consecutive integer. For HuaPro P1, the maximum board index is 15, while for HuaPro P2E, it is 7. The board index is a required parameter.

- **"type": <board_type>**

Specifies the type of the specific board. It is a *required* parameter. Valid values are `TWO_CHIP_BOARD` and `FOUR_CHIP_BOARD` corresponding to HuaPro P1 and P2E, respectively.

- **"IP": <ip_number>**

Specifies the IP number of the board. It is a *required* parameter. In the non-VRM mode, vDbg connects to the hardware devices that are specified in the HDF. You can contact the system administrator to get the system HDF. In the VRM mode, vDbg connects to the hardware devices that VRM allocates.

- **"SN": <SN_number>**

Specifies the serial number of the board. Serial number is an identification number for the board. If the type of boards in the system is same, but the manufacturing time is different, you need to describe the serial number for each board. You can get the serial number from the back panel of the board or the purchasing contract.

- **"CLKIN0": <clk_board_index>**

Specifies the index of board that is the clock source of the CLKIN0 port of the board. This parameter is only used for HuaPro P2E system.

- **"CLKIN1": <clk_board_index>**

Specifies the index of board that is the clock source of the CLKIN1 port of the board. This option can be used for HuaPro P1 and P2E systems. In the P2E system, CLKIN1 is only used when the devices are secondary-cascaded.

- **"CLKIN2": <clk_board_index>**

Specifies the index of board that is the clock source of the CLKIN2 port of the board. This parameter is only used for secondary-cascaded HuaPro P1 system.

 TIP

- You can use CLK to replace CLKIN0 for HuaPro P1 and CLKIN1 for HuaPro P2E to simplify the HDF of a primary-cascaded system.
- If you do not describe the system clock configuration in the BOARD lines, the clock distributor source is the board itself by default.
- For more information about the clock connection in a cascaded system, refer to the "Cascade System" section in hardware-specific Reference Manual.

Examples:

The following example indicates that the system consists of four HuaPro P1 devices and the devices are not cascaded. Each board uses its own system clock.

```
BOARD += {"index":0, "type":"FOUR_CHIP_BOARD", "CLK":0, "IP":"11.11.11.11"}
BOARD += {"index":1, "type":"FOUR_CHIP_BOARD", "CLK":1, "IP":"22.22.22.22"}
BOARD += {"index":2, "type":"FOUR_CHIP_BOARD", "CLK":2, "IP":"33.33.33.33"}
BOARD += {"index":3, "type":"FOUR_CHIP_BOARD", "CLK":3, "IP":"44.44.44.44"}
```

The following example indicates that the system consists of four HuaPro P2E devices and the devices are primary-cascaded. Board 0 is the master and other boards are slaves.

```
BOARD += {"index":0, "type":"FOUR_CHIP_BOARD", "CLK":0, "IP":"11.11.11.11"}
BOARD += {"index":1, "type":"FOUR_CHIP_BOARD", "CLK":0, "IP":"22.22.22.22"}
BOARD += {"index":2, "type":"FOUR_CHIP_BOARD", "CLK":0, "IP":"33.33.33.33"}
BOARD += {"index":3, "type":"FOUR_CHIP_BOARD", "CLK":0, "IP":"44.44.44.44"}
```

The following example indicates that the system consists of four HuaPro P2E devices and the devices are secondary-cascaded. Board 0 is the master. Board 1 and Board 3 are sub-masters. Board 2 is the slave of Board 1. Board 4 is the slave of Board 3.

```
BOARD += {"index":0, "type":"FOUR_CHIP_BOARD", "CLKIN0":1, "IP":"11.11.11.11"}
BOARD += {"index":1, "type":"FOUR_CHIP_BOARD", "CLKIN0":1, "CLKIN1":0, "IP":"22.22.22.22"}
BOARD += {"index":2, "type":"FOUR_CHIP_BOARD", "CLKIN0":, "IP":"33.33.33.33"}
BOARD += {"index":3, "type":"FOUR_CHIP_BOARD", "CLKIN0":3, "CLKIN1":0, "IP":"44.44.44.44"}
BOARD += {"index":4, "type":"FOUR_CHIP_BOARD", "CLKIN0":3, "IP":"55.55.55.55"}
```

5.7.5 FPGA in HDF

By default, all FPGAs in the boards are online and available for use. If some FPGAs are defective, or you do not want to use all the FPGAs, describe the FPGAs you want to disable in the HDF using a line or lines in the following text format:

Syntax:

```
FPGA -= {"board": <board_index>, "fpga": <fpga_index>}
```

Where,

- **"board": <board_index>**

Specifies the board where the FPGA is located.

- **"fpga": <fpga_index>**

Specifies the FPGA that is to be disabled. For HuaPro P1 and P2E, the FPGA index is labeled on the top panel. Valid values of <fpga_index> for P1 are A and B. Valid values of <fpga_index> for P2E are A, B, C, and D. If <fpga_index> is a null string, all FPGAs on the board are offline.

Example:

```
FPGA -= {"board":1, "fpga":"B"}
FPGA -= {"board":2, "fpga":""}
```

5.7.6 CABLE in HDF

To describe inter-FPGA connection in the HDF, add a line or lines in the following text format:

Syntax:

```
CABLE += {"type": <type>, "connection": <connector_id_pair>}
```

Where,

- **"type": <type>**

Specifies the type of cables. Supported cable types are as follows:

- PHC interconnection cable
 - PHC_CABLE_500 (0.5m)
 - PHC_CABLE_1000 (1.0m)
 - PHC_CABLE_2000 (2.0m)
- Mini-SAS cable
 - MINI-SAS_CABLE_500 (0.5m)
 - MINI-SAS_CABLE_1000 (1.0m)

- PHC interconnection daughter boards
 - `INTERCONN_CARD_H`
 - `INTERCONN_CARD_V` (only used for P2E)
- HAWK cable
 - `HAWK_CABLE_3000` (3m)
 - `HAWK_CABLE_5000` (5m)
- **"connection": <connector_id_pair>**

Specifies a pair of connectors that connects two FPGAs. The format of connector ID pair is "`<connector_id_1>`, `<connector_id_2>`".

**TIP**

For more information about the cable selection, contact XEPIC for support.

HuaPro hardware has PHC connectors on the top panel and Mini-SAS HD connectors on the front and rear panels. Each connector in a cascade system has a unique ID `<connector_id>`, which can be described as:

```
<board_index>.<fpga_index>.<connector_index>
```

Where,

- `<board_index>` specifies the board where the connector is located.
- `<fpga_index>` specifies the FPGA where the connector is located.
- `<connector_index>` specifies the connector index that is labeled on the top panel or front panel.
 - For PHC connectors, the indexes are labeled on the top panel of P1 and P2E.
 - For Mini-SAS HD connectors, the indexes are labeled on the front panel of P1 and on the front as well as rear panel of P2E.

Example:

```
CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.B.5, 1.A.9)"}
CABLE += {"type": "INTERCONN_CARD_H", "connection": "(0.A.14, 0.B.23)"}
CABLE += {"type": "INTERCONN_CARD_V", "connection": "(0.A.18, 0.D.19)"}
CABLE += {"type": "MINI-SAS_CABLE_500", "connection": "(1.A.HSSI00, 0.A.HSSI5)"}

```

The following figure shows the FPGA and connector indexes of PHC and Mini-SAS HD connectors on HuaPro P1 and P2E.

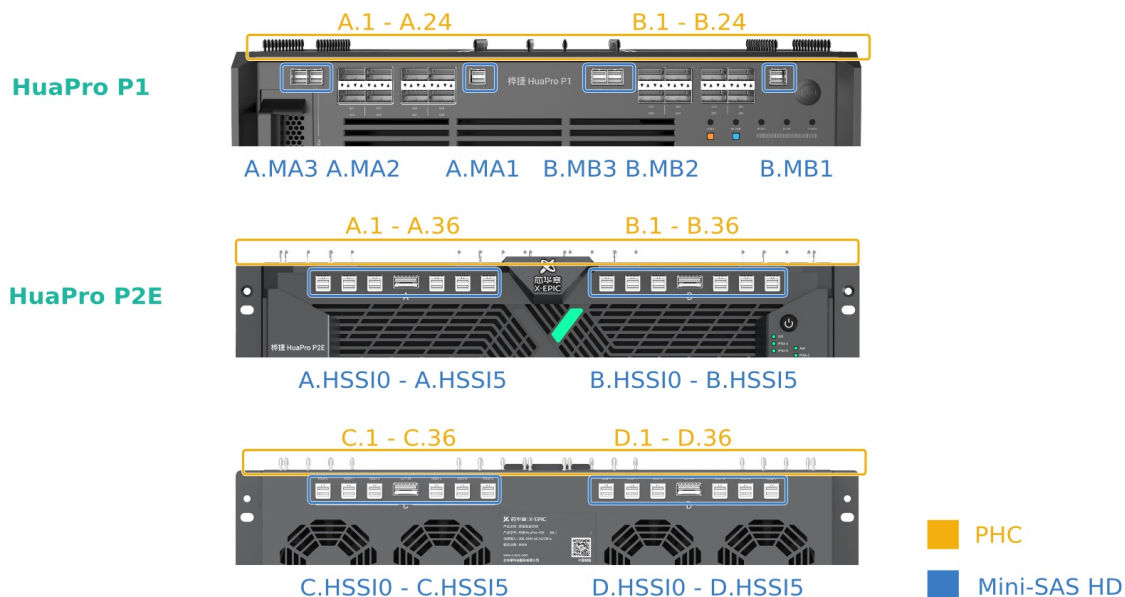


Figure 5-2: PHC and Mini-SAS HD Connectors on P1 and P2E

5.7.7 CARD in HDF

To describe the connection and use mode of daughter boards in HDF, add a line or lines in the following text format:

Syntax:

```
CARD += {"type": <card_type>, "connector": <connector_id>, "protocol":
<connection_mode>, "channel": <connection_parameter>, "peer": <card_host_ip>}
```

- **"type": <card_type>**

Specifies the daughter board type. Valid values are as follows:

- CAPC_8G_V1 and CAPC_16G_V1 specifies the DDR4 daughter board used for physical probe. The memory size is 8 GB and 16 GB, respectively. The version of the daughter board is v1.
- HAC_V1 specifies the HAC daughter board (introduced as Probe HAC in this document). protocol, channel, and peer are keys for HAC daughter board.
- HCM_16G_V1 specifies the DDR4 daughter board used for high capacity memory.
- XRAM_16G_V1 specifies the DDR4 daughter board used for XRAM.

- **"connector": <connector_id>**

Specifies a unique ID of the connector that the daughter board is connected to. The index is the same as that in [CABLE in HDF](#).

- **"protocol": <connection_mode>**

Specifies the connection mode of Probe HAC. Valid value is `stream`, indicating that the daughter board is used for streaming probe.

- **"channel": <connection_parameter>**

Specifies the probe channel configuration of Probe HAC. The connection parameter is in the format of `<card_index>.<channel>`, such as `"4.0"`.

- `<card_index>` specifies the index number that is configured on Probe HAC.
- `<channel>` specifies the probe channel of Probe HAC that is connected to the FPGA. Each Probe HAC has maximum four probe channels: 0-3.

- **"peer": <card_host_ip>**

Specifies the IP address of the host where the Probe HAC is installed.

Examples:

- The following example describes the card connection of two P1 devices.

```
CARD += {"type": "HAC_V1", "connector": "0.A.MA3", "protocol": "stream", "channel": "0.0",
"peer": "10.10.60.60"}
CARD += {"type": "HAC_V1", "connector": "0.B.MB3", "protocol": "stream", "channel": "0.1",
"peer": "10.10.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.A.MA3", "protocol": "stream", "channel": "0.2",
"peer": "10.10.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.B.MB3", "protocol": "stream", "channel": "0.3",
"peer": "10.10.60.60"}
CARD += {"type": "CAPC_16G_V1", "connector": "0.A.22"}
CARD += {"type": "CAPC_16G_V1", "connector": "0.B.1"}
CARD += {"type": "CAPC_16G_V1", "connector": "1.A.22"}
CARD += {"type": "CAPC_16G_V1", "connector": "1.B.1"}
```

- The following example describes the card connection of two P2E devices.

```
CARD += {"type": "HAC_V1", "connector": "0.A.HSSI4", "protocol": "stream", "channel": "0.0",
"peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "0.B.HSSI4", "protocol": "stream", "channel": "0.1",
"peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "0.C.HSSI4", "protocol": "stream", "channel": "0.2",
"peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "0.D.HSSI4", "protocol": "stream", "channel": "0.3",
"peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.A.HSSI4", "protocol": "stream", "channel": "7.0",
"peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.B.HSSI4", "protocol": "stream", "channel": "7.1",
"peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.C.HSSI4", "protocol": "stream", "channel": "7.2",
"peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.D.HSSI4", "protocol": "stream", "channel": "7.3",
"peer": "10.20.60.60"}
```

A DDR4 daughter board occupies three consecutive PHC connectors. The available PHC connectors to install the DDR4 daughter board are listed as follows:

- HuaPro P1: 1-3 and 22-24
- HuaPro P2E: 3-5, 9-11, 12-14, 26-28, and 29-31

 TIP

Only the first PHC connector needs to be specified on the CARD line. For example, if connector 1.A.1 is specified, the connectors 1.A.2 and 1.A.3 are also occupied and cannot be used for any other purpose.

5.8 Describing the Hardware Configuration

You need to describe the hardware for compilation by providing vCom with an HDF. The HDF defines the hardware board, cable, and interconnection needs. The content of the HDF must be in the valid format that is defined in [Hardware Description File](#). Otherwise, error message prints.

 IMPORTANT

If the hardware description in your specified HDF does not match the system, the design cannot run on the hardware. When you connect vDbg to hardware without VRM, you must take the system HDF as reference to prepare your HDF or directly specify the system HDF in the `emulator_spec` command. Contact the system administrator for the HDF of the system that you want to use.

Use the `emulator_spec` command to specify an HDF for vCom.

```
emulator_spec -add {file <file_name>}
```

Where `<file_name>` specifies the HDF.

When using auto cabling, you can add the `-reconstruct_interconnection` option in command. The `-add` option conflicts with the `-reconstruct_interconnection` option are mutually-exclusive. For more information about the auto cabling feature, see [Reconstructing Interconnection](#).

5.9 Setting Timing Constraints

Set timing constraints as introduced in the following topics.

5.9.1 Clocks in Emulation

HuaPro system has global clocks, a primary clock, a free root clock, and user clocks.

Global Clock

HuaPro system uses global clock (GC) network to distribute clocks among the FPGAs and connect the clock sources in the user design.

The HuaPro P1 system has one reserved system clock GC0 and eight usable global clocks (GC1-GC8). The HuaPro P2 system has one reserved system clock GC0 and seven usable global clocks (GC1-GC7).

- GC0

Global clock, GC0, is a reserved system clock. It is an input for all FPGAs with a fixed frequency of 100 MHz. GC0 is distributed on all PHC connectors. GC0 can not be used directly for the user design.

- GC1 - GC8

Global clocks GC1 - GC8 are also inputs for all FPGAs. HPE performs default frequency settings on the global clocks. You can also change the frequencies at runtime using the `set_clock` command. See [Setting the Frequencies of Global Clocks](#) for command usage.

Primary Clock

The primary clock is the global clock that a design uses. It is also called the slot clock. By default, vCom uses the global clock GC1 as the primary clock for the design. To change the default settings, use the command `primary_clock` at compile time. See [Defining the Primary Clocks](#) for command usage.

Free Root Clock

The free root clock drives the user clocks in DUT. The proportional relationship between the free root clock and the primary clock is determined by the divider count number. The default divider count number is set by vCom according to the SSTA result. You can use the command `set_div` to change the divider count number at runtime. See [Setting the Divider Count Number](#) for command usage.

$$\text{Frequency}_{frck} = \text{Frequency}_{pck} / N_{div}$$

User Clock

User clocks are user-defined clocks in the design. The frequencies of the input clocks which do not have drivers must be defined using the command `create_clock` at compile time. See [Defining the User Clocks](#) for command usage.

 NOTE

Make sure that all the clocks have drivers or are defined. Otherwise, vCom does not handle these clocks, which can cause design function errors.

If you define more than one user clock. The actual fastest user clock in the FPGAs is affected by the primary clock and the divider count number. The frequencies of the other clocks are reduced by a consistent proportion.

$$\text{frequency}_{\text{fastest_uck}} = \text{frequency}_{\text{pck}} / N_{\text{div}} / 2$$

For example, you define multiple user clocks with different frequencies as shown below. At runtime, `clk1` runs with the actual fastest frequency on hardware, which might be higher or lower than 5MHz and the frequency of `clk2` is one-fifth that of `clk1`.

```
create_clock -sig_name clk1 -frequency 5Mhz
create_clock -sig_name clk2 -frequency 1Mhz
```

5.9.2 Defining the User Clocks

In the emulation mode, use the following command line to declare the input clocks in a user design and configure the frequencies of these clocks:

```
create_clock -sig_name <signal_name> -frequency <number><unit>
```

Where `<signal_name>` specifies the user clock name and `<number><unit>` specifies the clock frequency and unit. Supported frequency units are KHz, MHz, and GHz. The units are case-insensitive. Be aware that your specified frequency is a string with no separating space between the number and the unit.

5.9.3 Defining the Primary Clocks

In the emulation mode, vCom assigns the global clock GC1 for user design by default. If required, you can use the following command line to define the source global clock that the design uses:

```
primary_clock -set <clock_name>
```

Where `<clock_name>` can be one of the global clocks GC1 - GC8 in P1 devices and one of the GC1 - GC7 in P2E devices GC1 - GC7. Global clock GC0 is reserved and cannot be specified on this command.

5.9.4 Excluding the Ports from the Clock Region

In the emulation mode, vCom separates the ports to clock regions and data regions automatically. The ports that belong to the clock region can be dumped into the *Clock_Region_Ports.log* file using the `dump_clock_region_ports` command without any option:

If you want to exclude some ports from the clock region to enhance the clock region detection, use the following command line:

Syntax:

```
clock_port -exclude {<port_path> ...}
```

Where `<port_path>` specifies the port that is to be excluded from the clock region.

5.9.5 Setting False Paths

You can set timing constraints of false paths to guide the timing analysis during compilation. A false path is a timing path that is ignored during SSTA and Vivado PnR. For false paths within one FPGA, vCom generates corresponding XDC commands for Vivado. For false paths that cross FPGA boundaries, vCom ignores the timing paths during pre-PR and post-PR STA.

To set false paths, use the `set_false_path` command before `design_edit`. When you specify a clock as the start point or end point of the timing path, you must annotate it with the `annotate_xdc_clock` command line.

Example:

Take the following design for example. The design netlist and schematics are shown as follows.

```
module top ();
...
wire clk1;
wire clk2;
FD fd_0(.D(w0),.Q(w1),.C(clk1));
FD fd_1(.D(w2),.Q(w3),.C(clk1));
FD fd_2(.D(w3),.Q(w4),.C(clk1));
FD fd_3(.D(w1),.Q(w5),.C(clk2));
FD fd_4(.D(w4),.Q(w6),.C(clk2));
...
endmodule
```

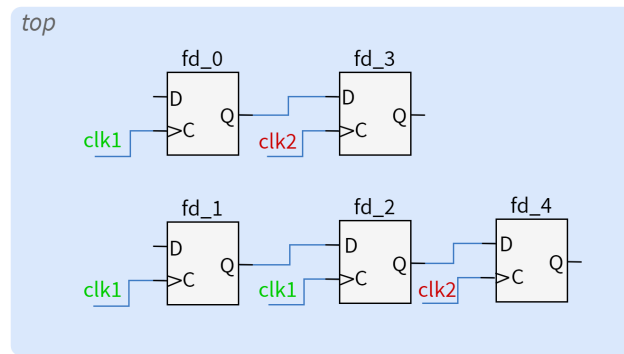



Figure 5-3: A Top Module Example for Setting False Paths

Either of the following three command line sets disables the timing analysis of the timing paths between the `clk1` domain and the `clk2` domain.

```
% annotate_xdc_clock -clock clk1 -wire clk1
% annotate_xdc_clock -clock clk2 -wire clk2
% set false_path -from clk1 -to clk2
```

```
% annotate_xdc_clock -clock clk1 -port {fd_0.C fd_2.C}
% annotate_xdc_clock -clock clk2 -port {fd_3.C fd_4.C}
% set_false_path -from clk1 -to clk2
```

```
% set false path -from {fd 0 fd 2} -to {fd 3 fd 4}
```

Either of the following two command lines disables the timing analysis between `fd_1` and `fd_2`.

```
% set_false_path -from {fd_1} -to {fd_2}
% set_false_path -from {fd 1.C} -to {fd 2.D}
```

The following command is not valid because `fd_1.Q` is not a valid start point of a timing path.

```
% set false path -from {fd 1.Q} -to {fd 2.D}
```

The following command is not valid because `clk1` and `fd_1.C` are specified for the same `-from` option, but they have different types. `clk` is a clock, while `fd_1.C` is a pin.

```
% set false path -from {clk1 fd 1.C} -to {fd 2.D}
```

5.9.6 Setting Maximum Delay

You can set maximum delay for timing paths using the `set_max_delay` command. The specified maximum delay value overrides the default setup constraint.

```
set_max_delay -from <object> -to <object> -through <object> <delay_value>
```

5.10 Terminal Assignment

HuaPro system provides connectors for interface expansion. If a design needs communications with an off-board target system, you can connect the hardware connectors with the target system using cables or daughter boards. And then assign IO terminals (ports) to specific pins of the connectors. vCom partitions the design in consideration of the terminal assignments.

A scenario of terminal assignment is shown as follows. The DUT uses two FPGAs and needs to communicate with two devices in the target system. The hardware are connected to the target devices physically. Output and input terminals pass between target system and the design through interfaces.

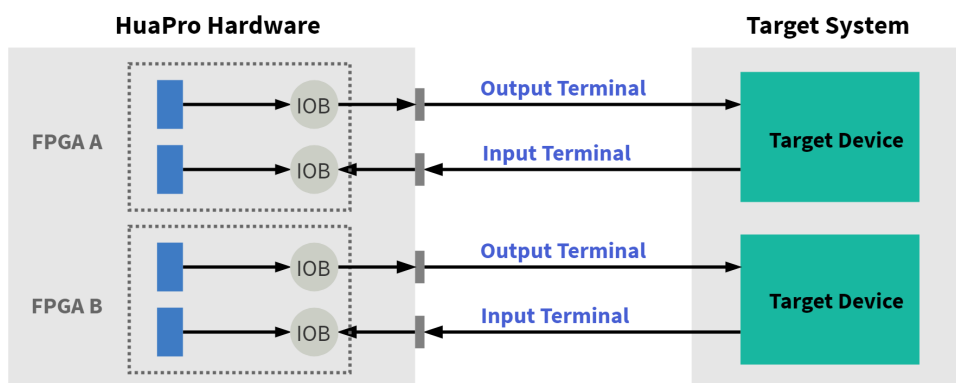


Figure 5-4: Typical Scenario of Terminal Assignment

In the terminal assignment feature, an *output terminal* means a top-level output signal of the design, which acts as an input signal for target devices. An *input terminal* means a top-level input signal of the design, which acts as an output signal for target devices.

For reference information about hardware cable interfaces, see the following topics.

- [Cable Interfaces on HuaPro P1](#)
- [Cable Interfaces on HuaPro P2E](#)

For the procedure of terminal assignment, see [Connecting Hardware to Target System](#).

5.10.1 Cable Interfaces on HuaPro P1

vCom supports two types of interfaces on the HuaPro P1, PHC and ORG, to which you can assign terminals. The following figure shows the location of the interfaces.



Figure 5-5: PHC and GPIO Connectors on HuaPro P1

PHC Connector

The PHC connectors can be used for inter-FPGA connection through X-EPIC interconnection boards or PHC cables and also for connection between the devices and the off-board target system through cables or daughter boards.

On HuaPro P1, each FPGA area is connected to 24 external PHC connectors. 23 of the 24 PHCs are connected to FPGA HP banks, which can run the high-performance transmission. One of the 24 PHCs are connected to FPGA HR banks. One P1 device has total 48 external PHC connectors.

GPIO Connector

On HuaPro P1, each FPGA area has a GPIO connector. GPIO connectors are used for connecting with target systems. Each GPIO connector has 14 pins.

TIP

- For more information about the pin mapping of PHC connectors, contact XEPIC for technical support.
- For more information about the pin mapping of GPIO connectors, see “GPIO Pin Mapping” in *HuaPro P1 Reference Manual*.

5.10.2 Cable Interfaces on HuaPro P2E

vCom supports only one type of interfaces on the HuaPro P2E, PHC, to which you can assign terminals. The following figure shows the location of the interfaces.



Figure 5-6: PHC Connectors on HuaPro P2E

On HuaPro P2E, each FPGA area is connected to 36 external PHC connectors. 34 of the 36 PHCs are HP banks. Two of the 36 PHCs are connected to four FPGA High Density (HD) banks. One P2E device has total 144 external PHC connectors on the top panel. HuaPro P2E does not have GPIO connectors.

TIP

For more information about the pin mapping of PHC connectors, contact XEPIC technical support.

5.10.3 Connecting Hardware to Target System

This topic describes a complete procedure of defining the connection between hardware and a target system at compile-time. Click the reference links for detail information about command usage of each step. You must add the commands for terminal assignments before the `design_edit` command in your compilation script file.

Prerequisite

Obtain the pin mapping of hardware connectors that the design uses.

Procedure

Perform the following compile-time steps to assign terminals.

1. Run `cable_connection` to define interfaces (see [Defining the Cable Connections](#)).
2. Run `terminal_assign` to assign terminal signals to the interfaces (see [Assigning Terminals](#)).
3. If you want to set delay constraints to terminal signals, use delay bins as follows:
 - a. Create a delay bin (see [Defining a Delay Bin](#)).
 - b. Insert the delay bin to a target signal (see [Inserting a Delay Bin](#)).
4. If you want to synchronize output signals, run `terminal_osync` by referring to [Synchronizing Output Signals](#).

5. If you want to keep some unassigned terminals from being optimized away, run `terminal_keep` by referring to [Keeping Unassigned Terminals](#).
6. If required, you can set global nets by referring to [Setting Global Nets](#).

Examples

For examples and usage of the terminal assignment feature, see [Examples for Interacting with External Targets](#).

5.10.4 Defining the Cable Connections

To define an interface for terminal assignment, use the `cable_connection` command line:

```
cable_connection -def {<interface> <type> <location>}
```

Where,

- `<interface>` specifies the interface name.
- `<type>` specifies the type of the physical connector in the system.

Valid values are as follows.

- `PHC` specifies PHC connectors on HuaPro P1 and P2E.
- `ORG` specifies GPIO connector on HuaPro P1.
- `<location>` specifies the location of physical connectors.

For PHC connectors, the location syntax is `<board_id>.<FPGA_id>.<connector_id>`. For GPIO connectors, the location syntax is `<board_id>.<FPGA_id>`.

After defining the interface, you can use the `terminal_assign` command to assign terminals to the interface. See [Assigning Terminals](#) for more information about the command usage.

5.10.5 Assigning Terminals

To assign a terminal to a specific pin of the interface that is already defined for a specific FPGA in the `cable_connection` command line, use the `terminal_assign` command line.

NOTE

When assigning terminals, you must keep all bits of a bus signal locating in the same FPGA.

```
terminal_assign -add {<interface> <pin> <signal>} [-weak_drive <value>] [-target_active_edge <value>]
```

Where,

- `<interface>` indicates an interface name that is already specified in the `cable_connection` command line.
- `<pin>` is the index of a valid pin on the connector of the interface.
- `<signal>` is a valid I/O signal in the design. Only port signals in the top module are supported. Signals such as `dut_top.cnt.clk_out_cnt0` and `dut_top.simple1.clk_in` are not supported.
- `-weak_drive <value>` specifies I/O constraints for the input signal or bidirectional/tristate primary IO signals that does not have weak driver. Valid values are listed as follows:
 - `pullup` : Applies a weak logic high level.
 - `pulldown` : Applies a weak logic low level.
 - `retain_state` : Applies a keeper.
 - `target_drive` (default): Applies nothing.
- `-target_active_edge <value>` specifies the clock edge which is used for timing analysis. If the target device works only on the positive edge or negative edge of the fastest user clock, you can set this option to relax the timing requirement. Valid values are listed as follows:
 - `pe` : Uses the positive edge of the fastest user clock.
 - `ne` : Uses the negative edge of the fastest user clock.
 - `be` (default): Uses both the positive edge and negative edge of the root clock.

5.10.6 Defining a Delay Bin

A delay bin indicates the minimum number of primary clock cycles that lags the reference signal. The actual signal transfer might have more delay than the defined delay bin. A common use model of the delay bin is to insert delay bins to data signals in reference to the clock signals so that to facilitate the clock sampling.

To define a delay bin in reference to a signal (reference signal) and label the relationship, use the `delay_bin` command line.

```
delay_bin -def {<bin> <delay> <reference_signal>}
```

5.10.7 Inserting a Delay Bin

To insert a delay bin to a target signal, use the `terminal_delay` command line:

```
terminal_delay -add {<target_signal> <bin>}
```

If any conflicts, vCom obeys the rules as described below:

Rule 1: If duplicate insertions of delay bin are present in the `terminal_delay` command lines, the last command line is effective.

Rule 2: If the delay bin insertion generates a loop, the insertion is forbidden and an error message prints.

5.10.8 Synchronizing Output Signals

When a design communicates with an off-board target system through the IOBs on the FPGA boards, you can control the delays of interacted output signals by adding output terminals to a sync-group. Signals in the same sync-group transfer out of the design at the same time. This method is more precise and useful for signals such as buses because vCom applies the timing constraint to all the output signals that are in a sync-group.

To add a list of output signals to a sync-group, use the `terminal_osync` command line:

```
terminal_osync -add {<signal> <group>}
```

If any conflicts, vCom obeys the rules as shown below:

- **Rule 1:** If you insert a delay bin to a target signal with regard to a reference signal, you cannot add both of the two signals to a sync-group.
- **Rule 2:** If you add the reference signal to a sync-group, vCom inserts the delay to the target signal by adding the delay that is inserted because of the sync-group and the delay that is defined by the delay bin.
- **Rule 3:** If you add the target signal to a sync-group, vCom inserts the delay by calculating the larger delay between the delay that is inserted because of the sync-group and the delay that is defined by the delay bin.
- **Rule 4:** Complicated cases with a conflict are not supported yet.

5.10.9 Keeping Unassigned Terminals

vCom deletes unassigned terminals for optimization during compilation. If you want vCom to keep certain terminals, use the `terminal_keep` command line. vCom connects these terminals untouched. For netlist macro ports that connect to existing DUT top ports directly, you must specify the ports on the command line.

```
terminal_keep -add {<signal_list>}
```

NOTE

You cannot assign the IO signals that are specified in the `terminal_keep` command line. Otherwise, vCom exits with an error message.

5.10.10 Setting Global Nets

For specific nets that need to be connected to the top level module, for example netlist macro ports that connect to non-existing DUT top ports directly, you can set these nets as global nets by using the `global_net` command.

```
global_net -port {<nm_port_name> <port_name>}
```

5.10.11 Examples for Interacting with External Targets

The following example demonstrates the command usage for synchronizing output signals. In this example, the design communicates with the target system with a bus signal, `dout`.

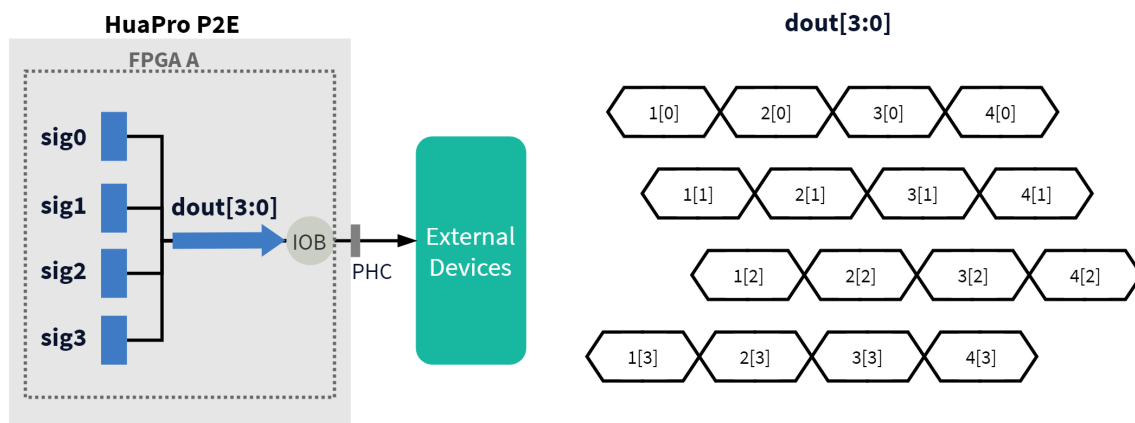


Figure 5-7: Assigning Bus Output Terminals

You can use the following command lines to assign the signals to pins of connectors.

```
% cable_connection -def {c1 PHC 1.A.3}
% terminal_assign -add {c1 A0 dout}
```

The four bits of the bus signal are misaligned when they pass the IOB because of transmission delays. To solve the misalignment problem, you can use the `terminal_osync` command to add the signals that compose the bus signal into a sync-group. The four bits of the bus signal will transfer out of the design at the same time.


```
% terminal_osync -add {sig0 grp}
% terminal_osync -add {sig1 grp}
% terminal_osync -add {sig2 grp}
% terminal_osync -add {sig3 grp}
```

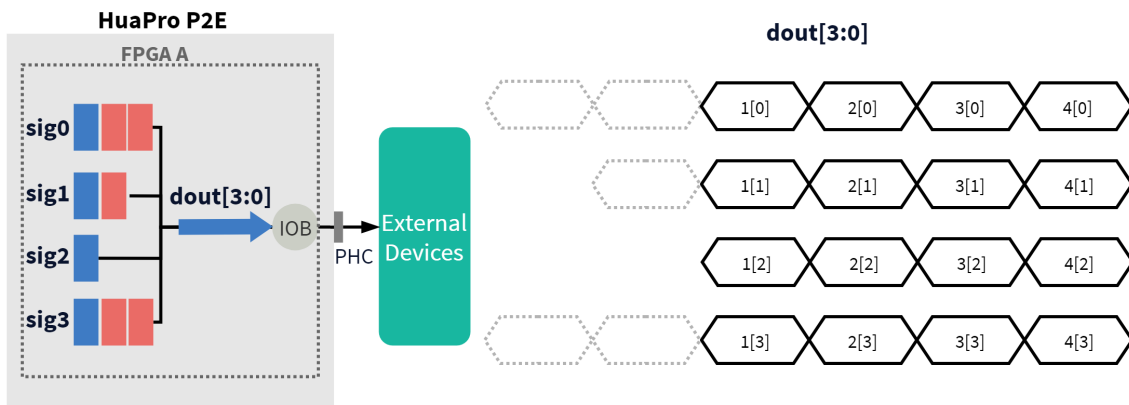


Figure 5-8: Synchronizing an Output Bus Signal

5.11 Netlist Macros

Netlist macro is a part of RTL or netlist that you do not want the compiler, vCom, to compile or modify. You can define netlist macros and use them in the compilation flow for the following purposes:

- Use daughter boards which work in a higher or lower frequency compared with user clocks.
- Use specific Xilinx primitives or encrypted code that cannot be recognized by vSyn and vCom, such as analog-based SerDes PHY, encrypted MIG core, and so on.
- Improve runtime performance or achieve the optimal resource utilization of FPGA by means of available Vivado synthesis or PnR strategies.

5.11.1 Comparison of the Use Flows of Netlist Macros

To use netlist macros, you can use either the simplified flow or standard flow with HuaPro P2E devices. It is not supported to use both of the flows for one compilation.

The simplified flow of netlist macros is also called *new netlist macro flow* and *EoU (Easy-of-Use) flow*. It is recommended using the simplified flow which has the following advantages:

- The interfaces of XEPIC-defined netlist macros are simplified, which greatly reduces the manual intervention and thus avoids the resulting errors.
- Source files of the netlist macro are provided in a package, which simplifies the creation of netlist macros. You can also create packages for user-defined netlist macros as you require.

**TIP**

This document does not cover how to create the packages of netlist macros by yourself. If you want the instruction, contact XEPIC for support.

The following table compares the simplified and standard use flow of netlist macros at compile time.

Step	Simplified Flow	Standard Flow
1. File preparation	You only need to connect the data interfaces of the netlist macro to the design top logic in the wrapper module. The other interfaces are handled by vCom automatically.	Apart from the data interfaces, you need to connect the physical pins and X-EPIC-dedicated signals to design top logic.
2. Synthesis	Same with standard flow.	Load the design source files and the wrapper module definition of the netlist macro.
3. Compilation	The flow is simplified to two steps: 1. Load the definition of the wrapper module. 2. Load a package which contains netlist and constraint files of the netlist macro.	The standard flow includes the following steps: 1. Load the definition of the wrapper module. 2. Load the netlist and constraint files of netlist macros. 3. Define netlist macros and specify the FPGA resource requirements. 4. Add timing constraints for netlist macros. 5. Handle the interfaces that connect to the top-level module, vCom netlist, or other netlist macros. For details, see Using Netlist Macros in the Standard Flow.
4. PnR	No more action needed.	In previous version, you might need to replace a Tcl script to solve known problems about PnR failures.

5.11.2 Preparing Files for Netlist Macros

To use netlist macros in a design, you must prepare Verilog wrapper files. In the standard flow, you need to prepare the netlist files. You can also add timing and physical constraints as the netlist macro needs. In the simplified flow, the netlist file and constraints are already provided in the netlist macro package.

- **Interface Definition**

To integrate your individual netlist macro, the first step is to define the interface for the netlist macro in an empty Verilog module and instantiate the module in the DUT top design. Note that one wrapper file can contain only the interface definition for one netlist macro.

- **Netlist Macro Package**

A netlist macro package consists of the encrypted netlist files, timing constraints, and compilation configuration scripts that relate to a netlist macro. HPE provides the following netlist macro packages in the ***netlist_macro_packages*** directory under the HPE installation directory.

- Netlist macro package for DDR4 daughter board

To use DDR4 daughter boards, check the netlist macro package corresponding to the installation location of daughter boards first.

Note that the voltage of corresponding PHC bank must be set to 1.2 V. Because of the installation conflict of bank 9,10,11 and bank 26,27,28, you cannot use both two locations.

- The ***xepic_ddr4_dc1*** directory stores the files for the DDR4 daughter board that is installed in Bank 3,4,5 on P2E device.
- The ***xepic_ddr4_dc2*** directory stores the files for the DDR4 daughter board that is installed in Bank 29,30,31 on P2E device.
- The ***xepic_ddr4_dc3*** directory stores the files for the DDR4 daughter board that is installed in Bank 9,10,11 on P2E device.
- The ***xepic_ddr4_dc4*** directory stores the files for the DDR4 daughter board that is installed in Bank 12,13,14 on P2E device.
- The ***xepic_ddr4_dc5*** directory stores the files for the DDR4 daughter board that is installed in Bank 26,27,28 on P2E device.

- Netlist macro package for DDR3 daughter board

To use DDR3 daughter boards, check the netlist macro package corresponding to the installation location of daughter boards first.

Note that the voltage of corresponding PHC bank must be set to 1.5 V. Because of the installation conflict of bank 9,10,11 and bank 26,27,28, you cannot use both two locations.

- The ***xepic_ddr3_dc1_bank030405*** directory stores the files for the DDR3 daughter board that is installed in Bank 3,4,5 on P2E device.
- The ***xepic_ddr3_dc2_bank091011*** directory stores the files for the DDR3 daughter board that is installed in Bank 9,10,11 on P2E device.
- The ***xepic_ddr3_dc3_bank121314*** directory stores the files for the DDR3 daughter board that is installed in Bank 12,13,14 on P2E device.
- The ***xepic_ddr3_dc4_bank262728*** directory stores the files for the DDR3 daughter board that is installed in Bank 26,27,28 on P2E device.
- The ***xepic_ddr3_dc4_bank293031*** directory stores the files for the DDR3 daughter board that is installed in Bank 29,30,31 on P2E device.
- The ***xepic_ddr3_dc5_bank262728*** directory stores the files for the DDR3 daughter board that is installed in Bank 26,27,28 on P2E device.

- Netlist macro package for PCIe RC daughter board

The ***xepic_axi_bridge_pcie3_rc*** directory stores the files for the PCIe RC daughter board. Note that the GT clock of bank 222 and 221 must be set to 100 MHz.

- Netlist macro package for PCIe EP daughter board

The ***xepic_pcie_ep*** directory stores the files for the PCIe EP daughter board. Note that GT clock of bank 222 must be set to 100 MHz.

- Netlist macro package for VIDEO daughter board

The ***xepic_video*** directory stores the files for the VIDEO daughter board. The VIDEO daughter board can display the image generated by DUT via HDMI interface.



TIP

Do not use the VIDEO daughter board and physical probe in the same FPGA. Otherwise, PnR will fail and report conflicts.

- Netlist macro package for ETH daughter board

The ***xepic_rgmiitri_mac*** directory stores the files for the ETH daughter board. Note that the frequency of global clock GC6 must be set to 100 MHz and the voltage of corresponding PHC bank must be set to 1.8 V.

• Netlist File

You must specify a source netlist file of the netlist macro for compilation. The netlist file of a netlist macro can be in any one of the following three formats:

- Structural HDL (Verilog or SystemVerilog) files from vSyn or Synplify.
- DCP files from Vivado.

You can generate DCP files by opening synthesized design in Vivado and then execute the `write_checkpoint` command. If the netlist macro contains MIG, execute the `implement_mig_cores` command before `write_checkpoint`.

- EDIF files produced by third-party synthesizers.

• Constraint File

If required, prepare timing constraint files for netlist macros. A netlist macro can have zero, one, or several XDC files. The XDC file(s) of a single netlist macro must be stored in a separate directory. You can create scoped XDC files and non-scoped XDC files.

- Scoped XDCs are constraints dedicated to the netlist macro, independent of the top-level constraints.
- Non-scoped XDC adds IO constraints for the ports of netlist macros that connect to the DUT terminals directly.

TIP

You must obey the Xilinx Constraints Scoping methodology to set the timing constraints of the netlist macros. Refer to the following Xilinx content for creating scoped XDC files and understanding XDC processing order.

- The “Constraints Scoping” section in *Vivado Design Suite User Guide: Using Constraints (UG903)*.
- The “Ordering Your Constraints” section in *Vivado Design Suite User Guide: Using Constraints (UG903)*.
- The “Constraint File Processing Order” section in *Vivado Design Suite User Guide: Designing with IP (UG896)*.

5.11.3 Using Netlist Macros in the Simplified Flow

Perform the following steps to use netlist macros in the simplified flow at compile time:

1. Define the data interface for the netlist macro in a wrapper and instantiate the module in the DUT top design.

Note that one wrapper file can contain only the interface definition for one netlist macro. Add the `/*synthesis syn_noprune=1 */` attribute to keep the logic from being optimized.

2. Synthesize the DUT and the wrapper using vSyn or the third-party synthesizer.
3. Run vCom compilation with the following `netlistmacro` command included to specify the properties of netlist macro instances.

```
netlistmacro -instance <instance_path> -package <package_path> [-location  
<fpga_id>] [-channel <channel_index>]
```

Where,

- `-instance <instance_path>` specifies the instance name of the netlist macro. This is a required option.
- `-package <package_path>` specifies the netlist macro package that corresponds to the daughter board you want to use. This is a required option. See [Preparing Files for Netlist Macros](#) for more information about the packages.
- `-location <fpga_id>` specifies the FPGA that the netlist macro instance corresponds to. This option is only required for DDR4 daughter boards.
- `-channel <channel_index>` specifies the channel that the backdoor interfaces of the netlist macro instance connects to. The valid channel index is an integer ranging from 0 to 3. This option is only used for DDR4 daughter boards. If this option is not specified, DDR4 backdoor is disabled. When multiple DDR4 daughter boards are used, the channel index identifies which one you want to backdoor access at runtime.

You can add other necessary vCom Tcl commands to complete the compilation script file. For details, see [List of vCom Tcl Commands](#).

4. Run PnR (see [Running FPGA PnR](#)).

5.11.3.1 Example for Defining Netlist Macros in the Simplified Flow

The following script example shows how to define netlist macros in the simplified netlist macro flow.

```

set cur_path [pwd]
set nnmf_path "$::env(HPE_HOME)/netlist_macro_packages"

design_read -netlist ${cur_path}/rev_1/pcie3_ddr4.vhdl

netlistmacro -instance pcie3_ddr4.chip0_wrapper -package
${nnmf_path}/xepic_ddr4_dc1 -location 0.A -channel 0
netlistmacro -instance pcie3_ddr4.ep_wrapper -package ${nnmf_path}/xepic_pcie_ep

design_load -top pcie3_ddr4

emulator_spec -add "file hw-config.hdf"
emulator_util -add {default 70}

create_clock -sig_name pcie3_ddr4.user_clk -frequency 10Mhz

design_edit
design_generation

```

5.11.4 Using Netlist Macros in the Standard Flow

Perform the following steps to prepare necessary files and define a netlist macro or netlist macros before executing the `design_load` command to load the design in vCom:

1. Prepare source files for the netlist macro.
 - Create a wrapper file.
 - Generate a netlist file for the netlist macro.
 - If the netlist macro needs timing and I/O constraints, create XDC files.

See [Preparing Files for Netlist Macros](#) for details.

2. Read the wrapper files of netlist macros.

See [Reading Interface Definitions for Netlist Macros](#) for details.

3. Load the source files of netlist macros by defining parameters, `NL_FILE_PATH`.

See [Loading Netlist Files for Netlist Macros](#) for details.

4. Define netlist macros and specify the FPGA resource requirements.

See [Defining Netlist Macros](#) for details.

5. If the netlist macros have input clocks, define the clocks.

See [Adding Timing Constraints for Netlist Macros](#) for details.

6. If the netlist macros have interfaces with the top-level module, vCom netlist, or other netlist macros, refer to the suggestions listed in [Adding I/O Constraints for Netlist Macros](#) to add I/O constraints.

5.11.4.1 Reading Interface Definitions for Netlist Macros

To read the interface definition for netlist macros, specify the Verilog files of wrappers on the `design_read` command line.

For example, either of the following scripts reads the wrapper file of a netlist macro and the netlist file of the design top.

```
% design_read -netlist dut_src/nm_wrapper.sv
% design_read -netlist rev_1/dut_top.vm
```

```
% design_read -netlist dut_src/nm_wrapper.sv rev_1/dut_top.vm
```

TIP

If you use netlist macros, you must read the wrapper files before reading the netlist file of the design top. Otherwise, vCom cannot find the wrapper files of the netlist macros.

5.11.4.2 Loading Netlist Files for Netlist Macros

Define the `NL_FILE_PATH` parameter to load the source netlist file of the netlist macros. For more information about how to define the parameter, see [Defining Netlist Macro Parameters](#).

5.11.4.3 Netlist Macro Parameters

The following table lists the parameters that can be used for netlist macro definition.

Table 5-2: Netlist Macro Parameter Description

Parameters	Description	Values
<code>NL_FILE_PATH</code>	Specifies the netlist files or the DCP files for the netlist macro.	Absolute path of files. Valid environment variables can be used in the path.
<code>XDC_DIR_PATH</code>	Specifies the directory path of the constraint files for the netlist macro specified on the same command line.	Absolute path of directory. Valid environment variables can be used in the path.

Parameters	Description	Values
<code>IO_XDC_FILE_PATH</code>	Specifies the non-scoped constraint files for the netlist macro pins that are connected to FPGA I/Os. This parameter is only for HuaPro P2E.	Absolute path of XDC files. Valid environment variables can be used in the path.

5.11.4.4 Defining Netlist Macro Parameters

vCom provides [Netlist Macro Parameters](#) for loading source files and constraints of netlist macros. You can specify the parameter values in any one of the following three ways:

- Command Execution
- Definition in Top-level Module
- Definition in Netlist Macro Module.

The `netlistmacro_param` command overwrites the parameter definitions in the source RTL. vCom generates `run_netlistmacro_pnr.tcl` for Vivado to perform the PnR. Netlist macro parameter values that are defined on `netlistmacro_param` command lines are used in `run_netlistmacro_pnr.tcl`.

Command Execution

Specify the module or instance name and parameter values by executing the `netlistmacro_param` command.

Syntax:

```
netlistmacro_param -module <module_name> -attribute <parameter> <value>
netlistmacro_param -instance <instance_name> -attribute <parameter> <value>
```

Definition in Top-level Module

Define the parameters in the top-level module definition with the `defparam` statement, which allows assignment to parameters using their hierarchical names.

Here shows an example that the module `B0X` is instantiated with two parameters defined in the top module.

```

module dut_top (
    input clk,
    input rst,
    output out
);

wire p1;
wire p2;

.....

BOX i_box(.p1(p1), .clk(clk), .rst(rst), .p2(p2));

defparam i_box.NL_FILE_PATH = "./i_box1.vm, ./i_box2.vm";
defparam i_box.XDC_DIR_PATH = "${XDC_PATH}/box";

endmodule

```

Definition in Netlist Macro Module

Define the parameters in the netlist macro module definition.

Here is an example that shows an empty definition file for module `BOX`. In this example, two parameters are defined to specify the source netlist and the timing constraint file.

```

module BOX (
    input p1,
    input clk,
    input rst,
    output p2
);

parameter NL_FILE_PATH = "./box.vm"
parameter XDC_DIR_PATH = "${XDC_PATH}/box"

endmodule

```

5.11.4.5 Defining Netlist Macros

Before compilation, use the following command line to notify the compiler which module is a netlist macro, specify the resource requirements, and set the default voltage of driverless ports. The `netlistmacro` command must be executed before `design_load`. Otherwise, vCom ignores the `netlistmacro` command lines.

Syntax:

```

netlistmacro -add {<library> <module>}

```

```
-resource {<resource_type>:<number> ...}
[-set_driverless high|low]
```

The supported resource types are as follows:

- LUT
- LUTM (LUTs used as distributed RAM)
- DSP
- FF
- BRAM

vCom partitions the FPGA design in consideration of the FPGA resource. You can specify the approximate resource utilization for each netlist macro module to help vCom perform the partitioning. Specifying resource utilization can also help avoid the PnR failure caused by excessive use of specific FPGA resource.

5.11.4.6 Clock Management Strategies for Netlist Macros

When vCom works in the default auto mode, vCom transfers the design to a single-clock design, which is driven by a single root clock. Therefore, you need to control how vCom handles the clock domain crossings between the vCom netlist and netlist macros. *vCom netlist* is the netlist generated by vCom, which includes the netlist of DUT top design and the netlist of debug IP. vCom netlist and the netlist of netlist macros compose the entire FPGA top netlist. vCom creates the project of the FPGA top design in the *\$WORK_PATH/fpgaCompDir/part_b*_f** directory.

vCom provides a new method to handle the clock domain crossing. The netlist macro can directly use the fastest user clock. The new method consumes the lowest manual effort because vCom adds timing constraints on clock domain crossings automatically. But the new method might take a relatively longer time period for PnR and lead to possibly worse runtime performance.

The old methods introduced in previous versions of the document require manual logic modification and addition of timing constraints, such as using the fast clock of the netlist macro to sample the slow clock of vCom netlist, and using the root clock, free root clock, and primary clock.

5.11.4.7 Adding Timing Constraints for Netlist Macros

In the emulation mode, you must specify the input clock ports of the netlist macros with the `create_generated_nmclock` command. Otherwise, the input ports are regarded as data ports. If you define only one user clock with the `create_clock` command, vCom uses this user clock for the netlist macro. If you define two or more user clocks, vCom uses the fastest one for the

netlist macro.

To add extra timing constraints for netlist macros, you can define the `XDC_DIR_PATH` parameter as instructed in [Defining Netlist Macro Parameters](#).

5.11.4.8 Supported Port Connection of Netlist Macros

Ports of netlist macros can be types of input, output, and input. vCom supports multiple connection scenarios of the netlist macro ports. The debugging capacity varies for different port connections. The debug capacity includes signal access, probe tracing, and trigger control. Debug support means that the ports cannot be specified as read nets, write nets, probe signals, and trigger nets in vCom compilation scripts.

The following table lists the supported connection scenarios and corresponding debugging capacities.

Table 5-3: Supported Port Connection of Netlist Macros

Port Connection	Port Type	Connection Type	Debug Capability
Netlist Macro <-> Netlist Macro	nm1.input <- nm2.output nm1.output -> nm2.input nm1.inout <-> nm2.inout	Direct	Unsupported
Netlist Macro <-> PI/PO	nm.input <- top.input nm.output -> top.output nm.inout <-> top.inout	Direct	Unsupported
Netlist Macro <-> vCom netlist	nm.input <- vcom_signal	Indirect connection with holdfix inserted	Supported
	nm.output -> vcom_signal	Direct	Unsupported

5.11.4.9 Adding IO Constraints for Netlist Macros

This topic provides some suggestions on handling IO constraints in different connection scenarios of the netlist macro ports.

- Use the `terminal_keep` command lines to specify the I/O ports of the netlist macro that connect to the top level ports.

If you want to assign the netlist macro ports target interfaces, you can add the `-disable_ff_insert` option on the `terminal_assign` command to improve timing.

- When netlist macro ports connect to some special cells in vCom netlist, vCom inserts FFs to fix the hold time violation. You can remove the holdfix to shorten the timing path by using the `nmport` command line, but the debugging of the input port will become not supported any longer.
- You cannot set output ports of netlist macros as write nets and are not recommended debugging the output ports of netlist macros.

If you have a strong need to set output ports of netlist macros as read nets, probe signals, and trigger nets, you must manually handle the clock domain crossings by adding extra XDC timing constraints for the ports. That is because netlist macros and debug IP use different clocks.

- To add extra IO constraints for netlist macros, you can define the `IO_XDC_FILE_PATH` parameter as instructed in [Defining Netlist Macro Parameters](#).

5.11.4.10 Example for Defining Netlist Macros in the Standard Flow

The following script example shows how to define netlist macros in the standard netlist macro flow.

```

set cur_path [pwd]
puts "cur_path: ${cur_path}"
design_read -netlist ${cur_path}/bbox/pcie3_ep_user_wrapper_stub.sv
design_read -netlist ${cur_path}/bbox/ddr_wrapper_stub.sv
design_read -netlist ${cur_path}/dut/pcie3_ddr4.vm

netlistmacro -add {work pcie3_ep_user_wrapper} -resource {LUT:80 BRAM:8}
netlistmacro -add {work ddr4_wrapper} -resource {LUT:80 BRAM:8}
netlistmacro_param -module pcie3_ep_user_wrapper -attribute NL_FILE_PATH
${cur_path}/bbox/pcie3_ep_user_wrapper.dcp
netlistmacro_param -module pcie3_ep_user_wrapper -attribute XDC_DIR_PATH
${cur_path}/bbox/
netlistmacro_param -module ddr4_wrapper -attribute NL_FILE_PATH
${cur_path}/bbox/ddr_wrapper.dcp
netlistmacro_param -module ddr4_wrapper -attribute XDC_DIR_PATH
${cur_path}/bbox/

design_load -top pcie3_ddr4
emulator_spec -add "file hw-config.hdf"
terminal_keep -add {m_pci_exp_txn}
terminal_keep -add {m_pci_exp_txp}
terminal_keep -add {m_pci_exp_rxn}
terminal_keep -add {m_pci_exp_rxp}
terminal_keep -add {m_sys_clk_n}
terminal_keep -add {m_sys_clk_p}
terminal_keep -add {m_sys_rst_n}
terminal_keep -add {pcie_dir}
terminal_keep -add {c0_sys_clk_p}
terminal_keep -add {c0_sys_clk_n}
terminal_keep -add {c0_ddr4_act_n}
terminal_keep -add {c0_ddr4_adr}
terminal_keep -add {c0_ddr4_ba}
terminal_keep -add {c0_ddr4_bg}
terminal_keep -add {c0_ddr4_cke}
terminal_keep -add {c0_ddr4_odt}
terminal_keep -add {c0_ddr4_cs_n}
terminal_keep -add {c0_ddr4_ck_t}
terminal_keep -add {c0_ddr4_ck_c}
terminal_keep -add {c0_ddr4_reset_n}
terminal_keep -add {c0_ddr4_dm_dbi_n}
terminal_keep -add {c0_ddr4_dq}
terminal_keep -add {c0_ddr4_dqs_c}
terminal_keep -add {c0_ddr4_dqs_t}

create_clock -sig_name pcie3_ddr4.user_clk -frequency 10Mhz
create_generated_nmclock -name pcie_axi_clk -port pcie3_ddr4.pcie3_ep.axi_clk
create_generated_nmclock -name ddr_axi_clk -port pcie3_ddr4.ddr4_chip0.axi_clk
design_edit
design_generation

```

5.11.5 Binding Netlist Macro Ports to AXI4 Interface

To use the DDR4 memory backdoor functionality, you must define netlist macros (see [Netlist Macros](#)) and then use the `bind_ports` command. The command binds the netlist macro instance to the debug logic instance. Note that DDR4 memory backdoor only supports HuaPro P2E system.

```
bind_ports -netlist_macro_ports {<nm_port_name>} -uni_debug_ports
{<board_index>.<fpga_index>.<channel_index> <port_name>}
```

Where `<channel_index>` indicates the location where the DDR4 daughter board installs. The location must be in the format of `<board_index>.<fpga_index>.<channel_index>`.

- `<board_index>` specifies the board index.
- `<fpga_index>` specifies the FPGA index. Valid values are `A`, `B`, `C`, and `D`.
- `<channel_index>` specifies the channel index. See the table below for the mapping of channels and PHC connectors.
- `<port_name>` specifies the AXI4 interface signals. See [Appendix B - Interface Signals for DDR4 Memory Backdoor](#) for required signals.

5.12 Inserting Instrumentation Logic for Debugging

You can insert instrument logics for runtime signal control, memory access, three types of probes, and two types of triggers. The following table lists the compile-time commands for inserting the debug logic.

Adhere to the rules in [Guidelines for Specifying Names on Command Lines](#) when you specify the instances or nets on the compile-time command lines.

Debugging Operation	Commands
Signal Access	
Signal Read	<code>read_net</code>
Signal Write	<code>write_net</code>
Memory Access	
Memory Backdoor Read and Write	<code>memory_access</code>
Waveform Generation	
Physical Probe	<code>trace_net</code>
Streaming Probe	<code>streaming_trace_net</code>

Debugging Operation	Commands
Trigger Control	
ATT trigger	<code>dynamic_trigger</code> , <code>trigger_net</code> , <code>trigger_net_group</code>

Table: List of Compile-time Commands for Debugging

5.12.1 Adding Nets for Signal Access

You can control the signals values with vDbg.

- To specify the nets that you want to read at runtime, use the `read_net` command line at compile time. HPE supports controlling maximum 16k bits signals for each FPGA.
- To specify the nets that you want to write, force, or release at runtime, use the `write_net` command line at compile time.

TIP

- vCom automatically adds all top-level input and inout as write nets and all top-level output ports as read nets. You do not need to specify top-level IO signals on the `write_net` and `read_net` command lines.
- Check the compilation log file, `vcom.log`, to make sure that no warning messages reporting the failure of command execution.

5.12.2 Adding Memories for Memory Access

HPE supports the DUT memory reading and writing features. To specify the DUT memories that you want to backdoor read and write at runtime, use the `memory_access` command line at compile time.

```
memory_access -add <memory>
```

TIP

- Depending on the primitive used, some memories might not support backdoor access. vCom will report an error if you specify a memory, which does not support backdoor access.
- Check the compilation log file, `vcom.log`, to make sure that no warning messages reporting the failure of the `memory_access` command execution.

If you use the netlist that is generated by the third-party synthesizer, get the list of memories by referring to the section [Classifying Memories](#).

For more information about how to use memory access at runtime, see [Accessing Design Memories](#).

5.12.3 Specifying High-capacity Memories in vCom

vCom recognizes the HCMs that are marked in vSyn and calculates the HCM resource utilization. If the HCMs are of unsupported RAM types or the HCM resources specified in [Hardware Description File](#) are used up, vCom reports errors in the log file, *vcom.log*.

To mark additional memories as HCMs during compilation, you can use the following vCom command:

```
memory_options -add {use_xdram <memory_instance_path>}
```

After compilation, vCom generates a file, *\$WORK_DIR/vcomMemDir/HCM_MEM_INFO.mi*, which records the memories that are specified as HCMs in the design.

5.12.4 Inserting Logic for Waveform Generation

You can insert instrument logics for runtime waveform generation as instructed in the following topics.

- [Adding Nets for Physical Probes](#)
- [Adding Nets for Streaming Probes](#)

5.12.4.1 Adding Nets for Physical Probes

HPE provides physical probe feature for trace data and generating waveforms. To debug a design using waveforms, you need to add a list of signals as trace nets at compile time. Then at runtime, you can add a list of probes and export the trace data as a waveform file.

The physical probe feature uses the external DDR daughter board to store the status of specific signals. The content in the DDR can be uploaded when users request.

 NOTE

HPE supports massive probe to expand the probe capacity. The maximum number of trace nets per FPGA for physical probe changes from 5,000 to 524,288 bits (256*2048), where 256 is TDM ratio. If the number of trace nets still exceeds the limitation for one FPGA, vCom partitions the design automatically. If no FPGA resources are available for physical probe logic, vCom report errors.

To debug a design using waveforms in the emulation mode, you need to define trace nets by using the `trace_net` command at compile time. vDbg can only generate the waveforms for those defined probes at runtime.

You can add trace nets using any one of or a combination of the following methods.

- Add specific nets.

```
trace_net -add <net>
```

- Add the nets in a specific instance for physical probes.

By default, only nets in the same hierarchy of the instance are added. You can use the `-depth` option to add nets in the sub-instances with your desired depth level.

For the netlist generated by the third-party synthesizer, you can use the `-third_party_netlist` option to filter out ports that might cause PnR failure or incorrect functionality of design.

```
trace_net -add <inst_path> [-depth <n>] [-third_party_netlist]
```

- Add the ports of a specific instance for physical probes.

By default, only ports in the same hierarchy of the instance are added. You can use the `-depth` option to add ports in the sub-instances with your desired depth level.

```
trace_net -port <inst_path> [-depth <n>]
```

For more information about using physical probes at runtime, see [Generating the Waveform for Physical Probes](#).

5.12.4.2 Adding Nets for Streaming Probes

To specify the nets that you want to trace with the streaming probe feature, use the `streaming_trace_net` command line:

```
streaming_trace_net -add <net>
```

 NOTE

The maximum number of streaming probe bits for one FPGA is 2000. A large number of streaming probe bits may cause performance penalties at runtime.

5.12.5 Adding Nets for ATT Control

To specify the nets that you want to work with ATT features, you must run `dynamic_trigger -enable` first and then use the following command line before `design_edit` :

```
trigger_net -add <nets> -clk <clks> [-inst_num <instance_num>]
```

Currently, you must place signals in a trigger instance on the same FPGA. To add trigger nets and set a constraint to place them in one FPGA, use the following command line:

```
trigger_net_group -name <group> -add <nets> -clk <clks> [-inst_num  
<instance_num>]
```

 NOTE

- The number of trigger nets is limited. The maximum number of trigger nets in one FPGA is 65536 (8192 nets per instance x 8 instances per FPGA).
- If you specify different maximum numbers for instances on multiple `trigger_net` and `trigger_net_group` command lines, vCom uses the largest from your specified values.
- Check the compilation log file, `vcom.log`, to make sure that no warning messages reporting the failure of command execution.

For more information about using ATTs at runtime, see [Controlling the Run with ATT](#).

5.13 Compiling with Optimization

vCom supports several optimization techniques for the trade-off among timing and area to achieve design goals. Most of the optimization techniques are enabled with default settings. You can disable undesired optimization techniques or change the settings according to your requirements.

5.13.1 Breaking Loops

Combinational loops in a DUT affect the partitioning, STA, and PnR. For a successful compilation, you need to handle the loops.

5.13.1.1 Detecting Loops

Automatic loop detection is enabled by default. vCom reports the results of loop detection in two log files in the compile-time working directory. You can refer to the log files for details about the detected loops in the DUT.

- *loop_pre_transform.rpt* reports the detected loops in the original RTL.
- *loop_post_transform.rpt* reports the detected loops that are generated after the clock transformation.

For all detected loops, vCom generates Tcl script files for Vivado to handle these loops: *loop_constraint_pre_transform.tcl* and *loop_constraint_post_transform.tcl*. If you do not want vCom to detect the loops, set the environment variable `DISABLE_LOOP_DETECT` to `true` before the design edit process.

5.13.1.2 Break Loops Automatically

vCom supports two methods to automatically break loops: auto loop breaking and loop breaking in Shannon. If both are enabled, vCom gives the priority to the auto loop breaking.

- Auto loop breaking

The auto loop breaking feature breaks all loops that are detected in the DUT. By default, this feature is enabled. You can set the environment variable `AUTO_LOOP_BREAK` to `0` before the design edit process to disable this feature.

- Loop breaking in Shannon

The loop breaking in Shannon feature inserts multiplexers and an oscillator to break the loop and detects the oscillation. The oscillation detection results can assist in debugging DUT. By default, this feature is disabled. You can set the environment variable `LOOP_BREAK_IN_SHANNON` to `1` before the design edit process to enable this feature.

5.13.1.3 Break Loops Manually

Besides automatic loop breaking, vCom also supports manual loop breaking. You can add the following user-defined loop breaking constraints before `design_edit`.

- Use the `-add` option in the `break_all_loops` command to break the loops in a specific instance scope or scopes. For example,

```
% break_all_loops -add {top.inst1 top.inst2}
```

- Use the `delay_cell` command to break a specific loop or loops in the DUT. vCom adds an FD cell as a driver of the target leaf instances before loop detection. For example,

```
% delay_cell -add {cg_top.block_cg.cg_inst.data_out_Z.I2}
```

5.13.1.4 Keeping Loops

In some cases, the loops in the DUT are expected and need to be kept. You can add the following user-defined loop breaking constraints before the design edit process.

- Use the `-keep` option in the `break_all_loops` command to keep the loops in a specific instance scope or scopes. If you enable auto loop breaking or loop breaking in Shannon, the other detected loops that are not in the specified scopes are broken automatically. For example,

```
% break_all_loops -keep {top.inst1 top.inst2}
```

- If required, you can set the environment variable `LOOP_DETECT_REMOVE_VCOMBREAKER` to `true` to make vCom change the inserted FD back to LUT1 so that to keep the loops.

5.13.2 Improving Timing

The following optimization techniques are used to improve timing.

5.13.2.1 Using Logic Replication

In a multi-FPGA system, a user design is partitioned into several partitions. A hop is a path from one partition to another partition. Timing paths that have multiple hops can significantly affect the timing result. To effectively improve the timing, you can control vCom to perform the logic replication in some timing paths to minimize the number of hops.

By default, the logic replication feature is disabled. Before running the `design_edit` command, you can use the `logic_replication` command line to enable the feature and guide the auto logic replication.

```
logic_replication -enable [-slack_threshold <value>] [-delay_threshold <value>]
```

Example:

The following figure illustrates the effect of threshold settings on the logic replication. The

design is assumed to have a total of four timing paths. The calculated delay for these paths is 10, 30, 50, and 60ns, respectively. Note that logic replication is based on calculated delay. The calculated delay equals to 10 times of the hop number of the timing path, instead of the actual delay. The calculated slack of a timing path is the difference between the delay for the timing path and that of the most critical timing path `4`.

When the slack threshold is higher or the delay threshold is lower, vCom performs logic replication on more timing paths. Therefore, if you want to achieve better timing result, you can try to set a higher slack threshold or a lower delay threshold at compile time.

Timing Paths in Design	Calculated Delay (ns)	Calculated Slack (ns)	Slack Threshold Settings			Delay Threshold Settings		
			5	20	40	15	35	55
#1	10	50	N	N	N	N	N	N
#2	30	30	N	N	Y	Y	N	N
#3	50	10	N	Y	Y	Y	Y	N
#4	60	0	Y	Y	Y	Y	Y	Y

Y : With Logic Replication
N : Without Logic Replication

Figure 5-9: Setting Threshold for Logic Replication

5.13.2.2 Using Routing Localization

vCom performs routing localization by default to decrease the number of cut nets so as to optimize the timing of the design.

Routing localization may result in a larger area. If you care about the area, you can run `routing_localization -disable` to manually disable the feature.

5.13.2.3 Using Clock Recognition

The clock recognition feature is used to detect clock-gate enable signals and specific logic that can be transformed to the data region. Using clock recognition can shorten the timing path and thereby increase the performance.

By default, the clock recognition feature is disabled. To enable the feature, use the `clock_recognition -enable` command line at compile time.

Clock gates can be both latch-based clock gates and latch-free clock gates. Clock recognition cannot detect the following clock gates:

- The clock gate that only consists of an AND gate.
- The clock gate that is inserted with additional DFT logic.

- The clock gate that has FDs with asynchronous ports.

Example:

As shown in the following figure, the design netlist has latch-free clock-gate logic. If the clock recognition feature is enabled, vCom keeps the FDRE primitive unchanged. If the feature is disabled, vCom transforms the FDRE primitive to LUT3, which makes the timing path longer.

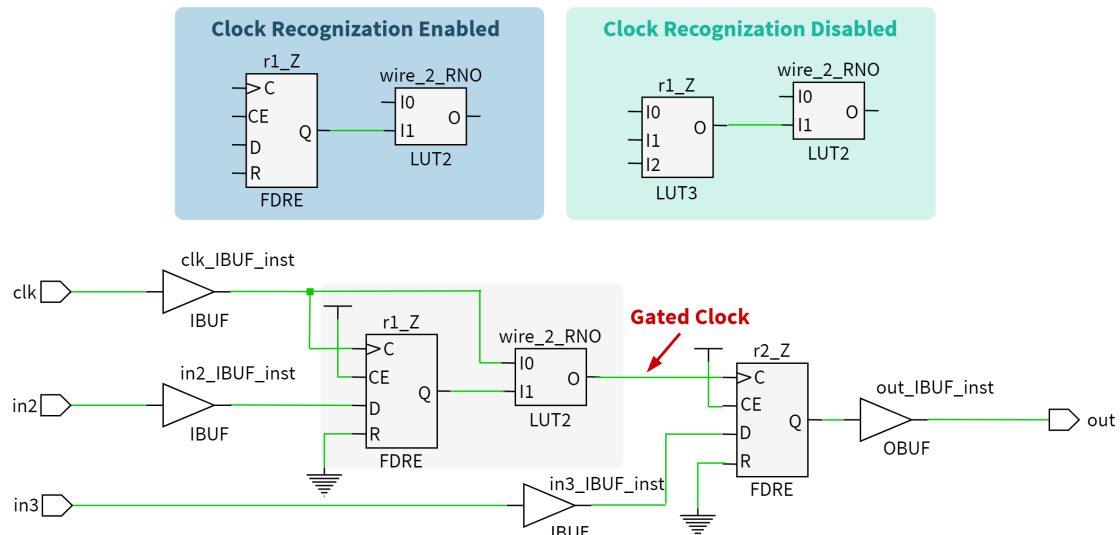


Figure 5-10: An Example of the Clock-gate Logic

5.13.2.4 Using Clock Localization

For a multi-FPGA design that has a complex clock tree, vCom automatically triggers the clock localization feature to improve the timing by duplicating cells in multi-hop clock paths.

Clock localization is enabled by default for better timing. However, cell duplication can result in a larger area. If you care about the area, use the `clock_localization -disable` command line to disable the feature.

5.13.2.5 Handling Asynchronous Resets

For the designs that contain asynchronously reset registers, vCom can change asynchronous register behavior to synchronous register behavior to shorten the timing path and thereby increase runtime performance. The optimization efficiency is dependent on the design logic.

The synchronous reset feature is enabled by default. You can make vCom not perform the optimization with the `synchronous_reset -disable` command line.

5.13.3 Optimizing the Netlist

Netlist optimization can help simplify netlists and reduce FPGA resources. Netlist optimization

includes the following methods.

- [Loadless Logic Optimization](#)
- [Common Logic Elimination](#)
- [Constant Propagation](#)

You can check *opt_insts.txt* in the compile-time working directory for all nets and cells that are removed by netlist optimization.

5.13.3.1 Loadless Logic Optimization

Loadless logic includes loadless gates, registers, and memories. The removal of loadless logic can reduce the FPGA resources needed by a design. By default, vCom removes loadless gates. You can use the `design_edit_option` command to control whether vCom removes loadless registers and memories or not.

```
design_edit_option -add {removeLoadless <N>}
```

Valid values of `<N>` are `0` (default) and `1`. `0` removes only loadless gates. `1` removes all loadless logic, including gates, registers, and memories.

NOTE

If a net or cell is removed by loadless logic optimization:

- You cannot assign the placement of the net or cell with the `par_assign` command
- The net cannot be backdoor written, forced, or released. But you can get the value of the net.

The *vcom.log* log file records the number of removed loadless logic cells as follows:

```
13001 Number of removed dead logic cells is: <number>
```

5.13.3.2 Common Logic Elimination

Common logic elimination is used to reduce area utilization by merging common logic that has same inputs and parameters. Currently, common logic supports FD, LUT, GND, and VCC. By default, this feature is enabled, and it runs recursively until the design does not have any more common logic.

If you want to shorten the compilation time, you can change the effort level of optimization by using the `elim_common_logic -enable -times <value>` command line before `design_edit`. The fewer times you set, the more common logic may remain.

The *vcom.log* log file records the number of eliminated common logic boxes as follows:

```
55000 Number of merged common logic boxes is: <number>
```

5.13.3.3 Constant Propagation

Constant propagation analyzes netlists during the compilation for constants (VCC or GND) and then optimizes accordingly. vCom supports constant propagation on LUT, DSP, memory, LD, FD, and TRI. You can use the following command line to enable or disable the optimization method.

```
const_prop_option -enable|-disable <method>
```

Where *<method>* can be one of the followings or multiple methods separated by space:

- `all` specifies all supported methods for constant propagation.
- `LUT_reduction` specifies the constant propagation optimization on LUT.
- `DSP_const_prop` specifies the constant propagation optimization on DSP.
- `MEM_const_prop` specifies the constant propagation optimization on memory.
- `LD_downgrade` specifies the constant propagation optimization on LD.
- `FD_LD_downgrade` specifies the constant propagation optimization on FD and LD.
- `TRI_downgrade` specifies the constant propagation optimization on V\$TRI in vSyn-generated netlist.



NOTE

- If a net or cell is removed by constant propagation, you cannot assign the placement of the net or cell with the `par_assign` command.
- You cannot use the `dump_mem` command at runtime to dump the data of the memories that have been optimized by constant propagation.

You can check *vcom.log* for the number of logic cells that are removed by constant propagation.

```
13002 Number of removed constant propagation cells is: <number>
```

5.13.4 Handling Driverless Logic

By default, vCom ties all driverless logic (also called sourceless logic) to GND. You can use `design_edit_option -add tieDriverlessHigh` to tie driverless nets to VCC.

5.13.5 Stubbing Modules or Instances

If the design contains some unnecessary modules or instances, you can remove modules or instances in the vCom compilation to reduce design size. You can also remove a particular module or instance to estimate the possible improvement of timing.

Use the `module_stub` command to remove all the instances of a module. Use the `instance_stub` command to remove an instance. The modules and instances specified on the commands are stubbed out of all of the contents.

```
module_stub -add {<module_name>} [-keep_input] [-keep_output]
```

```
instance_stub -add {<instance_name>} [-keep_input] [-keep_output]
```

Where,

- `-add {<module_name>}` specifies a module name.
- `-add {<instance_name>}` specifies the hierarchy name of an instance.
- `-keep_input` retains the inputs and makes vCom treat them as read nets. If not specified, the loadless inputs will be removed from the netlist.
- `-keep_output` retains the outputs and makes vCom treat them as write nets. If not specified, vCom ties the loaders of stubbed instances to GND or VCC as specified by the `design_edit_option -add tieDriverlessHigh|tieDriverlessLow` command.

NOTE

Currently, this feature has the following limitations:

- You can only stub modules or instances in the lowest hierarchy level.
- The outputs of stubbed modules or instances cannot be traced or used as trigger nets.
- Specifying multiple instance names or module names on one command line is not supported.
- You cannot stub the instance in which the internal signal is used by another instance across hierarchy.

5.13.6 Remapping LUTs

Excessive use of LUT5 and LUT6 might reduce the routability which results in PnR failure. In vCom compilation, you can enable automatic LUT remapping optimization or customize the proportion of LUT2, LUT3, LUT4, LUT5, and LUT6.

This section describes how to use LUT remapping in the following topics.

- Checking LUT Reports
- Enabling Automatic LUT Remapping Optimization
- Customizing LUT Remapping for All Instances
- Customizing LUT Remapping for Specific Instances
- Dissolving Modules in LUT Remapping

5.13.6.1 Checking LUT Reports

vCom provides the LUT report, *de_detail_lut_report*, which records the number of each type of LUT resources used in instances. The LUT report is generated in the compile-time working directory after design edit completes. You can refer to the report to determine whether LUT remapping is needed or to adjust the settings of LUT remapping.

The following example shows a part of a LUT report.

vCom LUT Report												
06/04/24 13:54:14												
Edited design after design_edit												
Instance	Module	LUT	LUT1	LUT2	LUT3	LUT4	LUT5	LUT6	LUT6_2	LUTM		
[cmp_top	cmp_top	5938242.0	200388.0	902892.0	1326749.0	408526.0	1129930.0	2128424.0	990198.0	2220993.0		
[cmp_top.system	system	5938120.0	200388.0	902066.0	1326688.0	408516.0	1129921.0	2128414.0	990186.0	2220993.0		
[cmp_top.system.chip	chip	5902370.0	197434.0	897957.0	1323460.0	407089.0	1125674.0	2117065.0	886408.0	2149779.0		
[cmp_top.system.chip pll_top	pll_top		0.0	3.0	0.0	1.0	0.0	1.0	0.0	0.0		
[cmp_top.system.chip pll_top.pll	fake_pll	8.0	0.0	3.0	0.0	1.0	0.0	1.0	0.0	0.0		
[cmp_top.system.chip.chip_intf.chip_bridge	chip_bridge	360.0	16.0	0.0	761.0	0.0	542.0	170.0	134.0	70.0		
[cmp_top.system.chip.chip_intf.chip_fpga_out	chip_bridge_out	1892.0	7.0	183.0	525.0	40.0	27.0	490.0	89.0	570.0		
[cmp_top.system.chip.chip_intf.chip_fpga_out.separator	bridge_network_choser	127.0	1.0	15.0	19.0	19.0	12.0	73.0	12.0	0.0		

Figure 5-11: Example of a LUT Report

5.13.6.2 Enabling Automatic LUT Remapping Optimization

vCom supports automatic LUT remapping to improve the routability. By default, the automatic LUT remapping is disabled. You can use the `remap_optimization -enable` command before `design edit` to enable it.

```
remap optimization -enable
```

If the automatic LUT remapping cannot meet your requirements, you can customize LUT remapping by setting the maximum cost of LUTn and area threshold for dissolving. See the following topics for more information about how to set customized LUT remapping constraints. Note that if you use the `remap_optimization -disable` command after all commands for customizing LUT remapping, vCom ignores the settings and disables LUT remapping.

 TIP

To further decrease the proportion of LUT5s and LUT6s, you can choose the following methods tentatively in some cases:

- Enable the dump free map at synthesis time by adding the `-dup-free-map` option in the `vsyn` command. This method can result in an increase in the area of vSyn-generated netlist.
- Use the `remap_optimization -effort {high}` command in vCom compilation scripts. This method can result in a significant increase in the running time of LUT remapping.

5.13.6.3 Customizing LUT Remapping for All Instances

You can customize LUT remapping of all instances in a design using the `-global` option in the `remap_optimization` command. Choose one of the following ways to set LUT cost or LUT size globally. Do not use a combination of the three ways.

- To control the proportion of LUTs, from LUT2 to LUT6, specify the maximum cost of LUT2-6 in turn as shown the following command line.

```
remap_optimization -global {lut_cost <lut2_cost> <lut3_cost> <lut4_cost>
<lut5_cost> <lut6_cost>}
```

The sum of specified LUT cost values must be larger than or equal to 100. The valid value for each LUT cost is a positive integer between 0 and 100, including 100.

The cost of `LUTn` primitives is calculated using the following equation, where `n` ranges from 2 to 6.

The cost of `LUTn` = (The number of `LUTn`) / (The sum of LUT2-6) * 100

 NOTE

The actual LUT cost might not be equal to the LUT cost that you set and depends upon the design logic.

- If you want to specify the maximum cost of LUT6 only or specify the maximum cost of both the LUT6 and LUT5, use the following command line.

```
remap_optimization -global {lut_cost [LUT5 <lut5_cost>] LUT6 <lut6_cost>}
```

You cannot set the maximum cost of LUT5 individually. The sum of specified LUT cost values must be smaller than 100.

- If you want to disable the mapping of specific large-size LUTs, specify the largest LUT that can be used with the following command line.

```
remap_optimization -global {lut_size <n>}
```

Valid values of `<n>` are 4, 5, and 6. The default value is 6. For example, setting the value to 5 disables the mapping of LUT6. Note that disabling the mapping of LUT n , where n is 4, 5, or 6, does not mean that the netlist will not contain any LUT n primitives. Some LUT n , such as those with the dont touch attribute attached, are kept and do not participate in LUT remapping.

Examples:

The following command line limits the maximum proportion of LUT2, LUT3, LUT4, LUT5, and LUT6 to 20%, 20%, 20%, 30%, and 10%, respectively, for the whole design.

```
% remap_optimization -global {lut_cost 20 20 20 30 10}
```

The following command line limits the maximum proportion of LUT6 to 10% for the whole design.

```
% remap_optimization -global {lut_cost LUT6 10}
```

The following command line disables the mapping of LUT6 for the whole design.

```
% remap_optimization -global {lut_size 5}
```

5.13.6.4 Customizing LUT Remapping for Specific Instances

You can remap LUTs of a specific instance using the `-instance` option in the `remap_optimization` command. Obey the same rules of [Customizing LUT Remapping for All Instances](#) as for specific instances. Note that the constraint added to specific instances has higher priority than global settings.

Choose one of the following ways to set LUT cost or LUT size for an instance. Do not use a combination of the three ways for a single instance. Only one instance can be specified on one command line. To specify multiple instances, use separate `remap_optimization` commands.

- Set the cost of all LUT types for a specific instance.

```
remap_optimization -instance {<inst> lut_cost <lut2_cost> <lut3_cost>
```

```
<lut4_cost> <lut5_cost> <lut6_cost>}
```

- Set the cost of LUT6, or LUT5 together with LUT6 for a specific instance.

```
remap_optimization -instance {<inst> lut_cost [LUT5 <lut5_cost>] LUT6  
<lut6_cost>}
```

- Set the LUT size for a specific instance.

```
remap_optimization -instance {<inst> lut_size <n>}
```

Examples:

The following command line limits the proportion of LUT2, LUT3, LUT4, LUT5, and LUT6 to 20%, 20%, 20%, 30%, and 10%, respectively, for the `top.a` instance.

```
% remap_optimization -instance {top.a lut_cost 20 20 20 30 10}
```

The following command line limits the maximum proportion of LUT5 and LUT6 to 20% and 10%, respectively, for the `top.a` instance.

```
% remap_optimization -instance {top.a lut_cost LUT5 20 LUT6 10}
```

The following command line disables the mapping of LUT5 and LUT6 for the `top.a` instance.

```
% remap_optimization -instance {top.a lut_size 4}
```

5.13.6.5 Dissolving Modules in LUT Remapping

By default, the modules whose FPGA resource utilization is smaller than 0.001% are dissolved in LUT remapping. You can customize the area threshold or disable module dissolving using the following command line.

```
remap_optimization -dissolve_util {<N>}
```

Where `<N>` specifies the area threshold. For example, setting `<N>` to 0.002 means that the modules whose FPGA resource utilization is smaller than 0.002% are dissolved in LUT remapping. Setting the area threshold to 0 disables the module dissolving in LUT remapping.

The maximum area threshold you can set is one tenth of the minimum FPGA resource utilization limitation set by the `emulator_util` command. For example, if you set the FPGA resource utilization as follows, the maximum area threshold you can set for module dissolving in LUT remapping is 3.

```
emulator_util -add {0.A 30}
emulator_util -add {0.B 40}
emulator_util -add {0.C 50}
emulator_util -add {0.D 0}
```

5.13.7 Setting Memory Optimization Strategies

5.13.7.1 Setting Memory Balance Strategy

The current memory balance optimization is only used for scenarios that too many BRAM resources are used and can be converted to LUTM and URAM resources. vCom provides two strategies for memory balance, Smart and Adaptive. The default is Smart. You can use the following command to set your desired strategy. Note that the option is case-sensitive.

```
memory_options -add {bram_balance SMART|ADAPTIVE}
```

- **SMART** : Balances the usage of memory resources as much as possible.
- **ADAPTIVE** : Converts BRAM resources as little as possible to make the resource utilization lower than upper limit.

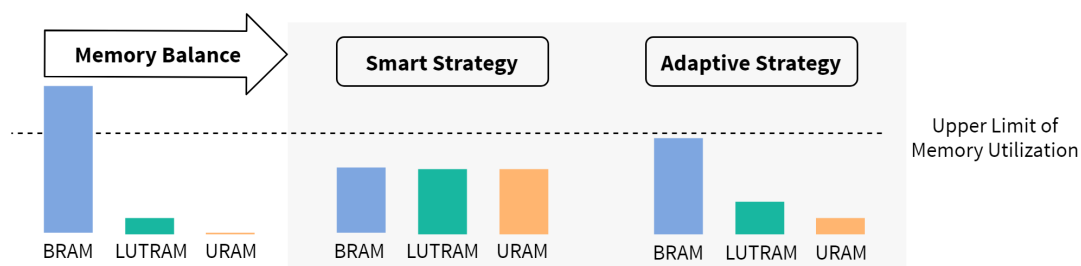


Figure 5-12: Strategies of Memory Balance

You can check whether memory balance succeeds and the number of transformed memory instances from the log file, *vcom.log*. To get details about transformed memory instances, refer to the *memoryBalance.txt* file in the *vcomMemDir* directory.

Example:

Assume that the current memory resource utilization of the design is BRAM: 90%, LUTM: 10% and URAM: 0% and is to be limited to BRAM: 60%, LUTM: 60% and URAM: 60%.

If the Smart strategy is selected for memory balance, the memory resource utilization might be optimized to BRAM: 40%, LUTM: 40% and URAM: 40%.

If the Adaptive strategy is selected for memory balance, the memory resource utilization might be optimized to BRAM: 59%, LUTM: 20% and URAM: 10%.

5.13.7.2 Setting Memory Split

In vSyn flow, vCom enables memory split by default to split memory leaf instances into small memories for the following purposes.

- Be capable to partition large memories that exceed the upper limit of FPGA resources.
- Increase the possibility of passing PnR by removing MUXF7 and MUXF8.

For now, memory balance does not transform BRAM to LUTRAM if memory split is enabled. If you want to disable memory split, unset the `MEMORY_SPLIT` environment variable or set it to `0` or `false`.

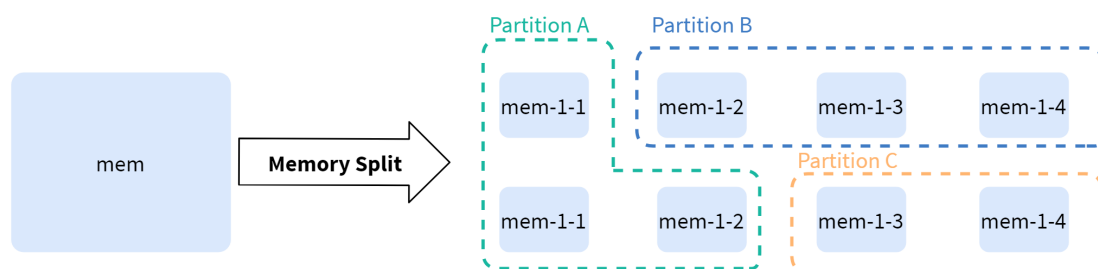


Figure 5-13: Memory Split

5.13.8 Optimizing Control Sets

vCom supports control set optimization to improve the utilization of register and CLB. By default, the control set optimization feature is disabled. To enable the feature, use the `control_set_optimization -enable` command. When the feature is enabled, the LUT consumption increases.

Currently, vCom performs the control set optimization when the control set utilization for the design is larger than 0%. The utilization of control set is defined as the ratio of the number of control sets in the design to the number of control sets provided by FPGAs. Note that vCom calculates the control set utilization without considering LUTMs, so the utilization might be smaller than the value provided by Vivado.

By default, vCom optimizes the control sets whose loader number is less than or equal to 8. You can use the following commands to set the loader number threshold globally or for specific instances. The loader number threshold that you set for an instance has higher priority than that for the global.

```
control_set_optimization -global {loader_num <N>}
```

```
control_set_optimization -instance {<instance_path_name> loader_num <N>}
```

The log file, `vcom.log`, reports the number of control sets before and after optimization. For

more information about optimized registers, refer to *control_set_optimized_inst.log*, which is generated in the working directory.

5.13.9 Shortening FD-related Timing Paths for Hold-time Fix

Sometimes, a design might contain several short FD-related timing paths. Short FD-related timing paths refer to the timing paths starting from and ending at FDs with a few number of combinational logic levels in between. During the PNR stage, Vivado might take a long time to fix the hold-time for these paths.

In order to solve the problem, vCom supports detecting the short FD-related timing paths and inserting data-path delays to these timing paths. This feature is off-by-default. You can enable it using the following command.

```
local_fpga_hold_fix -enable
```

To customize the optimization, you can use the following options:

```
local_fpga_hold_fix [-lut_levels <value>] [-direct_only] [-hierarchies]
```

Where,

- `-lut_levels <value>` sets the threshold of LUT level. The default LUT level is 3, which means that LUT1 chains are inserted to timing paths with less than 3 logic levels until the level number is not less than 3.
- `-direct_only` controls vCom to insert LUT1 chains to only the timing paths with directed connected FDs, that is, timing paths with zero combinational logic level.
- `-hierarchies` controls vCom to process only the timing path end points within the provided hierarchies.

5.14 Performing Design Edit and Design Generation

vCom performs design edit based on your design source files and compilation constraints. The design-edit flow includes six steps which you can run to as shown in the following figure. Design generation is a critical process in the compilation flow. The design-generation flow includes four steps (pre-generation, partitioning, routing, and implementation) for a multi-FPGA design, and two steps (pre-generation and implementation) for a single-FPGA design. The following figure shows the design-edit and design-generation flow for multiple FPGAs and a single FPGA.

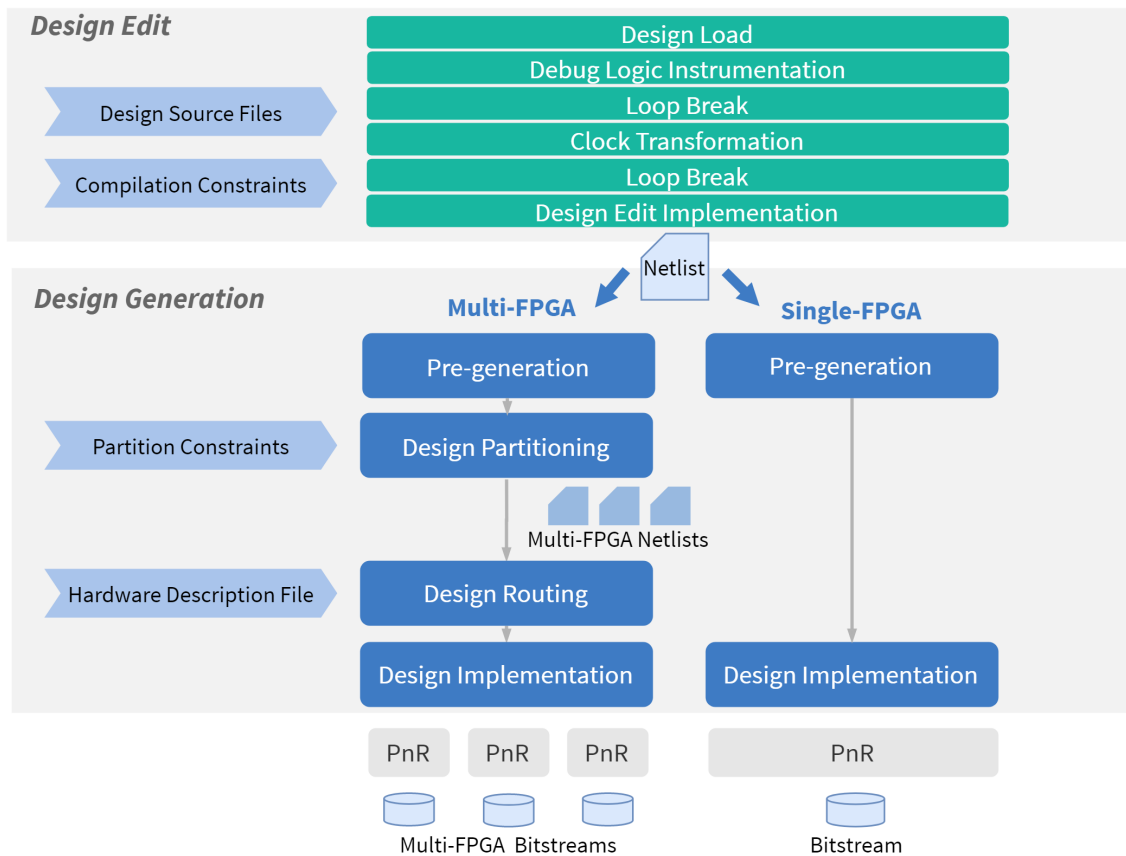


Figure 5-14: Design Generation and PnR Flow

You can perform design edit and design generation in one step or multiple steps:

- Perform one-step design edit and design generation with the `design_edit` and `design_generation` commands without any option.

```
design_edit
design_generation
```

- Use the `run_to` command to run to a sub-step in the flow chart of design edit and design generation. The flow chart will stop after the specified step completes.

```
run_to <step_id>|<step_name>
```

To check the sub-steps in the flow chart which you can run to and the step ID and name of each step, execute the `display` command in the vCom interactive prompt. The close angle parenthesis (>) points to the step that you are going to run.

All the sub-steps in design edit and the sub-step 2.2.2 (NSplit) support save and restore. The [Saving and Restoring Partitioning Results](#) topic shows how to save and restore in the step 2.2.2. You can refer to the topic to learn how to save and restore in the flow chart of design edit.

```

vcom>display

> 1          Design Edit (DEdit)
  1.1        DesignLoad (DeLoad)
  1.2        Dbg Instrument (DbgInstrument)
  1.3        LoopBreak Pass1 (LoopBreak1)
  1.4        Clock Transform (ClockTrans)
  1.5        LoopBreak Pass2 (LoopBreak2)
  1.6        DesignEdit Implement (DEImpl)
2          Design Generation (Dgen)
  2.1        PreGeneration (PreDgen)
  2.2        Design Partition (DPar)
    2.2.1    Partitioning (Par)
    2.2.2    Netlist Split (NSplit)
    2.2.3    Logic Replication (LRepl)
  2.3        Design Routing (DRoute)
    2.3.1    Clock Localization (CLocal)
    2.3.2    Routing (Route)
    2.3.3    Net Group Assignment (GAssign)
  2.4        Design Implement (DImpl)

```

5.15 Checking the Design Area Reports

vCom provides two area reports for you to check FPGA resources and debug logic resources the design uses. Based on the area results, you can adjust the compilation optimization strategies.

- The pre-design-edit area report, *design_original_report*, shows the number of each resource that the design uses originally.
- The post-design-edit area report, *de_area_report*, shows the number of each resource that the design uses after design edit.

By default, the area reports show the area information of all resource types for the modules whose area is equal to or lower than 0.1% of the design area at any level of hierarchy. You can customize the area reports to only address certain types of resources and list the modules that have a larger or smaller area at certain levels of hierarchy.

Use the `design_edit_option` command before `design_edit` to customize the pre-design-edit area report and post-design-edit area report. You can view the two area reports in the compile-time working directory after the design edit process completes.

```

design_edit_option -add {area_report [layer:<level>] [util:<value>]
[res_type:all|fpga|<res_type>]}

```

Where,

- `layer:<level>` specifies the level of design hierarchy. By default, the level of hierarchy has no limit. Setting the level to zero disables the generation of area reports.
- `util:<value>` specifies the percentage value of resource utilization. Default value is `0.1`.
- `res_type:all|fpga|<res_type>` specifies the resource types. Default value is `all`. The parameter values are case-insensitive. Valid values are listed as follows:
 - `all` reports the resources of all types, including FPGA resources (FF, LUT, LUTM, DSP, BRAM, URAM) and debug logic resources.
 - `fpga` reports only FPGA resources.
 - `<res_type>` specifies a resource type or multiple types separated by the separator bar (|) without any space, for example, `FF|LUT|dsp`.

The following example shows the post-design-edit area report customized by the `design_edit_option -add {area_report layer:3 util:0 res_type:fpga}` command line.

```
% cat design_original_report

05/15/24 13:23:01
Original design before design_edit
```

Instance	Module	FF	LUT	LUTM	DSP	BRAM	URAM
cmp_top	cmp_top	3334045.0	5702526.0	23851.0	1344.0	5002.0	0.0
cmp_top.system	system	3333971.0	5702421.0	23851.0	1344.0	5002.0	0.0
cmp_top.system.chip	chip	3310047.0	5669669.0	20017.0	1344.0	4992.0	0.0
cmp_top.system.chipset	chipset	23917.0	32751.0	3834.0	0.0	10.0	0.0

```
% cat de_area_report

05/15/24 13:57:14
Edited design after design_edit
```

Instance	Module	FF	LUT	LUTM	DSP	BRAM	URAM
cmp_top	cmp_top	3556172.0	5951220.0	222093.0	1344.0	7506.0	0.0
cmp_top.system	system	3556085.0	5951098.0	222093.0	1344.0	7506.0	0.0
cmp_top.system.chip	chip	3533428.0	5915189.0	214979.0	1344.0	7488.0	0.0
cmp_top.system.chipset	chipset	22650.0	35908.0	7114.0	0.0	18.0	0.0

5.16 Running Formal Equivalence Check for Compilation

vCom performs netlist optimization in the design edit process, including constant propagation, clock transformation, and so on. To avoid the functional problems that caused by the inequivalence between netlists before and after optimization, it is recommended to perform equivalence check using GalaxFV (X-EPIC formal verification tool) or VC Formal (third-party tool).

The following figure illustrates the work flow of vCom formal check. vCom performs seven rounds of netlist optimization. You can specify a specific round or all rounds that you want to check when compiling a design with vCom. HPE provides the `auto_formal_check` command to split a large design into several cases and then generates an executable file, `run.sh`, which you can use to start the formal check. The summary results of all cases are saved in the log file, `result.log`.

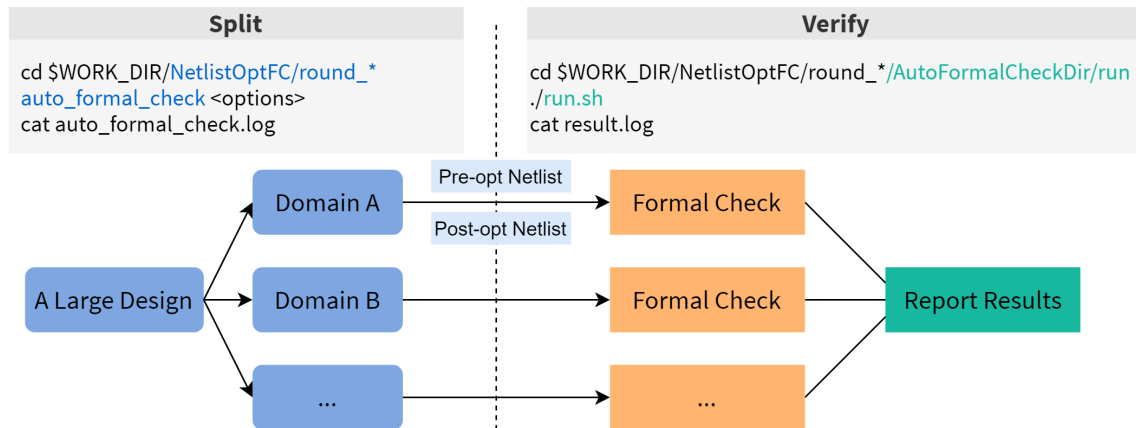


Figure 5-15: Work Flow of vCom Formal Check

Step 1: Set up the environment for vCom formal check

Before start the formal check, make sure that the following tools are installed and required environment variables are configured, such as `GALAXFV_HOME` and `PATH`.

- HPE 24.09.00
- At least one of the following formal verification tools:
 - SEC Application in GalaxFV
 - VC Formal
- Vivado 2022.2 or later version

Step 2: Select the netlist optimization round

Use the following command before `design_edit` to specify a specific round or all rounds that you want to check. After compilation completes, directories named `round_<n>` are created in the `NetlistOptFC` directory under the current working directory.

```
netlist_optimization_option -auto_formal_check all|1|2|3|4|5|6|7
```

NOTE

- vCom formal check supports only vSyn flow.
- vCom formal check supports only the simplified netlist macro flow.
- Do not rely on the formal check results for round 4.

Step 3: Split the design

Enter the *NetlistOptFC/round_<n>* directory and use the following command line to split a large design into several small designs.

```

auto_formal_check -o|--output_dir <dir>\
                  -t|--tool VCF|GalaxFV \
                  -s|--max_ff_size <ff_num> \
                  -l|--verify_level rough|detail \
                  -u|--nprocess_for_unparser <num> \
                  -p|--nprocess_for_verify <num> \
                  -n|--nthreads <num> \
                  -v|--nthread_for_verify <num> \
                  -c|--cut_point \
                  -h|-help

```

Where,

- `-o <dir>` or `--output_dir <dir>` specifies the output directory. The default output directory is *AutoFormalCheckDir* under the current working directory.
- `-t VCF|GalaxFV` or `--tool VCF|GalaxFV` specifies the formal verification tool. The default tool is VCF (VC Formal).
- `-s <ff_num>` or `--max_ff_size <ff_num>` specifies the maximum number of registers for each small design. The default number is 1000.
- `-l rough|detail` or `--verify_level rough|detail` specifies the level of formal check, rough or detail. The default level is rough.
- `-u <num>` or `--nprocess_for_unparser <num>` sets the number of process for unparser. The default number is 8.
- `-p <num>` or `--nprocess_for_verify <num>` sets the number of process for verification. The default number is 4.
- `-n <num>` or `--nthreads <num>` sets the thread number. The default number is 16.
- `-v <num>` or `--nthread_for_verify <num>` sets the thread number for one verification process. The default number is 4.
- `-c` or `--cut_point` enables cut points to improve performance.
- `-h` and `-help` print help messages.

The following gives a command example:

```

% cd /home/userA/vcom-run/NetlistOptFC/round_1
% auto_formal_check -t GalaxFV -s 800 --cut_point

```

The log file of the Split phase is saved in the *auto_formal_check.log* file in the directory where the `auto_formal_check` command is executed. If the log records Error or Fatal, an exception occurs during design split.

Step 4: Start vCom formal check

Enter the *run* directory under the *AutoFormalCheckDir* directory, or the directory you specify at Split phase and then execute the *run.sh* file.

The following gives a command example:

```
% cd /home/userA/vcom-run/NetlistOptFC/round_1/AutoFormalCheckDir/run
% chmod +x run.sh
% ./run.sh
```

Step 5: View formal check results

The log file of the Verify phase is saved in the *result.log* file in the directory where *run.sh* is executed. The results are of the following types:

- PASS: The equivalence checks of all compare points pass.
- FAILED: At least one equivalence check of compare points fails.
- ERROR: The formal verification tool reports error messages.
- INCONCLUSIVE: The equivalence check is inconclusive, possibly because the designs after split are still too large or complex.

To debug failed cases, you can check the *<case_name>.log* files in the *logs* directory for detailed information.

5.17 Running FPGA PnR

After the design generation, vCom generates the *Makefile* file in the compile-time working directory for you to quickly start the back-end synthesis and PnR.

Enter the directory *fpgaCompDir* and then execute *Makefile* to perform the back-end synthesis and PnR process.

```
% cd fpgaCompDir
% make all
```


**NOTE**

- You must place the *fpgaCompDir* folder in its original location of the compile-time and runtime working directory. Otherwise, errors might occur because of the hard-coded locations that are written in the *Makefile* file.
- Before executing the Makefile, you must set the `VCOM_HOME` environment variable and add the path of the executable file *vivado* to the `PATH` environment variable.
- Make sure that the Vivado version is 2022.2 or later.

The PnR tool generates the bitstream file that has *.bin* extension for each FPGA in the *fpgaCompDir* directory. If the bitstream file is not generated, check the errors in the log files of the PnR tool.

**TIP**

- The PnR process takes a few minutes or hours depending on the design.
- If you add nets for trigger control at compile time, a debug probe file (*.ltx*) is generated for each FPGA.

5.17.1 Estimating CPU and Memory Requirements for Running PnR

Before running PnR, refer to the following table to estimate the CPU, memory, and disk requirements for the PnR you are going to run. The table below lists the hardware requirements.

Table 5-4: Hardware Requirements for Running PnR

Requirement	Peak	Average	Minimum
CPU Core and Speed	32 cores @4.0GHz	8 cores @3.5GHz	4 cores @3.5 GHz
Memory	64 GB	48 GB	/
Disk Space	4 GB per FPGA	1 GB per FPGA	

5.17.2 Relocating PHC Pblocks

HPE supports manual PHC Pblock relocation to improve PnR results when PHC works in TDM mode. This optimization method is useful when the PnR fails or the timing closure cannot be achieved because of uneven or suboptimal PHC Pblock distribution. You can use the method to get PnR results that meet your requirements. The effect of the method depends on specific design logic.

You can use PHC Pblock relocation in the following two common scenarios:

- Too many PHC Pblocks located in the same SLR can cause congestion, which might lead to PnR failure. In this case, one way of increasing routability is to distribute the PHC Pblocks to different SLRs.
- The connection between PHC and DUT crosses multiple SLRs and the delays are too long to make timing closure difficult or the timing results unacceptable. In this case, one way of shortening the timing paths is to move the PHC Pblocks close to the DUT.

5.17.2.1 Use Flow of PHC Pblock Relocation

The following figure shows the use flow of PHC Pblock relocation.

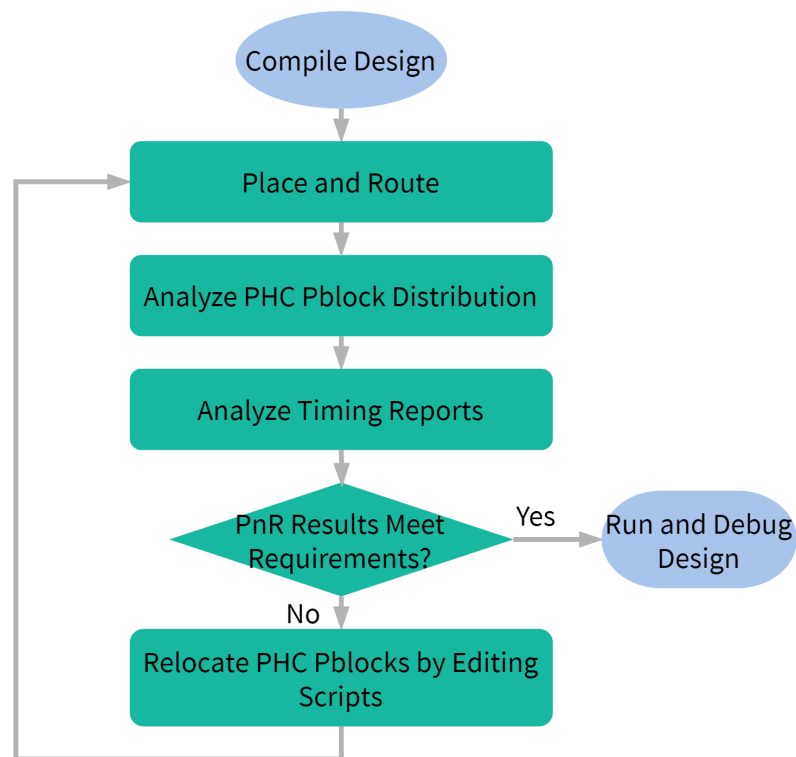


Figure 5-16: PHC Pblock Relocation Flow

Perform the following steps to slightly relocate PHC Pblocks:

1. Make sure that you have run PnR to generate necessary files that analysis and relocation need.
2. Analyze PHC Pblock usage and distribution in either of the following ways:
 - View the `<fpga_top>.phc_pblock_origin_location` text file that is stored in the `<work_path>/fpgaCompDir/part_b*_f*/pnrDir/rptDir` directory.

In the following example, the PHC Pblock information file, `dut_top_0_0.phc_pblock_origin_location`, stored in `/home/user1/work/fpgaCompDir/part_b0_f0/pnrDir/rptDir/` shows that FPGA 0.A uses PHC Pblocks which map to Bank 11 and 13-17. You can get the location of the PHC Pblocks by checking the diagram in [Location of PHC Pblock Banks](#).

```
Origin PHC pblock list :
pblock_phc_bank11
pblock_phc_bank13
pblock_phc_bank14
pblock_phc_bank15
pblock_phc_bank16
pblock_phc_bank17
```

- View the distribution of PHC Pblocks directly with Vivado in the following steps:
 - a. Click **File > Checkpoint > Open** and select the `<fpga_top>_link_design.dcp` text file in the `<work_path>/fpgaCompDir/part_b*_f*/pnrDir` directory.

TIP

To view details about PHC and DUT connection, you can view the `<fpga_top>_route.dcp` text file in the same directory as `<fpga_top>_link_design.dcp`. If your design is large, it might take a relatively long time to open the DCP file.

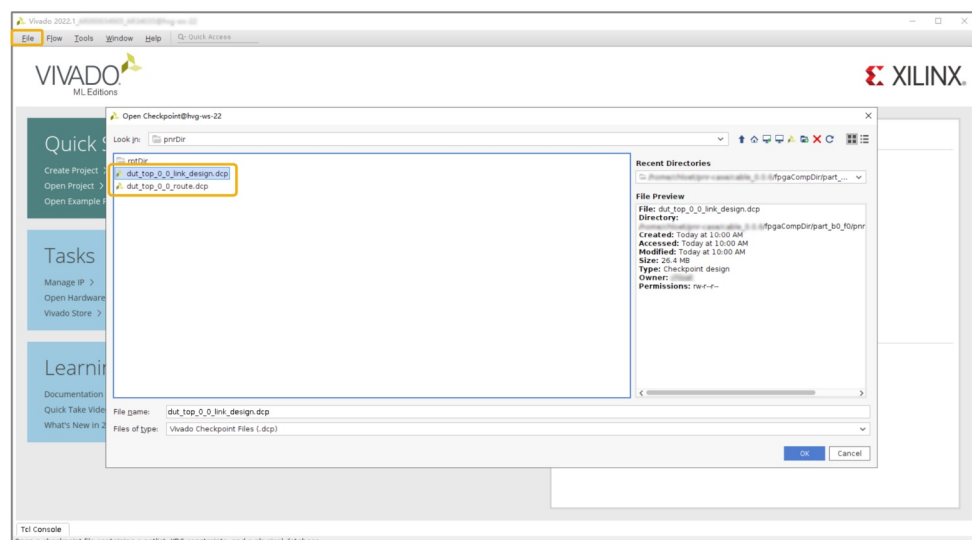


Figure 5-17: Opening DCP Files

- b. Click **Window > Physical Constraints**. The *Physical Constraints* window lists the FPGA banks where PHC Pblock logic is placed. The *Device* window highlights the PHC Pblocks and physical connections.

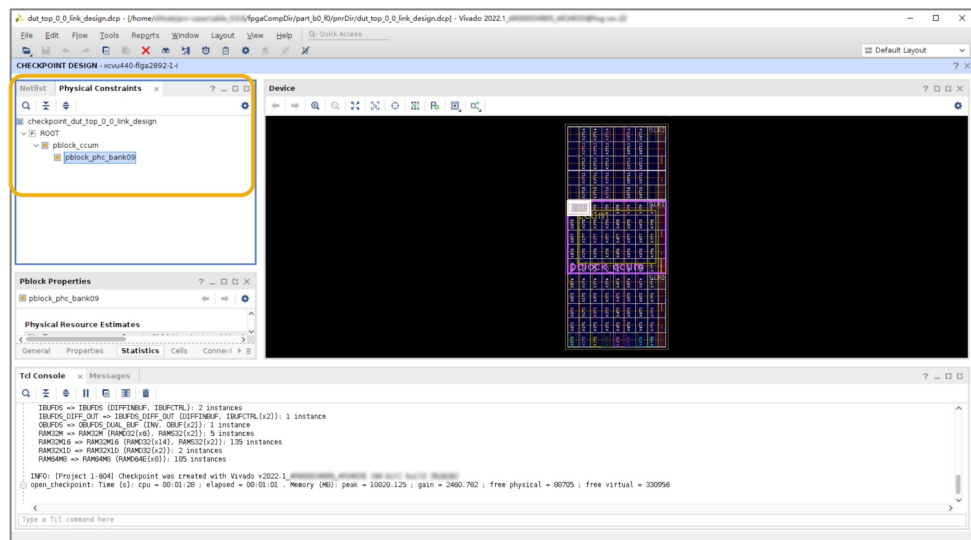


Figure 5-18: Displaying Physical Constraints

3. (Optional) View the `<fpga_top>_timing_summary.rpt` file in the `rptDir` directory to check the timing report.

TIP

Back up the PHC Pblock information files, DCP files, and timing reports if you want to compare the results after relocating PHC Pblocks.

4. Relocate PHC Pblocks by editing the `phc_pblock_relocation.tcl` Tcl script file in the `part_b*_f*` directory. The syntax of the script file is different for the device type.
 - For HuaPro P1, add the following lines for each location change.

Syntax:

```
pblock_phc_bank<target_bank> <original_bank>
phc_io_bank<target_bank> <original_bank>
delete_pblock [get_pblocks pblock_phc_bank<original_bank>]
```

Where,

- `<target_bank>` specifies the index of the target bank.
- `<original_bank>` specifies the index of the original bank.

The three command lines in the Tcl script file add the following constraints, respectively:

- Move the original bank to the target bank.
- Move IO assignments of the original bank to the target bank.
- Delete the original bank.

In the following example, *phc_pblock_relocation.tcl* in */home/user1/work/fpgaCompDir/part_b0_f0/* moves a PHC Pblock from Bank 09 to Bank 05 in HuaPro P1 FPGA 0.A.

```
pblock_phc_bank05 09
phc_io_bank05 09
delete_pblock [get_pblocks pblock_phc_bank09]
```

- For HuaPro P2E, add the following line for each location change.

Syntax:

```
phc_io_bank<target_bank>_target <original_bank>_origin
```

Where,

- `<target_bank>` specifies the index of the target bank.
- `<original_bank>` specifies the index of the original bank.

The command line in the Tcl script file adds the following constraints:

- Move the original bank to the target bank.
- Move IO assignments of the original bank to the target bank.
- Delete the original bank.

In the following example, *phc_pblock_relocation.tcl* in */home/user1/work/fpgaCompDir/part_b0_f0/* moves a PHC Pblock from Bank 09 to Bank 05 in HuaPro P2E FPGA 0.A.

```
phc_io_bank05_target 09_origin
```

5. In the *fpgaCompDir* directory, run PnR flow.

- For HuaPro P1, run PnR flow in either of the following ways:
 - Run PnR for each FPGA individually. This method is useful when you change the PHC Pblock location of a small number of FPGAs.

```
make pnr_part_b<x>_f<y>
```

Where `<x>` specifies the board index and `<y>` specifies the FPGA index.

- Run PnR for all FPGAs. This method is useful when you change the location of PHC Pblocks in many FPGAs.

```
make pnr_all
```

- For HuaPro P2E, run PnR flow in either of the following ways:
 - Run PnR for each FPGA individually. This method is useful when you change the PHC Pblock location of a small number of FPGAs.

```
make pnr_part_b<x>_f<y>
make phc_relocation
```

Where `<x>` specifies the board index and `<y>` specifies the FPGA index.

- Run PnR for all FPGAs. This method is useful when you change the location of PHC Pblocks in many FPGAs.

```
make pnr_all
make phc_relocation
```

6. Check the PHC Pblock usage and distribution after relocation in either of the following ways:

- View the `<fpga_top>.phc_pblock_post_location` text file that is stored in the `<work_path>/fpgaCompDir/part_b*_f*/pnrDir/rptDir` directory.

In the following example, the PHC Pblock information file,

`dut_top_0_0.phc_pblock_post_location`, stored in

`/home/user1/work/fpgaCompDir/part_b0_f0/pnrDir/rptDir/` shows that FPGA 0.A PHC

Bank 16 is relocated to Bank 05 and Bank 17 to Bank 06. You can get the physical

location of the PHC Pblocks by checking the diagram in [Location of PHC Pblock Banks](#).

```
PHC Relocation Information:
pblock_phc_bank16 -> pblock_phc_bank05
pblock_phc_bank17 -> pblock_phc_bank06
```

- View the distribution of PHC pblocks directly with Vivado as instructed in Step 2.

7. When the PHC Pblock relocation meets the timing requirements, contact the system administrator to reconnect PHC cables on hardware.

8. Complete the following two sub-steps before you download bitstream files to FPGAs on HuaPro P1.

 TIP

If you use HuaPro P2E, you can try to skip this step. HPE supports an early access version of automatic post-PnR process for Pblock relocation on P2E. The automatic post-pnR process is under test currently.

- a. Manually update the hardware configuration file, *hw-config.json*.
- b. If you relocate PHC Pblocks to the PHC banks that are not used in the original distribution, set the voltages of these PHC banks to 1.8V by using the `set_phc_vol` runtime command. For more information about the command usage, see [Setting the PHC Voltages](#).

For example, the original PHC banks are FPGA 0.A bank 5 and FPGA 0.B bank 5 of P1. The PHC Pblocks are relocated to FPGA 0.A bank 10 and FPGA 0.B bank 10. Then you need to set PHC voltages as follows.

```
vdbg> set_phc_vol -id 0.0 -bank 10 -voltage 1.8  
vdbg> set_phc_vol -id 0.1 -bank 10 -voltage 1.8
```

9. Download bitstream files to FPGAs and run the design with vDbg.

5.17.2.2 Location of PHC Pblock Banks

The following figure shows the FPGA banks in HuaPro P1 that can be used for PHC Pblock relocation. Internal banks and HR banks are not available for the relocation and not indicated in the figure.

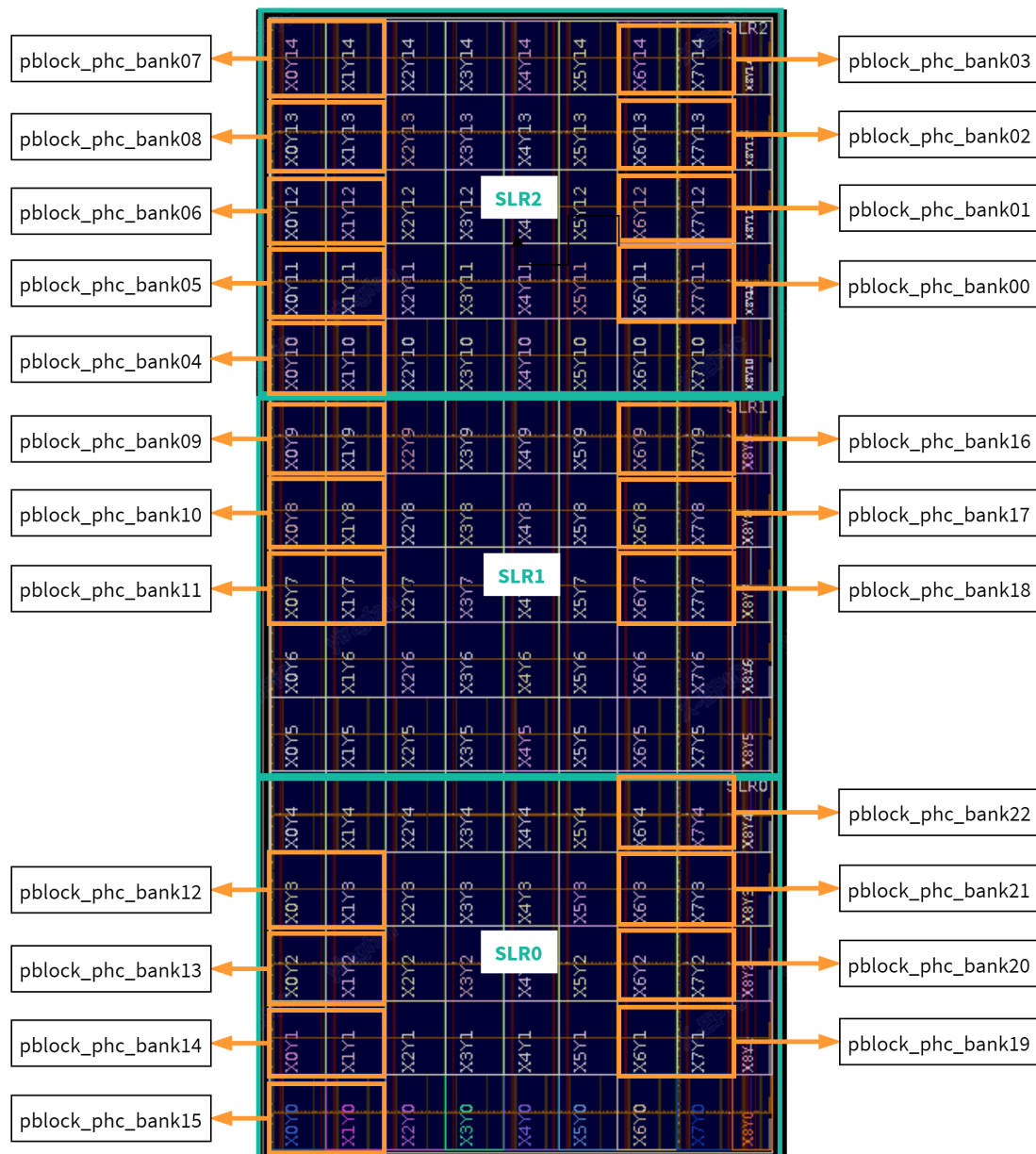


Figure 5-19: Location of PHC Pblock Banks in HuaPro P1

The following figure shows the FPGA banks in HuaPro P2E that can be used for PHC Pblock relocation. HD banks are not available for the relocation and not indicated in the figure.

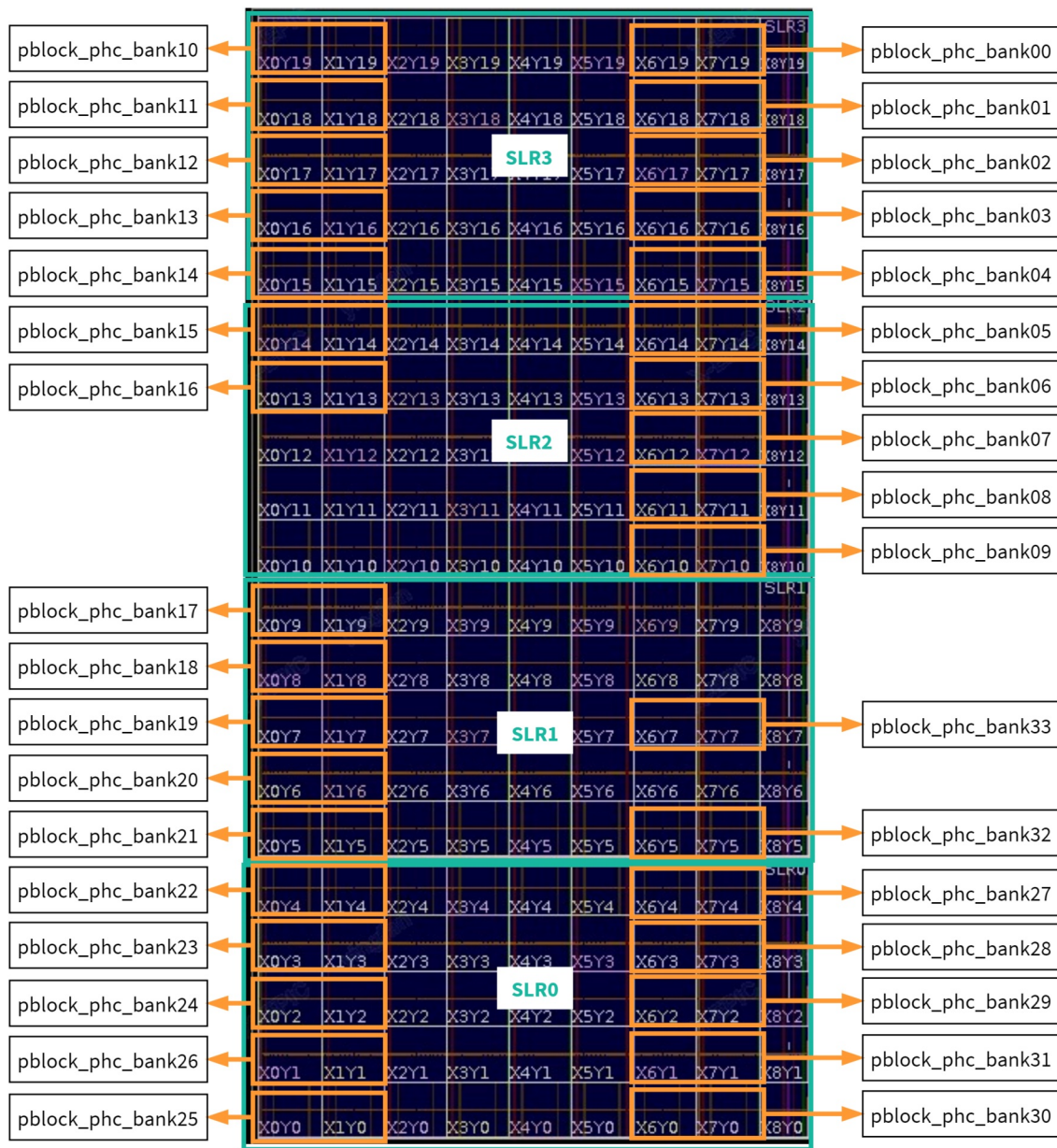


Figure 5-20: Location of PHC Pblock Banks in HuaPro P2E

For both HuaPro P1 and P2E, banks that are used for direct PHC are not available for Pblock relocation. You can check these banks from the *Netlist* pane in Vivado and their locations from the above two figures.

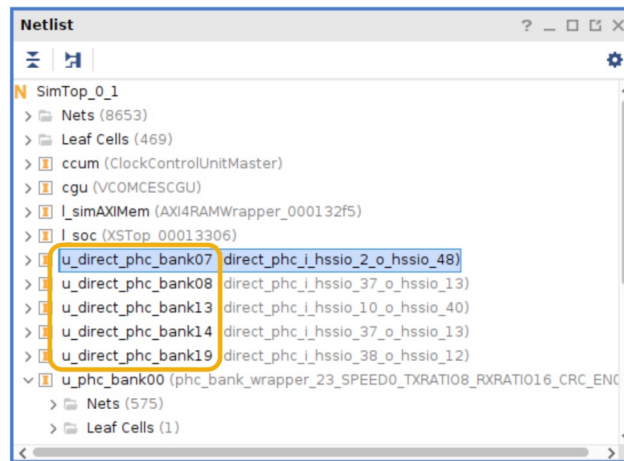


Figure 5-21: Banks for Direct PHC

5.17.3 Running PnR with Pre-defined Strategies

HPE supports several PnR strategies to help achieve desired PnR results. Different pre-defined optimization strategies are used depending on the PnR strategy. The [Selecting the PnR Strategy](#) topic describes the differences among the PnR strategies and how to run PnR in your desired strategy.

The [Running MPAR](#) topic provides an overview of MPAR use flow. The other topics in this section guide you through setting configurations for MPAR.

5.17.3.1 Selecting the PnR Strategy

You can select PnR strategy from default, runtimeopt, and MPAR.

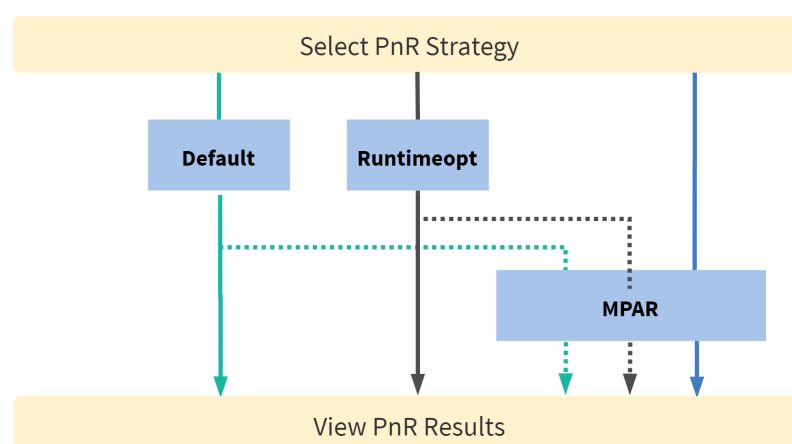


Figure 5-22: PnR Strategies in the Emulation Mode

By default, HPE runs default PnR first and when the PnR completes with a non-timing-met result starts MPAR automatically. If you do not want HPE to start MPAR, you can disable auto-

MPAR before you start PnR. You can also skip the others and directly run MPAR.

Default PnR

When you run `make all` or `make pnr_part_b*_f*` without the `pnr_strategy` option to start PnR, the PnR works with the default strategy.

```
make all
```

```
make pnr_part_b<x>_f<y>
```

Runtimeopt PnR

You can start runtimeopt PnR by adding the `pnr_strategy=runtimeopt` option on the `make all` or `make pnr_part_b*_f*` command line. When Runtimeopt is selected, HPE configures the PnR run to help speed up PnR.

```
make all pnr_strategy=runtimeopt
```

```
make pnr_part_b<x>_f<y> pnr_strategy=runtimeopt
```

MPAR

MPAR can try seven different PnR strategies with parallel runs. By default, when the PnR completes with a non-timing-met result in default, smart, or runtimeopt PnR, MPAR is automatically started.

To directly use MPAR, specify the `pnr_strategy=MPAR` option on the `make` command. You can run PnR for all FPGA partitions or an individual partition. You can also set MPAR configurations on the `make` command line.

- Run MPAR for all FPGA partitions by executing the following command line in the *fpgaCompDir* directory.

Syntax:

```
make all pnr_strategy=MPAR [n_jobs=<max_parallel_run>]
```

Where,

- `all` runs PnR for all partitions in parallel.
 - `pnr_strategy=MPAR` enables MPAR.
 - `n_jobs=<max_parallel_run>` specifies the maximum number of PnR runs which can be run in parallel.
- Run MPAR for an individual FPGA partition by executing the following command line in the

fpgaCompDir directory:

Syntax:

```
make pnr_part_b<x>_f<y> pnr_strategy=MPAR [n_jobs=<max_parallel_run>]
[apply_best_result=<boolean>]
```

Where,

- `pnr_part_b<x>_f<y>` runs PnR for an individual partition. `<x>` specifies the board index and `<y>` specifies the FPGA index.
- `pnr_strategy=MPAR` enables MPAR.
- `n_jobs=<max_parallel_run>` specifies the maximum number of PnR runs which can be run in parallel.
- `apply_best_result=<boolean>` specifies whether to apply the best strategy that you have got for the PnR run of an individual partition.

To disable auto-MPAR, add the following command line in the PnR configuration file, *PNR_settings.tcl*, and put the file in the compile-time working directory where you start vCom.

```
set ena_auto_MPAR 0
```

TIP

The *xepic_pnr.log* file reports the timing result for timing-not-met PnR, including WNS, TNS, WHS, and THS. If the timing result is acceptable, you can use the generated bitstream files in the *fpgaCompDir/part_b*_f*/pnrDir* directory before MPAR finishes.

5.17.3.2 Running MPAR

MPAR can try seven PnR strategies with parallel runs, which is useful for achieving timing closure and getting a desired PnR result. MPAR can launch up to seven PnR runs simultaneously with separate pre-defined strategies by default. Individual runs are launched on each processor on the local machine or on remote hosts as you configure. You can change the maximum number of parallel runs to avoid using up CPU resources.

During the MPAR progress, you can get a real-time summary report of runs. Once the first timing-met run completes, the strategy used for the run is reported and corresponding PnR result is stored. The other runs are stopped and related files are deleted. If you get the best strategy for an FPGA partition, MPAR supports reuse of the reported best strategy to the subsequent PnR runs of that partition.

The following figure shows the use flow of MPAR and how MPAR selects the best result.

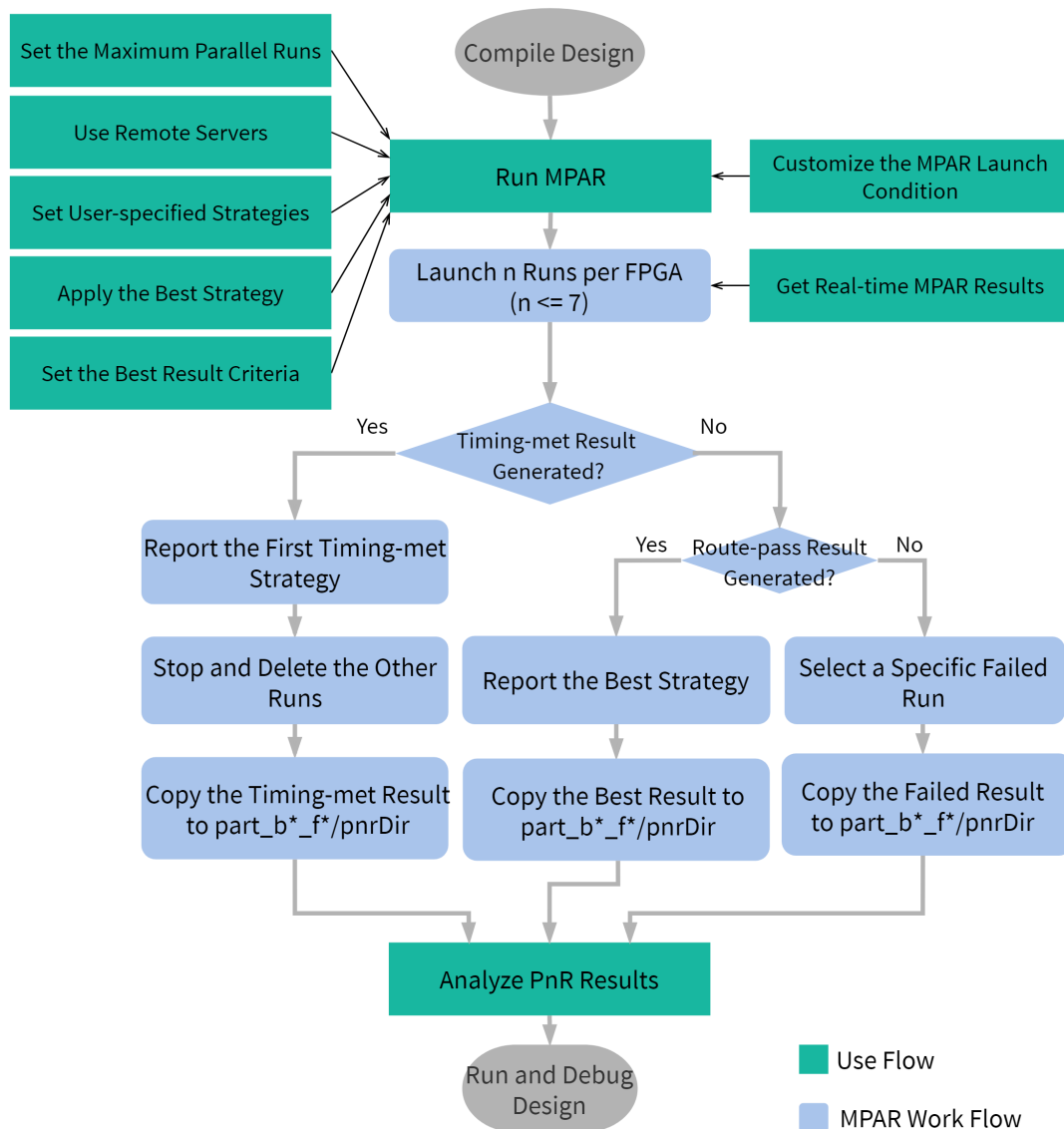


Figure 5-23: Use Flow of MPAR

Prerequisites

[Performing Design Edit and Design Generation.](#)

Procedure

Perform the following steps to run MPAR after you compile a design:

1. Enable MPAR.

- By default, MPAR is automatically started when smart PnR completes with a non-timing-met result (WNS<0 and WHS<0). If required, you can customize the MPAR launch condition by setting the WNS and WHS thresholds. For more information about how to set the thresholds, see [Customizing the MPAR Launch Condition](#).

- To directly use MPAR, specify the `pnr_strategy=MPAR` option on the `make` command..

2. (Optional) You can add configuration for MPAR as follows:

- [Setting the Maximum Number of Parallel Runs](#) by using the `n_job` option on the `make` command line.
 - [Launching MPAR Runs on Remote Hosts](#) by saving your cluster configuration in a separate file and specifying the file in the PnR configuration file.
 - [Setting User-specified MPAR Strategies](#) by using the `user_par_strategy` option in the PnR configuration file.
3. (Optional) Check the real-time status of the MPAR progress as you require (see [Checking the MPAR Progress in Real Time](#)).

MPAR starts parallel runs for each partition as you specify on the `make` command line.

MPAR selects the best result based on [Selection Criteria of the Best Run Result](#).

If required, you can make MPAR select the best result based on the hold slack, ignoring the setup slack (see [Customizing the Best Result Criteria](#)).

4. Analyze the PnR result for the best strategy that MPAR reports. When the result meets your requirements, download the bitstream files to FPGAs.

If you get the best strategy for a partition, you can apply the strategy to later runs of the partition (see [Applying the Best Strategy for Subsequent PnR Runs](#)).

5.17.3.3 Customizing the MPAR Launch Condition

MPAR is automatically started when PnR running with the other strategies does not give a timing-met result ($WNS < 0$ and $WHS < 0$) by default. If the worst slack, for example, -0.05 ns, is acceptable and the design function is not affected, you can pass MPAR and generate the bitstream file directly.

Add the following options in the PnR configuration file, *PNR_settings.tcl* to customize the MPAR launch condition. You can lower the threshold of WNS, WHS, or both as you require. Note that MPAR is launched when $TPWS < 0$, regardless of how much the two thresholds are.

- `MPAR_ignore_WNS_THR` : Specifies the WNS threshold. MPAR is launched if the actual WNS value with the sign is lower than the specified WNS threshold.
- `MPAR_ignore_WHS_THR` : Specifies the WHS threshold. MPAR is launched if the actual WHS value with the sign is lower than the specified WHS threshold.

Example:

```
set MPAR_ignore_WNS_THR -1.0
set MPAR_ignore_WHS_THR -0.2
```

You can check whether MPAR is launched or not with the threshold settings from the following messages in *part_b*_f*/xepic_pnr.log*.

```
[PNR_INFO]: MPAR is launched as the default PNR timing result is worse than the
user-defined MPAR timing threshold.
[PNR_INFO]: MPAR is not launched as the default PNR timing result is better than
the user-defined MPAR timing threshold.
[PNR_INFO]: MPAR is launched as Pulse Width check is not met (TPWS < 0), despite
of the user-defined MPAR timing threshold.
```

5.17.3.4 Setting the Maximum Number of Parallel Runs

For each FPGA partition, HPE can launch at most seven PnR runs with different pre-defined strategies in MPAR. You can use the `n_job` option to limit the maximum number of parallel runs to free up CPU resources for other tasks.

Syntax:

```
make all|pnr_part_b<x>_f<y> pnr_strategy=MPAR n_jobs=<max_parallel_run>
```

Where `<x>` specifies the board index and `<y>` specifies the FPGA index. The default value of `<max_parallel_run>` is 7, which launches at most seven runs in parallel at the same time with separate pre-defined strategies.

Your configuration of user-specified MPAR strategies affects the actual maximum number of parallel runs. For example, if you set three MPAR strategies for a partition and set the maximum number of parallel runs to seven, HPE can launch only three PnR runs at most in parallel.

Example:

The following command line launches the first four runs in parallel for `part_b0_f0` and then starts the subsequent runs after previous run completes.

```
make pnr_part_b0_f0 pnr_strategy=mpar n_jobs=4
```

5.17.3.5 Launching MPAR Runs on Remote Hosts

MPAR uses the pre-defined cluster configuration of the Vivado tool to make use of cluster server. The Vivado tool supports three pre-defined cluster configurations: LSF, SGE, and SLURM.

You can save your cluster configuration in a file and run MPAR on cluster servers in the following steps:

1. Specify your cluster configuration in the PnR configuration file, *PNR_settings.tcl*, in the compile-time working path using the following command line format.

Syntax:

```
set mpar_cluster_submit "<submit_cmd>"
set mpar_cluster_type "<type>"
```

Where,

- `<submit_cmd>` specifies the submit command of cluster configuration. Enclose the string in double quotes (" ").
- `<type>` specifies the type of cluster configuration, `LSF` , `SGE` , or `SLURM` . Enclose the string in double quotes (" ").

Example:

The following command lines in the PnR configuration file submit jobs in the SLURM cluster.

```
% set mpar_cluster_submit "sbatch --partition p1 -c 4 --mem-per-cpu=16G"
% set mpar_cluster_type "SLURM"
```

2. Start MPAR with the cluster configuration specified in the PnR configuration file.

If no cluster configuration is specified in the PnR configuration file, PnR runs are launched on the local machine by default.

5.17.3.6 Setting User-specified MPAR Strategies

MPAR launches seven runs by default with the following different pre-defined strategies:

- `MPAR_Performance_Explore`
- `MPAR_Performance_ExtraTimingOpt`
- `MPAR_Performance_NetDelay_low`
- `MPAR_Congestion_SpreadLogic_low`
- `MPAR_Flow_RuntimeOptimized`
- `MPAR_Performance_SpreadSLLs`
- `MPAR_Performance_EarlyBlockPlacement`

You can specify only one or more out of the seven strategies to be launched for any partition by setting the `user_par_strategy` option in the PnR configuration file, *PNR_settings.tcl*.

Syntax:

```
set user_pnr_strategy {part_b<x>_f<y>=<strategy>}
```

Where,

- `part_b<x>_f<y>` sets the strategy for an individual partition. `<x>` specifies the board index and `<y>` specifies the FPGA index. Use a space to separate the strategy declaration for multiple partitions.
- `<strategy>` specifies one or a number of strategies from the supported seven pre-defined strategies for a partition. Use a comma to separate multiple strategies of a single partition.

Example:

The following example sets two MPAR runs with `Performance_Explore` and `Performance_SpreadSLLs` strategies for `part_b0_f0` and one MPAR run with `Performance_NetDelay_low` strategy for `part_b2_f0`. The other parts have the default seven runs launched if they are set to run MPAR.

```
% set user_pnr_strategy {
part_b0_f0=MPAR_Performance_Explore,MPAR_Performance_SpreadSLLs
part_b2_f0=MPAR_Performance_NetDelay_low }
```

5.17.3.7 Checking the MPAR Progress in Real Time

You can run the following `make` command to check the MPAR progress in real time.

Syntax:

```
make get_mpar_results
```

The terminal output and a report show summary information about each FPGA partition for which MPAR is enabled. The report is saved as the `MPAR_summary.rpt` file in the `fpgaCompDir/MPAR_rpt` directory. You can get the current best strategy in the terminal or the report.

TIP

The best result is updated when the run proceeds. Each time you run `make get_mpar_results`, you might get different best results before all seven runs complete.

The summary report uses the following columns for tracking information:

- **Name:** Displays the name of the PnR run.
- **Status:** Indicates real-time status as not started, running, complete, or fails with error.
- **WNS:** Displays the worst negative slack.
- **TNS:** Displays the total negative slack.
- **WHS:** Displays the worst hold slack.
- **THS:** Displays the total hold slack.

- **TPWS:** Displays the total pulse width negative slack.
- **Failed Routes:** Displays the number of nets that fail to route, are partially routed, or have conflicts.
- **Runtime:** Displays the running time.
- **Strategy:** Indicates the strategy that the run uses.
- **Step:** Displays the step of the PnR flow.
- **Directive:** Displays the directive that the step uses.

The following example of the *MPAR_summary.rpt* file shows real-time MPAR information.

```

1  MPAR Results
2  -----
3
4  +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
5  | Name      | Status      | WNS  | TNS  | WHS  | THS  | TPWS  | Failed Routes | Runtime | Strategy |
6  +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
7  | MPAR_run1 | Not started | -    | -    | -    | -    | -    | -             | 00:00:00 | Performance_Explore |
8  | MPAR_run2 | Not started | -    | -    | -    | -    | -    | -             | 00:00:00 | Performance_ExtraTimingOpt |
9  | MPAR_run3 | Not started | -    | -    | -    | -    | -    | -             | 00:00:00 | Performance_NetDelay_low |
10 | MPAR_run4 | Not started | -    | -    | -    | -    | -    | -             | 00:00:00 | Congestion_SpreadLogic_low |
11 | MPAR_run5 | route_design Complete! | 0.097 | 0.000 | 0.006 | 0.000 | 0.000 | 0             | 02:56:03 | Flow_RuntimeOptimized |
12 | MPAR_run6 | Not started | -    | -    | -    | -    | -    | -             | 00:00:00 | Performance_SpreadSLs |
13 | MPAR_run7 | Not started | -    | -    | -    | -    | -    | -             | 00:00:00 | Performance_EarlyBlockPlacement |
14 | MPAR_run5 | route_design Complete! | 0.097 | 0.000 | 0.006 | 0.000 | 0.000 | 0             | 02:56:03 | Flow_RuntimeOptimized |
15  +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
16
17 Performance_EarlyBlockPlacement
18 [MPAR progress]: 7 run(s) in total. 1 run(s) completed. 0 run(s) failed with error.
19 [MPAR result] : Current Best Result: MPAR_run5.
20 +-----+-----+-----+
21 | Step      | Directive | |
22 +-----+-----+-----+
23 | [MPAR result] | RuntimeOptimized | |
24 | OPT_DESIGN  | RuntimeOptimized | |
25 | PLACE_DESIGN | RuntimeOptimized | |
26 | ROUTE_DESIGN | RuntimeOptimized | |
27

```

Figure 5-24: Example of MPAR Summary Report

5.17.3.8 Checking the Result of Each Run

The seven runs of MPAR are named from *MPAR_run1* to *MPAR_run7*. You can check the result of each run, the best run, or failed runs as you require.

- During the MPAR process, you can check the real-time results of each run in either of the following ways:
 - Run `make get_mpar_results` to get a summary report.
 - View the files in the `part_b*_f*/<fpga_top>.runs/MPAR_run*` directories. The `runme.log` file in the directory records the PnR logs.
- Once the timing-met strategy or the best strategy is selected, you can check the corresponding result by viewing the files that are listed in [Checking the Best Run Result](#).
 - When the first timing-met result is generated, the other six runs are stopped. And the files in the *MPAR_run** directories of the other runs are removed.
 - When the route-pass run with the best timing result is reported as the best result, you can analyze not only the best result but also all of the other run results. That is because the databases of all runs are kept for analysis purpose if no timing-met results are generated for all seven runs.

- If any run fails with errors, you can check the *MPAR_run*_error.log* file in the *part_b*_f** directories, which is a copy of the PnR log file, *runme.log*. If all seven runs fail, the failed run that has the highest degree of completion and the shortest running time is selected for analysis. You can check the result of this failed run in the *part_b*_f*/pnrDir* directory.

5.17.3.9 Checking the Best Run Result

The first timing-met strategy or the best strategy is reported in the *MPAR.log* and *MPAR_results.rpt* files in the *part_b*_f*/MPAR_rpt* directory. You can refer to the following files in the *part_b*_f** directory to check whether the best run result that MPAR reports meet your requirements.

File Name	Target Directory	File Description
<fpga_top>.10_worst_path_info	pnrDir/rptDir	Report of 10 worst Setup and 10 Hold paths
<fpga_top>_timing_summary.rpt	pnrDir/rptDir	Post-route timing summary report
<fpga_top>_post_route_utilization.rpt	pnrDir/rptDir	Post-route utilization report
<fpga_top>_pre_place_utilization.rpt	pnrDir/rptDir	Pre-place utilization report
<fpga_top>_opt_design.dcp	pnrDir	Post-opt DCP
<fpga_top>_route.dcp	pnrDir	Post-route DCP
<fpga_top>.bit	pnrDir	bit file
<fpga_top>.bin	pnrDir	bin file

5.17.3.10 Applying the Best Strategy for Subsequent PnR Runs

Once you get the best PnR strategy for an individual FPGA partition, you can use this strategy for later PnR runs of this partition. It is useful when you make a few changes of the design, which do not affect the selection of the best strategy.

Use either of the following two ways to run PnR with the best strategy:

- Use the following `make` command line. This way is commonly used when you want to reuse the best strategy for a single partition.

Syntax:

```
make pnr_part_b<x>_f<y> pnr_strategy=MPAR apply_best_result=1
```

Where `<x>` specifies the board index and `<y>` specifies the FPGA index. The `apply_best_result` option is default to 0. You need to set it `1` to reuse the best strategy.

- Add the following command line to the PnR configuration file, *PnR_settings.tcl*, in the compile-time working path. This way is commonly used when you want to reuse the best strategy for multiple partitions.

Syntax:

```
set user_pnr_strategy { part_b<x>_f<y>=MPAR_best }
```

Where `<x>` specifies the board index and `<y>` specifies the FPGA index. Use a space to separate the declaration for multiple partitions.

TIP

Currently, you can use the `apply_best_result` option only with the `pnr_part_b<x>_f<y>` option, instead of the `all` option. To apply the best strategy for all partitions, set `user_pnr_strategy` for each partition in the PnR configuration file and then run `make all`.

Examples:

- The following example runs MPAR with the best strategies that are reported in the previous runs for `part_b0_f1`.

```
% make pnr_part_b0_f1 pnr_strategy=MPAR apply_best_result=1
```

- The following example runs MPAR with the best strategies that are reported in the previous runs for `part_b0_f1` and `part_b0_f2`, and runs smart PnR with the HPE-default strategy for the other partitions.

Tcl scripts in the PnR configuration file:

```
% set user_pnr_strategy { part_b0_f1=MPAR_best part_b0_f2=MPAR_best }
```

The `make` command line:

```
% make all
```

5.17.3.11 Selection Criteria of the Best Run Result

MPAR selects the best run result based on the following criteria:

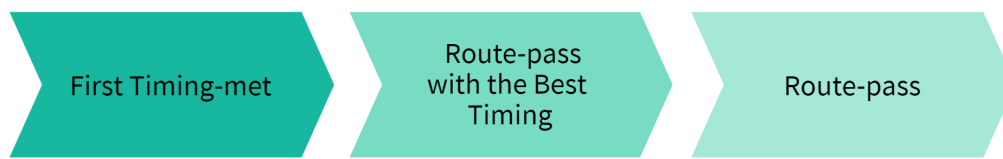


Figure 5-25: Selection Criteria of the Best Run Result

- **Rule 1:** The result of the *first* timing-met run is the best result. Once any of the parallel runs completes with timing requirements met, this run result is reported as the best result and the strategy that is used for the run is the best strategy. If by any chance two timing-met runs complete with the same running time, the one with better timing results of WNS and WHS is the best.
- **Rule 2:** If all runs finish without a timing-met result, the route-pass run (Unrouted Routes = 0) with the best timing result is selected as the best.
- **Rule 3:** If only one run succeeds in route, the run is selected as the best. If all seven runs are of route-fail (Unrouted Routes > 0) results, no best result is reported.

5.17.3.12 Customizing the Best Result Criteria

By default, the first result with both the setup and hold timing met is the MPAR best result. If you want MPAR to select the first hold-timing-met result as the best result regardless of how much the setup slack is, set the `MPAR_mode` option to `hold_meet` in the PnR configuration file, `PNR_settings.tcl`, before running PnR.

```
set MPAR_mode "hold_meet"
```

To use the default best result criteria, remove the settings of `MPAR_mode` from `PNR_settings.tcl` or set the option to `all_meet` explicitly.

```
set MPAR_mode "all_meet"
```

5.17.4 Launching Parallel PnR Runs in a Cluster

When many Vivado jobs run in parallel on the local machine, Vivado synthesis and PnR runs might fail because of insufficient resources of memory. To make the parallel jobs work correctly and stably, you can use a cluster, such as Slurm and LSF, to distribute the jobs to existing resources.

Set up the server hosts of a cluster and make sure that the cluster is running correctly. Then you can perform the following steps to run PnR in a cluster:

1. Set the `FARM_CMD` environment variable to a command line of the cluster, such as the `srun` command of Slurm and the `bsub` command of LSF.
2. Run PnR (see [Running FPGA PnR](#)).

Examples:

- The following two examples are equivalent. They both submit the PnR run of a single FPGA partition as a job to the queue named *normal* and run the job on the *turbo01* host. Each job requests a minimum of two tasks and 14336 KB memory.

- Set the `FARM_CMD` environment variable and run the `pnr_part_b0_f0` option with the `make` command.

```
% setenv FARM_CMD "bsub -q normal -m turbo01 -n 2 -R rusage[mem=14336]"
% cd /home/user1/compile/fpgaCompDir
% make pnr_part_b0_f0
```

- Directly run the `bsub` command.

```
% bsub -q normal -m turbo01 -n 2 -R rusage[mem=14336] make -f
/home/user1/compile/fpgaCompDir/part_b0_f0/Makefile pnr
pnr_strategy=default n_jobs=7 apply_best_result=0
mpar_cluster_config=local
```

- For a single-FPGA design, the following command lines submit the back-end synthesis and PnR runs to a Slurm job in sequence. Each job requests one CPU and 1 GB of memory per task and uses the `hpc` partition for resource allocation.

```
% setenv FARM_CMD "srun --cpus-per-task 1 --partition hpc --mem-per-cpu=1G"
% cd fpgaCompDir
% make all
```

- For a two-FPGA design, the following example submits the back-end synthesis of two FPGA partitions to two LSF jobs in parallel. When the synthesis jobs complete, the PnR runs of two FPGA partitions are submitted to two jobs. Each job requests one CPU and 1 GB of memory per task and uses the `hpc` partition for resource allocation.

```
% setenv FARM_CMD "srun --cpus-per-task 1 --partition hpc --mem-per-cpu=1G"
% cd fpgaCompDir
% make all

.....
srun --cpus-per-task 1 --partition hpc --mem-per-cpu=1G make -f
/home/user1/compile/fpgaCompDir/part_b0_f0/Makefile pnr pnr_strategy=default
n_jobs=7 apply_best_result=0 >
/home/user1/compile/fpgaCompDir/part_b0_f0/.pnr.log & srun --cpus-per-task 1
--partition hpc --mem-per-cpu=1G make -f
/home/user1/compile/fpgaCompDir/part_b0_f1/Makefile pnr pnr_strategy=default
n_jobs=7 apply_best_result=0 >
/home/user1/compile/fpgaCompDir/part_b0_f1/.pnr.log & wait
.....
```

5.18 Checking Timing Analysis Results

To verify that the timing of a single-FPGA design is properly constrained, you can check the PnR summary report which includes inner-FPGA STA results. To check the timing of a multi-FPGA design, you can refer to the system-level STA (SSTA) reports besides the inner-FPGA STA report.

STA results including the following reports:

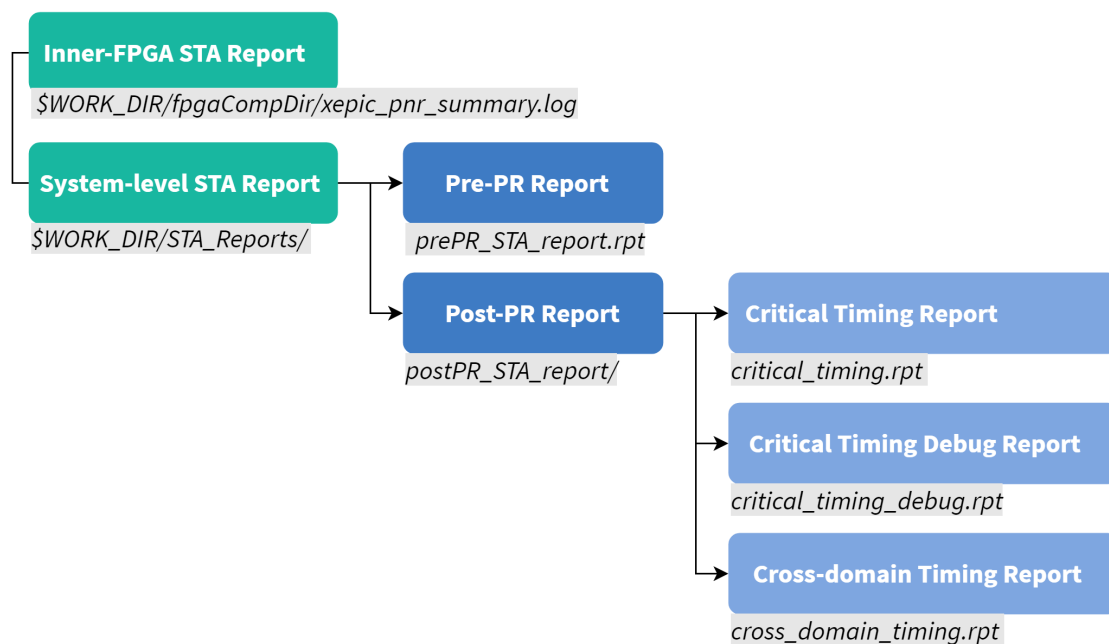


Figure 5-26: Timing Reports

- **Inner-FPGA STA Report**

The inner-FPGA STA report is generated by the third-party PnR tool, Vivado, for each FPGA part. vCom provides a PnR summary report, *fpgaCompDir/xepic_pnr_summary.log*, for you to quickly get an overview of the timing results of all partitions. For more information about the report, see [Checking the PnR Summary Report](#).

- **System-level STA Report**

vCom performs the SSTA before PnR (pre-PR) and after PnR (post-PR) during the compilation of the netlist. SSTA helps vCom optimize the timing-driven partitioning and routing of a multi-FPGA design. SSTA also helps vCom find the worst delay and estimate the root clock frequency of the system. You can check the SSTA timing reports and modify the compilation to get better runtime performance.

All system-level STA reports are stored in the `$WORK_DIR/STA_Reports` directory.

- **Pre-PR Report**

Pre-PR STA focuses on the timing path across the parts and provides an estimated result of timing for reference. For accurate timing results, you can refer to the post-PR report.

The pre-PR report is stored in the `prePR_STA_report.rpt` file and is generated after vCom compilation finishes.

- **Post-PR Report**

Post-PR STA calculates the worst timing in the system level. vCom only analyzes the inter-FPGA timing paths in the post-PR timing analysis. For the timing segment within one FPGA, vCom reports only the two ends of the segment and does not report the intermediate combination cells.

You can get the following types of post-PR timing reports when the bitstream file of each part is generated. All reports are stored in the `postPR_STA_report` directory.

- **Critical Timing Report**

The critical timing report only analyzes the PHC connections that are critical timing paths. The report is stored in the `critical_timing.rpt` file and is generated after PnR finishes.

If you restart PnR and want to update the critical timing report, you can generate the critical timing report individually by executing the `make sta_report` command in the `fpgaCompDir` directory.

Each time you update the critical timing report, vCom saves the older `postPR_STA_report` directory as `postPR_STA_report.bak.<num>` for backup, where `<num>` is the system clock number.

- **Critical Timing Debug Report**

As a supplement to the critical timing report, vCom can generate the critical timing debug report. The report provides detailed information about the inter-FPGA timing paths and accurate path delay estimation, which can help debug the timing.

The critical timing debug report is stored in the *critical_timing_debug.rpt* file and can be generated only when you execute the `make sta_debug` command in the *fpgaCompDir* directory after executing `make all` or `make sta_report`.

5.18.1 Checking the PnR Summary Report

HPE provides a target, `post_pnr_summary`, in the *fpgaCompDir/Makefile* for you to quickly check PnR results. When the target is run, a summary report, *xepic_pnr_summary.log* is generated in the *fpgaCompDir* director and you can also check the summary table from the terminal.

You can check timing results and FPGA resource usage after performing PnR.

- To start PnR and get the PnR summary report for all partitions, execute the following command line.

```
make all
```

- To get the PnR summary report for specific partitions, complete PnR runs for the partitions and then execute the following command line.

```
make post_pnr_summary
```

The summary report consists of two tables. One summarizes the timing results and is displayed on the terminal. The other summarizes the resource utilization. Following is an example for the PnR summary report.

```

1 *****
2 ||||| PNR_summary_table:
3 *****
4
5 [INFO]: 1.No data for utilization means PNR failed before place_design. Please refer to part_b*_f*/xepic_pnr.log for details.
6 [INFO]: 2.No data for WNS/TNS/WHS/THS/TPWS/TPWS means PNR failed before or in route_design. Please refer to part_b*_f*/xepic_pnr.log for details.
7 [INFO]: 3.If the Timing_result is Timing not met, please refer to part_b*_f*/pnrDir/rptDir/*_timing_summary.rpt for details.
8
9 +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
10 | PNR_runtime | PNR_result | Timing_result | WNS | TNS | WHS | THS | WPWS | TPWS |
11 +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
12 | part_b0_f3 | 1:35:16 | Bitfile generated | Timing met | 0.014 | 0.000 | 0.004 | 0.000 | 0.000 | 0.000 |
13 | part_b1_f3 | 1:27:22 | Bitfile generated | Timing met | 0.064 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
14 | part_b0_f2 | 1:12:29 | Bitfile generated | Timing met | 0.034 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
15 | part_b1_f0 | 1:49:16 | Bitfile generated | Timing met | 0.666 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
16 | part_b1_f2 | 2:1:54 | Bitfile generated | Timing met | 0.038 | 0.000 | 0.010 | 0.000 | 0.000 | 0.000 |
17 | part_b1_f1 | 1:25:2 | Bitfile generated | Timing met | 0.046 | 0.000 | 0.010 | 0.000 | 0.000 | 0.000 |
18 | part_b0_f1 | 1:33:34 | Bitfile generated | Timing met | 0.556 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
19 | part_b0_f0 | 1:17:40 | Bitfile generated | Timing met | 0.034 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
20 +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
21 | LUT | LUTM | FF | BRAM | URAM | DSP | CARRY | F7_Muxes | F8_Muxes | F9_Muxes | CLB | Uni Ctrl Sets |
22 +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
23 | part_b0_f3 | 80564(1.97%) | 7984(0.84%) | 125870(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0(0.00%) | 0(0.00%) | 22903(4.48%) | 9517(0.93%) |
24 | part_b1_f3 | 80573(1.97%) | 7984(0.84%) | 125834(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0(0.00%) | 0(0.00%) | 23138(4.53%) | 9495(0.93%) |
25 | part_b0_f2 | 80587(1.97%) | 7968(0.83%) | 125888(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0(0.00%) | 0(0.00%) | 23273(4.56%) | 9491(0.93%) |
26 | part_b1_f0 | 79546(1.95%) | 7968(0.83%) | 125850(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0(0.00%) | 0(0.00%) | 22431(4.39%) | 9495(0.93%) |
27 | part_b1_f2 | 80593(1.97%) | 7984(0.84%) | 125820(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0(0.00%) | 0(0.00%) | 22736(4.45%) | 9479(0.93%) |
28 | part_b1_f1 | 80596(1.97%) | 7984(0.84%) | 125861(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0(0.00%) | 0(0.00%) | 22860(4.48%) | 9494(0.93%) |
29 | part_b0_f1 | 79611(1.95%) | 7984(0.84%) | 125841(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0(0.00%) | 0(0.00%) | 23960(4.69%) | 9518(0.93%) |
30 | part_b0_f0 | 80590(1.97%) | 7963(0.83%) | 125838(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0(0.00%) | 0(0.00%) | 23100(4.52%) | 9484(0.93%) |
31 +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
32

```

Figure 5-27: Example of a PnR Summary Report

The first table in the PnR summary report uses the following columns to track information for each partition:

- **PNR_runtime:** Displays the running time of PnR.

- **PNR_result:** Indicates whether the PnR run completes successfully. If the bitstream file is not generated, the PnR summary table shows the step of the PnR flow in which the PnR run fails.
- **Timing_result:** Indicates whether the PnR result is timing-met or not.
- **WNS:** Displays the worst negative slack.
- **TNS:** Displays the total negative slack.
- **WHS:** Displays the worst hold slack.
- **THS:** Displays the total hold slack.
- **WPWS:** Displays the worst pulse width slack.
- **TPWS:** Displays the total pulse width negative slack.

The second table in the PnR summary report summarizes the FPGA resource utilization in different partitions. Following are descriptions of the columns in the table.

- **LUT:** Displays the LUT utilization number and percentage.
- **LUTM:** Displays the LUTRAM utilization number and percentage.
- **FF:** Displays the flip-flop utilization number and percentage.
- **BRAM:** Displays the block RAM utilization number and percentage.
- **URAM:** Displays the URAM utilization number and percentage.
- **DSP:** Displays the DSP utilization number and percentage.
- **CARRY8:** Displays the CARRY8 utilization number and percentage.
- **F7_Muxes:** Displays the MUXF7 utilization number and percentage.
- **F8_Muxes:** Displays the MUXF8 utilization number and percentage.
- **F9_Muxes:** Displays the MUXF9 utilization number and percentage.
- **CLB:** Displays the CLB utilization number and percentage.
- **Uni Ctrl Sets:** Displays the utilization number and percentage of unique control sets.

TIP

No data for resource utilization means that the PnR fails before the place_design step in PnR flow. You can check the *xepic_pnr.log* file in the *part_b*_f** directory for details.

For a PnR run failure or PnR run success with timing violation, you can refer to other log and report files for details.

- If the PnR fails, you can check the *xepic_pnr.log* file in the *part_b*_f** directory for detailed run status information.
- If the bitstream file is generated, but the timing is not met, you can check the *xepic_pnr.log* file and also the *<fpga_top>_timing_summary.rpt* file in the *part_b*_f*/pnrDir/rptDir* directory for detailed timing information.

Here are two examples of the PnR summary report:

- In the following example, PnR for *part_b1_f0* fails in the *opt_design* step. You can refer to the *xepic_pnr.log* file stored in the *part_b1_f0* directory for error messages and logs of the failure.

```

1  *****
2  ||||| PNR_summary_table:
3  *****
4  [INFO]: 1.No data for utilization means PNR failed before place_design. Please refer to part_b*_f*/xepic_pnr.log for details.
5  [INFO]: 2.No data for WNS/TNS/WHS/TPWS means PNR failed before or in route_design. Please refer to part_b*_f*/xepic_pnr.log for details.
6  [INFO]: 3.If the Timing_result is Timing not met, please refer to part_b*_f*/pnrDir/rptDir/*_timing_summary.rpt for details.
7
8  *****
9  | - | PNR_runtime | PNR_result | Timing_result | WNS | TNS | WHS | THS | WPWS | TPWS |
10 |-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
11 | part_b1_f3 | 1:27:22 | Bitfile generated | Timing met | 0.064 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
12 | part_b1_f1 | 1:25:12 | Bitfile generated | Timing met | 0.046 | 0.000 | 0.010 | 0.000 | 0.000 | 0.000 |
13 | part_b0_f2 | 1:12:29 | Bitfile generated | Timing met | 0.034 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
14 | part_b0_f0 | 1:17:40 | Bitfile generated | Timing met | 0.034 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
15 | part_b0_f1 | 1:33:34 | Bitfile generated | Timing met | 0.556 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
16 | part_b1_f0 | 1:49:16 | opt_design failed | Timing met | 0.050 | 0.000 | 0.010 | 0.000 | 0.000 | 0.000 |
17 | part_b1_f2 | 2:22:29 | Bitfile generated | Timing met | 0.034 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
18 | part_b0_f3 | 1:35:16 | Bitfile generated | Timing met | 0.014 | 0.000 | 0.004 | 0.000 | 0.000 | 0.000 |
19
20 *****
21 | - | LUT | LUTM | FF | BRAM | URAM | DSP | CARRYB | F7_Muxes | F8_Muxes | F9_Muxes | CLB | Uni Ctrl Sets |
22 |-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
23 | part_b1_f3 | 80573(1.97%) | 7984(0.84%) | 125834(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 23138(4.53%) | 9495(0.93%) |
24 | part_b1_f1 | 80586(1.97%) | 7984(0.84%) | 125861(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 22860(4.48%) | 9494(0.93%) |
25 | part_b0_f2 | 80587(1.97%) | 7958(0.83%) | 125888(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 23273(4.56%) | 9491(0.93%) |
26 | part_b0_f0 | 80590(1.97%) | 7963(0.83%) | 125838(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 23100(4.52%) | 9484(0.93%) |
27 | part_b0_f1 | 79611(1.95%) | 7984(0.84%) | 125841(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 23960(4.69%) | 9518(0.93%) |
28 | part_b1_f0 | 79546(1.95%) | 7968(0.83%) | 125850(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 22431(4.39%) | 9495(0.93%) |
29 | part_b1_f2 | 80593(1.97%) | 7984(0.84%) | 125820(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 22736(4.45%) | 9479(0.93%) |
30 | part_b0_f3 | 80564(1.97%) | 7984(0.84%) | 125870(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 22903(4.48%) | 9517(0.93%) |
31

```

Figure 5-28: Example of a PnR Summary Report with Failed PnR Run

- In the following example, PnR for *part_b1_f2* succeeds to generate a bitstream file but fails to achieve timing closure. You can refer to the *dut_top_1_2_timing_summary.rpt* file stored in the *part_b1_f2/pnrDir/rptDir* directory for more timing information.

```

1  *****
2  ||||| PNR_summary_table:
3  *****
4  [INFO]: 1.No data for utilization means PNR failed before place_design. Please refer to part_b*_f*/xepic_pnr.log for details.
5  [INFO]: 2.No data for WNS/TNS/WHS/TPWS means PNR failed before or in route_design. Please refer to part_b*_f*/xepic_pnr.log for details.
6  [INFO]: 3.If the Timing_result is Timing not met, please refer to part_b*_f*/pnrDir/rptDir/*_timing_summary.rpt for details.
7
8  *****
9  | - | PNR_runtime | PNR_result | Timing_result | WNS | TNS | WHS | THS | WPWS | TPWS |
10 |-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
11 | part_b1_f3 | 1:27:22 | Bitfile generated | Timing met | 0.064 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
12 | part_b0_f1 | 1:33:34 | Bitfile generated | Timing met | 0.556 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
13 | part_b1_f0 | 1:49:16 | Bitfile generated | Timing met | 0.066 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
14 | part_b1_f1 | 1:25:12 | Bitfile generated | Timing met | 0.046 | 0.000 | 0.010 | 0.000 | 0.000 | 0.000 |
15 | part_b1_f2 | 2:1:54 | Bitfile generated | Timing not met | -0.038 | -0.126 | 0.010 | 0.000 | 0.000 | 0.000 |
16 | part_b0_f3 | 1:35:16 | Bitfile generated | Timing met | 0.014 | 0.000 | 0.004 | 0.000 | 0.000 | 0.000 |
17 | part_b0_f2 | 1:12:29 | Bitfile generated | Timing met | 0.034 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
18 | part_b0_f0 | 1:17:40 | Bitfile generated | Timing met | 0.034 | 0.000 | 0.009 | 0.000 | 0.000 | 0.000 |
19
20 *****
21 | - | LUT | LUTM | FF | BRAM | URAM | DSP | CARRYB | F7_Muxes | F8_Muxes | F9_Muxes | CLB | Uni Ctrl Sets |
22 |-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
23 | part_b1_f3 | 80573(1.97%) | 7984(0.84%) | 125834(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 23138(4.53%) | 9495(0.93%) |
24 | part_b0_f1 | 79611(1.95%) | 7984(0.84%) | 125841(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 23960(4.69%) | 9518(0.93%) |
25 | part_b1_f0 | 79546(1.95%) | 7968(0.83%) | 125850(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 22431(4.39%) | 9495(0.93%) |
26 | part_b1_f1 | 80586(1.97%) | 7984(0.84%) | 125861(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 22860(4.48%) | 9494(0.93%) |
27 | part_b1_f2 | 80593(1.97%) | 7984(0.84%) | 125820(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 22736(4.45%) | 9479(0.93%) |
28 | part_b0_f3 | 80564(1.97%) | 7984(0.84%) | 125870(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 22903(4.48%) | 9517(0.93%) |
29 | part_b0_f2 | 80587(1.97%) | 7958(0.83%) | 125888(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 23273(4.56%) | 9491(0.93%) |
30 | part_b0_f0 | 80590(1.97%) | 7963(0.83%) | 125838(1.54%) | 45.5(2.11%) | 0.0(0.0%) | 0.0(0.0%) | 1014(0.20%) | 275(0.01%) | 0.0(0.0%) | 0.0(0.0%) | 23100(4.52%) | 9484(0.93%) |
31

```

Figure 5-29: Example of a PnR Summary Report with Timing-not-met PnR Run

5.18.2 Checking Timing Analysis Results

Here is an example of the critical timing report in the emulation mode. Only the information about *path_0* is shown.

```

----- <path_0> -----
Name: simple_spi_top_0_1/u_sys_ctrl_top/i_int_phc_user_txdata[123].grp_1.driver(part_0_1)
<root_clock> -> simple_spi_top_0_0/u_sys_ctrl_top/o_int_phc_user_rxdata[136].grp_2.sink(part_0_0)
<root_clock>
Launch clock: <root_clock>
Launch clock MCP: 1
Launch clock edge: Positive
Capture clock: <root_clock>
Capture clock MCP: 2
Capture clock edge: Positive
Capture clock delay: 0
Normalized path delay: 263.548
Path details:
    simple_spi_top_0_1/u_sys_ctrl_top/i_int_phc_user_txdata[123].grp_1.driver(part_0_1)
<root_clock>
    0
    simple_spi_top_0_1/u_sys_ctrl_top/i_int_phc_user_txdata[123]                16.18
    simple_spi_top_0_0/u_sys_ctrl_top/o_int_phc_user_rxdata[123]                75.2
    simple_spi_top_0_0/u_sys_ctrl_top/i_int_phc_user_txdata[119]                8.267
    simple_spi_top_0_1/u_sys_ctrl_top/o_int_phc_user_rxdata[119]                75.2
    simple_spi_top_0_1/u_sys_ctrl_top/i_int_phc_user_txdata[136]                9.819
    simple_spi_top_0_0/u_sys_ctrl_top/o_int_phc_user_rxdata[136]                75.2
    simple_spi_top_0_0/u_sys_ctrl_top/o_int_phc_user_rxdata[136].grp_2.sink(part_0_0)<root_clock>
3.682

.....

```

Where **Name** indicates the inter-FPGA timing path **path_0**. You can get the following information about the start point and the end point of the timing path.

Take the start point

simple_spi_top_0_1/u_sys_ctrl_top/i_int_phc_user_txdata[123].grp_1.driver(part_0_1)<root_clock> for example.

- **simple_spi_0_1** indicates the top module.
- **u_sys_ctrl_top/i_int_phc_user_txdata[123]** indicates the PHC pin that the cell is connected to.
- **grp_1** indicates that the cell belongs to the clock domain group with the multi-cycle ratio 1.
- **driver** indicates that the cell is a signal driver.
- **(part_0_1)** indicates that the partition location of the cell is FPGA A Board 0.
- **root_clock** indicates that the cell is currently in the root clock domain.

The following table lists the descriptions for the keys in the critical timing report. Time unit in the reports is **ns**.

Table 5-5: Description of Keys in Critical Timing Reports in Emulation Mode

Keys	Descriptions
------	--------------

Keys	Descriptions
MCP	Multi-cycle path (MCP) ratio of a timing path indicates the equivalent number of root clock cycles of the timing requirement of the path, which means the minimum period of time to guarantee the correct function of the signal.
Launch clock	Indicates the source clock. The launch clock can be <code><root_clock></code> , <code><slot_clock></code> , <code><free_root_clock></code> , or other tool-defined clock.
Launch clock MCP	Indicates the timing requirement of the launch clock path.
Launch clock edge	Indicates the active edge of the source clock that triggers the data launching. The launch clock edge can be <code>Positive</code> or <code>Negative</code> .
Capture clock	Indicates the destination clock. The capture clock can be <code><root_clock></code> , <code><slot_clock></code> , <code><free_root_clock></code> , or other tool-defined clock.
Capture clock MCP	Indicates the timing requirement of the capture clock path.
Capture clock edge	Indicates the active edge of the destination clock that triggers the data capturing. The capture clock edge can be <code>Positive</code> or <code>Negative</code> .
Capture clock delay	Indicates the calculated delay from the capture clock point to the end point of the timing path.
Normalized path delay	Indicates the normalized delay for the path.
Path details	Provides detailed information about the partitions, PHC pins, and delay of each timing arc.

 TIP

For more information about the report, contact XEPIC for support.

6 Partitioning the Design

You can use automatic partitioning or add constraints to partition your design. This chapter describes how to partition the design and check results in the following topics:

- [Partitioning a Design Automatically or Manually](#)
- [Setting the PHC Connection Mode](#)
- [Setting the Partitioning Strategy](#)
- [Setting Constraints for Manual Partitioning](#)
- [Saving and Restoring Partitioning Results](#)
- [Using Auto Cabling](#)
- [Checking Partition Results](#)
- [Checking Routing Results](#)

6.1 Use Flow for Single-FPGA and Multi-FPGA Designs

Depending on the number of FPGAs that the design uses and whether you need to debug the design onboard, the following four use flows are supported.

- Single-FPGA implementation flow
- Single-FPGA debug flow
- Multi-FPGA implementation flow
- Multi-FPGA debug flow

For more information about the use flows, see the following topics:

- [Single-FPGA Flow](#)
- [Multi-FPGA Flow](#)

6.1.1 Single-FPGA Flow

When the calculated size of your design is less than the system-default resource utilization of single FPGA, you can run the compilation depending on the task needs:

- Implement single-FPGA design without debug.
- Instrument single-FPGA design at compile time and debug at runtime.

The following figure shows the steps in the use flow to implement or debug a single-FPGA design.

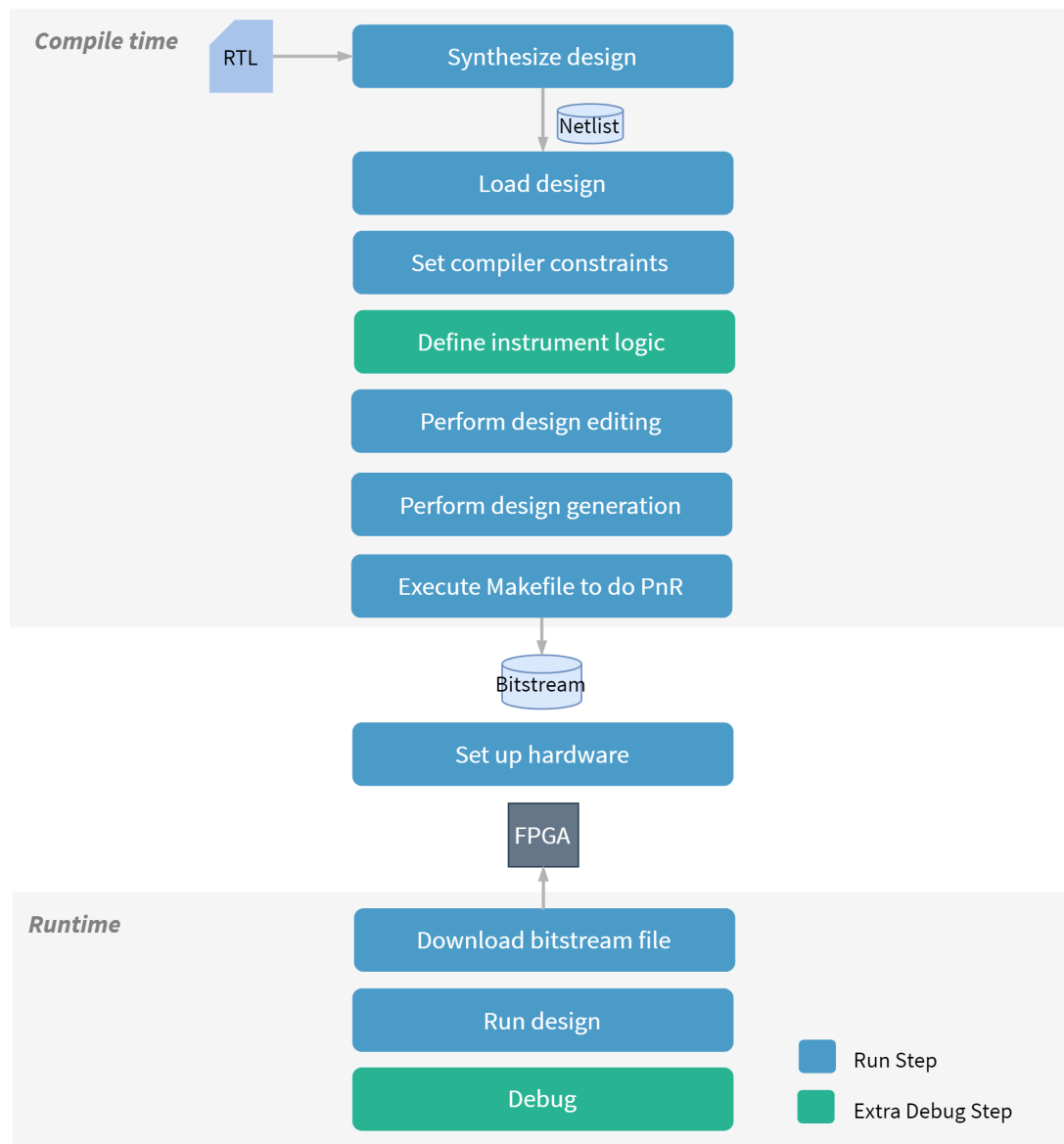


Figure 6-1: Single-FPGA Implementation and Debug Flow

For more information about each step, see the references in the following table:

Table 6-1: Steps in Single-FPGA Flow

Steps	References
Synthesize design	Chapter Synthesizing the Design
Load design	Section Adding Compilation Inputs
Set compiler constraints	Sections Describing the Hardware Configuration , Setting Timing Constraints , and Netlist Macros
Define instrument logic	Section Inserting Instrumentation Logic for Debugging
Perform design edit	Section Performing Design Edit and Design Generation

Steps	References
Perform design generation	Section Performing Design Edit and Design Generation
Execute Makefile to do PnR	Section Running FPGA PnR and User Guide of the third-party tool
Set up hardware	Reference manuals of corresponding hardware
Download bitstream file	Section Downloading the Design to the Hardware
Run design	Section Running the Design on Hardware
Debug	-

6.1.2 Multi-FPGA Flow

For a large design, you can partition the design across multiple FPGAs with the advanced automatic partitioning technology or as constrained as you require.

The following figure shows the steps in the flow to implement or debug a design in multiple FPGAs.

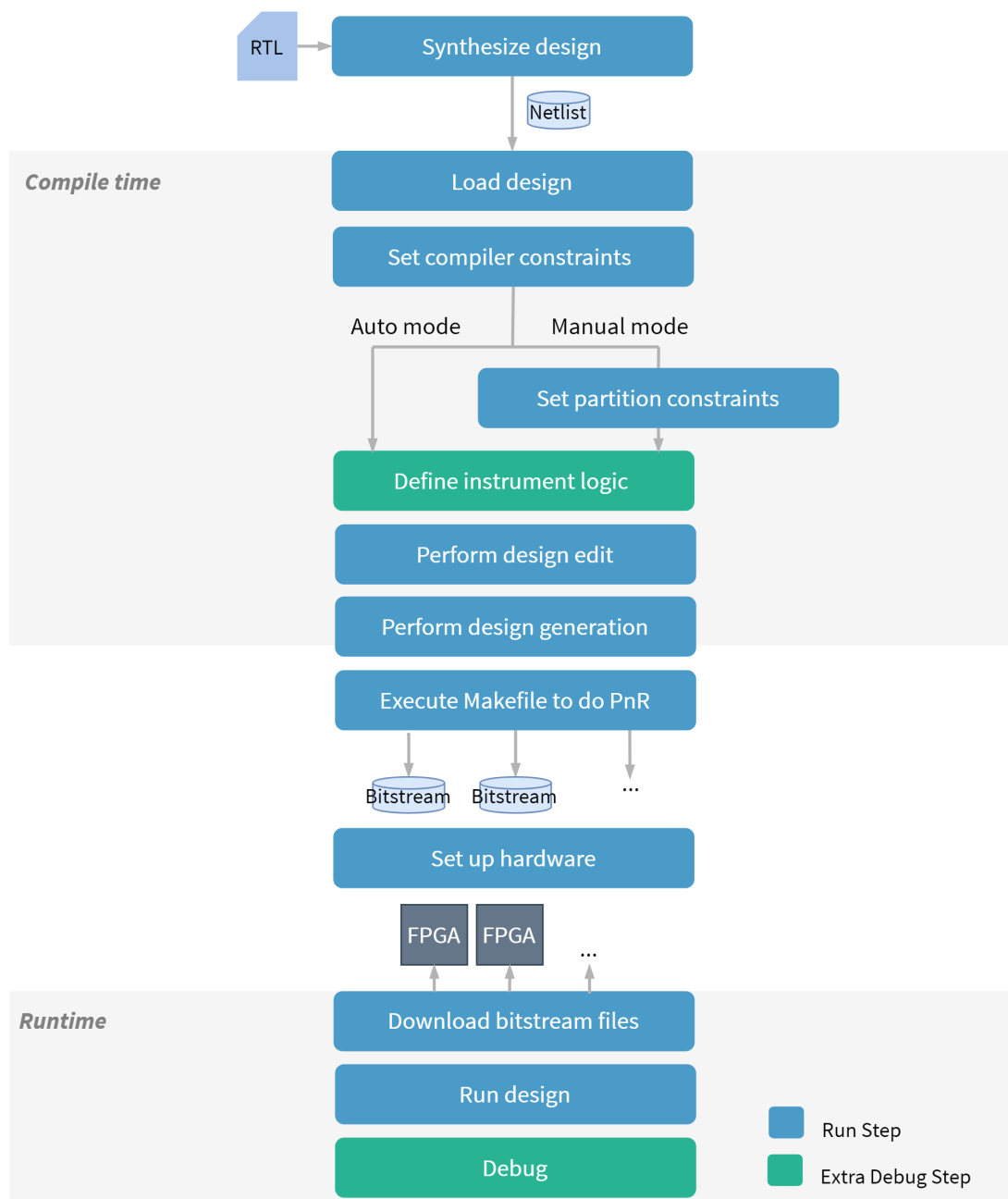


Figure 6-2: Multi-FPGA Implementation and Debug Flow

For more information about each step, see the references in the following table:

Table 6-2: Steps in Multi-FPGA Flow

Steps	References
Synthesize design	Chapter Synthesizing the Design
Load design	Section Adding Compilation Inputs
Set compiler constraints	Sections Describing the Hardware Configuration , Setting Timing Constraints , and Netlist Macros

Steps	References
Set partition constraints	Section Setting Constraints for Manual Partitioning
Define instrument logic	Section Inserting Instrumentation Logic for Debugging
Perform design edit	Section Performing Design Edit and Design Generation
Perform design generation	Section Performing Design Edit and Design Generation
Execute Makefile to do PnR	Section Running FPGA PnR and User Guide of the third-party tool
Set up hardware	Reference manuals for corresponding hardware
Download bitstream file	Section Downloading the Design to the Hardware
Run design	Section Running the Design on Hardware
Debug	-

6.2 Partitioning a Design Automatically or Manually

Partitioning is part of the design generation. In the design generation, vCom first reads the netlist file and compile-time commands to check if the partitioning step is needed.

- If the partitioning is needed, vCom partitions the design as required by user-defined constraints and generates multiple-FPGA netlists. Then in the routing step, vCom connects all cut inter-FPGA signals according to the hardware description file.
- If the partitioning is not needed, vCom bypasses both the partitioning and the inter-FPGA routing, but other operations in the design-generation process continue.

vCom provides two modes for partitioning a design to multiple FPGAs:

- **Automatic Mode**

If you do not specify any partition constraint commands, vCom performs the partitioning in default mode. vCom partitions the input gate-level design netlist automatically if the calculated size is larger than the system-default resource utilization of single FPGA (60%).

- **Manual Mode**

You can customize the partitions by setting partition constraints. You must execute the constraint commands before the `design_edit` command.

In both auto and manual modes, vCom partitions a design based on the following rules:

- Do not partition leaf instance, which means vCom places the logic in a leaf instance into one FPGA.
- If the size of the logic is larger than the system-default resource utilization (60%) of single FPGA, vCom tries to portion the logic into the other FPGA on the same board.

- If the size of the logic is larger than one board, a warning message is displayed and user-defined indivisible rules are invalid.

6.3 Setting the Partitioning Strategy

vCom supports two partitioning strategies:

- **Min-cut partition:** Reduces the cut number between partitions.
- **Timing-driven partition:** Improves design performance by considering timing information.

The default strategy is min-cut partition. To change the partition strategy, use the `par_opt_priority` command.

To get better runtime performance, you can try to set the partition strategy to timing-driven partition. Performing timing-driven partition will increase the compilation time.

6.4 Setting Constraints for Manual Partitioning

You can use one of or a combination of the following methods to partition a design to several parts as you require. vCom and the PnR tool implement the parts according to the requirements.

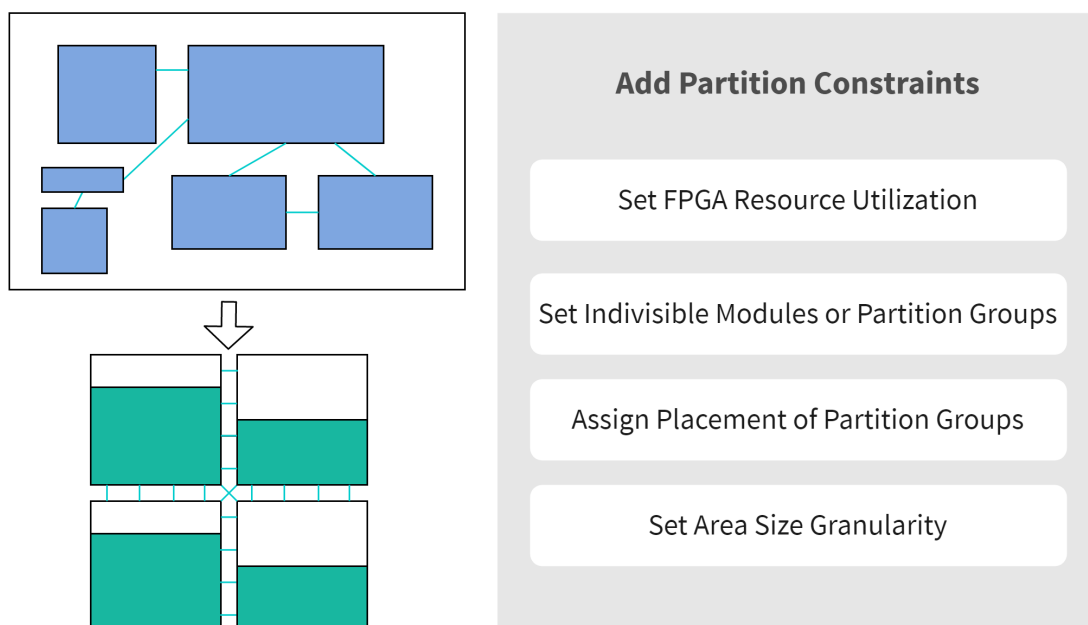


Figure 6-3: Ways of Adding Partition Constraints

- Set FPGA resource utilization for FPGAs with the `emulator_util` command. See [Setting the FPGA Resource Utilization](#) for details.
- Keep modules and groups from being partitioned.

- Keep modules from being partitioned with the `indivisible_module` command.
- Keep groups from being partitioned with the `indivisible_group` command.

See [Defining Indivisibility Rules for Specific Modules](#) for details.

- Assign groups to specific hardware resource in the following two steps.
 1. Define groups and associate portion of the design (instances or net-drivers) with a specific group by using the `par_group` command. See [Defining Partition Groups](#) for the command usage.
 2. Assign the partition group to a specific hardware resource by using the `par_assign` command. See [Assigning the Placement of Partition Group](#) for the command usage.
- Set the area size granularity for partitioning. See [Setting Area Thresholds for Partitioning](#) for details.

6.4.1 Setting the FPGA Resource Utilization

By default, when the utilization of any resource types (FF, LUT, LUTM, DSP, BRAM, and URAM) of a design is larger than 60% of a single FPGA, vCom partitions the design into several parts. If you want to change the number of parts, you can add partition constraints in the following two ways.

- Limit the utilization of all FPGA resource types for all enabled FPGAs or for a specific FPGA.

```
emulator_util -add {default|<location> [<min>] <max>}
```

In the following example, the first command line disables all FPGAs. The second command line sets the maximum resource utilization of FPGA 1.A. The combination of the two command lines specifies that only FPGA 1.A is to be used and the utilization of all resource types is limited.

```
% emulator_util -add {default 0}
% emulator_util -add {1.A 50}
```

- Limit the utilization of specific resource type. The supported resource types are FF, LUT, LUTM, DSP, BRAM, and URAM.

```
emulator_util -add {default|<location> <resource_type>:<max> ...}
```

The following command example sets the maximum utilization for each available type of resource in FPGA 0.A.

```
% emulator_util -add {0.A FF:10 BRAM:25 LUTM:25 LUT:20 DSP:30 URAM:20}
```

 TIP

- vCom adds the constraints of resource utilization considering the resources that the DUT uses and also the instrumentation logic uses for debugging features. For example, when you add trace nets for physical probes and set the maximum utilization of LUT to 50% in FPGA 0.A, vCom counts the total LUT resources used by the DUT and physical probes.
- The actual maximum utilization of each type of resources is increased by a margin of 5% by default. It means that if you set the maximum utilization to 60%, the actual utilization might be 63% ($60\% \times 1.05$).

6.4.2 Defining Indivisibility Rules for Specific Modules

You can set user-defined indivisibility rules for certain modules in any one of or a combination of the following ways.

- Specify the module for which the instantiation you do not want to partition. Assuming that a module has several instances, indivisibility means that each instance itself will not be partitioned while the instances are allowed to be placed in different parts.

```
indivisible_module -add {<module_name>}
```

- Specify the group that you do not want to be partitioned.

```
indivisible_group -add {<group_name>}
```

Note that you cannot specify same instances on the `indivisible_group` and `indivisible_module` command lines or on multiple `indivisible_group` command lines.

For more information about how to use indivisibility rules together with placement constraints, see [Combining User-defined Indivisibility Rules with Placement Constraints](#).

6.4.3 Defining Partition Groups

You can partition your design into different FPGAs by defining groups, keeping some groups from being partitioned, and assigning the groups to specific FPGAs.

You can define groups in the following ways:

- Grouping by including instances.
- Excluding instances from a group.
- Grouping by net name.

6.4.3.1 Grouping by Including Instances

To define a partition group and associate the group with specific netlist hierarchies, use the following command line:

```
par_group -instance {<group_name> <instance>}
```

Examples:

The following command line puts all the netlist in and under the path `dut_top.a` into group `group0` :

```
% par_group -instance {group0 dut_top.a}
```

The following command line puts the memory instances at and under the path `top.fifo` into two groups:

```
% par_group -instance {gp0 top.fifo.mem_1 top.fifo.mem_2 top.fifo.mem_3  
top.fifo.mem_4}  
% par_group -instance {gp1 top.fifo.mem_5 top.fifo.mem_6 top.fifo.mem_7  
top.fifo.mem_8}
```

The following command line specifies the path using Verilog-style escaping:

```
% par_group -instance {gp0 dut_top.\abc.def[0] }  
% par_group -instance {gp1 dut_top.\xyz[1].qrs[0] }
```

6.4.3.2 Excluding Instances from a Group

The compiler supports nesting. You can assign an instance to one group and assign its sub-instances (the children of an instance) to another group. Before assigning the sub-instances into another group, you need to use the `par_group -exclude` command line to exclude the children from the parent group. Otherwise, a conflict occurs.

```
par_group -exclude {<group_name> <instance>}
```

Examples:

The following command lines define a group and then remove a specific sub-instance from the group:

```
% par_group -instance {group1 top.u_rgu}  
% par_group -exclude {group1 top.u_rgu.sub}
```

The execution sequence of `par_group -instance` and `par_group -net` does not matter. The above example is equivalent to the following command lines:

```
% par_group -exclude {group1 top.u_rgu.sub}
% par_group -instance {group1 top.u_rgu}
```

The following command lines define a group with a specific sub-instance excluded and include the sub-instance to another group:

```
% par_group -instance {gp1 top.u_rgu}
% par_group -exclude {gp1 top.u_rgu.sub}
% par_group -instance {gp2 top.u_rgu.sub}
```

6.4.3.3 Grouping by Net Names

The partition group is defined based on instances for logic, flip-flops, memories, and so on. Sometimes you might want to add signal probes into a group and place the group in an FPGA for further debugging. However, specifying the path of the net drivers is not easy. In this case, you can specify the instance by a net name. The compiler associates the net drivers (the leaf instances that drive the specified net) with the defined group.

```
par_group -net {<group_name> <net>}
```

TIP

- Use the net name with caution because the same net might appear in many places with different names.
- This method should be taken as an addition to the regular grouping method which specifies the instance path, instead of an alternative.
- Leaf instance cannot be partitioned. If you assign a bus driver which is a leaf instance to different FPGAs, vCom keeps the bus driver in an FPGA and report a warning in the log file.

Examples:

The following command lines define groups with specific submodules or instances:

```
% par_group -net {group1 top.u_core.ren}
% par_group -net {group1 top.u_core.rdata}
```

The following command line includes the leaf instance that drives one bit of the bus signal `sa01_sub` in a group:

```
% par_group -net {gp0 dut_top.us10.sa01_sub[0]}
```

The following command line includes the leaf instance that drives the net `VMA` in a group:

```
% par_group -net {gp1 system.i_pram.VMA}
```

The `VMA` net and the `system.i_cpu.i_CCU` instance are highlighted in red color in the following schematic. vCom includes `system.i_cpu.i_CCU` instead of `system.i_cpu` in the `gp1` group.

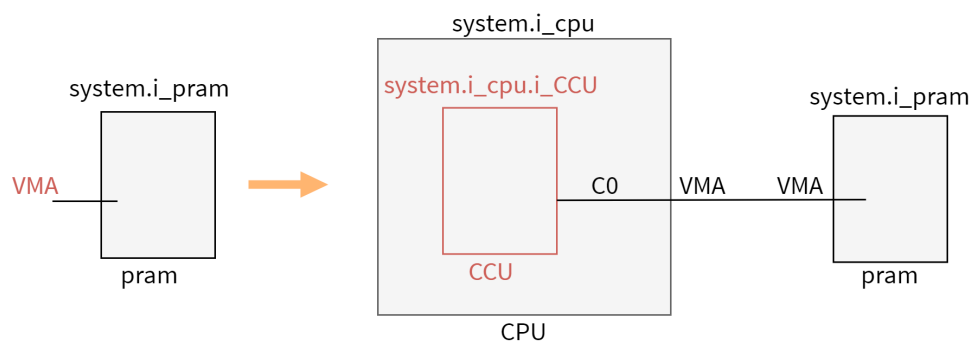


Figure 6-4: Selecting the Net Driver in the Schematic

6.4.4 Assigning the Placement of Partition Group

To restrict the placement of a specific group that is already defined, use the following command line:

```
par_assign -add {<group_name> <board_index>.<FPGA_id>}
```

You can restrict the placement of a specific group to single FPGA, multiple FPGAs, single board, multiple boards, or a combination of FPGA(s) and board(s). If the logic size of one group is larger than a single FPGA, you can assign the group to a set of FPGAs or boards as necessary. For example, if you assign a group to `0.A+0.B`, vCom can place the group on either FPGA 0.A, FPGA 0.B, or both of the two FPGAs. The actual partitioning results depend on the group size and other partitioning constraints.

Examples:

The following command lines define two groups and assign the groups to a single FPGA, respectively:

```
% par_group -instance {gp0 dut_top.gen}
% par_group -instance {gp1 dut_top.error}
% par_assign -add {gp0 0.A}
% par_assign -add {gp1 0.B}
```


The following command line assigns the group to a single board:

```
% par_assign -add {group3 0}
```

The following command line restricts the location of the group to 0+1.A+2 .

```
% par_assign -add {group5 0+1.A+2}
```

The group might be assigned to either of the following locations:

- Board 0
- FPGA 1.A
- Board 2
- Board 0 and FPGA 1.A
- Board 0 and Board 2
- Board 1.A and Board 2
- Board 1.A, FPGA 1.A, and Board 2

6.4.5 Combining User-defined Indivisibility Rules with Placement Constraints

For more precise placement constraints of groups and instances, vCom supports a combination of user-defined indivisibility rules and placement constraints of partition groups. You can use the `indivisible_group` or `indivisible_module` command together with the `par_assign` command to control the placement.

For example, when you assign a group, which consists of multiple instances, to two FPGAs, the instances in the group can be placed to one of the FPGA or partitioned. If you want to keep certain instances from being partitioned, you can set corresponding modules indivisible.

Note that the `par_assign` command can override user-defined indivisibility rules. If you set a group indivisible but assign the group to multiple FPGAs, the group might be partitioned. To keep a group from being partitioned, you can either set the group indivisible or assign it to a single FPGA. The latter method can restrict the placement of the group.

The following RTL example defines two blocks in the top module and instantiates the `sub` , `a` , and `b` modules. Detailed module definition is not shown.

```

module top
    block inst_block_0
    block inst_block_1
endmodule

module block
    sub inst_sub
    c inst_c
endmodule

module sub
    a inst_a
    b inst_b
endmodule

module a
    ...
endmodule

module b
    ...
endmodule

module c
    ...
endmodule

```

Examples:

- The following command lines assign the `inst_block_0` instance to FPGA `0.A+0.B` and set the `sub` module indivisible. The `inst_block_0` instance contains one of the instances of the `sub` module.

```

par_group -instance {gp0 top.inst_block_0}
par_assign -add {gp0 0.A+0.B}
indivisible_module -add sub

```

vCom adds constraints as follows:

- Place the `inst_block_0` instance in FPGA 0.A or 0.B, or partition the instance to the FPGAs.
- Place the target instance, `top.inst_block_0.inst_sub`, in FPGA 0.A or 0.B.
- Place the other instance, `top.inst_block_1`, in any FPGA in the system or partition the instance. Place its child instance, `top.inst_block_1.inst_sub` in a single FPGA.
- The following command lines assign the `top.inst_block_0.inst_sub` instance and set the `sub` module indivisible. The `top.inst_block_0.inst_sub` instance is one of the

instances of the `sub` module in the RTL.

```
par_group -instance {gp0 top.inst_block_0.inst_sub}
par_assign -add {gp0 0.A}
indivisible_module -add sub
```

vCom adds constraints as follows:

- Place the instance, `top.inst_block_0.inst_sub`, in the assigned FPGA 0.A.
 - Place the other instance, `top.inst_block_1.inst_sub`, in a single FPGA in the system.
- The following command lines assign the `inst_a` instance to a single FPGA and set the `sub` module indivisible. The `inst_sub0` instance is the parent instance of `inst_a` and is an instantiation of the `sub` module. vCom places the whole instance, `top.inst_block_0.inst_sub0`, in the assigned FPGA 0.A.

```
par_group -instance {gp0 top.inst_block_0.inst_sub0.inst_a}
par_assign -add {gp0 0.A}
indivisible_module -add sub
```

- The following command lines add two child instances in the `inst_block_0` instance in a group, set the group indivisible, and assign the `inst_block_0` instance to FPGA 0.A+0.B.

```
par_group -instance {gp0 top.inst_block_0.inst_sub_0}
par_group -instance {gp0 top.inst_block_0.inst_c}
indivisible_group -add gp0
par_group -instance {gp1 top.inst_block_0}
par_assign -add {gp1 0.A+0.B}
```

vCom adds constraints as follows:

- Place the `top.inst_block_0` instance in either FPGA 0.A or 0.B or partition the instance.
 - Place both two instances in the `gp0` group in a single FPGA, FPGA 0.A or 0.B.
- The following command lines assign the `inst_block_0` instance to a single FPGA and set itself indivisible. vCom places the whole instance in the assigned FPGA 0.A.

```
par_group -instance {gp0 top.inst_block_0}
par_assign -add {gp0 0.A}
indivisible_group -add gp0
```

- The following command lines assign the `inst_a` instance to a single FPGA and set its parent instance indivisible. vCom places the `gp1` group in the assigned FPGA 0.A.

```

par_group -instance {gp0 top.inst_block_0.inst_sub0.inst_a}
par_assign -add {gp0 0.A}
par_group -instance {gp1 top.inst_block_0.inst_sub0}
indivisible_group -add gp1

```

6.4.6 Setting Area Thresholds for Partitioning

You can add partition constraints for modules by setting FPGA resource utilization thresholds. Two area thresholds can be used to control the indivisibility and dissolving of modules, *Area Threshold H (High)* and *Area Threshold L (Low)*.

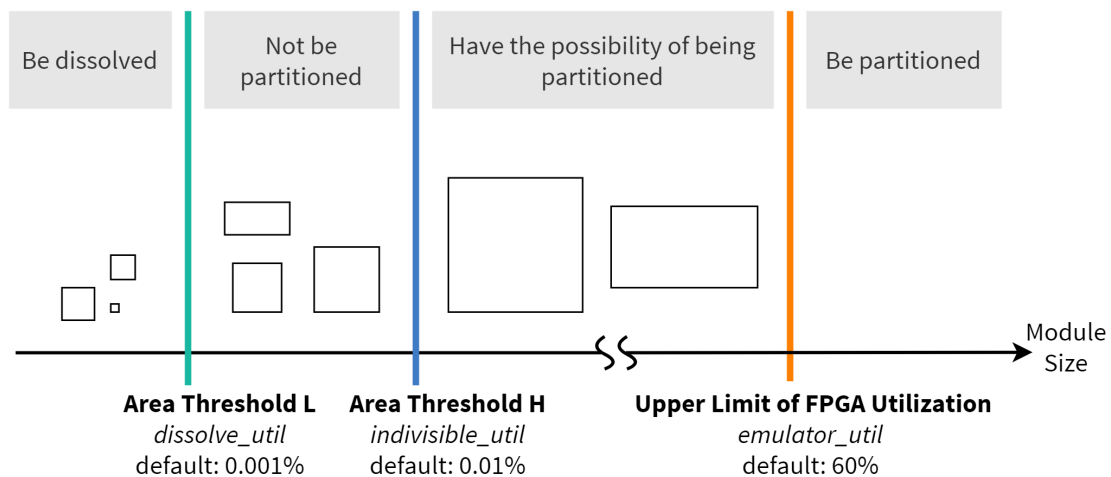


Figure 6-5: Setting Area Thresholds for Partitioning

- The **Area Threshold H** is set by the `indivisible_util -add {<n>}` command and defaults to 0.01%. The Area Threshold H must be lower than the FPGA resource utilization limitation set by the `emulator_util` command. If an invalid value is specified, one tenth of the the maximum FPGA resource utilization is used as the Area Threshold H.
- The **Area Threshold L** is set by the `prepartition_option -add {dissolve_util <n>}` command and defaults to 0%. You can set the Area Threshold L to a non-zero value to enable the tiny module dissolving feature for partition. The Area Threshold L must be smaller than the Area Threshold H. If an invalid value is specified, one tenth of the Area Threshold H is used as the Area Threshold L.

The Area Thresholds H and L divide the modules into three categories.

- **Large module:** The module whose FPGA resource utilization is larger than the Area Threshold H. Large modules might be partitioned or not depending on the partition algorithm analysis.
- **Small module:** The module whose FPGA resource utilization is between the Area Thresholds H and L. A small module is constrained to be placed in a single FPGA, or in other words, is not partitioned.

- **Tiny module:** The module whose FPGA resource utilization is smaller than the Area Threshold L. Tiny modules are dissolved which means that the behavior of a tiny module and its submodules is the same as that of its parent module for partition.

TIP

If a design has many modules with very small area, it is recommended to find out the appropriate value of area threshold L by referring to pre- and post-design-edit area reports. Using tiny module dissolving properly can help improve the partition performance and shorten the critical timing paths.

6.4.7 Example for Manually Partitioning a Design

In this example, the compilation Tcl script file specifies the following definitions and settings:

- Defines module `even_wrapper` as a netlist macro.
- Specifies the hardware description file. Refer to the following example of `hw.hdf`.
- Specifies the maximum resource utilization for FPGA 0.B and 1.A and disables the other FPGAs in the two boards.
- Restricts module `registerPos` to be unpartitioned.
- Defines two groups and set odd-related instances and even-related instances to different groups.
- Assigns the two groups to specific FPGAs respectively.

```
design_read -netlist {./dut_top.vm}
netlistmacro -add {work even_wrapper} -resource {LUT:8000}
netlistmacro_param -module even_wrapper -attribute {NL_FILE_PATH
./dut_src/even_wrapper.vm}
design_load -top dut_top

emulator_spec -add "file hw.hdf"
create_clock -sig_name dut_top.clk -frequency 100Mhz

emulator_util -add {default 0}
emulator_util -add {0.B 60}
emulator_util -add {1.A 50}

par_group -instance {gp0 dut_top.odd_wrapper}
par_group -instance {gp0 dut_top.odd_monitor}
par_group -instance {gp1 dut_top.even_wrapper}
par_group -instance {gp1 dut_top.even_monitor}

indivisible_module -add registerPos
```

```

par_assign -add {gp0 0.B}
par_assign -add {gp1 1.A}

write_net -add {reset_n}
read_net -add {error_odd}
read_net -add {error_even}
read_net -add {cnt_out}
read_net -add {cnt_odd}
read_net -add {cnt_even}

set_dr_mode -add enable
design_edit
design_generation

```

In the hardware description file `hw.hdf`, the cable connections between FPGA `0.B` and `1.A` are defined. This file defines two DDR cards for the system external memory and signal probe.

```

VERSION += 1.0
DEVICE += P1
BOARD += {"index":0, "type":"TWO_CHIP_BOARD"}
BOARD += {"index":1, "type":"TWO_CHIP_BOARD"}
CABLE += {"type":"P1_CABLE_1000", "connection":"(1.A.13, 0.B.6)"}
CABLE += {"type":"P1_CABLE_2000", "connection":"(0.B.8, 1.A.9)"}
CARD += {"type":"DRAM_16G_V1", "connector":"1.A.1"}
CARD += {"type":"CAPC_16G_V1", "connector":"0.B.1"}

```

The following is a part of the `partition_report`. You can get the FPGA location and resources of each module and instance.

Instances :: Part :: Resource

Instance	Location	Module	Total LUTs
partition_0	0.B	partition_0	5297
partition_0.dut_top	0.B	dut_top	5297
partition_0.dut_top.odd_wrapper	0.B	odd_wrapper	1540
partition_0.dut_top.odd_monitor	0.B	cnt_monitor	3757
partition_1	1.A	partition_1	17914
partition_1.dut_top	1.A	dut_top	17914
partition_1.dut_top.cnt_pos	1.A	registerPos_WIDTH_1024	4098
partition_1.dut_top.even_monitor	1.A	cnt_monitor	3757

For the information about the partition results, see [Checking Partition Results](#).

6.5 Setting the PHC Connection Mode

Each FPGA on the HuaPro hardware has external PHC connectors on the top panel and internal PHC connectors. vCom supports three PHC connection modes, normal PHC, lite PHC, and direct PHC.

- **Normal PHC**

Normal PHC can provide high-bandwidth for data transmission. vCom inserts logic to multiplex the pins when cut-nets exceed the available FPGA I/O capacity. During the inter-FPGA routing process, vCom automatically selects an optimal TDM ratio according to the inter-FPGA connection capacity and the number of cut-nets.

TDM ratio supports 8, 16, 24, ..., and 256. Using normal PHC, one bank can transmit up to 5888 (23×256) nets with the same number of PHC pins assigned to each direction (RX and TX).

- **Lite PHC**

Lite PHC is a lightweight version of normal PHC, which provides lower latency of data transmission paths between different FPGAs with less bandwidth. TDM ratio 4 is supported. Using lite PHC, one bank can transmit up to 188 (47×4) nets. The PHC pins can be assigned to RX or TX direction as required.

**TIP**

Lite PHC has not been fully tested in the HuaPro P1 system. You can try to enable it and contact XEPIC for support if you encounter any problems.

- **Direct PHC**

Direct PHC can help achieve better timing results. Using direct PHC, cut-nets can be propagated directly across the FPGA boundaries with no multiplexing, which can shorten the inter-FPGA delay for critical nets.

TDM ratio is 1 if direct PHC is used. Compared with normal PHC and lite PHC, using direct PHC lowers latency, but also decreases data bandwidth. One bank can transmit at most 50 nets. In direct PHC, the PHC pins can be assigned to RX or TX direction as required.

By default, vCom can use normal PHC, lite PHC, or both for PHC connection. You can customize the range of available PHC connection modes from which vCom can select. Note that you must set the PHC connection mode before partitioning a design.

- Normal PHC is enabled by default and not supported to be disabled.
- Lite PHC is enabled by default. To disable it, use the `lite_phc -disable` command line.
- Direct PHC is disabled by default. To enable it, use the `direct_phc -enable` command line.

You can check the routing results in `vcom.log` for the TDM ratio and the numbers of pins and

cut-nets per PHC connection. See [Checking Routing Results](#) for details about the routing results.

The default bit rate for all external and internal PHC connection is 1400 Mbps. In some cases, you might need to change the TDM bit rates. To set the TDM bit rates of the PHC cables, use the following command line at compile time.

```
set_phc_speed -add {<cable_type> <bit_rate>}
```

Where `<cable_type>` can be `PHC_CABLE_500` (0.5m), `PHC_CABLE_1000` (1.0m), and `PHC_CABLE_2000` (2.0m). Valid values of `<bit_rate>` can be 1000, 1200, and 1400 for HuaPro P1 systems and 1000, 1200, 1400, and 1600 for HuaPro P2E systems.

6.6 Using Auto Cabling

As introduced in [Describing the Hardware Configuration](#), you need to provide vCom with an HDF to describe the hardware before the design edit process. In the design generation process, the design is partitioned based on the specification of inter-FPGA connection (interconnection) in the HDF. This specification also has significant effects on routing results.

For a multi-FPGA design on hardware with limited connection resources, one method to get better runtime performance is to re-organize the interconnection configuration. However, when the number of cut nets is large, it is difficult to get an optimal solution manually. vCom provides the auto cabling feature to automatically generate and apply an optimal solution for interconnection. The connectors for daughter boards are fixed as they are specified in the HDF.

To improve the auto interconnection results, you can define interconnection constraints as you require or control whether the interconnection configuration in your specified HDF are kept or ignored.

6.6.1 Dumping the Optimized Interconnection Configuration

You can use the `interconn_dump` command after the `run_to 2.1` or `run_to PreDgen` command to enable auto cabling. When you execute `interconn_dump`, vCom dumps the interconnection configuration file and applies the optimized configuration at the same time.

Note that you must specify the file extension as `.json`. Otherwise, vCom cannot identify the specified file during design generation. If you specify an existing file name on the `interconn_dump` command, vCom overwrites the file without any backup. Therefore, if you want to keep the original interconnection configuration file, you must specify a non-existing file name.

You can enable auto cabling right after 2.1 (PreDgen) or before any step prior to 2.3.2 (Route) in the flow chart. The dumped interconnection configuration file varies depending on when you

enable auto cabling.

- If you enable auto cabling before design partition, vCom generates symmetrical interconnection without considering partitioning results.
- If you enable auto cabling right after partitioning completes, vCom optimizes interconnection configuration according to partitioning results.
- You can also enable auto cabling right after clock localization completes. With the optimization results of logic replication and clock localization, vCom can better optimize interconnection configuration. In this way, runtime performance is more likely to be improved.

Examples:

The following command lines enable auto cabling right after design edit and dump an interconnection configuration file, *auto-conn1.json*.

```
% design_edit
% interconn_dump -file auto-conn1.json
% design_generation
```

The following command lines enable auto cabling right after 2.2.2 (NSplit) and dump an interconnection configuration file, *auto-conn2.json*.

```
% design_edit
% run_to NSplit
% interconn_dump -file auto-conn2.json
% run_to DImpl
```

The following command lines enable auto cabling right after 2.3.1 (CLocal) and dump an interconnection configuration file, *auto-conn3.json*.

```
% design_edit
% run_to CLocal
% interconn_dump -file auto-conn3.json
% run_to DImpl
```

For more information about the flow chart, see [Performing Design Edit and Design Generation](#).

6.6.2 Adding or Removing User-defined Interconnection Constraints

Auto cabling also supports user-defined interconnection constraints. You can add or remove constraints to modify the interconnection for a better result. To add or remove constraints for interconnection, use the `interconn_constraint` command line and then execute the `interconn_dump` command to make the modification take effect.

Example:

The following compilation script file adds interconnection constraints and enables auto cabling. The interconnection configuration file is dumped as *auto_conn_1.json*.

```

design_read -netlist dut_top.g.v
design_load -top dut_top
emulator_spec -add {file ../design/hw-config.hdf}
create_clock -sig_name clk -frequency 100Mhz
design_edit
run_to NSplit
interconn_constraint -add -disabled_connector {0.A.3 0.A.5 0.B.4 1.A.7 1.A.21
1.B.26}
interconn_constraint -add -disabled_interconnection {0.A.2:0.B.8 0.B.10:1.B.10}
interconn_constraint -add -fixed_interconnection {PHC_CABLE_500:0.A.11:0.B.14
PHC_CABLE_1000:1.A.5:1.B.20}
interconn_dump -file auto_conn_1.json
run_to DImpl

```

6.6.3 Displaying User-defined Interconnection Constraints

To display all the user-defined constraints for auto cabling, use the `interconn_constraint -display` command line.

6.6.4 Reconstructing Interconnection

When you load an HDF with `emulator_spec -add`, the interconnection configuration in your specified HDF has a higher priority than auto interconnection generation. You can use the `interconn_constraint` command to add fixed interconnections, but you cannot disable the connections or connectors that are specified in the original HDF. That means, if you want to ignore the original interconnection configuration, you need to manually comment out or delete certain lines in the HDF.

To leverage the auto cabling technique and generate optimized interconnection automatically, you can load an HDF with `emulator_spec -reconstruct_interconnection .vCom` reconstructs the interconnection defined in the original HDF and tries to generate a symmetrical interconnection for FPGAs.

6.7 Checking Partition Results

You can check partition results from the compilation log file and the partition area report.

6.7.1 Checking the Compilation Log File

If partitioning fails during compilation, you can check the partitioning errors in *vcom.log* and restart the compilation.

If the partitioning succeeds, check the partitioning results in *vcom.log* to see if the results meet your needs.

- Check the numbers of cut nets which might have an impact on the DUT with high timing requirements.
- Check the reported FPGA utilization values for each part. If the utilization values cannot achieve your utilization goals, you can set additional partition constraints and restart the compilation or adjust your utilization goals.
- Check whether the partition goals are satisfied with your constraints.

An example of a 4-way partition result in *vcom.log* is as follows:

```

20020 -----Summary for the 4-way partition-----
20023 Hyperedge Cut:      4
20029 K-1 Cut:           6
20030 Sum of External Degrees:    10
20024 Part Sizes & External degrees:
20025 (12)[2] (12)[2] (12)[4] (11)[2]
25106 Write fpga map.
25105 Chosen fpgas are: 0.A 0.B 1.A 1.B
40057 Create ccu instance successfully!
25150 0.A: FF: 28(0.01%) LUT: 31(0.01%) LUTM: 0(0.00%) DSP: 0(0.00%) BRAM: 0(0.00%) URAM:
0(0.00%) READ_NET: 9(0.05%) WRITE_NET: 1(0.01%) PHYSICAL_PROBE: 0(0.00%) STREAM_PROBE:
0(0.00%) TRIGGER_NET: 0(0.00%) MEMORY_ACCESS: 0(0.00%)
25150 0.B: FF: 25(0.01%) LUT: 25(0.01%) LUTM: 0(0.00%) DSP: 0(0.00%) BRAM: 0(0.00%) URAM:
0(0.00%) READ_NET: 8(0.05%) WRITE_NET: 0(0.00%) PHYSICAL_PROBE: 0(0.00%) STREAM_PROBE:
0(0.00%) TRIGGER_NET: 0(0.00%) MEMORY_ACCESS: 0(0.00%)
25150 1.A: FF: 27(0.01%) LUT: 25(0.01%) LUTM: 0(0.00%) DSP: 0(0.00%) BRAM: 0(0.00%) URAM:
0(0.00%) READ_NET: 8(0.05%) WRITE_NET: 0(0.00%) PHYSICAL_PROBE: 0(0.00%) STREAM_PROBE:
0(0.00%) TRIGGER_NET: 0(0.00%) MEMORY_ACCESS: 0(0.00%)
25150 1.B: FF: 24(0.01%) LUT: 25(0.01%) LUTM: 0(0.00%) DSP: 0(0.00%) BRAM: 0(0.00%) URAM:
0(0.00%) READ_NET: 8(0.05%) WRITE_NET: 0(0.00%) PHYSICAL_PROBE: 0(0.00%) STREAM_PROBE:
0(0.00%) TRIGGER_NET: 0(0.00%) MEMORY_ACCESS: 0(0.00%)
20032 MLRB Partitioning Wall Time: 0.084033 sec
12000 Design Area: FF:104 LUT:106 LUTM:0 DSP:0 BRAM:0 URAM:0 READ_NET:33 WRITE_NET:1
PHYSICAL_PROBE:0 STREAM_PROBE:0 TRIGGER_NET:0 MEMORY_ACCESS:0
29999 *****Design Partitioning ends*****
10101 [Partitioning] SUMMARY: CPU time: 0.98 s, Wall time: 0.20 s, Max memory usage: 110 MB.

```

6.7.2 Checking the Partition Area Report

vCom provides the area report, *partition_report*, for you to check the number of each resource that each part of the design uses after partition. You can view the area report in the compile-time working directory after partition completes.

By default, the area report shows the area information of all resource types for the modules whose resource utilization is equal to or lower than 0.1% at any level of hierarchy. You can customize the area reports to only address certain types of resources and list the modules that have a higher or lower value of resource utilization at certain levels of hierarchy. Use the

`partition_option` command before `design_edit` to customize the area report.

```
partition_option -add {area_report [layer:<level>] [util:<value>]
[res_type:all|fpga|<res_type>]}
```

Where,

- `layer:<level>` specifies the level of design hierarchy. By default, there is no limitation on the levels of hierarchy. Setting the level to zero disables the generation of area report.
- `util:<value>` specifies a percentage value, which is used to customize the area report by area size. If the module area is equal to or smaller than the specified percentage of the design area, the module is listed in the partition area report.
- `res_type:all|fpga|<res_type>` specifies the resource types. The parameter values are case-insensitive. Valid values are listed as follows:
 - `all` reports the resources of all types, including FPGA resources (FF, LUT, LUTM, DSP, BRAM, URAM) and debug logic resources.
 - `fpga` reports only FPGA resources.
 - `<res_type>` specifies a resource type or multiple types separated by the separator bar (|) without any space, for example, `FF|LUT|dsp`. Currently, this option is not fully tested.

The following example shows the post-design-edit area report customized by the

`partition_option -add {area_report layer:2 util:1 res_type:fpga}` command line.

Instance	Location	Module	FF	LUT	LUTM	DSP	BRAM	URAM
partition_0	0.A	partition_0	92.0	100.0	0.0	0.0	0.0	0.0
partition_0.dut_top	0.A	dut_top	92.0	100.0	0.0	0.0	0.0	0.0
partition_0.dut_top.u_ram_rw	0.A	ram_rw	78.0	80.0	0.0	0.0	0.0	0.0
partition_1	0.B	partition_1	36.0	46.0	0.0	0.0	1.0	0.0
partition_1.dut_top	0.B	dut_top	36.0	46.0	0.0	0.0	1.0	0.0
partition_1.dut_top.blk_ram_0	0.B	ram_signal	16.0	17.0	0.0	0.0	1.0	0.0
partition_1.dut_top.u_ram_rw	0.B	ram_rw	5.0	7.0	0.0	0.0	0.0	0.0
partition_2	1.A	partition_2	6.0	9.0	0.0	0.0	0.0	0.0
partition_2.dut_top	1.A	dut_top	6.0	9.0	0.0	0.0	0.0	0.0

6.8 Checking Routing Results

vCom automatically performs the routing process with an inter-FPGA routing tool after the partitioning. Inter-FPGA routing plays an important role in the execution speed of the final design and can maximize the system frequency by minimizing the TDM ratio and the hop number.

If the routing fails, you can check the routing errors in *vcom.log* and restart the compilation.

If the routing succeeds, check the routing result in *vcom.log* for PHC connectors which are

configured in the hardware description file.

The routing result lists the following information:

- Cable connections in routing.
- The optimal PHC TDM ratio. TDM ratio can be 4, 8, 16, 24, ..., 256. If direct PHC is enabled, TDM ratio can be 1. TDM ratio represents the maximum number of signals that a physical wire can accommodate.
- Pin numbers.
- The numbers of cut nets.

vCom routes the internal bank to corresponding banks of the FPGA on the same device with fixed connection. vCom gives priority to the internal banks to perform the routing step.

You can use the external banks through the PHC connectors on the top panel of the board. To connect the PHC connectors between two FPGAs, you can use X-EPIC PHC cables or interconnection daughter boards. The PHC connection affects the routing result and the PnR runtime. For recommended solutions to connect the PHC connectors, contact XEPIC for supports.

As shown in the following example of the routing result, no cut net is connected from the 0.A.26 bank to the 0.B.26 bank. 152 cut nets are connected from 0.B.26 bank to 0.A.26 bank using 19 pins and for each pin the maximum number of signals is eight.

```
40022 Routing result:
40025 0.A.25 <=> 0.B.27 TDM ratio: 8/8, pins: 19/19, cutnets: 3/152
40025 0.A.26 <=> 0.B.26 TDM ratio: 0/8, pins: 0/19, cutnets: 0/152
40025 0.A.27 <=> 0.B.25 TDM ratio: 0/8, pins: 0/19, cutnets: 0/152
40025 0.A.24 <=> 0.B.7 TDM ratio: 0/8, pins: 0/23, cutnets: 0/58
```

6.9 Saving and Restoring Partitioning Results

The flow chart mode of the design generation progress allows you to save the partitioning results and restore them later.

Perform the following steps use the `fc_save` and `fc_load` commands to save and restore partitioning results:

1. Run to Step 2.2.2 (NSplit).

```
run_to 2.2.2
```

2. Save the partition result to a specified file path. The results will be saved as `result_pool_<index>` under the specified file path, where `<index>` starts from 0.

```
fc_save {<file_path>}
```

3. Exit vCom and restart it as you require.
4. Load the partition result from specified files.

```
fc_load {<file_path>}/result_pool_<index>
```

5. Continue to run to 2.4 (DImpl).

```
run_to 2.4
```

7 Running the Design on Hardware

This chapter introduces how to start the debugger vDbg and how to run the design on the hardware with vDbg runtime commands. Before using vDbg, the hardware and required peripheral equipment must be installed and configured as your requirements.

7.1 vDbg Flow

The vDbg flow has five stages:

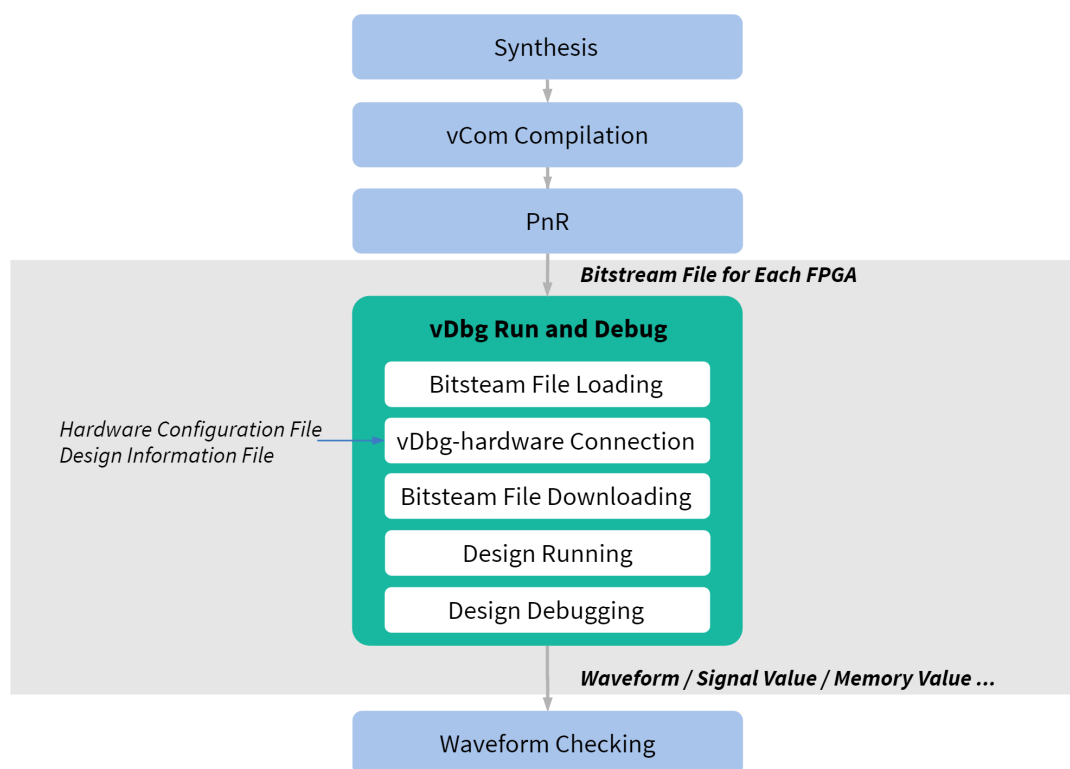


Figure 7-1: vDbg Flow

- **Bitstream File Loading**

The first step in vDbg flow is to load a design, which includes bitstream files and other necessary input files.

- **vDbg-hardware Connection**

vDbg supports two ways of connecting to the hardware, non-VRM mode and VRM mode.

- Non-VRM Mode

In the non-VRM mode, vDbg directly starts SimCtl and connects to the hardware that is configured by users (see [Using the Hardware in the Non-VRM Mode](#)).

- VRM Mode

In the VRM mode, vDbg applies to VRM for available resources and connects to the hardware that is allocated by VRM (see [Using the Hardware in the VRM Mode](#)).

- **Bitstream File Downloading**

After bitstream files and other necessary databases are loaded and vDbg can successfully connect to hardware, vDbg downloads the existing bitstream files to the boards and program the FPGAs.

- **Design Running**

vDbg controls the running of designs. You can start and stop the running in different ways. If your task requires, you can run a design without debugging at runtime. You can also stop the running in specific conditions by setting triggers.

- **Design Debugging**

vDbg provides powerful debugging capabilities, including signal access, memory access, probe tracing, and trigger stop. vDbg can integrate with Fusion Debug to assist you with waveform checking.

Input files of vDbg include the following:

- For FPGA programming: the bitstream files that have the *.bin* extension.
- [Input Files for System Connection](#).

Output values or files of vDbg include the following:

- Signal values.
- Memory values and files.
- Waveforms of probes.

7.2 Starting the Runtime Debugger vDbg

You can start vDbg, the HPE debugger, in three ways:

- Execute the `vdbg` command to start the interactive prompt and then execute the runtime commands and other Tcl commands in the prompt.

```
% vdbg
vdbg>
```

- Specify the run script file in the `vdbg` command line to start vDbg in the batch mode. For more information about creating your run script file, see [Customizing Your Run Script File](#).

```
vdbg <run_script>
```


- Execute `vdbg -gui` to start vDbg in the GUI mode. Then you can operate in the GUI or use Tcl commands in the Console part of the GUI. For more information about using the GUI, see [Debug Station GUI](#).

```
vdbg -gui
```

vDbg does not have automatic logout time. vDbg logs you out of the system only when you use the `exit` command on the vDbg prompt or in the run script.

The debug processor SimCtl starts at the same time as vDbg starts, and stops automatically when you log out of vDbg.

You can get help information in two ways:

- Use the command `help` on the vDbg prompt to get a list of all available runtime commands.

```
% vdbg
vdbg> help
```

- Specify the command in the `help` command line on the vDbg prompt to get usage information about each command.

```
vdbg> help <runtime_command>
```

7.3 List of vDbg Tcl Commands

The following table lists the runtime commands for running and debugging the design on the hardware. You can read the related references for more information.

Table 7-1: vDbg Tcl Commands for Running and Debugging a Design

Command	Reference
Design Running	
<code>set_vrm</code>	Setting the User Mode
<code>design</code>	Opening and Closing Design
<code>set_clock</code>	Setting the Frequencies of Global Clocks and Setting the Frequencies of GT Clocks
<code>get_clock</code>	Setting the Frequencies of Global Clocks
<code>hw_server</code>	Connecting vDbg to the Hardware
<code>set_phc_vol</code>	Setting the PHC Voltages

Command	Reference
<code>get_phc_vol</code>	Setting the PHC Voltages
<code>download</code>	Programming the FPGAs
<code>run</code>	Starting the Clock
<code>after_blocking</code>	Starting the Clock
<code>stop</code>	Stopping the Clock
<code>get_time</code>	Getting Current Cycle Number or Time
<code>set_div</code>	Setting the Divider Count Number
<code>get_div</code>	Setting the Divider Count Number
<code>set_time_unit</code>	Setting the Default Time Unit
Design Debugging	
<code>force</code>	Forcing and Releasing the Signal Values
<code>release</code>	Forcing and Releasing the Signal Values
<code>deposit</code>	Depositing the Signal Values
<code>signal_group</code>	Controlling Signal Values by Group
<code>get_value</code>	Getting the Signal Values
<code>memory</code>	Accessing Design Memories
<code>dump_mem</code>	Dumping Memories without Extra Logic Inserted at Compile Time
<code>trace_signals</code>	Managing Probes
<code>tracedb</code>	Generating the Waveform for Physical Probes
	Generating the Waveform for Streaming Probes
	Generating the Waveform for GRP Probes
<code>get_max_depth_cycles</code>	Getting the Maximum Depth Cycle
<code>set_trace_size</code>	Setting the Trace Size
<code>trigger</code>	Creating and Enabling ATT Trigger Instances and Managing Trigger Instances
<code>set_post_trigger_samples</code>	Setting Post-trigger Samples
<code>exit</code>	Exits vDbg.

Command	Reference
help	Prints help information.

7.4 Customizing Your Run Script File

vDbg supports to run in the batch mode. So you can include vDbg runtime commands in a Tcl script file and execute the Tcl script to run and debug the design. The Tcl script is called the *run script file* in this document.

Procedure

Perform the following steps to customize your run script file for running and debugging the design. See [Appendix A - File Templates and Examples](#) for examples of the run script file.

1. Use a text file editor or other editors to create a new Tcl script file, such as *run.tcl* and *.vdbg_run*.
2. Start the script by adding command lines for loading a design (see [Downloading the Design to the Hardware](#)).
 - a. Add a command line for specifying the design bitstream files.
 - b. Add a command line for starting the hardware server.
 - c. (Optional) To set the PHC voltages, add command lines for setting the voltages before downloading the bitstream files. If not specified, vDbg does not set the PHC voltages.
 - d. Add commands to download the bitstream files.
3. (Optional) Set the global clock frequencies.
If not specified, vDbg uses the default frequencies. See [Setting the Frequencies of Global Clocks](#).
4. (Optional) Force the values of input signals that affect design running (see [Forcing and Releasing the Signal Values](#)).
5. Add command lines for running and debugging the design (see [List of vDbg Tcl Commands](#)).
6. (Optional) If required, add other Tcl commands for running and debugging the design. Check the Tcl script syntax before saving the run script file.
7. Execute the run script file in two ways:
 - Execute the run script file in the batch mode.

```
vdbg <run_script>
```

- Source the run script file in the interactive prompt.

```
% vdbg  
vdbg> source <run_script>
```

7.5 Opening and Closing Design

You can open a design with the `design` command in vDbg.

To run a design in vDbg, you must open a debug session and specify the directory path of compilation database. The debug session is the runtime working directory, where vDbg creates the soft links of design files and saves log files. The default design session is the directory where you launch vDbg. You can use the `-session` option to create a new debug session.

Opening a Design with the Default Debug Session

Open a design with the default debug session in either of the following ways as you require.

- Use the same directory for compilation database and runtime database. In this way, soft links of design files will not be created.

For example, the debug session is the `caseA` directory.

```
% cd /home/userA/compile/caseA  
% vdbg  
vdbg> design .
```

- Use different directories for compilation database and runtime database. In this way, the files created at runtime are separated from the design files and compilation results.

For example, the debug session is the `caseA` directory that is under the `debug` directory.

```
% cd /home/userA/debug  
% vdbg  
vdbg> design ../compile/caseA
```

Opening a Design with a User-specified Debug Session

Open a design with a debug session specified by the `-session` option in one of the following ways as you require.

- Open different debug sessions for a single design in its compile-time working directory.

For example, debug session directories, `test_run_*.session`, are created in the `caseA` directory.

```
% cd /home/userA/compile/caseA  
% vdbg  
vdbg> design . -session test_run_1  
...
```

```
vdbg> design -close  
vdbg> design . -session test_run_2
```

- Open different debug sessions for multiple designs in the same directory.

For example, debug session directories, *test_run_case*.session*, are created in the *compile* directory for two cases.

```
% cd /home/userA/compile  
% vdbg  
vdbg> design caseA -session test_run_case_a  
...  
vdbg> design -close  
vdbg> design caseB -session test_run_case_b
```

**TIP**

If you launch vDbg in the compile-time working directory of design A and you want to open design B, you must use the `-session` option to create a new session or specify an existing session.

You can use the `-overwrite` option to overwrite the session directory when the specified session name exists. If a directory named *abc.session* already exists without the `-overwrite` option specified, the existing session directory will be renamed to *abc.session.bak*.

Closing a Design

Use `design -close` to close the design. If you specify a session name when opening the design, use `design -close` will switch the current working path to the father directory of session and close the design.

7.6 Downloading the Design to the Hardware

When PnR generates the bitstream files, you can download them to the hardware with vDbg.

Prerequisites

Make sure to meet the following requirements before you start the procedure.

- [Running FPGA PnR](#)
- [Starting the Runtime Debugger vDbg](#)
- [Opening and Closing Design](#)

Procedure

Perform the following steps to download the design to the FPGAs:

1. Select the user mode (see [Setting the User Mode](#)).

Non-VRM mode is enabled by default. If you want to switch to the VRM mode, execute the `set_vrm -on` command.

2. (Optional) To change the frequencies of the global clocks, use the `set_clock` command (see [Setting the Frequencies of Global Clocks](#)).
3. Use the `hw_server` command to specify the server and enable the connection between your host PC/server and the hardware (see [Connecting vDbg to the Hardware](#)).
4. (Optional) If an off-board target system is connected to the PHC connector and needs power supply, use the `set_phc_vol` command to set the PHC voltages (see [Setting the PHC Voltages](#)).
5. Use the `download` command to download the bitstream files to FPGAs (see [Programming the FPGAs](#)).

7.6.1 Setting the User Mode

By default, vDbg works in the non-VRM mode. To enable the VRM mode, use either of the following methods before loading a design:

- Execute the `set_vrm -on` command line.

Syntax:

```
set_vrm -on [-relocation|-relocationoff]
```

By default, relocation is enabled when you execute `hw_server` to connect to hardware. You can use `-relocationoff` to disable it. For more information about relocation, see [Design Relocation](#)

Example:

```
vdbg> set_vrm -on -relocation
[info] 11763 set vrm on.
```

- Set the `XPVRM_OFF` environment variable to `0`.

To switch to the non-VRM mode, execute the `set_vrm -off` command line or set the `XPVRM_OFF` environment variable to `1`.

7.6.1.1 Using the Hardware in the Non-VRM Mode

The following figure shows the main steps of using vDbg in the non-VRM mode. For more information about each step, see the procedure.

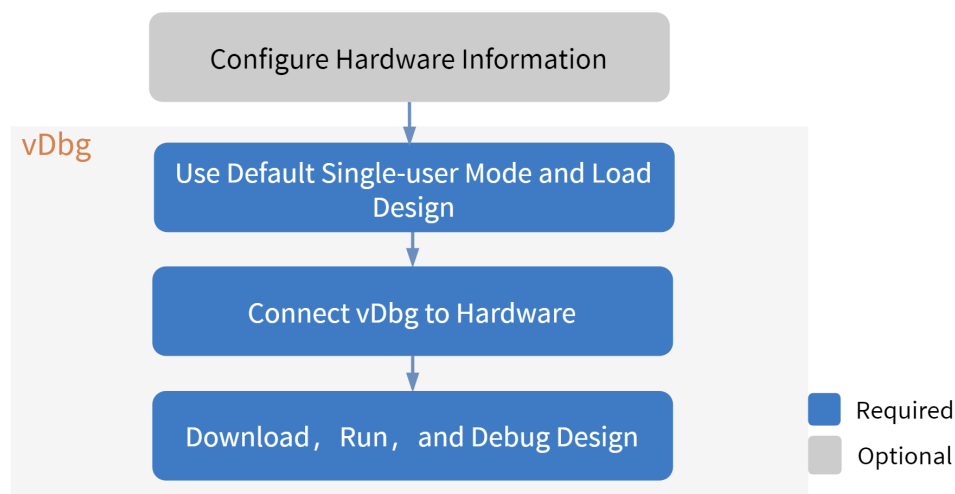


Figure 7-2: Use Flow of the Non-VRM Mode

Perform the following steps to use the hardware in the non-VRM mode:

1. Invoke vDbg and load a design.
2. Connect to the hardware that the system administrator has configured.

Usually, the system administrator configures the hardware information, you can directly start vDbg and load the design without any configuration operation.

3. Download, run, and debug the design.

IMPORTANT

When vDbg works in the non-VRM mode, you must download the design with caution to avoid unexpected overriding of the design that is running on the hardware.

TIP

If the FPGAs cannot be programmed because of system disconnection, make sure that the devices are powered on and the hardware configuration information in the *etc* directory corresponds to the hardware.

7.6.1.2 Using the Hardware in the VRM Mode

The following figure shows the main steps of using vDbg in the VRM mode. For more information about each step, see the procedure.

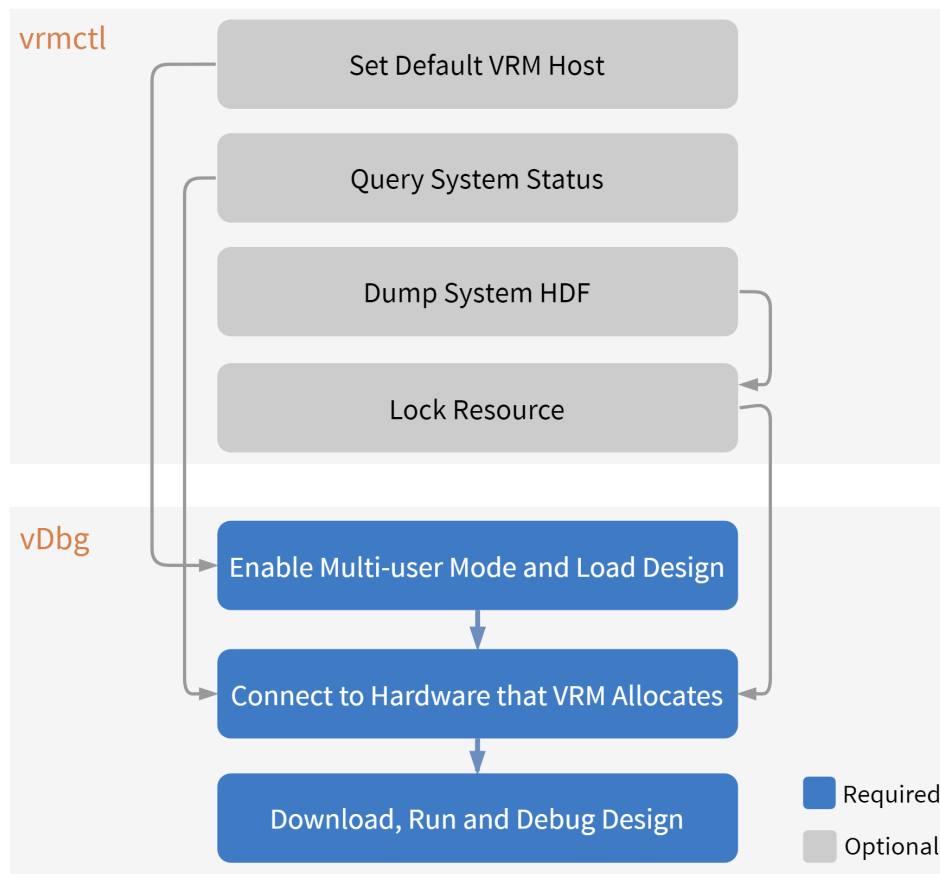


Figure 7-3: Use Flow of the VRM Mode

Perform the following steps to use the hardware in the VRM mode:

1. Make sure that you can access the default VRM host or get the available VRM host information from the system administrators.
2. (Optional) To use the hardware more efficiently, you can optionally invoke `vrmctl` to do the following operations.
 - Manage your VRM host list and set the default. If you do not add any VRM host information, `vDbg` connects to the VRM host that the system administrator sets.
 - Query the hardware resource status.
 - Dump the system HDF to prepare the user-defined HDF.
 - Lock specific resource for later usage.

See [Managing Hardware Resources](#) for more information about `vrmctl`.

3. Invoke `vDbg` and enable the VRM mode by using the `set_vrm` command.
4. Connect to the hardware that VRM allocates.

If you have locked resource, you can use the locked resources by specifying the locking key or locking file on the `hw_server` command.

If the resources you want to use is unavailable, you can specify the `-relocation` option on the `set_vrm` command to allow VRM to search for an alternative.

5. Download, run, and debug the design.

7.6.2 Setting the Frequencies of Global Clocks

The global clock that the design uses is GC1 if you have not changed it by using the `primary_clock` command. When you download the bitstream files into the hardware without any global clock settings, vDbg completes the following settings automatically:

- Setting the frequencies of the global clock that the design uses to 125 MHz.
- Setting the maximum frequencies of user clocks according to the compilation results.

In case you need to change the setting of the global clock, vDbg provides the `set_clock -clock` command and option for setting the frequencies of global clocks. vDbg sets the frequencies of global clocks as you specified when you download bitstream files into hardware.

- Set the frequency of the global clock that the design uses by using the following command line after the `hw_server` command and before `download`.

```
set_clock -clock <number_of_frequency><frequency_unit>
```

- Set the frequencies of the global clocks that the design does NOT use by using the following command line before the `hw_server` command.

```
set_clock -clock <clock_id> <number_of_frequency><frequency_unit>
```

`<clock_id>` specifies the index of the global clock that the design does not use.

vDbg also provides the `get_clock` command for you to check the current frequencies of global clocks.

- Get the current frequencies of global clocks that the design uses by using the following command line after the `download` command.

```
get_clock -clock
```

- Get the current frequencies of global clocks that the design does not use by using the following command line after the `download` command.

```
get_clock -clock <clock_id>
```

Examples:

The following command lines set the frequency of the global clock that the design uses to 30 MHz in the emulation mode:

```
vdbg> design .
vdbg> hw_server .
vdbg> set_clock -clock 30MHz
vdbg> download
[info] 11043 Download successfully! Design is ready to run.
vdbg> get_clock -clock
30000 kHz
```

The following command lines set the frequency of the global clock that the design uses to 20MHz, instead of 10MHz in the emulation mode:

```
design .
set_clock -clock 10MHz
set_clock -clock 20MHz
hw_server .
download
```

The following command lines set the frequency of the global clock GC2 that the design does not use to 30MHz in the emulation mode:

```
design .
set_clock -clock GC2 30mhz
hw_server .
download
```

7.6.3 Setting the Frequencies of GT Clocks

HuaPro P1 and P2E provide Gigabyte Transceiver (GT) interfaces for data transmission. These interfaces are organized into several banks in each FPGA. The reference clocks of these banks are called *GT clocks*.

HuaPro P1 provides eight QSFP+ and three Mini-SAS HD interfaces for each FPGA, which correspond to 11 FPGA banks. In HuaPro P1, the GT clocks are configurable. You can set the frequency as you require.

HuaPro P2E provides one QSFP28 interface and six Mini-SAS HD interfaces for each FPGA, which correspond to seven FPGA banks. Each bank has two reference clocks, RefCLK0 and RefCLK1. The reference clock RefCLK0 is configurable. You can set the frequency of RefCLK0 as you require. Except where noted, the GT clock for HuaPro P2E refers to the reference clock RefCLK0. The frequency of the reference clock RefCLK1 is fixed to 156.25 MHz and cannot be changed.

By default, the frequencies of GT clocks are set to 156.25 MHz when the hardware devices are powered on. The following operations are supported for GT clocks.

- Configure the frequencies of GT clocks with the `set_clock` command and `-gtbank` option. The command line can be used before or after design download.

```
set_clock -gtbank -id <fpga_id> -bank <band_id_or_interface_name> -frequency <freq_setting>
```

- Get the current frequencies of GT clocks with the `get_clock` command and `-gtbank` option.

```
get_clock -gtbank -id <fpga_id> -bank <band_id_or_interface_name>
```

Where,

- `-id <fpga_id>` specifies the FPGA ID in the format `<board_index>.<fpga_index>`.
- `-bank <band_id_or_interface_name>` specifies the bank ID or interface name. Refer to the following two tables for supported banks in HuaPro P1 and P2E. You can specify bank IDs, interface names, or both and separate them with commas.

Table 7-2: Mappings of GT Interfaces and FPGA banks on HuaPro P1

GT Interface	FPGA Bank	Interface Name
QSFP+	219	QA3 for FPGA A, QB3 for FPGA B
	220	QA1 for FPGA A, QB1 for FPGA B
	221	QA4 for FPGA A, QB4 for FPGA B
	222	QA2 for FPGA A, QB2 for FPGA B
	229	QA7 for FPGA A, QB7 for FPGA B
	230	QA5 for FPGA A, QB5 for FPGA B
	231	QA8 for FPGA A, QB8 for FPGA B
	232	QA6 for FPGA A, QB6 for FPGA B
Mini-SAS HD	227	MA1 for FPGA A, MB1 for FPGA B
	226	MA2 for FPGA A, MB2 for FPGA B
	225	MA3 for FPGA A, MB3 for FPGA B

Table 7-3: Mappings of GT Interfaces and FPGA banks on HuaPro P2E

GT Interface	FPGA Bank	Interface Name
QSFP28	231	QSFP28 for FPGA A-D
Mini-SAS HD	221	HSSI-0 for FPGA A-D

GT Interface	FPGA Bank	Interface Name
	222	HSSI-1 for FPGA A-D
	226	HSSI-2 for FPGA A-D
	232	HSSI-3 for FPGA A-D
	236	HSSI-4 for FPGA A-D
	237	HSSI-5 for FPGA A-D

- `-frequency <freq_setting>` specifies the frequency setting in the format `<number_of_frequency><frequency_unit>`. Valid frequency ranges from 60 MHz to 350MHz. Setting the frequency to zero means to disable the GT clock.

Examples:

- Sets the frequency of the GT clock of FPGA 0.A Bank 220 to 100 MHz.

```
set_clock -gtbank -id 0.A -bank 220 -frequency 100MHz
```

- Sets the frequencies of the GT clocks of FPGA 1.B Bank 231, 221, and 222 to 100 MHz.

```
set_clock -gtbank -id 1.B -bank 231,221,HSSI-1 -frequency 100 mhz
```

7.6.4 Connecting vDbg to the Hardware

Depending on the working mode you select, single-user or multi-user, vDbg can connect to hardware devices as specified in vDbg hardware configuration or as VRM allocates.

Configuration for system connection is usually completed by system administrators. Only if you want to override the settings, you can see [Input Files for System Connection](#) to know the required input files and how to change the settings.

Use the `hw_server` command to connect vDbg to the hardware. See [Specifying the Hardware Server](#) for details.

7.6.4.1 Specifying the Hardware Server

After opening a design, you can use the `hw_server` command to specify the hardware server where the debug processor, SimCtl, runs. At present, you can start the debug processor on the same host as vDbg.

```
hw_server <hostname>
```

The `<hostname>` can be specified as a period character (`.`), `localhost` , or `127.0.0.1` . You can use `hw_server -list` to print out the available hostnames when you open the design. Currently, only `localhost` is listed.

Use `hw_server -release` to release the hardware resource.

If you want to lock resources and use them at runtime by VRM, see [Locking Hardware Resources](#) and [Using Locked Resources](#).

7.6.4.2 Input Files for System Connection

The following figure summarizes the configuration files that vDbg needs to build connection with hardware when it works in the non-VRM mode and the VRM mode.

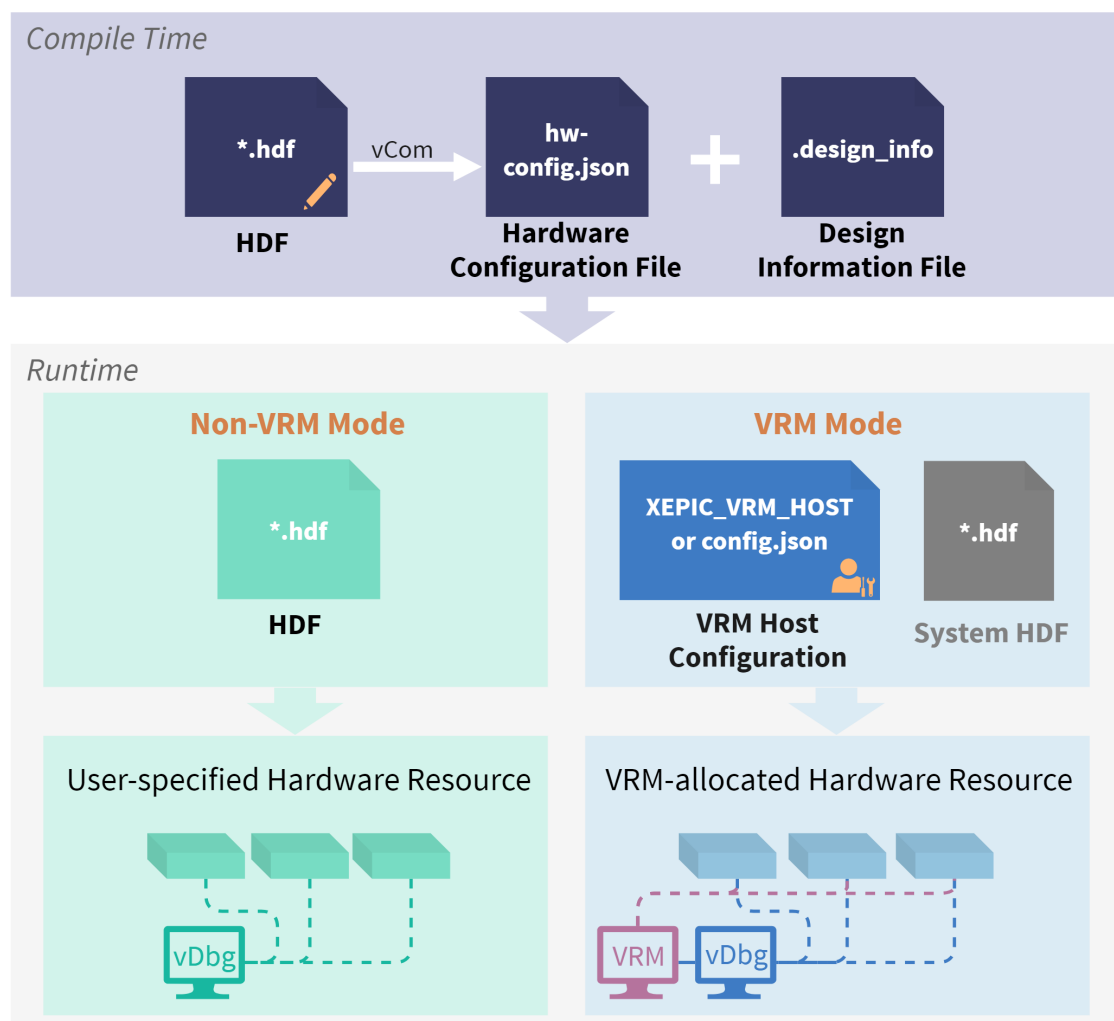


Figure 7-4: Configuration Files for System Connection

HDF

HDF describes the hardware resource and connection requirements for the DUT. You need to

load it for compilation. vCom compiles the design according to the requirements in the HDF. For more information about the syntax of HDF, see [Hardware Description File](#).

Hardware Configuration File

The hardware configuration file describes the hardware resource and connection requirements for the compiled design. vCom generates it as *hw-config.json* in the compile-time working directory. It is a mandatory input for vDbg to build system connection in both the non-VRM mode and the VRM mode.

Design Information File

The design information file contains information about the design. vCom generates it as *.design_info*, which is a hidden file in the compile-time working directory. It is a mandatory input for vDbg to build system connection in both the non-VRM mode and the VRM mode.

VRM-host Configuration

When vDbg works in the VRM mode, VRM-host configuration must be set up before vDbg starts the hardware server. The VRM-host configuration specifies the IP addresses and ports of the default VRM host. vDbg sends a resource request to the VRM host. If the request succeeds, vDbg starts the hardware server and connects to hardware that VRM allocates or you have locked.

Usually, the system administrator sets up the VRM-host configuration for all users. To override the setting, you can manage your VRM host list by using `vrnctl` or setting an environment variable referring to [VRM Host Configuration](#).

System HDF

The system HDF describes the actual hardware resource and connection of the hardware system. You can dump the system HDF by using `vrnctl` referring to [Dumping the System HDF](#).

7.6.4.3 Using Locked Resources

In the VRM mode, if you do not specify any locking key, you use the resources that VRM allocates. The resources are released after you execute `exit` or `hw_server -release` in vDbg.

You can use the resources that you have locked by specifying a locking key or file in vDbg. For more information about how to use `vrnctl` to create a locking key or file, see [Resource Locking](#).

vDbg supports three ways of using the locked resources:

- Use the `-key` option on the `hw_server` command line to specify the locking key value directly.

Syntax:

```
hw_server <hostname> -key <key_value>
```

Example:

```
vdbg> hw_server . -key locking-key1
```

- Use the `-keyfile` option on the `hw_server` command line to specify the locking key file.

Syntax:

```
hw_server <hostname> -keyfile <file_path>
```

Example:

```
vdbg> hw_server . -keyfile .top_module.lkey
```

- Set the `XPVRM_RESOURCE_KEY` environmental variable as the locking key value. The environmental variable setting has the highest priority. For example,

```
% setenv XPVRM_RESOURCE_KEY locking_key2
vdbg> hw_server .
```

If you use locked resources, VRM releases the resources depending on the locking mode that you choose when locking resources.

- **Single Session Mode**

VRM releases the locked resources when you succeed to apply for the resources by using the `hw_server` command.

- **Multi Session Mode**

VRM releases the locked resources until the specified timeout expires even if you disconnect vDbg from SimCtl or exit vDbg.

Examples:

The following `vrnctl` and `vdbg` command lines directly run a design by using the resources that VRM allocates. VRM releases the resources after you execute `exit`.

```

vrctl>> add_vrm_host("10.10.70.70", 9000)
vrctl>> set_default_vrm_host("10.10.70.70", 9000)
vdbg> set_vrm -on
vdbg> design .
vdbg> hw_server .
vdbg> download
vdbg> run 1000rclk
vdbg> exit

```

The following `vrctl` and `vdbg` command lines creates a locking key and run a design by using the resources that VRM locks. The resources are locked in the multi session mode, so the locking key is still valid even if you execute `exit`. VRM releases the resources after two hours.

```

vrctl>> add_vrm_host("10.10.70.70", 9000)
vrctl>> set_default_vrm_host("10.10.70.70", 9000))
vrctl>> lock_resource("./", timeout = "2h", savekey = 'true', mode = "single-
session")
vdbg> set_vrm -on
vdbg> design .
vdbg> hw_server . -keyfile .top_module.lkey
vdbg> download
vdbg> run 1000rclk
vdbg> exit

```

7.6.5 Setting the PHC Voltages

You can set and get the voltages of PHC connectors as follows:

- To set the bank voltage, use the `set_phc_vol` command. The `set_phc_vol` command must be executed before downloading the bitstream files. Otherwise, the voltage settings are ignored.

IMPORTANT

Set PHC voltages with caution because inappropriate operations may cause damage to daughter boards.

```

set_phc_vol -id <board_index>.<FPGA_id> -bank <PHC_id> -voltage
<voltage_number>

```

- You can get the current bank voltage by using the `get_phc_vol` command.

```

get_phc_vol -id <board_index>.<FPGA_id> -bank <PHC_id>

```


 TIP

- Get the installation location of daughter boards or external devices before you set the voltages.
- Daughter boards provided by XEPIC may have different voltage requirements. For information about daughter boards, see the daughter-board-specific reference manual or contact XEPIC for support.

Each FPGA in HuaPro P1 has 24 banks that are connected to PHC connectors. Valid voltages of HR banks are different from those of HP banks. The following table lists the valid voltage settings for the PHC connectors on P1.

Table 7-4: Valid Voltages of P1 PHC Connectors

Bank	P1 PHC Connector	Valid Voltage (V)
HR	17	1.0 / 1.2 / 1.35 / 1.5 / 1.8 / 2.5 / 3.3
HP	1-16, 18-24	1.0 / 1.2 / 1.35 / 1.5 / 1.8

Each user FPGA in HuaPro P2E has 36 banks that are connected to PHC connectors. Valid voltages of HD banks are different from those of HP banks. The following table lists the valid voltage settings for the PHC connectors on P2E.

Table 7-5: Valid Voltages of P2E PHC Connectors

Bank	P2E PHC Connector	Valid Voltage (V)
HD	1, 36	1.2 / 1.35 / 1.5 / 1.8 / 2.5 / 3.3
HP	2-35	1.0 / 1.2 / 1.35 / 1.5 / 1.8

Example:

The following command lines set the voltages of five PHC connectors and check the actual voltages after downloading.

```
vdbg> set_phc_vol -id 0.0 -bank 1 -voltage 1.2
vdbg> set_phc_vol -id 0.0 -bank 2,3,4,5 -voltage 1.2
vdbg> download
[info] 11043 Download successfully! Design is ready to run.
vdbg> get_phc_vol -id 0.0 -bank 1,2,3,4,5

bank_id:  voltage(mV):
1         1234
2         1210
3         1213
```

4	1221
5	1217

7.6.6 Programming the FPGAs

After you load the design and connect to hardware successfully, you can download the bitstream files that are generated in the *fpgaCompDir* directory. vDbg downloads the bitstream files to the hardware in parallel and to the on-board FPGAs sequentially.

To download the bitstream files into the connected hardware, use the `download` command without any option.

7.7 Running the Design for Emulation

After you download the bitstream files to the hardware, you can run the design on the hardware using vDbg.

Prerequisites

Make sure that you download the bitstream files to the hardware by referring to [Downloading the Design to the Hardware](#).

Procedure

Perform the following steps to run the design on hardware in emulation mode:

1. (Optional) If required, configure the design running before you start the user clocks.
 - Change the divider count number which affects the user clock frequency. See [Setting the Divider Count Number](#).
 - Change the default time unit for runtime. See [Setting the Default Time Unit](#).
 - Initialize signals and memories. See [Accessing Signal and Memory Values](#).
 - Set probes. See [Debugging Designs with Probes](#).
 - Set triggers. See [Debugging Designs with Triggers](#).
2. Use the `run` command to start the user clocks. See [Starting the Clock](#).
3. (Optional) You can get the current running time of the design with the `get_time` command. See [Getting Current Cycle Number or Time](#).

Design running stops automatically if the running time or number of cycles set on the `run` command line exceeds or a trigger happens.

4. (Optional) To stop the running of design manually, use the `stop` command. See [Stopping the Clock](#).

7.7.1 Starting the Clock

In the emulation mode, you can use the `run` command to start the run of design in any one of the following four ways:

- Run the design in the blocking mode and specify the maximum running time. You can use either clock cycles or the length of time.

```
run <number_of_cycle> rclk
run <time_value> [<time_unit>]
```

`<time_unit>` specifies the time unit for the number. Valid values for the time unit are `fs`, `ps`, `ns`, `us`, `ms`, and `s`. If you do not specify the time unit, vDbg use the time unit that is set by the `set_time_unit` command. For more information, see [Setting the Default Time Unit](#).

- Run the design in the blocking mode without specifying the maximum running time. The design will stop at 2^{48} rclk if no trigger event occurs, or you do not manually stop the clocks.

```
run
```

- Run the design in the non-blocking mode (no wait mode). In the non-blocking mode, the vDbg prompt is not frozen, so you can execute vDbg Tcl commands during the running.

```
run -nowait
```

- Run the design in the non-blocking mode but selectively block out command execution for a period of time.

```
run -nowait
after_blocking [<time_value>] [<time_unit>]
```

Valid time units are `ms` and `s`. The default time unit is `s` if you only specify the time value. If no timeout is specified, the command execution is blocked out indefinitely.

This way is useful when you do not want Tcl scripts that follow the `run -nowait` command line to be executed immediately after the start of run. The blocking enabled by the `after_blocking` command stops in the following conditions:

- A trigger event occurs.
- The specified timeout is reached.
- You enter the Ctrl+C key combination on the keyboard.

The following example runs the design in the non-blocking mode and executes a command to enable a trigger instance after one second.

```
% run -nowait
% after_blocking 1s
% trigger -at -enable inst1
```



NOTE

- If the clock is already running when you use the `run` command, vDbg only start the data capturing.
- vDbg converts your specified time to run on the `run` command into an equivalent number of rclk according the clock frequency. The specified blocking timeout on the `after_block` command is not equivalent to the time that the design runs.

7.7.2 Stopping the Clock

The user clock will be stopped in the following conditions:

- The debug instrumentation reaches the number of cycles or time that are specified for running.
- The debug instrumentation reaches the trigger point.
- You use the `stop` command to stop the user clock manually. You can use the `stop` command when the debug instrumentation does not reach the trigger point or the number of cycles that are specified to run.
- The `$stop` system task is triggered.

7.7.3 Getting Current Cycle Number or Time

To get the run time of design since the beginning of the `run` in either of the following ways:

- Get the current cycle number.

```
get_time [rclk]
```

- Get the current running time.

```
get_time <timeunit>
```

Valid values for the time unit are `fs`, `ps`, `ns`, `us`, `ms`, and `s`.

Example:

The following example shows command lines to run the design in no wait mode and get the

current running time and specific signal value during running.

```
% run -nowait
% get_time ns
62575ns
% get_value sig_a
'b0
% get_time ns
93582ns
% get_value sig_a
'b0
% stop
```

7.7.4 Setting the Divider Count Number

Divider count number defines the multiple relationship between the primary clock and the root clock. The root clock determines the actual frequency of the fastest user clock on the hardware.

vCom sets a default divider count number according to the STA results. You can check the current divider count number that the design uses by using the `get_div` command without any option.

vDbg uses the default divider count number. You can also change the divider count number as required by using the `set_div` command after bitstream download finishes.

```
set_div <div_number>
```

The number can be any integer that is equal to or more than the divider count number that vCom sets.



NOTE

- Use the `set_div` command with caution because the settings of divider count number can result in DUT function errors.
 - If the divider count number is set to a number greater than the default one, the DUT function might be incorrect. Execute `reset` after changing the divider count number to check the function correctness.
 - If the divider count number is set to a number smaller than the default one, design function errors might occur because of overclocking.
- Each time you change the divider count number at runtime, the DUT needs to be reset.

Example:

The following example sets the divider number, which is 4 by default, to 8.

```
vdbg> download
[info] 11043 Download successfully! Design is ready to run.
vdbg> get_div
4
vdbg> set_div 8
vdbg> get_div
8
```

7.7.5 Setting the Default Time Unit

If you do not specify the `<time_unit>` option for the `run` command, vDbg uses a default time unit. You can run `set_time_unit` to change the default time unit. If you do not use the `set_time_unit` command, the time unit is ns by default.

Syntax:

```
set_time_unit [<value>]
```

Where `<value>` specifies the time unit. Valid time units are fs , ps , ns , us , ms , and s . If you run `set_time_unit` without the `<value>` parameter, vDbg prints the current default time unit that you set.

Example:

```
vdbg> set_time_unit
ns
vdbg> set_time_unit ps
ps
vdbg> set_time_unit
ps
```

8 Accessing Signal and Memory Values

This chapter introduces methods of access signal values and design memories and describes the procedures and command usage in the following topics:

- [Signal Name and File Text Formats](#)
- [Accessing Signal Values](#)
- [Accessing Design Memories](#)
- [Dumping Memories without Extra Logic Inserted at Compile Time](#)
- [Using DDR4 Memory Backdoor](#)

8.1 Signal Name and File Text Formats

Obey the following rules to specify signal names and prepare memory text files.

8.1.1 Supported Signal Formats

vDbg supports the following signal formats:

- Bit-blast, such as:
 - `dut.a`
 - `dut.abc[1]`
- Vector or bus signal `bus_signal[x:y]`, such as:
 - `dut.abc[7:0]`
 - `dut.abc`
 - `dut.abc[3:0]`
 - `dut.abc[0:3]`
 - `dut.abc[4:1]`
 - `dut.abc[1:4]`
- Signal group that is defined by the `signal_group` command, such as `s1` which is defined by the following command line:

```
% signal_group -add s1 top.a dut.abc[3:2]
```

8.1.2 Memory File Formats

DUT memory backdoor supports the following formats of memory content:

- `%fread`
- `%raw`

- `%readmemb`
- `%readmemh`
- `%b` , `%o` , `%h`

DDR4 memory backdoor supports only `%fread` and `%readmemh` formats.

`%fread` Format

This format is consistent with Verilog standard `$fread` format. You can use the `-format %fread` option on the `memory` command line to read memory data from a binary file or write memory data to a binary file.

`%raw` Format

The raw format is a binary file format. You can write your own C/C++ program to read the raw format file or to generate the raw format file. The raw format is a type of the left-align file format. You can specify the raw format of the memory file using `-format %raw` on the `memory` command line.

The following table describes the raw format.

Table 8-1: Left-align Raw Format

Offset	Description	Value
0-7	Signature	QTMEM\0\0\0
8-9	Version	0x0100
10-17	Reserved	
18-21	Depth of the memory instance	Depends on the specific memory instance.
22-25	Width of the memory in number of bits	Depends on the specific memory instance.
26-29	Number of bytes for each memory word. Calculated as $(\text{memory_width}+7)/8$	Depends on the specific memory instance.
30-33	Start address	Depends on the specific memory instance.
34-37	End address	Depends on the specific memory instance.
38-41	Length of the memory instance name	Depends on the specific memory instance.
42-xx	Memory instance name	Depends on the specific memory instance.

Offset	Description	Value
yy-zz	Memory data (left justified)	Depends on the specific memory instance.

%readmemb Format

You can use the memory content format which is supported Verilog system task `$readmemb` by using `-format %readmemb` on the `memory` command line. Note that the content to be loaded and dumped must adhere to the following rules:

- Each number must be binary. You can use the unknown value (x or X), the high-impedance value (z or Z), and the underscore (_) to specify a number.
- Use white spaces or comments, or a combination of them to separate the numbers.
 - White spaces, including spaces, newlines, tabs, and form feeds.
 - Comments, including one-line comments and block comment.
- Only data values can be included in the memory file. Data addresses can be specified with the `-start` and `-end` options on the `memory` command line. If the command line does not contain the start address, vDbg uses the 0 address by default.

Example:

The following two text files are equivalent. Both of the text files can be written into a DUT memory such as `reg [3:0] mem[0:7]`.

```
1111
0011
1100
1011
0000
1110
0101
1010
```

```
1111 0011 1100 1011 0000 1110 0101 1010
```

%readmemh Format

You can use memory content format which is supported by Verilog system task `$readmemh` by using `-format %readmemh` on the `memory` command line. The content to be loaded and dumped must adhere to the following rules:

- Each number should be hexadecimal. You can use the unknown value (x or X), the high-impedance value (z or Z), and the underscore (_) to specify a number.

- For DUT memory backdoor, adhere to the following additional rules:
 - Use white spaces or comments, or a combination of them to separate the numbers.
 - White spaces, including spaces, newlines, tabs, and form feeds.
 - Comments, including one-line comments and block comment.
 - Only data values can be included in the memory file. Data addresses can be specified with the `-start` and `-end` options on the `memory` command line. If the command line does not contain the start address, vDbg uses the 0 address by default.

For example, the following text file can be written into a DUT memory such as `reg [3:0]`
`mem[0:7]` .

```
f 3 c b 0 e 5 a
```

- For DDR4 memory backdoor, adhere to the following additional rules:
 - A line can contain either one data address or one data value. That is, use tabs to separate the numbers.
 - Lines beginning with an `@` sign are considered as address lines. The addresses can be hexadecimal values only.
 - The number of hexadecimal digits in a data line must be two to represent one byte, for example, `AB` .
 - The data values must always follow the start address that they relate to. A data write starts from the first data value and ends at the last data value related to the start address.
 - You can specify the start address either in the file or with the `-start` option. If neither the file nor the command line contains the start address, vDbg uses the 0 address by default. If both the file and command line contain the start address, the address specified in the file takes effect.
 - When you use DDR4 memory write, the `-end` option is optional. If the `-end` option is not specified, vDbg calculates the end address according to the data size in the specified file.

The following is an example of a memory file in the supported `%readmemh` format which can be backdoor written into the DDR4 memory:

```
@0
AB
0a
@0x234
CD
12
56
```

%b , %o , %h Formats

You can specify the memory file formats as binary, octal, and hexadecimal using `-format %b` , `-format %o` , and `-format %h` on the `memory` command line, respectively. Files in these formats must adhere to the following rules:

- Specify the memory name, format, start address, and end address at the beginning of the file.
- Specify the data according to the rules of the `%readmemb` and `%readmemh` formats.

The syntax of the content is as follows:

```
Memory name: <memory_name>
Format: %b|%o|%h
Start address: <start_address>
End address: <end_address>
<data>
```

Here is an example of text file in binary radix format that can be written into a DUT memory:

```
Memory name: dut.mem
Format: %b
Start address: 0x128
End address: 0x130
0010
1100
0101
1110
xxxx
xxxx
xxxx
xxxx
```

8.2 Accessing Signal Values

This topic introduces how to control and get signal values. Signal values can be obtained without any compile-time preset or by signal backdoor read.

8.2.1 Forcing and Releasing the Signal Values

You can set the signal value and force the value until you release the signal value using the `release` command. To set a given value to a specified signal, use the following command line:

```
force <signal> <value>
```

All accessible signals must be specified by the `write_net` command at compile time. For more information about the command usage, see [Adding Nets for Signal Access](#).

To release a signal or all signals that are previously forced by using the `force` command, use the `release` command. vDbg releases the value of the specified signal in the next cycle and then the design can drive the signal.

```
release <signal>|-all
```

8.2.2 Depositing the Signal Values

To set a signal to a specified value for one clock cycle, use the `deposit` command line. The signal value is hold for one clock cycle.

```
deposit <signal> <value>
```

All accessible signals must be specified by the `write_net` command at compile time. For more information about the command usage, see [Adding Nets for Signal Access](#).

8.2.3 Getting the Signal Values

Getting signal values is one of the common debugging operations for emulation. Signals that can be read at runtime depend on your instrumentation at compile time. By default, you can directly get the values of the top-level output ports in the RTL and all registers without any extra logic added during compilation.

If you want to get the value of specific signals at runtime, you can use the signal backdoor read feature. By manually defining read nets at compile time (see [Adding Nets for Signal Access](#)), you can backdoor read signals at runtime. HPE provides the capability of defining all types of signals as read nets and getting their values at runtime in the emulation mode. You can define a limited number of read nets because of the limitation of FPGA resources. Up to 16k-bit read nets are supported for each FPGA.

Note that if you get values of a bus when clocks are running in the emulation mode, the time coherency between the values of signals is not guaranteed. If you get values of single-bit nets when clocks are running, except for the state device outputs and read nets, the returned values might be incorrect.

Use the `get_value` command to get values of signals at runtime in vDbg.

```
get_value [<value_format>] <signal>
```

Valid `<value_format>` are `%b`, `%o`, `%d`, and `%h`. If the value format is not specified, binary

format is used.

Examples:

- The following example gets the value of signal `cnt1` in the default binary format.

```
.....  
vdbg> get_value cnt1  
'b000000010
```

- Either of the following command lines can get the value of signal `cnt1` in the specified decimal format.

```
.....  
vdbg> get_value %d cnt1  
'd2
```

Or

```
.....  
vdbg> get_value cnt1 %d  
'd2
```

- The following example gets the value of partial signal bits of signal `cnt1` in the default binary format.

```
.....  
vdbg> get_value {cnt1[4:2]}  
'b000
```

8.2.4 Controlling Signal Values by Group

vDbg supports group operations on a collection of signals at a time. You can use the function to improve efficiency and to simplify the input runtime Tcl scripts.

TIP

Currently, you can control signal values by group in the emulation mode only.

Add signals to a group and use the signal group in the following steps.

1. Define a signal group and add signals into the group with the `signal_group` command and the `-add` option (see [Defining a Signal Group](#)).

2. Control the values of the signal groups using the `force`, `deposit`, `release`, and `get_value` commands (see [Controlling the Values of Signal Groups](#)).
3. Manage the signal groups with the `signal_group` command and the `-list` and `-rm` options.
 - `-list` gets signal names in a specific group or groups (see [Listing Signal Names in a Group](#)).
 - `-rm` removes the defined signal groups (see [Removing a Signal Group](#)).

For more information about an example, see [Example for Using Signal Groups](#).

8.2.4.1 Defining a Signal Group

To define a signal group and add signals to the group, use the `signal_group` command line:

```
signal_group -add <signal_group> <signal1> <signal2> ...
```

Signals on the `signal_group` command line are processed in the left-to-right order.

- You can add any signals in the design into a group.
- The bit widths of signals in a group can be different. For example, `sig[4:3]` and `sig[2:0]` can be defined in the same signal group.
- You can include interval overlaps of signals in a signal group. For example, `sig[10:0]` and `sig[15:5]` can be defined in the same signal group.

8.2.4.2 Controlling the Values of Signal Groups

You can control the values of signal groups like one-bit signals and bus signals by specifying a signal group with the `-g` or `-group` option on the `force`, `deposit`, `release`, and `get_value` command lines.

You can get the values of the signal groups containing signals that defined as read nets or not. You can force, deposit, and release signal groups containing only signals that defined as read nets.

Forcing a Signal Group

Set the signal value of a signal group and force the value until you release the value.

```
force -g|-group <signal_group> <value>
```

Depositing a Signal Group

Set a signal group to a specified value for one clock cycle.

```
deposit -g|-group <signal_group> <value>
```

Releasing a Signal Group

Release the signal value of a signal group that is previously forced by using the `force` command.

```
release -g|-group <signal_group>  
release -all
```

Getting the Value of a Signal Group

Get the current value of a signal group using the `get_value` command with the `-g` or `-group` option. vDbg treats the signal group as a bus signal when you get the value of the signal group.

```
get_value [<value_format>] -g|-group <signal_group>
```

Valid formats are `%b` (default), `%o`, and `%h`. `%d` is not supported.

When you force or deposit the value of a signal group, vDbg set your specified value for each of the signals that are added in the signal group. For example, you add `sig1[0]`, `sig2[0]`, and `sig3[0]` into a group. Forcing the group value to `'b1` is equivalent to forcing the values of the three signals to `'b1` individually.

Usually, you add signals with same bit widths in a group and force a value with matched bit width. When signal controls conflict, vDbg obeys the following rules:

- If the bit width of your specified value is greater than that of the signals in a group, vDbg omits the overflowing high bits of the value and uses the remaining bits.
- If the bit width of your specified value is smaller than that of the signals in a group, vDbg pads out the value with `0` from the highest bit.
- If the signals in the same group have interval overlaps and also different bit widths, the value of latter signal bits on the `signal_group` command line is effective.

Example 1

In this example, the signal group `s1` includes three parts of the signal `sig`. The three parts are all 2-bits signals. `sig[2:3]` has different bit order with the other signals. The value of signal `sig` is `'b000000` before forcing.

```

.....
vdbg> get_value sig
'b000000
vdbg> signal_group -add s1 {sig[1:0]} {sig[2:3]} {sig[5:4]}
[info] 12205 Create signal group : s1 succeeded.

```

If the forced value is 'b01 , the bit width of the forced value matches that of the signals in group s1 . Therefore, s1 is 'b010101 and sig is 'b011001 after forcing.

```

.....
vdbg> force -g s1 'b01
vdbg> get_value -g s1
'b010101
vdbg> get_value sig
'b011001

```

If the forced value is 'b101 , the bit width of the forced value is larger than that of the signals in group s1 . vDbg omits the overflowing high bits 1 . The actual execution results are the same with 'b01 .

If the forced value is 'b1 , the bit width of the forced value is smaller than that of the signal group s1 . vDbg pads out the forced value with 0 from the highest bit. The actual execution results are also the same with 'b01 .

Example 2

In this example, the specified signal parts have interval overlaps of sig[2] and sig[1] and have different bit widths. The signal sig is 'b0000 before forcing.

```

.....
vdbg> get_value sig
'b0000
vdbg> signal_group -add s1 {sig[3:2]} {sig[2:0]} {sig[1]}
[info] 12205 Create signal group : s1 succeeded.

```

vDbg performs the following operations when you force the group value to 'b01 .

- Force sig[3:2] to 'b01 .
- Pad out the forced value with 0 from the highest bit and then force sig[2:0] to 'b001 , which overwrites sig[2] .
- Omit the overflowing high bits 0 and then force sig[1] to 'b1 , which overwrites sig[1] .

Therefore, s1 is 'b0011 and sig is 'b000111 after forcing.

8.2.4.3 Listing Signal Names in a Group

To list the signal names in a signal group, use the following command line:

```
signal_group -list [<signal_group>|<reg_exp>]
```

Where,

- `-list <signal_group>` specifies an exact signal group name.
- `-list <reg_exp>` specifies the regular expression. Only the matched signal groups are listed.

If you specify only the `-list` option without the signal group name or the regular expression, all signal group names with corresponding signal names are listed.

Examples

Take the following signal group definition for example.

```
% signal_group -add x0 {a[0]} {b[0]} {c[0]}
% signal_group -add x1 {a[1]} {b[1]} {c[1]}
% signal_group -add y0 {a[2]} {b[2]} {c[2]}
% signal_group -add z1 {a[3]} {b[3]} {c[3]}
```

- The following command line prints out the signals in all signal groups:

```
.....
vdbg> signal_group -list
signal_group: x0: {a[0]} {b[0]} {c[0]}
signal_group: x1: {a[1]} {b[1]} {c[1]}
signal_group: y0: {a[2]} {b[2]} {c[2]}
signal_group: z1: {a[3]} {b[3]} {c[3]}
```

- The following command line prints out the signals in the signal group `x0`:

```
.....
vdbg> signal_group -list x0
signal_group: x0: a[0] b[0] c[0]
```

- The following command line prints out the signal group that match the specified regular expression, `[x|y]`, with corresponding signal names:

```
.....
vdbg> signal_group -list {[x|y].}
signal group: x0: a[0] b[0] c[0]
signal group: x1: a[1] b[1] c[1]
signal group: y0: a[3] b[3] c[3]
```

8.2.4.4 Removing a Signal Group

To remove the definition of a signal group or a collection of signal groups, use the following command line:

Syntax:

```
signal_group -rm <sig_group_name>|<reg_exp>|-all
```

Where,

- `-rm <sig_group_name>` specifies the defined signal-group to be removed.
- `-rm <reg_exp>` specifies the regular expression. vDbg removes only the matched signal group.
- `-rm -all` removes all signal groups.

8.2.4.5 Adding Signals to a Signal Group

To add a signal or signals to an existing signal group, use the `signal_group` command with the `-append` option.

Syntax:

```
signal_group -append <signal_group> <signal1> <signal2> ...
```

Where,

- `-append <signal_group>` specifies the signal group name. The specified signal group name must exist.
- `<signal1>`, `<signal2>`, and so on specify the names of signals or a part of the signal bits separated by a space. Signals on the command line are processed in the left-to-right order.

Example:

The following example defines a signal group named `g1` and appends the signal `cnt_out[3:0]` to the signal group.

```
vdbg> signal_group -add g1 {data_in[3:0]}
[info] 12205 Create signal group : g1 succeeded.
```

```
vdbg> signal_group -append g1 {cnt_out[3:0]}
[info] 14954 Signal cnt_out[3:0] is appended to group g1 sucessfully.

vdbg> signal_group -list
signal group: g1: data_in[3:0] cnt_out[3:0]
```

8.2.4.6 Example for Using Signal Groups

This example shows you how signal groups simplifies the run script file and guides you through creating signal groups and getting values of signal groups.

Assume that the `in` signal has been defined as a write net and the `out1` and `out2` signals as read nets at compile time.

The original run script file is show as follows. To control the values of multiple signals, you need to specify a command line for each signal.

```
design .
hw_server localhost
download

force in[4:0]    'b11111
force in[14:10] 'b11111
force in[24:20] 'b11111

run 100rclk
get_value out1[3:0]
get_value out1[13:10]
get_value out1[23:20]
get_value out2[3:0]
get_value out2[13:10]
get_value out2[23:20]
exit
```

With the signal group feature, you can group signals as required and control the values of the groups at runtime with ease. In the following example, the `s1` group consists of signals that need to be assigned values. The `s2` group consists of signal parts that you want to get values after the design runs 100 rclk.

```

design .
hw_server localhost
download

signal_group -add s1 {in[4:0]} {in[14:10]} {in[24:20]}
signal_group -add s2 {out1[3:0]} {out1[13:10]} {out1[23:20]} {out2[3:0]}
{out2[13:10]} {out2[23:20]}

force -g s1 'b11111
run 100rclk
get_value %b -g s2
exit

```

8.3 Accessing Design Memories

You can backdoor access the memory values in the following steps:

1. Make sure that the memories that you want to access are specified on the `memory_access` command line at compile time (see [Adding Memories for Memory Access](#)).
2. Download the design to the hardware.
3. If you want to get all the DUT memories or specific memories, use the `memory` command with the `-list` option (see [Listing the DUT Memories](#)).
4. Use the `memory` command with the `-write` option to write memory files into the DUT memories (see [Writing User Data into DUT Memory](#)).

vDbg supports several memory file formats. Refer to [Memory File Formats](#) for more information about the valid file formats.

5. Run the design.
6. Use the `memory` command with the `-read` option to export the DUT memories to memory files (see [Reading Data from DUT Memory](#)).

8.3.1 Writing User Data into DUT Memory

To write the user data in a specified file into the DUT memory, use the following command line. Only one `memory` command can be executed at a time. A new command request is blocked until the previous one finishes. In other words, vDbg does not support multiple memory operations to happen simultaneously.

```

memory -write {-format <file_format>} \
              {-name <mem_name>} \
              {-file <filename> | -mode <mode_type>} \
              [-start <start_addr>] [-end <end_addr>]

```

 TIP

If the memory write operation is blocked for a long time without any timeout response, you can use Ctrl+C to abort the operation.

8.3.2 Reading Data from DUT Memory

To save the data in the DUT memory into a specified file, use the following command line.

```
memory -read {-format <file_format>} {-name <mem_name>} {-file <filename>} [-start <start_addr>] [-end <end_addr>]
```

Only one `memory` command can be executed at a time. A new command request is blocked until the previous one finishes. In other words, vDbg does not support multiple memory operations to happen simultaneously.

 TIP

- If the memory reading operation is blocked for a long time without any timeout response, Ctrl+C can be used to abort the operation.
- Only use the memory backdoor reading function at runtime if necessary because the function might affect performance and routability of a design.

8.3.3 Listing the DUT Memories

To print out all or a collection of the DUT memories, use the `-list` option on the `memory` command line:

- List all DUT memories

```
memory -list
```

- List the DUT memories whose name match the specified pattern.

```
memory -list -pattern <pattern_exp>
```

8.3.4 Accessing High-capacity Memories

You can read and write HCMs in the following steps:

1. Install DDR4 daughter boards and configure the card type `HCM_16G_V1` in [Hardware Description File](#).
2. (Optional) By default, memories over 1GB are considered as HCMs. If required, you can specify memories as HCMs during synthesis, compilation, or both.
 - In vSyn stage, you can attach the `xpvs_use_xdram` attribute in RTL code or use the `-hcm <N>` option. For details, see [Specifying High-capacity Memories in vSyn](#).
 - In vCom stage, you can use the `memory_options` command. For details, see [Specifying High-capacity Memories in vCom](#).
3. Run the design with vDbg and use the `memory` command to access the HCMs.

Syntax:

```
memory -read [-format {<file_format>}] \
  -name <mem_name> \
  {-file <file_name> | -mode {all0|all1}} \
  [-start <start_addr>] [-end <end_addr>]

memory -write -format {<file_format>} \
  -name <mem_name> \
  -file <file_name> \
  [-start <start_addr>] [-end <end_addr>]
```

Where `<file_format>` supports `%fread`, `%readmemb`, and `%readmemh`. For details, see [Memory File Formats](#). It is recommended to use the `%fread` and `%readmemh` memory file formats because the memory file of the `%readmemb` format is large.

8.4 Dumping Memories without Extra Logic Inserted at Compile Time

In addition to [Reading Data from DUT Memory](#), you can also dump memory data by using the `dump_mem` command. The advantage of `dump_mem` is that it does not need any instrumentation logic during compilation. You have the visibility to LUTM and BRAM memories in the design. URAM memories are not supported to be dumped.

```
dump_mem [-format <value_format>] -name <memory_name> -file <file_name> [-start <mem_addr>] [-end <mem_addr>]
```

8.5 Using DDR4 Memory Backdoor

HuaPro P2E supports DDR4 memory backdoor in the non-VRM mode of vDbg. This topic describes how to backdoor access DDR4 memory at runtime.

Prerequisites

Make sure that you have completed the following compile-time tasks before using DDR4 memory backdoor.

- Install DDR4 daughter boards on HuaPro P2E.
- Define netlist macros in vCom compilation.
 - If you use the simplified netlist macro flow, define the netlist macro instructed in [Using Netlist Macros in the Simplified Flow](#).
 - If you use the standard netlist macro flow, define the netlist macro as follows:
 1. Setting netlist macros as instructed in [Using Netlist Macros in the Standard Flow](#).
 2. [Binding Netlist Macro Ports to AXI4 Interface](#).

Procedure

Perform the following runtime steps to write data to and read data from the DDR4 memory.

1. Open the design using the `design` command.
2. Set the frequencies of global clocks using the `set_clock` command.
 - a. Use the default settings or set the frequencies of the global clocks that the DUT uses.
 - b. Set the frequencies of the global clocks that the DDR4 module uses as follows.
 - In the simplified netlist macro flow, you do not need to configure the clock of the DDR4 module.
 - In the standard netlist macro flow, you must set the frequency of the global clock GC2 to 100MHz by using the `set_clock -clock 100 MHz` command line.
3. Connect vDbg to hardware without VRM using the `hw_server` command.
4. Set the voltages of the PHC connectors that DDR4 daughter boards use to 1.2 V using the `set_phc_vol` command.

The following example sets the voltages for a DDR4 daughter board installed in bank 3-5 of FPGA 0.A:

```
% set_phc_vol -id 0.A -bank 3,4,5 -voltage 1.2
```

5. Download the design to FPGAs using the `download` command.
6. Run the design by using the `run` command.
7. Write data into the DDR4 memory using the `memory` command and the `-write` option.

NOTE

Make sure that both frontdoor memory write and read are stopped when a backdoor memory write or read starts.

See [Writing Data into DDR4 Memory](#) for the command usage of `memory`.

8. Read data from the DDR4 memory using the `memory` command and the `-read` option.

See [Reading Data from DDR4 Memory](#) for the command usage of `memory`.

Example[Examples for Using DDR4 Memory Backdoor](#)**8.5.1 Writing Data into DDR4 Memory**

You can use the `memory` command to backdoor write data into external DDR4 memories in either of the following two ways:

- Specify a memory file in the `%fread` or `%readmemh` format. For more information about the memory file, see [Memory File Formats](#).

```
memory -write -fpga <fpga_id> -channel <channel_index> -file
<ddr4_mem_filename> [-format <ddr4_mem_file_format>] [-start <start_addr>]
[-end <end_addr>]
```

The `-channel` option specifies the location of DDR4 daughter board for DDR4 memory backdoor. If the content of the specified memory file is the `%fread` binary data, you must add the `-format %fread` option.

Examples:

```
% memory -write -fpga 0.A -channel 0 -file init.mem
```

```
% memory -write -fpga 0.A -channel 0 -file init.mem -format $fread
```

- Clear the entire memory with `0` or `1`.

```
memory -write -fpga <fpga_id> -channel <channel_index> -mode all0|all1 [-
format <ddr4_mem_file_format>] [-start <start_addr>] [-end <end_addr>]
```

Example:


```
% memory -write -fpga 0.A -channel 0 -mode all0
```

8.5.2 Reading Data from DDR4 Memory

To save the data in the DDR4 memory into a specified file, use the following command line.

Only one `memory` command can be executed at a time.

```
memory -read -fpga <board_index>.<fpga_index> -channel <channel_index> -file
<ddr4_mem_filename> -start <start_addr> -end <end_addr> [-format
<ddr4_mem_file_format>] [-append]
```

Where,

- `-fpga <fpga_id>` specifies the FPGA where the DDR4 memory locates. The FPGA ID must be in the format of `<board_index>.<fpga_index>`.
 - `<board_index>` specifies the board index.
 - `<fpga_index>` specifies the FPGA index. Valid values are A-D or 0-3.
- `-channel <channel_index>` specifies the location of DDR4 daughter board that is to be used for DDR4 memory backdoor.
- `-file <ddr_mem_filename>` specifies the text file to store the data read from the DDR4 memory. For more information about the text format, see [Memory File Formats](#).
- `-start <start_addr>` specifies the start address of the accessible memory. The start address must be in hex format, such as 0xa1f. If you do not specify the start address, vDbg uses the default start address of memory (Instance Start).

The start address and the end address must not cross the board of the memory. The start address must be less than or equal to the end address.

- `-end <end_addr>` specifies the end address of the accessible memory. Same with the start address, the end address must be in hex format. If you do not specify the end address, vDbg uses the default end address of memory (Instance End).
- `-format <ddr4_mem_file_format>` specifies the format of the file. The valid value of `<ddr4_mem_file_format>` is `%fread` currently. This option is only required when the content is the `$fread` binary data.
- `-append` appends the dumped data to the end of the content in the memory file. If this option is not set, the new data overwrites the old data.

8.5.3 Examples for Using DDR4 Memory Backdoor

This section describes two examples to show how to use DDR4 memory backdoor on HuaPro

P2E in the simplified netlist macro flow and standard netlist macro flow.

8.5.3.1 Using DDR4 Memory Backdoor in Simplified Netlist Macro Flow

The following example shows the Tcl scripts that are used for DDR4 memory backdoor in the simplified netlist macro flow. In addition to the DDR4 netlist macro, this example also uses the PCIe netlist macro.

1. Define the data interface for the netlist macro in a wrapper and instantiate the module in the DUT top design. Then synthesize the DUT and the wrapper using vSyn or the third-party synthesizer. In this example, the third-party synthesizer is used.
2. Get the netlist macro package corresponding to the DDR4 daughter board you use.

You can find the netlist macro package from HPE installation path. In this example, the netlist macro package is the *xepic_ddr4_dc1* directory, which corresponds to the DDR4 daughter board installed on Bank 3,4,5.

3. Run compilation with vCom.

```
% vcom vcom_compile.tcl
```

The vCom compilation scripts are shown as follows. As indicated in the scripts, the `pcie3_ddr4.chip0_wrapper` instance is defined as a netlist macro, the DDR4 daughter board is located at FPGA 0.A, and backdoor channel 0 is enabled. You can add scripts to define ISA nets or assign terminals as you require.

```

1  set cur_path [pwd]
2  set nrmf_path "${env(HPE_HOME)}/netlist_macro_packages"
3
4  ## define netlist macros
5
6  design_read -netlist ${cur_path}/rev_1/pcie3_ddr4.vm
7
8  netlistmacro -instance pcie3_ddr4.chip0_wrapper -package ${nrmf_path}/xepic_ddr4_dc1 -location 0.A -channel 0
9  netlistmacro -instance pcie3_ddr4.ep_wrapper -package ${nrmf_path}/xepic_pcie_ep
10
11 design_load -top pcie3_ddr4
12
13 emulator_spec -add "file hw-config.hdf"
14 emulator_util -add {default 70}
15
16 ## define clocks
17 create_clock -sig_name clock1 pcie3_ddr4.user_clk -frequency 10Mhz
18
19 design_edit
20 design_generation

```

Annotations in the image:

- Line 8: **NM package provided in HPE installation package** (points to `-${nrmf_path}/xepic_ddr4_dc1`)
- Line 8: **The FPGA where the DDR4 locates** (points to `-location 0.A`)
- Line 8: **Enable DDR4 backdoor channel** (points to `-channel 0`)
- Line 17: **Define user clock** (points to `create_clock`)

Figure 8-1: Example of vCom compilation Scripts

4. Run PnR.

```
% cd fpgaCompDir && make all
```

5. Run the design on HuaPro P2E with vDbg.

```
% vdbg debug.tcl
```

The vDbg runtime scripts are shown as follows. Note that only in the simplified netlist macro flow, you can backdoor read and write DDR4 memory before design running.

```

1 design .
2
3 ## set clock
4 set_clock -clock 100 Mhz
5
6 hw_server . Set DDR4 voltage
7 set_phc_vol -id 0.0 -bank 3,4,5 -voltage 1.2
8
9
10 download
11 run
12
13 ## backdoor Backdoor access DDR4 memory
14 memory -write -fpga 0.A -channel 0 -file channel1.ref
15 memory -read -fpga 0.A -channel 0 -file channel1.result -start 5 -end 10000003
16
17 exit

```

Figure 8-2: Example of vDbg Runtime Scripts

8.5.3.2 Using DDR4 Memory Backdoor in Standard Netlist Macro Flow

The following example shows the Tcl scripts that are used for DDR4 memory backdoor in the standard netlist macro flow. In addition to the DDR4 netlist macro, this example also uses the PCIe netlist macro.

1. Prepare the wrapper files and netlist files for netlist macros. Then synthesize the DUT and the wrapper using vSyn or the third-party synthesizer. In this example, the third-party synthesizer is used.
2. Run compilation with vCom.

```
% vcom vcom_compile.tcl
```

The vCom compilation scripts are shown as follows:

```

1 set cur_path [pwd]
2 puts "cur_path: ${cur_path}"
3
4 design_read -netlist ${cur_path}/bbox/pcie3_ep_user_wrapper_stub.sv
5 design_read -netlist ${cur_path}/bbox/ddr4_wrapper_dc1_stub.sv
6 design_read -netlist ${cur_path}/rev_1/pcie3_ddr4.vm
7
8 netlistmacro -add {work pcie3_ep_user_wrapper} -resource {LUT:80 BRAM:8}
9 netlistmacro -add {work ddr4_wrapper_dc1} -resource {LUT:80 BRAM:8}
10
11 netlistmacro_param -module pcie3_ep_user_wrapper -attribute NL_FILE_PATH ${cur_path}/bbox/pcie3_ep_user_wrapper.dcp
12 netlistmacro_param -module pcie3_ep_user_wrapper -attribute XDC_DIR_PATH ${cur_path}/bbox/
13
14 netlistmacro_param -module ddr4_wrapper_dc1 -attribute NL_FILE_PATH ${cur_path}/bbox/ddr4_wrapper_dc1.dcp
15 netlistmacro_param -module ddr4_wrapper_dc1 -attribute XDC_DIR_PATH ${cur_path}/bbox/
16
17 design_load -top pcie3_ddr4
18
19 emulator_spec -add "file hw-config.hdf"
20 emulator_util -add {default 70}
21
22
23 create_clock -sig_name pcie3_ddr4.user_clk -frequency 10Mhz
24 create_generated_nmclock -name pcie_axi_clk -port pcie3_ddr4.pcie3_ep.axi_clk
25 create_generated_nmclock -name ddr_dc1_axi_clk -port pcie3_ddr4.ddr4_chip0.axi_clk
26
27 terminal_keep -add {m_pcie_exp_txn}
28 terminal_keep -add {m_pcie_exp_txp}
29 terminal_keep -add {m_pcie_exp_rxn}
30 terminal_keep -add {m_pcie_exp_rxp}
31 terminal_keep -add {m_sys_clk_n}
32 terminal_keep -add {m_sys_clk_p}
33 terminal_keep -add {m_sys_rst_n}
34 terminal_keep -add {pcie_dir}
35 .....
36 ## hide part of the Tcl scripts
37
38 proc connect_dbg {bbox_port board_id fpga_id channel_id } {
39 set dest_name "$board_id.$fpga_id.$channel_id dut_ddr${channel_id}_calib_complete"
40 set bbox_name "pcie3_ddr4.ddr4_chip${bbox_port}.c0_init_calib_complete"
41 bind_ports -netlist_macro_ports "$bbox_name" -uni_debug_ports "$dest_name"
42 set dest_name "$board_id.$fpga_id.$channel_id dut_axi_s${channel_id}_clk"
43 set bbox_name "pcie3_ddr4.ddr4_chip${bbox_port}.c0_ddr4_ui_clk"
44 bind_ports -netlist_macro_ports "$bbox_name" -uni_debug_ports "$dest_name"
45 set dest_name "$board_id.$fpga_id.$channel_id dut_axi_s${channel_id}_awid[3:0]"
46 set bbox_name "pcie3_ddr4.ddr4_chip${bbox_port}.s1_ddr4_s_axi_awid[3:0]"
47 bind_ports -netlist_macro_ports "$bbox_name" -uni_debug_ports "$dest_name"
48 set dest_name "$board_id.$fpga_id.$channel_id dut_axi_s${channel_id}_awaddr[63:0]"
49 .....
50 ## hide part of the Tcl scripts
51
52 }
53
54 design_edit
55 design_generation

```

Read files of netlist macros

Define netlist macros and set parameters

Define user clocks

Keep pins of netlist macros

Bind interface ports

Figure 8-3: Example of vCom compilation Scripts

3. Run PnR.

```
% cd fpgaCompDir && make all
```

4. Run the design on HuaPro P2E with vDbg.

```
% vdbg debug.tcl
```

The vDbg runtime script file is shown as follows:

```
1 design .
2
3 hw_server .
4 set_clock -gtbank -id 0.A -bank 221 -frequency 100 Mhz
5
6 set_user_clock -clock GC2 100MHZ
7 Set DDR4 voltage
8 set_phc_vol -id 0.0 -bank 3,4,5 -voltage 1.2
9
10 download
11 run -nowait
12 Backdoor access DDR4 memory
13 memory -write -fpga 0.A -channel 1 -file channel1.ref
14 memory -read -fpga 0.A -channel 1 -file channel1.result -end 10000000
15
16 exit
```

Figure 8-4: Example of vDbg Run Scripts

9 Debugging Designs with Probes

Probes are the signals that you are interested in at runtime. vDbg can capture the waveforms of probes or reconstruct waveforms to help debug the design. The capture progress is also called tracing. Depending on your familiarity of the DUT and runtime performance requirements, you can select different types of probes.

The topic introduces the types of probes, covers the command usage of tracing flow, and provides reference information for viewing XEDB waveforms.

For more information about generating waveforms from the GUI, see [Generating Waveforms from the GUI](#).

9.1 Ways of Waveform Generation

HPE supports four ways of waveform generation: physical, streaming, and GRP probe.

- **Physical Probe Mode**

The physical probe mode is useful when you have already identified a range of signals that you are interested in and you care about the runtime performance.

In the physical probe mode, you must specify the signals of interest as trace nets at compile time. Tracing of physical probe uses the external DDR memory daughter boards of P1 devices and internal DDR memories of P2E devices to store trace data.

At runtime, you can upload the content in the DDR with vDbg commands or from GUI. vDbg supports generating VCD, XEDB, and CWDB waveform files for the physical probes.

- **Streaming Probe Mode**

The streaming probe mode is useful when you have already identified the signals that you are interested in and you want to continuously capture the waveforms. It does not affect the operation of peripherals and not occupy DDR memory resources.

In the streaming probe mode, you must specify the signals of interest as trace nets at compile time. HAC probe daughter boards are required for data transmission.

At runtime, the trace data is automatically dumped into the waveform files that are saved in the host PC. vDbg uploads the values of specific signals into the host PC continuously. vDbg supports generating XEDB waveform files for streaming probes.

- **Global Register Probe (GRP) Probe Mode**

You are recommended using GRP probes as a supplement of physical probes, even though the GRP probe mode can be used individually. GRP probes can help you debug some critical signals that you forget to define as trace nets at compile time.

In the GRP probe mode, you do not need to define trace nets or generate any database at compile time, which saves you from re-compilation and time-consuming PnR.

vDbg automatically uploads the values of specific signals into the host PC continuously. vDbg supports generating XEDB waveform files for the GRP probes. GRP probes can negatively affect the runtime performance. Therefore, you are recommended debugging in a small time window. GRP probe mode supports generating waveforms in the XEDB format only.

The following table lists the compile-time and runtime flow of the three types of probes.

Table 9-1: Step References of Probe Flows

Type	Compile-time Flow	Runtime Flow
Physical Probe	Adding Nets for Physical Probes	Generating the Waveform for Physical Probes
Streaming Probe	Adding Nets for Streaming Probes	Generating the Waveform for Streaming Probes
GRP Probe	No operation required	Generating the Waveform for GRP Probes

TIP

All the probe modes are only supported in the emulation mode.

NOTE

To use the physical probe mode on HuaPro P1, there must be memory daughter board specification in the HDF for the FPGA which trace nets are assigned to. To use the streaming probe mode on both HuaPro P1 and P2E, there must be Probe HAC daughter board specification in the HDF for the FPGA which trace nets are assigned to. For example, the driver port of trace net `dut_top.a` is assigned into FPGA `0.B`. However, the Probe HAC daughter board is only installed for FPGA `0.A`. vCom reports a partition failure as follows.

```
[error] 25117 Total area is larger than max resource the fpgas can provide.
[error] 25117 STREAM_PROBE [7] exceed resource limit [0]
```

9.2 Selecting the Probe Mode

This topic introduces how to select the waveform generation method to meet your runtime requirements. The following table lists the recommended types of probes for different requirements.

Table 9-2: Comparison of Probe Modes

Specification / Probe Type	Physical	Streaming	GRP
Supported working mode	Prototyping and Emulation	Emulation	Emulation
Probe quantity limitation	524,288 bits per FPGA	2,000 bits per FPGA	No limitation
Impact on runtime performance	Small	Medium	Great
Debug time window	Medium	Large	Narrow
Signal visibility	Signals that are defined as trace nets	Signals that are defined as trace nets	The entire registers
Supported waveform format	CWDB, VCD, and XEDB	XEDB	XEDB
Synthesizer	vSyn or the third-party tool	vSyn or the third-party tool	vSyn or the third-party tool
Instrumentation	Definition of trace nets	Definition of trace nets	–
Data upload	Manual	Automatic	Automatic
Daughter board installation	DDR4 for P1	Probe HAC for both P1 and P2E	–

9.3 Generating the Waveform for Physical Probes

This topic guides you through generating the waveform using the physical probe feature.

Prerequisite

Make sure you have completed the following steps before enabling the data capture:

- [Adding Nets for Physical Probes](#). This step is required.
- [Configuring the DDR4 Daughter Board](#) if you use HuaPro P1 devices.
- [Setting Up the Environment for Generating CWDB Format Waveforms](#) if you want

waveforms in the CWDB format.

- [Downloading the Design to the Hardware](#). This step is required.

Procedure

Perform the following steps to manipulate the data capture and generate the waveforms for physical probes:

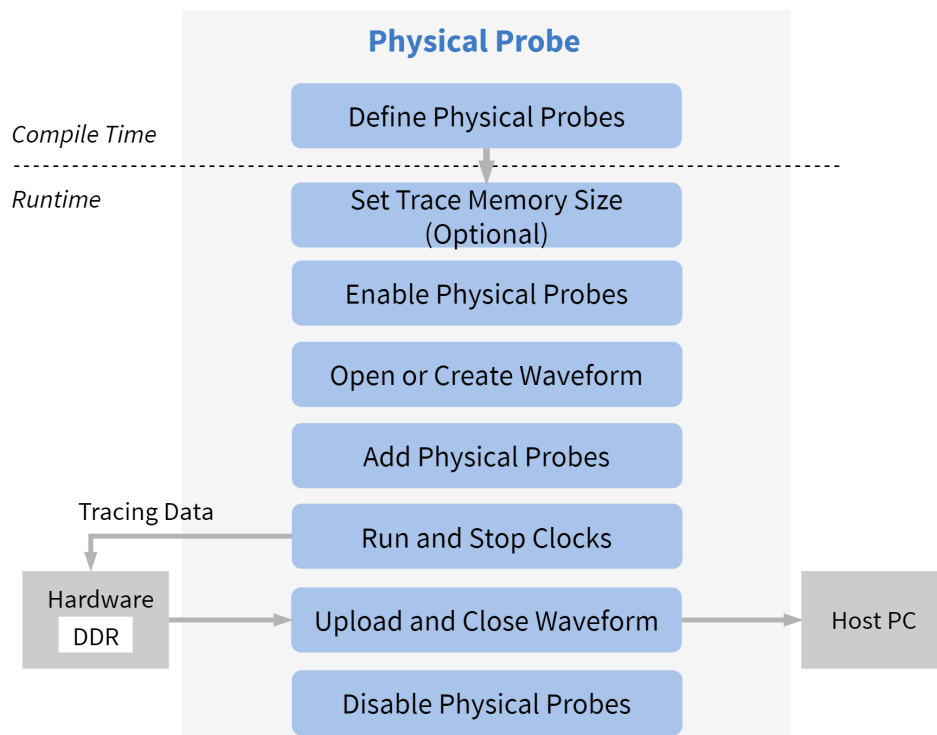


Figure 9-1: Flows of Using Physical Probes

1. (Optional) Configure the capture control as you require.
 - The tracing size is 100,000 rclk by default. You can set the maximum trace cycle by using the `set_trace_size` command. See [Setting the Trace Size](#) for details.
 - If you use triggers, vDbg stops clocks when a trigger happens by default. You can use the `set_post_trigger_samples` command to enable post-trigger running and set post trigger samples. See [Setting Post-trigger Samples](#) for details.

2. Enable the data capture.

```
tracedb -enable
```

3. Open a waveform file.

```
tracedb -open <db_name> [-vcd|-xedb|-cwdb] [-overwrite]
```

4. Define a physical probe or physical probes.

```
trace_signals -add {[-name <probe_name>] <signals>} [-errorok]
```

For more information about how to define physical probes, see [Managing Physical Probes](#).

5. Run the design in the blocking mode.

For more information about clock start and stop, see [Starting the Clock](#) and [Stopping the Clock](#).

6. Disable the data capture.

```
tracedb -disable
```

7. Upload the trace data to generate the waveform file in the host PC.

```
tracedb -upload
```

8. Close the current waveform file.

```
tracedb -close
```

9. Use Fusion Debug or xWave to check the VCD and XEDB waveform. You can also use third-party waveform viewer to check the waveform.

Guideline

The followings include the notes that need to be considered when using physical probes and the tips that help you flexibly use the `tracedb` and `trace_signals` commands.

- If you do not open a waveform file before adding probes, vDbg creates the `physicalprobe.xvcf` directory in the working path by default to store waveform files in the XEDB format.
- If you do not enable the data capture before adding probes, an empty waveform file or directory is created. The data capture is only started until you enable it.
- You can add additional probes after running the design and before uploading the trace data.
- You are not recommended running the design in non-blocking mode because this way cannot control exactly how much of the running cycles for which probes are traced.
- vDbg stores the trace data in DDR memories. When the trace memory size reaches the maximum value you set or the default one, new trace data overwrites the old data.
- If you want to restart the data capture, execute `tracedb -upload` every time before executing `tracedb -enable` to prevent loss of existing waveform data.

Example

For more information about how to use the commands, see [Example for Tracing Physical Probes](#).

9.3.1 Configuring the DDR4 Daughter Board

To use the DDR4 daughter board for debugging on HuaPro P1 devices, you must configure the type and connection of the DDR4 daughter board at compile time. For HuaPro P2E devices, on-chip DDR memories are used for data capture, so no DDR4 daughter board is needed.

Procedure

Perform the following steps to configure the DDR4 daughter board at compile time.

1. Include the DDR settings in the hardware description file. Use the following two card types to describe the DDR4 daughter boards that are used for physical probes.
 - `CAPC_8G_V1` is used for probed signal capture. The memory size is 8 GB and the version of the daughter board is v1.
 - `CAPC_16G_V1` is used for probed signal capture. The memory size is 16 GB and the version of the daughter board is v1.

Example:

The following line in the hardware description file *hw.hdf* describes a DDR4 daughter board that is used for signal data capture:

```
CARD += {"type": "CAPC_16G_V1", "connector": "1.B.1"}
```

For more information about how to describe the connection of DDR daughter boards in the hardware description file, see [CARD in HDF](#).

2. Specify the hardware description file at compile time using the `emulator_spec` command line:

The following example command line specifies the hardware description file *hw.hdf*.

```
% emulator_spec -add {file hw.hdf}
```

Reference

For more information about the DDR4 daughter board, refer to *HuaPro DDR4 Daughter Board Reference Manual*.

9.3.2 Getting the Maximum Depth Cycle

To get the maximum number of trace cycles for which DDR can capture the data, use the `get_max_depth_cycles` command line.

The maximum number of trace cycles is directly proportional to the memory size of DDR daughter boards and inversely proportional to the number of physical probes that you define at compile time. Up to 98% of DDR memory can be used for physical probe tracing. In the same DDR configuration, the more physical probes you add, the less the maximum trace cycles you can set.

9.3.3 Setting the Trace Size

HPE stores trace data and uploads the data from the DDR memory into a trace file. You can set the trace memory size as required. When the trace data exceeds the specified trace memory size, new trace data overwrites the old data. If not specified, the tracing size is 100,000 rclk by default. You can get the maximum trace cycle by using the `get_max_depth_cycles` command.

Setting the tracing memory size affects all subsequent `run` and `tracedb -upload` commands. The memory size might also affect the number of additional clock cycles that are counted after a trigger happens and before the clock or tracing stops.

To set the trace memory size for the physical probe, use the `set_trace_size` command line before executing the `tracedb -enable` command line:

Syntax:

```
set_trace_size [<size>]
```

Where `<size>` specifies the amount of traced data to be uploaded into the trace file. If not specified, the current value of the `set_trace_size` parameter returns. The minimum trace size is 100 rclk.

The `<size>` can be specified in one of the following format:

- `<number>rclk` : Number of probe samples.
- `<number><time_unit>` : Simulation time. Valid values of the time unit: fs, ps, ns, us, ms, and s.
- `<number>%` : Percentage value of the maximum possible trace memory size.
- `<number>` : A number without units. The unit is the same as the current setting of the `set_time_unit` parameter.

**NOTE**

- You can use the floating point number for all the format of `<size>` instead of `<number>rclk` and `<number><time_unit>`.
- You can use the integer or floating point number only in decimal.
- You cannot use octal and hexadecimal formats.

9.3.4 Setting Up the Environment for Generating CWDB Format Waveforms

Common Wave Database (CWDB) is an XEPIC-defined waveform format, which is compatible with the FSDB waveform format. Only waveforms of physical probes can be generated in CWDB format.

To generate CWDB format waveforms with the vDbg `tracedb` command and the `-cwdb` option, you need to set up the environment in the following steps before starting vDbg:

1. Prepare required files for CWDB waveform generation.
 - FSDB include files: *ffwAPI.h* and *fsdbShr.h*.
 - FSDB library: *libnffw.so*.
 - An implementation of CWI functions. Contact XEPIC technical support for more information about CWI.
2. Compile CWI functions into a dynamic library with g++.

```
#!/bin/bash

#set the include/lib path.
cwdb_include_root="<include_path>"
cwdb_lib_root="<lib_path>"

FLAGS="-I. -I./include/ -I${cwdb_include_root} -fPIC -shared"
g++ ${FLAGS} <impl_file_path> -o ./<dy_lib_file> ${cwdb_lib_root}/libnffw.so
-ldl -pthread
```

Where,

- `<include_path>` specifies the path of the FSDB include files.
- `<lib_path>` specifies the path of the FSDB library.
- `<impl_file_path>` specifies your implementation of CWI functions.
- `<dy_lib_file>` specifies the dynamic library to be created.

In the following example, *libcwi_writer_impl.so* is created from *cwdb_writer_impl_cwi.cc*.

```
cwdb_include_root="/home/user1/cwdb"
cwdb_lib_root="/home/user1/cwdb"
```

```

FLAGS="-I.. -I../include/ -I${cwdb_include_root} -fPIC -shared"
g++ ${FLAGS} /home/user1/cwdb/cwdb_writer_impl_cwi.cc -o
./libcwi_writer_impl.so ${cwdb_lib_root}/libnffw.so -ldl -lpthread

```

3. Set the `LD_LIBRARY_PATH` environment variable to point to the dynamic library.

Example:

```
setenv LD_LIBRARY_PATH "/home/user1/cwdb/libcwi_writer_impl.so"
```

You can use waveform tools, for example, Verdi, to view the CWDB format waveforms directly without any format conversion. Set the file filter to `*.*` as shown in the following figure and then you can find CWDB waveform files.

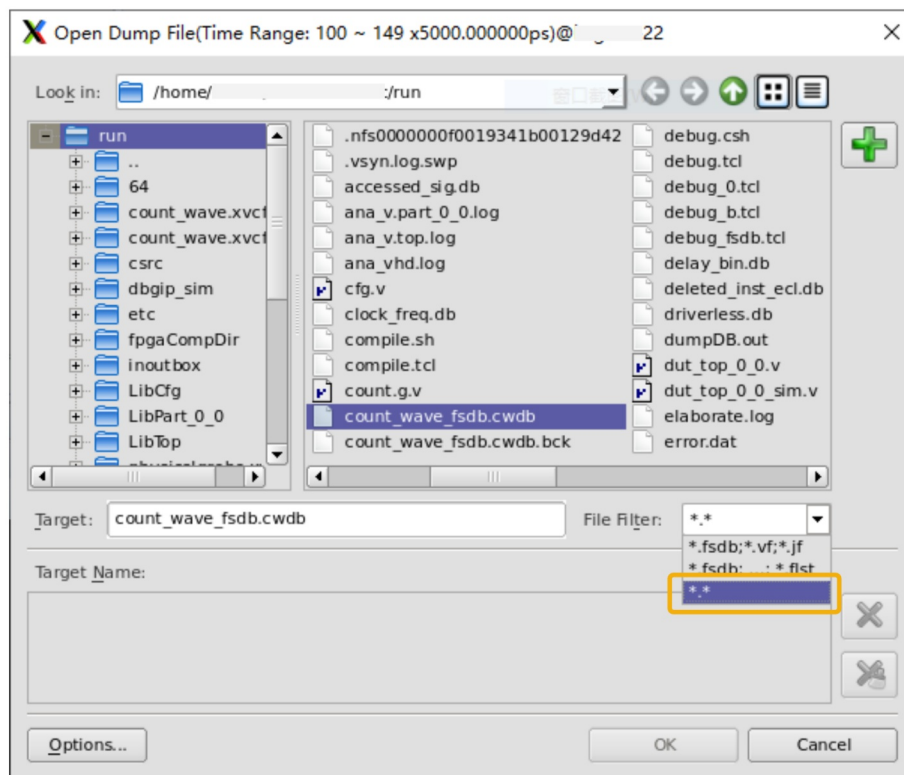


Figure 9-2: Viewing CWDB Waveforms in Verdi

9.3.5 Example for Tracing Physical Probes

This topic takes an example of HDF, vCom compilation script file, and vDbg run script file to show how to set trace nets and generate waveforms of physical probe signals for a two-FPGA design from one P1 device.

Prerequisites

Two DDR4 memory daughter boards are installed on the P1 device.

Procedure

Perform the following steps generate waveforms of physical probe signals:

1. Synthesize the design with vSyn or the third-party tool.
2. Prepare an HDF which describes the connection of DDR4 daughter boards.

An example of HDF, *hw-config.hdf*:

```
VERSION += 1.0
DEVICE += P1
BOARD += {"index": 0, "type": "TWO_CHIP_BOARD", "SN": "<SN_number>",
"CLK": 0}
CARD += {"type": "CAPC_16G_V1", "connector": "0.A.22"}
CARD += {"type": "CAPC_16G_V1", "connector": "0.B.1"}
```

Where,

- VERSION indicates that the HDF is version 1.
 - DEVICE indicates the device type is P1.
 - BOARD indicates that the system consists of one P1 device.
 - CARD indicates that two DDR4 daughter boards are connected to FPGAs for data capturing.
3. Compile the design with vCom with trace nets specified.

vCom compilation script file, *compile.tcl*:

```
design_read -netlist dut_top.vm
design_load -top dut_top

emulator_spec -add "file ./hw-config.hdf"

create_clock -sig_name dut_top.clk1 -frequency 100Mhz
create_clock -sig_name dut_top.clk2 -frequency 50Mhz

read_net -add {a[7:0] b c}
write_net -add {d e f g}

trace_net -add {a[7:0] b c d e f g}
trace_net -depth 1 -add dut_top.sub1
trace_net -add {dut_top.sub2.m1.m2.data[63:0]}
trace_net -add {dut_top.sub2.m1.m3.data[63:0]}
trace_net -add {dut_top.sub2.m2.data[63:0]}
trace_net -depth 1 -port dut_top.sub3

design_edit
design_generation
```

Where trace_net command lines set the following signals as trace nets.

- Signals in the top-level module: `a[7:0]` , `b` , `c` , `d` , `e` , `f` , and `g` .
 - All signals in the submodule of the `sub1` module with the depth level 1.
 - Data signal `data[63:0]` in different sub-modules of the `sub2` module.
 - All ports in the submodule of the `sub3` module with the depth level 1.
4. Generate the bitstream files of the design.
 5. Launch vDbg to download the bitstream files to FPGAs, run the design, and upload the trace data.

vDbg run script file:

```
design .
hw_server .
download

set_trace_size 2000rclk

tracedb -enable
trace_signals -add *

force b 0
force c 0
force d 1
force e 1
force f 1
run 100rclk
force e 0
run 1000rclk
force c 1
run 1000rclk

tracedb -upload
tracedb -disable
tracedb -close

exit
```

Where,

- `set_trace_size` sets the maximum trace size to 2000rclk, which means that only the latest trace data that captured in 2000rclk are retained.

```
% set_trace_size 2000rclk
```

- `tracedb -enable` enables the physical probe tracing.

- `tracedb` is not used, so vDbg creates the `physicalprobe.xvcf` directory in the working path by default to store waveform files.
- `trace_signals -add *` adds all signals that are set as trace nets into waveforms.
- `force` and `run` set up the running environment and start the clocks.
- `tracedb -upload` uploads the trace data stored in the buffer DDR to host PC.
- `tracedb -disable` disables the physical probe tracing.
- `tracedb -close` closes the waveform database.

The `physicalprobe.xvcf` directory is successfully generated in the working directory. The `physicalprobe_0_0.xedb` and `physicalprobe_0_1.xedb` waveform files are included in the directory.

6. Load the `physicalprobe.xvcf` directory in Fusion Debug or xWave waveform viewer and then add signals to display the wave.

9.4 Generating the Waveform for Streaming Probes

This procedure guides you through generating the waveform using the streaming probe feature. You can generate waveforms of streaming probes in the emulation mode only.

Use Flow

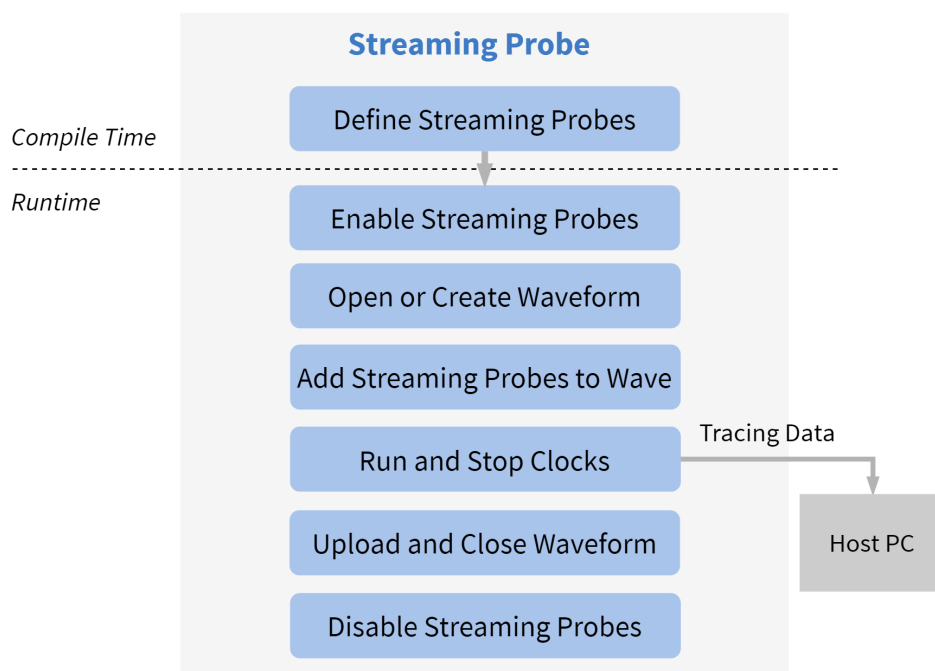


Figure 9-3: Flows of Using Streaming Probes

Prerequisite

- [Adding Nets for Streaming Probes](#).

- [Configuring the Probe HAC Daughter Board.](#)
- [Downloading the Design to the Hardware.](#)

 TIP

To achieve better performance when using a large number of streaming probes, the suggested hardware requirements of host PC for one FPGA are as follows.

- Threads ≥ 2
- Memory ≥ 0.5 GB
- Disk data rate ≥ 64 MB/s
- CPU speed ≥ 3 GHz

Procedure

Perform the following steps to manipulate the data capture and generate the waveform for the streaming probes at runtime:

1. Enable the data capture.

```
tracedb -stream -enable
```

2. Open an XEDB waveform file.

```
tracedb -stream -open <db_name> -xedb [-overwrite]
```

3. Define a streaming probe or streaming probes.

```
trace_signals -stream -add {[-name <probe_name>] <signals>} [-errorok]
```

For more information about how to define streaming probes, see [Managing Streaming Probes](#).

4. Run the design as instructed in [Starting the Clock](#).

When the design is running, vDbg uploads the trace data into a trace buffer in the host PC continuously. When the trace buffer fills up, vDbg might lower the user clock frequency before the trace data is uploaded completely. The more streaming probes you add, the greater the impact.

You might not view a completed waveform for the streaming probes because of remaining trace data in the trace buffer. To manually upload the trace data from the trace buffer to the waveform file, you can use the `tracedb -stream -flush` command line.

5. Disable the data capture function.

```
tracedb -stream -disable
```

IMPORTANT

Use the streaming probe function with caution. When the data capture is enabled, explosion of disk and memory might occur if there is a large amount trace data in a short time. Disable the data capture if you do not need the function.

If required, you can use `tracedb -stream -enable` to restart the data capturing.

6. Close the current waveform file.

```
tracedb -stream -close
```

vDbg closes the waveform file after finishing uploading the trace data including the remaining data in the trace buffer.

7. View the waveforms.

You can open the Fusion Debug GUI or xWave GUI to view the waveforms and also convert the waveform file format as you require. For more information about viewing waveforms, see [Displaying XEDB Waveform Files](#).

For more information about how to use the commands, see [Example for Tracing Streaming Probes](#).

9.4.1 Configuring the Probe HAC Daughter Board

To use the streaming probes, you must configure the connection and channel of Probe HAC daughter board. The Probe HAC functions as an interface daughter board which bridges the communication between the host and hardware.

Before using the streaming probes, make sure that the Probe HAC daughter boards are installed and the streaming probe drivers are running.

TIP

Refer to *Probe HAC Reference Manual* to get detailed information about the installation and configuration of the Probe HAC daughter boards.

The following figure shows the Probe HAC connection of two P1 devices. The Probe HAC daughter board contains four Mini-SAS HD ports that work as the streaming probe ports.

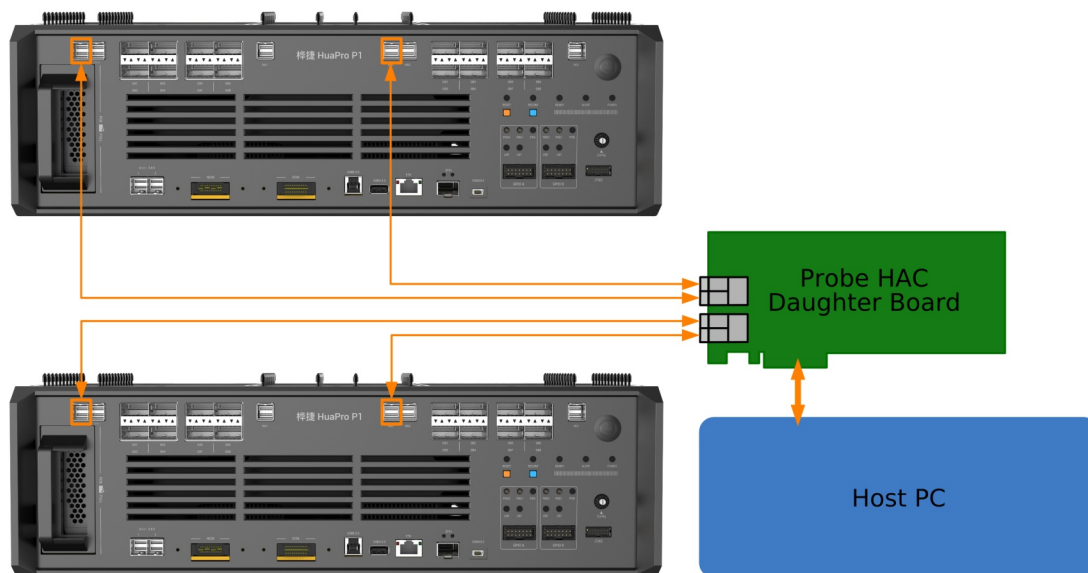


Figure 9-4: Installation of a Probe HAC with Two HuaPro P1 Devices

Include the Probe HAC settings in the hardware description file. Use the `HAC_V1` card type to describe the Probe HAC daughter boards.

For HuaPro P1, only MA3 and MB3 Mini-SAS ports can be used as the streaming probe ports. For HuaPro P2E, only HSSI-4 Mini-SAS ports can be used as the streaming probe ports.

The following is an example of card description for two P1 devices:

```
CARD += {"type": "HAC_V1", "connector": "0.A.MA3", "protocol": "stream", "channel": "0.0", "peer": "10.10.60.60"}
CARD += {"type": "HAC_V1", "connector": "0.B.MB3", "protocol": "stream", "channel": "0.1", "peer": "10.10.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.A.MA3", "protocol": "stream", "channel": "0.2", "peer": "10.10.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.B.MB3", "protocol": "stream", "channel": "0.3", "peer": "10.10.60.60"}
```

The following is an example of card description for two P2E devices:

```
CARD += {"type": "HAC_V1", "connector": "0.A.HSSI4", "protocol": "stream", "channel": "0.0", "peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "0.B.HSSI4", "protocol": "stream", "channel": "0.1", "peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "0.C.HSSI4", "protocol": "stream", "channel": "0.2", "peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "0.D.HSSI4", "protocol": "stream", "channel": "0.3", "peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.A.HSSI4", "protocol": "stream", "channel": "7.0", "peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.B.HSSI4", "protocol": "stream", "channel": "7.1", "peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.C.HSSI4", "protocol": "stream", "channel": "7.2", "peer": "10.20.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.D.HSSI4", "protocol": "stream", "channel": "7.3", "peer": "10.20.60.60"}
```

For more information about how to describe the connection of DDR daughter boards in the hardware description file, see [CARD in HDF](#).

In the VRM mode, the Probe HAC channels are configured by VRM administrators. If the hardware cannot provide Probe HAC channel resources, you cannot lock resources or connect to hardware.

9.4.2 Example for Tracing Streaming Probes

This topic shows an example of generating waveforms for a two-FPGA design from one P1 device using streaming probe mode and non-VRM mode.

Procedure

Perform the following steps to generate waveforms of streaming probe signals:

1. Synthesize the design with vSyn or the third-party tool.
2. Prepare an HDF which describes the connection of the Probe HAC daughter board.

An example of HDF, *hw-config.hdf*:

```
VERSION += 1.0
DEVICE += P1
BOARD += {"index": 0, "type": "TWO_CHIP_BOARD", "SN": "RRD00000026AFE0",
"CLK": 0}

CARD += {"type": "HAC_V1", "connector": "0.A.MA3", "protocol": "stream",
"channel": "0.0", "peer": "10.10.60.60"}
CARD += {"type": "HAC_V1", "connector": "0.B.MB3", "protocol": "stream",
"channel": "0.1", "peer": "10.10.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.A.MA3", "protocol": "stream",
"channel": "0.2", "peer": "10.10.60.60"}
CARD += {"type": "HAC_V1", "connector": "1.B.MB3", "protocol": "stream",
"channel": "0.3", "peer": "10.10.60.60"}
```

Where,

- **VERSION** indicates that the HDF is version 1.
- **DEVICE** indicates that the device type is P1.
- **BOARD** indicates that the system consists of one P1 device.
- **CARD** indicates that one Probe HAC daughter board is connected to FPGAs for data capturing.

3. Compile the design with vCom with streaming trace nets specified.

vCom compilation script file, *compile.tcl*:

```
design_read -netlist dut_top.vm
design_load -top dut_top

emulator_spec -add "file ./hw-config.hdf"

create_clock -sig_name dut_top.clk1 -frequency 100Mhz
create_clock -sig_name dut_top.clk2 -frequency 50Mhz

read_net -add {a[7:0] b c}
write_net -add {d e f g}

streaming_trace_net -add {a[7:0] b c d e f g}
streaming_trace_net -depth 1 -add dut_top.sub1
streaming_trace_net -add {dut_top.sub2.m1.m2.data[63:0]}
streaming_trace_net -add {dut_top.sub2.m1.m3.data[63:0]}
streaming_trace_net -add {dut_top.sub2.m2.data[63:0]}

design_edit
design_generation
```

Where `streaming_trace_net` command lines set signals as trace nets.

4. Generate the bitstream files of the design.
5. Launch vDbg to download the bitstream files to FPGAs, run the design, and upload the trace data.

vDbg run script file:

```

design .
hw_server .
download

tracedb -stream -enable
tracedb -stream -open wave -overwrite -xedb

trace_signals -stream -add {a[7:0]}
trace_signals -stream -add {b}
trace_signals -stream -add {c}
trace_signals -stream -add {d}
trace_signals -stream -add {e}
trace_signals -stream -add {f}
trace_signals -stream -add {g}
trace_signals -stream -add {sub1.Q[127:0]}
trace_signals -stream -add {sub1.A[18:0]}
trace_signals -stream -add {sub1.CEN}
trace_signals -stream -add {sub1.CLK}
trace_signals -stream -add {sub1.D[127:0]}
trace_signals -stream -add {sub1.WEN[15:0]}
trace_signals -stream -add {sub2.m1.m2.data[63:0]}
trace_signals -stream -add {sub2.m1.m3.data[63:0]}
trace_signals -stream -add {sub2.m2.data[63:0]}

force b 0
force c 0
force d 1
force e 1
force f 1
run 100rclk
force e 0
run 1000rclk
force c 1
run 1000rclk

tracedb -stream -disable
tracedb -stream -close

exit

```

Where,

- `tracedb -stream -enable` enables the streaming probe tracing.
- `tracedb -stream -open` opens the waveform database `wave` no matter the file exists or not and saves the waveform in XEDB format.

```
% tracedb -stream -open wave -overwrite -xedb
```

- `trace_signals -stream -add` adds signals that are set as trace nets into waveforms.
- `force` and `run` set up the running environment and start the clocks.
- `tracedb -stream -disable` disables the streaming probe tracing.
- `tracedb -stream -close` closes the waveform database.

The `wave.xvcf` directory is successfully generated in the working directory. The `wave_0_0.xedb` and `wave_0_1.xedb` waveform files are included in the directory.

6. Load the `wave.xvcf` directory in Fusion Debug or xWave waveform viewer and then add signals to display the wave.

9.5 Generating the Waveform for GRP Probes

This procedure guides you through generating the waveform using the GRP (Global Register Probe) feature. You can generate waveforms of GRP probes in the emulation mode only.

Use Flow

The following figure shows the compile-time and runtime use flow of GRP probes.

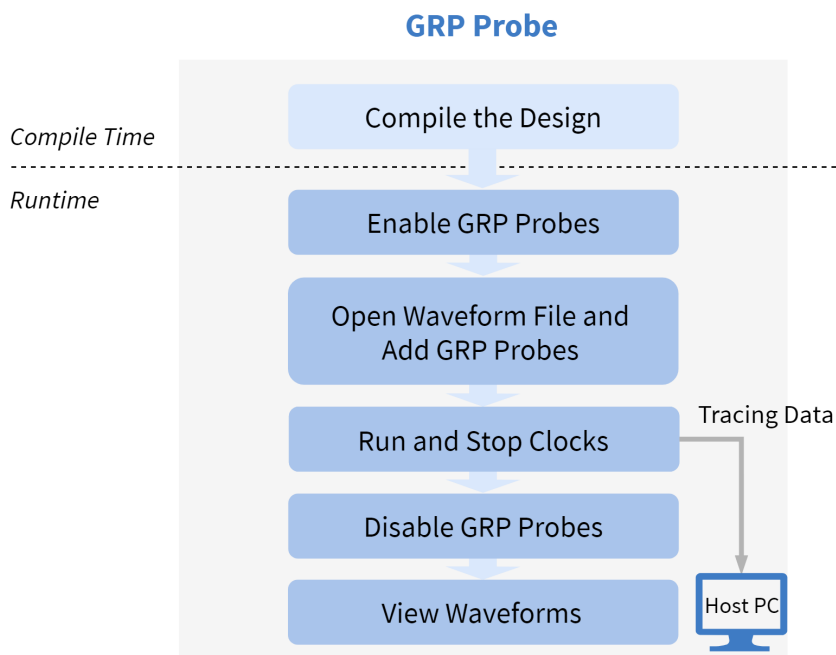


Figure 9-5: Flows of Using GRP Probes

Step 1: Compile the Design

1. Use vSyn or the third-party synthesizer to synthesize the design.
2. Use vCom to compile the design with the trace nets of physical probes defined or not.

You can use GRP probes alone or take it as a supplement of physical probes.

3. Run PnR with Vivado by using the *Makefile* file.

Step 2: Run and Debug the Design

1. After [Downloading the Design to the Hardware](#), enable the data capture.

```
trace_signals -ff -enable
```

2. Open an XEDB waveform file and define a GRP probe or GRP probes.

```
trace_signals -ff -add [-waveform_file <probe_name>] <signals> [-errorok]
```

Both the waveform file and the probe are named as `<name>`. If you specify an existing name, vDbg renames the existing waveform file with the *.bak* extension. If the `- waveform_file <name>` option is not specified, vDbg uses a default name, *flopflip<n>*.

If required, you can open multiple waveform files and add signals to each waveform file.

For more information about how to define GRP probes, see [Managing GRP Probes](#).

3. Run the design in the blocking mode (see [Starting the Clock](#) and [Stopping the Clock](#)).

NOTE

GRP have a significant impact on the runtime performance. The more GRP probes you add, the greater the impact is. Therefore, you are recommended generating waveforms of GRP probes for a small time window.

4. Disable the data capture and close the waveform file.

```
trace_signals -ff -disable
```

IMPORTANT

Use the GRP function with caution. When the data capture is enabled, explosion of disk and memory might occur if there is a large amount trace data in a short time. Disable the data capture if you do not need it.

vDbg closes the waveform file after finishing uploading the trace data including the remaining data in the trace buffer.

5. View the waveforms.

You can open the Fusion Debug GUI or xWave GUI to view the waveforms and also convert the waveform file format as you require. For more information about viewing waveforms, see [Displaying XEDB Waveform Files](#).

Example

Example 1: The following vDbg runtime scripts enable GRP, add signals to the default probe, and generate the *flopflip0.xedb* waveform file.

```
design .
hw_server .
download

trace_signals -ff -enable
trace_signals -ff -add a23_core.u_fetch.i_wb_err a23_core.u_fetch.o_wb_adr
a23_core.globrst
run 100 rclk
trace_signals -ff -disable

exit
```

Example 2: The following vDbg runtime scripts enable GRP, save the trace signal names into a text file and then import the file. The trace signals are added to two probes. vDbg generates two waveform files, *flopflip0.xedb* and *myprobe.xedb*.

```
design .
hw_server .
download

trace_signals -ff -enable
trace_signals -ff -add a23_core.u_fetch.i_wb_err a23_core.u_fetch.o_wb_adr
trace_signals -ff -add -waveform_file myprobe a23_core.globrst
trace_signals -ff -save sig_list.txt
trace_signals -ff -disable
trace_signals -ff -enable
trace_signals -ff -import sig_list.txt
run 100 rclk
trace_signals -ff -disable

exit
```

The *sig_list.txt* file is shown as follows:

```
probe name: flopflip0
  a23_core.u_fetch.i_wb_err
  a23_core.u_fetch.o_wb_adr
probe name: myprobe
```

```
a23_core.globrst
```

9.6 Managing Probes

You must define probes before starting probe tracing. A probe can consist of a single trace signal or multiple trace signals. Grouping multiple trace signals into a probe is useful when you want to delete and query trace signals in batch.

vDbg provides several options of the `trace_signals` command for you to manage probes efficiently. Depending on the probing mode you select, the options are different.

You can manage physical and streaming probes to quickly and precisely define probes in the runtime script file.

- [Managing Physical Probes](#)
- [Managing Streaming Probes](#)
- [Managing GRP Probes](#)

9.6.1 Managing Physical Probes

You can manage physical probes as follows:

- Check the trace nets that can be added to physical probes.
 - List all available trace nets.

```
trace_signals -list_signals
```

- List the available trace nets in your specified instance.

```
trace_signals -list_signals <instance>
```

- List the available trace nets whose name match your specified pattern.

```
trace_signals -list_signals <pattern>
```

- Define physical probes.
 - Add a trace net or trace nets to a physical probe with a default name, *ph<n>*, where *<n>* starts from zero and is increased by one for each command execution.

```
trace_signals -add {<signal_names>}
```

- Add all available trace nets to a default physical probe.

```
trace_signals -add *
```

- Add a trace net or nets to a physical probe with a specified name.

```
trace_signals -add {-name <probe_name> <signals>}
```

To ignore the errors about the missing nets, you add the `-errorok` option on the command.

- Check the trace signals that already added to physical probes.

- List all physical probes and corresponding trace signals.

```
trace_signals -list
```

- List trace signals in your specified probe or probes.

```
trace_signals -list <probe_names>
```

- Delete probes that you do not need. Note that only emulation mode supports the deletion operation.

- Delete your specified probe or probes.

```
trace_signals -delete <probe_names>
```

- Delete all defined probes.

```
trace_signals -delete *
```

- Save and restore the command execution about physical probe tracing.

1. Save the executed command lines for opening a waveform file and defining physical probes into a specified Tcl script file.

```
trace_signals -save <tcl_file>
```

2. After you close the current waveform file or you restart debugging, execute the command lines in your specified Tcl script file.

```
trace_signals -import <tcl_file>
```

9.6.2 Managing Streaming Probes

You can manage streaming probes as follows:

- Define streaming probes.
 - Add a trace net or nets to a streaming probe with default name, *sp<n>*, where *<n>* starts from zero and is increased by one for each command execution.

```
trace_signals -stream -add {<signals>}
```

- Add a trace net or nets to a streaming probe with a specified name.

```
trace_signals -stream -add {-name <probe_name> <signals>}
```

To ignore the errors about the missing nets, you add the `-errorok` option on the command. Note that streaming probe mode does not support `-add *`.

- Check the trace signals that are already added to streaming probes.
 - List all streaming probes and corresponding trace signals.

```
trace_signals -stream -list
```

- List trace signals in your specified probe or probes.

```
trace_signals -stream -list <probe_name> ...
```

- Delete streaming probes that you do not need.
 - Delete your specified probe or probes.

```
trace_signals -stream -delete <probe_name> ...
```

- Delete all defined probes.

```
trace_signals -stream -delete *
```

- Save and restore the command execution about streaming probe tracing.
 1. Save the executed command lines for opening a waveform file and defining streaming probes into a specified Tcl script file.

```
trace_signals -stream -save <filename>
```

2. After you close the current waveform file or you restart debugging, execute the command lines in your specified Tcl script file.

```
trace_signals -stream -import <filename>
```

9.6.3 Managing GRP Probes

You can manage GRP probes as follows:

- Define GRP probes after enabling data capture.
 - Add a signal or signals to a GRP probe a default name, *flopflip<n>*, where *<n>* starts from zero and is increased by one for each command execution. The maximum number of probes are limited to 50. vDbg opens or creates a waveform with the default name at the same time as probe definition.

```
trace_signals -ff -add {<signals>}
```

- Add a signal or signals to a GRP probe with a specified name. vDbg opens or creates a waveform with the specified name at the same time as probe definition.

```
trace_signals -ff -add {-waveform_file <probe_name> <signals>}
```

To ignore the errors about the missing nets, you add the `-errorok` option on the command. Note that GRP probe mode does not support `-add *`.

- Check the trace signals that are already added to GRP probes.
 - List trace signals in your specified probe or probes.

```
trace_signals -ff -list <probe_names>
```

- List all GRP probes and corresponding trace signals.

```
trace_signals -ff -list
```

- Delete GRP probes that you do not need.
 - Delete your specified probe or probes.

```
trace_signals -ff -delete <probe_names>
```

- Delete all defined probes.

```
trace_signals -ff -delete *
```

 TIP

The data capture is disabled automatically when all GRP probes are deleted. If you want to define probes after deleting all, you must enable the data capture first.

- Save and restore the trace signals used in GRP tracing.
 1. Save the names of defined GRP trace signals into a specified text file.

```
trace_signals -ff -save <text_file>
```

2. After you close the current waveform file or you restart debugging, enable GRP and then add the signals that are listed in the text file into a GRP probe.

```
trace_signals -ff -import <text_file>
```

9.7 Displaying XEDB Waveform Files

HPE supports saving waveforms in the XEDB format. XEDB is a new X-EPIC self-developed event database. HPE package provides a standalone waveform debugging tool, xWave, for you to view the dumped waveforms. After [Setting Up Your Environment](#), use the following command line to invoke xWave.

```
xwave [-wdb <file>]
```

Where `-wdb <file>` specifies the waveform database file. If you do not specify any option, xWave opens without loading any database file. You can add a database file from the xWave GUI.

For more information about how to start xWave, refer to the “Starting xWave” section in *Fusion Debug User Guide*. For more information about how to use xWave, refer to the “Debugging in the Waveform View” section in *Fusion Debug User Guide*.

 TIP

To tryout more advanced debugging functions, such as source code debugging, schematic analysis, and coverage analysis, you can contact XEPIC for product trials of Fusion Debug.

If the physical probes or streaming probes are partitioned into multiple FPGAs, vDbg generates

a waveform directory (.xvcf) and stores an XEDB waveform file for each FPGA in the directory. If vCom partitions a multi-bit probe into two FPGAs, the waveform of the probe is saved in two files separately. Nevertheless, you can directly open the waveform directory in xWave and Fusion Debug. The tools merge XEDB files automatically.

The following command line invokes xWave to display the waveform database which is saved as a directory.

```
xwave -wdb user1_record.xvcf
```

The following figure shows how to select a directory when opening a database from the xWave GUI.

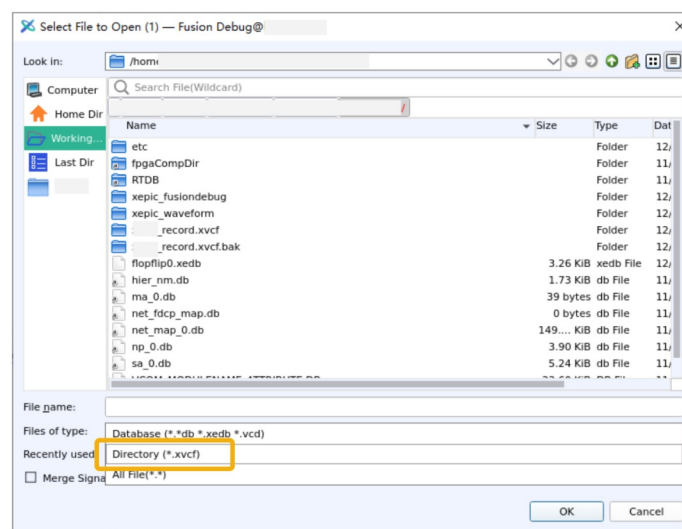


Figure 9-6: Opening a Waveform Directory in xWave GUI

10 Debugging Designs with Triggers

This section introduces how to use the Accuracy-Time Triggers (ATTs) to control the design running.

10.1 Stopping Clocks with Triggers

When a trigger event happens, vDbg stops user clocks and tracing at the same time even though you specify longer running time or a larger number of cycles on the `run` command line.

10.2 Setting Post-trigger Samples

By default, vDbg stops clocks when a trigger happens. If you want to run a design for a period of time after a trigger happens, you can use the `set_post_trigger_samples` command to enable post-trigger running and set post trigger samples. vDbg stops the clock and tracing until the design runs specified clock cycles, no matter how much running time restricted by the `run` command and whether new triggers occur during the post-trigger running.

```
set_post_trigger_samples <cycle_num>|<percentage_value>
```

Note that if you use triggers together with physical probes, you must set the post-trigger samples before enabling tracing. It means that you must execute the `set_post_trigger_samples` command before `tracedb -enable`.

By default, vDbg shows all messages of trigger occurrence and the execution of the `$cmd()` system tasks in the terminal. In some cases, numerous messages of trigger occurrence might print out during the post-trigger running which affects debugging operations.

If you want to hide these messages, use the `trigger -at -silent` command line before running a design. Then vDbg shows only the messages of `$cmd()` in the terminal. You can find trigger messages in the vDbg log file, `vdbg.log`. To explicitly show trigger messages in the terminal, use the `trigger -at -verbose` command line before running a design.

10.3 Examples for Controlling Clocks and Tracing Stop

You can use different combinations of `run` and `set_post_trigger_samples` commands to control how clocks and tracing are stopped. Depending on your configuration, triggers can stop user clocks, tracing, or both at the same time.

The following table lists and compares different configurations of running time, post-trigger

samples, and trigger stop mode.

Table 10-1: Examples of Using Triggers to Stop the Clock and Tracing

Run	Trigger	Sample	Trigger Time	Stop Clock and Tracing
-nowait	No	-	-	No stop
-nowait	Occur	0	Any	Trigger occurs
-nowait	Occur	100	Any	100 rclk after trigger occurs
1000 rclk	No	-	-	Run 1000 rclk
1000 rclk	Occur	0	Any	Trigger occurs
1000 rclk	Occur	100	0-999	100 rclk after trigger occurs
1000 rclk	Occur	100	>=1000	Run 1000 rclk

Where,

- **Run** indicates the option that is specified on the `run` command line.
- **Trigger** indicates whether a trigger occurs during the design running. “Occur” means a trigger occurs. “No” means that no trigger occurs.
- **Sample** indicates the clock cycles for post trigger tracing. “0” means that the cycle of post-trigger samples is not set or set to zero. A value means the number of cycles that is specified on the `set_post_trigger_samples` command. The time unit is rclk.
- **Trigger Time** indicates when the trigger occurs. The time unit is rclk.
- **Stop Clock and Tracing** indicates when the user clocks and the tracing are stopped.

10.4 Controlling the Run with ATT

ATT triggers provide the capability of defining trigger conditions with Verilog language. If trigger conditions need a change, you can directly modify the Verilog code at runtime and reload the source file with vDbg commands. When an ATT happens, vDbg can stop user clocks and tracing (waveform capture of probes) accurately with a minimized impact on runtime performance.

TIP

You can use ATT triggers in the emulation mode for HuaPro P2E devices only.

You can perform the following tasks by ATTs:

- Stop user clocks by using the `$trigger` system task in FSMs.

- Stop tracing.
- Change trigger conditions flexibly at runtime.
- Invoke vDbg Tcl commands in Verilog code to debug a design by using the `$cmd()` system task.

TIP

The syntax of Verilog is described using BNF in this section, which is a little bit different from that in other sections.

- Lowercase words containing embedded underscores indicate syntactic categories.
- Braces (`{}`) enclose a repeated item. You can specify the item zero or multiple times.

10.4.1 Use Flow of ATT

The complete use flow of ATTs is shown as follows:

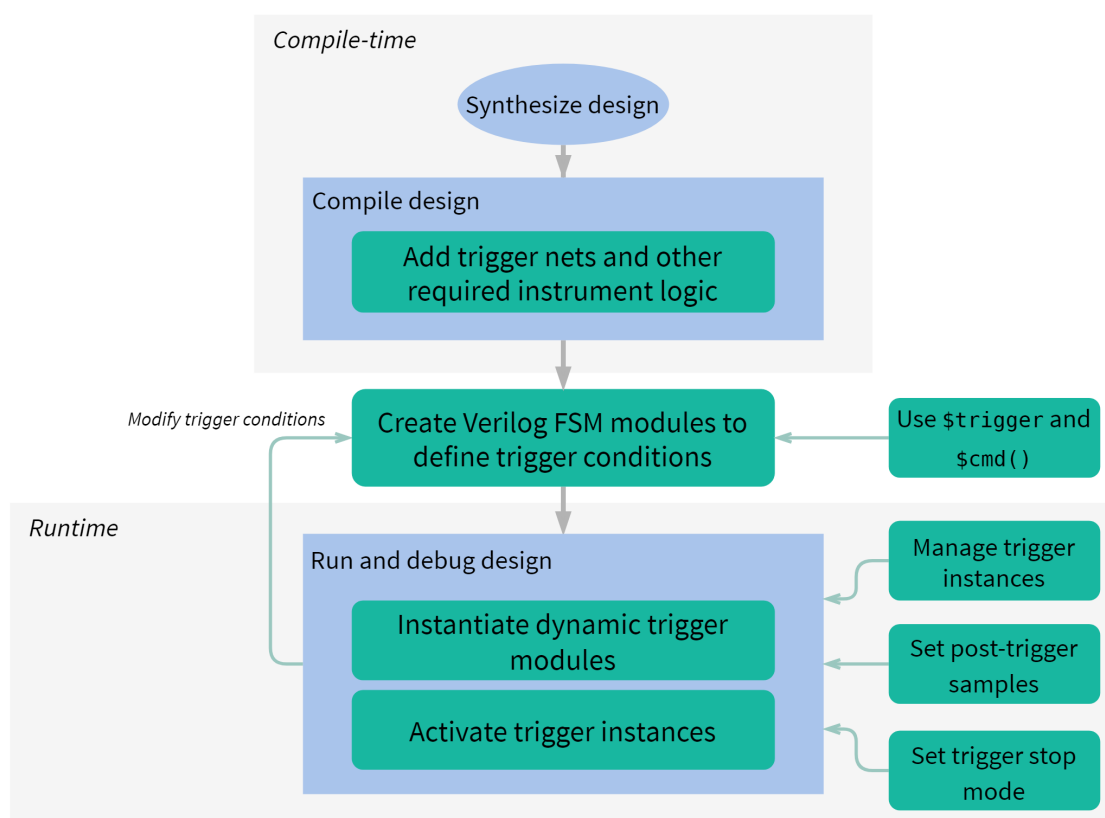


Figure 10-1: Use Flow of ATTs

Prerequisites

- [Adding ATT Nets at Compile Time.](#)
- [Setting Up the Environment for ATT.](#)

Procedure

You can control the running of designs and debug designs with ATTs in the following steps:

1. Create Verilog FSM modules in supported Verilog coding style and supported system tasks (see [Preparing ATT Trigger Modules](#)).

By [Executing Callbacks of vDbg Debugging Scripts](#), you can invoke vDbg commands inside Verilog code of trigger FSM.

2. (Optional) If required, you can set post-trigger samples (see [Setting Post-trigger Samples](#)).
3. Instantiate and activate trigger modules at runtime with vDbg Tcl commands (see [Creating and Enabling ATT Trigger Instances](#)).
4. (Optional) You can manage, such as list, delete, and overwrite, trigger instances as you require (see [Managing Trigger Instances](#)).

5. Run the design and wait for trigger occurrence or the execution of `$cmd()` system tasks.

When a trigger happens or `$cmd()` is executed, vDbg prints out messages to indicate the line number of source code and the clock cycle.

6. After a trigger occurs, you can perform common debugging operations on the design, such as accessing signals and memories.



NOTE

Currently, ATTs are not recommended being used along with probes.

7. (Optional) If required, you can modify the trigger conditions, reload module definitions, and restart the design running.

10.4.2 Setting Up the Environment for ATT

Verify the environment of the runtime host where you run vDbg before you use ATTs.

- Vivado 2022.2 or later version is installed.
- The `gnu/microblaze/lin/bin` path in the installation directory of Vivado is added to the `PATH` environment variable. For example:

```
% setenv PATH
"${PATH}:/nfs/tools/software/Vivado/2022.2/gnu/microblaze/lin/bin"
```

10.4.3 Adding ATT Nets at Compile Time

You must specify all ports of active trigger instances by using the `dynamic_trigger` and `trigger_net` commands at compile time. For more information about the command usage,

see [Adding Nets for ATT Control](#).

10.4.4 Preparing ATT Trigger Modules

To set up the conditions for ATT, you can define FSMs with a limited synthesizable subset of Verilog to describe the conditions and the state sequence. The FSM gets the values of inputs from associated signals in the design and performs state transition based on the defined state-transition behavior, and then triggers when a certain state reaches or a certain condition happens.

You can get the coding style guide of defining FSM modules by referring to [FSM Language Syntax](#).

10.4.5 Creating and Enabling ATT Trigger Instances

Use the `trigger` command with the `-at` option and other options to instantiate ATT trigger modules. Note that the `-at` option must follow the `trigger` command and do not use any other options between them.

Procedure

1. Load source files that contain the definition of trigger modules.

NOTE

If you reload the source file for a module, all trigger instances that are instantiated from the module are deleted. You need to create the instances of the module again.

```
trigger -at -readfile <files>
```

2. Instantiate trigger modules and bind ports.

```
trigger -at -addinst <inst_name> [-overwrite] -module <module_name> -  
bind_ports {<net0>} {<net1>} ...
```

3. Enable required trigger instances.

All trigger instances are disabled by default when you create them. You must enable required instances to make the trigger conditions active.

```
trigger -at -enable <instance_names>|*
```

Example

In the following example, two Verilog files, *trigger_fsm.v*, and *trigger_fsm_2.v*, are loaded and two trigger instances, `inst1` and `inst2` are created. After the `inst1` instance is enabled, the design starts running and the trigger happens.

```
vdbg> design .
vdbg> hw_server .
vdbg> download

[info] 11043 Download successfully! Design is ready to run.

vdbg> trigger -at -readfile trigger_fsm.v trigger_fsm_2.v

-- Analyzing Verilog file '/home/user1/vdbg-case/dy-trigger/cnter/trigger_fsm.v' (VERI-1482)
-- Analyzing Verilog file '/home/user1/vdbg-case/dy-trigger/cnter/trigger_fsm_2.v' (VERI-1482)
INFO: /home/user1/vdbg-case/dy-trigger/cnter/trigger_fsm.v(2): compiling module 'trigger_fsm'
(VERI-1018)
INFO: /home/user1/vdbg-case/dy-trigger/cnter/trigger_fsm_2.v(1): compiling module 'trigger_fsm_2'
(VERI-1018)

vdbg> trigger -at -addinst inst1 -module trigger_fsm -bind_ports {dut_top.clk} {dut_top.cnt[1:0]}

[info] 12785 Add trigger instance inst1 successfully.

vdbg> trigger -at -addinst inst2 -module trigger_fsm_2 -bind_ports {dut_top.clk} {dut_top.flag}

[info] 12785 Add trigger instance inst2 successfully.

vdbg> trigger -at -enable inst1
vdbg> force rst_n 'b1
vdbg> run 200rclk

$trigger:trigger_fsm.v(42) triggers at 477 rclk.
```

10.4.6 Managing Trigger Instances

After [Creating and Enabling ATT Trigger Instances](#), you can manage the trigger instances as follows:

- Query Trigger Modules
- Query Trigger Instances
- Disable Trigger Instances
- Delete Trigger Instances

Query Trigger Modules

You can query the list of ports for a specific trigger module or all trigger modules in the design by using the `-listmodule` option on the `trigger` command line.

```
trigger -at -listmodule [<module_names>|*]
```

Examples:

- The following command line lists the ports of trigger module `trigger_fsm`.

```
vdbg> trigger -at -listmodule trigger_fsm

module name:trigger_fsm, ports:clk rst_n cnt0[9:0] cnt1[4:0]
```

- The following command line lists all the trigger modules and their ports.

```
vdbg> trigger -at -listmodule *

module name:trigger_fsm, ports:clk cnt[1:0]
module name:trigger_fsm_2, ports:clk flag
```

Query Trigger Instances

You can query information about a specific trigger instance or all trigger instances in the design by using the `-listinst` option on the `trigger` command line.

```
trigger -at -listinst [<instance_names>|*]
```

You can get the following information about each trigger instance:

- Instance name
- Module name
- The list of ports for the instance
- Board ID
- FPGA ID
- Processor ID
- State: active or inactive

Examples:

- The following command line queries the information of trigger instance `inst1`.

```
vdbg> trigger -at -listinst inst1

instance name:inst1, module:trigger_fsm, port:dut_top.clk dut_top.cnt[1:0] ,
board id:0, fpga id:0, microBlaze id:0, state:active
```

- The following command line lists all the trigger instances and prints out their information.

```
vdbg> trigger -at -listinst

instance name:inst1, module:trigger_fsm, port:dut_top.clk dut_top.cnt[1:0] ,
board id:0, fpga id:0, microBlaze id:0, state:active

instance name:inst2, module:trigger_fsm_2, port:dut_top.clk dut_top.flag ,
board id:0, fpga id:0, microBlaze id:1, state:active
```

Disable Trigger Instances

You can disable instances by using the `-disable` option on the `trigger` command line. If some trigger instances are not required for run control, you can disable the trigger instances temporarily.

```
trigger -at -disable <instance_names>|*
```

Delete Trigger Instances

You can delete trigger instances permanently that are not required for run control by using the `-delete` option on the `trigger` command line.

```
trigger -at -delete <instance_name>|*
```

10.4.7 Executing Callbacks of vDbg Debugging Scripts

To facilitate the debugging of designs, you can use the `$cmd()` system task in [Simple Action Statements](#). The `$cmd()` system task is a specific system task provided by vDbg to invoke a majority of vDbg commands inside Verilog code of trigger FSM.

```
$cmd("vdbg_tcl_command_line")
```

Where `vdbg_tcl_command_line` specifies a supported vDbg Tcl command line. The supported commands and options are listed as follows. Unless noted, all options of the commands are supported.

- Signal access: `force`, `deposit`, `release`, `signal_group`, `get_value`
- Memory access: `memory`, `dump_mem`
- Run control: `get_time`
- Probe: `tracedb`, `trace_signals`
- Trigger instance management: Only the following options of the `trigger` command are supported.
 - `trigger -at -listinst`
 - `trigger -at -listmodule`
 - `trigger -at -enable`
 - `trigger -at -disable`

 NOTE

- If you use `$cmd()` in FSMs, to avoid possible design function errors, you are recommended to specify running time or clock cycles on the `run` command line, which runs a design in blocking mode.
- When the `$cmd()` system tasks are executed, the clocks are stopped. After execution completes, the clocks are started again. The clock stop and start as well as the command execution increase the amount of time it takes to run a design.

Example:

```
module trigger_fsm(
    input clk,
    input [9:0] cnt
);
    always @ (posedge clk) begin
        if (cnt == 5) begin
            $cmd("tracedb -enable");
        end else if (cnt == 8) begin
            $trigger;
        end
    end
endmodule
```

10.4.8 Limitations of ATT

Limitations of ATT are summarized as follows:

- Maximum eight trigger instances are supported for one FPGA. Maximum 8192 trigger nets are supported for one trigger instance.
- A subset of Verilog syntax is supported for FSM declaration.
- A part of vDbg commands are supported to be invoked in FSMs.

10.5 FSM Language Syntax

Obey the following coding style guide when defining FSM modules.

 NOTE

Do not rely on the availability and reliability of the syntax that is not introduced in this document.

- [Defining Verilog FSM Modules](#)

- [Port Declaration](#)
- [State Definition](#)
- [Counter Definition](#)
- [FSM Body Definition](#)
- [Case Statements](#)
- [If-then-else Statements](#)
- [Simple Action Statements](#)
- [Supported FSM Expressions and Operators](#)
- [Lexical Conventions in FSM](#)
- [Examples for FSM Programs](#)

10.5.1 Defining Verilog FSM Modules

To use ATTs at runtime, define Verilog FSM modules with Verilog language in the following coding style:

Syntax:

```
module fsm_name(input wire clk_sig, [input_ports_declarations])
[ state_definition ]
[ counter_definition ]
fsm_body_definition
endmodule
```

Where,

- `fsm_name` specifies the module name.
- `clk_sig` specifies the clock signal in the module. The clock signal declaration is required.
- `input_ports_declarations` specifies declarations of other input signals in the module. See [Port Declaration](#) for more information about limitations on port declaration.
- `state_definition` creates a state encoding and specifies how to store the current state of the FSM. If you define a state-less FSM program, the state definition is not required. See [State Definition](#) for details.
- `counter_definition` creates a counter variable as you require. See [Counter Definition](#) for details.
- `fsm_body_definition` specifies the trigger condition and state transition behavior in a `always@` block. See [FSM Body Definition](#) for details.

10.5.2 Port Declaration

Ports of trigger modules must obey the following rules:

- An input clock port is required.
- Only input ports are supported. Output and inout ports are not supported.
- Input ports must be scalar or vector type. Array type is not supported.
- The signed type of signals is not supported.

10.5.3 State Definition

You can create a state encoding with Verilog symbolic constants. For better clarity and flexibility, you can use `localparam` to define the state constant value. The state variables must be unsigned type and the width must be less than or equal to 32 bits.

Example:

```
localparam [3:0]    STATE_INIT    = 4'b0000,
                   STATE_LOAD    = 4'b0001,
                   STATE_FETCH    = 4'b0010,
                   STATE_EXECUTE = 4'b0011;
reg [31:0] cstate;
```

10.5.4 Counter Definition

If you want to use a counter or multiple counters in the FSM, you can define counter variables. The counter variables must be unsigned type and the width must be less than or equal to 32 bits. You can only operate the counter variables with non-blocking assignments and use the addition (+) and subtraction (-) binary operators to control the value of the counter.

Example:

```
reg [31:0] cnt1;
```

10.5.5 FSM Body Definition

The FSM body in the trigger module must have one, and only one, `always` block, which is sensitive to the posedge of the clock signal by default. Currently, you define two types of FSM body: state-less FSM and state-transition FSM.

A *state-less FSM* uses an if-then-else statement to define the condition for which the trigger occurs. No state definition and reset signal are needed.

Example:

```
always @(posedge clk) begin
  if_then_else_statement
end
```

In the example above, the if-then-else statement is omitted. See [If-then-else Statements](#) for details.

A *state-transition FSM* uses a case statement to control state transitions. The FSM uses the asynchronous reset mode by default and is sensitive to the negedge of the reset signal. You can specify the initial value of the state when the reset happens.

Example:

```
always @(posedge clk or negedge rstn)
if (!rstn)
    cstate <= rst_val;
else begin
    case_statement
end
```

In the example above, `rst_val` is the initial state value when the reset signal `rstn` is low. The case statement is omitted. See [Case Statements](#) for details.

10.5.6 Case Statements

Case statements define the state-to-state transition of FSM. Nesting of case statements is not supported. The supported syntax of the case statements is shown as follows:

Syntax:

```
case_statment ::= case(case_expression)
                    case_item {case_item}
                    endcase
case_item ::= case_item_expression : action_list
               | default : action_list
action_list ::= action_statement | begin action_statement_list end
```

Where,

- `case_expression` specifies an expression that is evaluated once and before any of the case item expressions. See [Supported FSM Expressions and Operators](#) for details of expressions.
- `case_item_expression` specifies an expression in supported that is evaluated and then compared with the case expression accordingly. Case statements in trigger modules must have one default branch.
- `action_statement` specifies an if-then-else statement or a simple action statement. An action statement can be an if-then-else statement or a simple action statement. See [If-then-else Statements](#) and [Simple Action Statements](#) for details.

- `action_statement_list` specifies a list of action statements in a `begin` block.

10.5.7 If-then-else Statements

An if-then-else statement selects and executes one or none of the action list, depending on the value of corresponding conditions. The supported syntax of the if-then-else statement is shown as follows:

Syntax:

```

if_then_else_statement ::= if_statement [ else_statement ] |
if_else_if_statement
if_statement ::= if (expression) action_list
else_statement ::= else action_list
if_else_if_statement ::= if (expression) action_list
                        { else if (expression) action_list }
                        [ else action_list ]
action_list ::= action_statement
              | begin action_statement_list end
action_statement ::= simple_action
                  | if_then_else_statement
action_statement_list ::= action_statement { action_statement }

```

Where,

- `if_statement` specifies a condition expression and an action list which is evaluated if the condition expression is true.
- `else_statment` specifies an action list which is evaluated if the condition expression is false.
- `if_else_if_statement` specifies the nesting of if statements.
- `expression` specifies an expression for the condition. See [Supported FSM Expressions and Operators](#) for details of expressions.
- `simple_statement` specifies a leaf statement. See [Simple Action Statements](#) for details of simple action statements.

10.5.8 Simple Action Statements

Only simple action statements can be leaf statements (terminals) and only non-blocking assignment is supported. A simple action statement can be any one of the following statement format:

- Assign a constant value to a state variable to define state transition.
- Assign a constant value to a counter variable.
- Change the value of a counter variable by adding or subtracting a constant value.
- Call the `$trigger` system task.

- Call the `$cmd()` system task. Only ATT supports the `$cmd()` system task.

The supported syntax of the simple action statement is shown as follows:

```
simple_action ::= state_var <= constant_value ;
              | counter_var <= constant_value ;
              | counter_var <= counter_var + constant_value ;
              | counter_var <= counter_var - constant_value ;
              | $trigger ;
              | $cmd() ;
```

10.5.9 Supported FSM Expressions and Operators

The syntax and semantic rules of FSM expressions are the same with the Verilog standard. However, only a subset of operators is supported. Bit selection after concatenation is not supported, such as `{a, b, c}[7:0]`.

ATTs support the following FSM expression syntax:

```
expression ::= variable_expression
              | value
              | unary_expression
              | binary_expression
              | ( expression )
              | { expression { , expression } }
variable_expression ::= simple_name
                     | simple_name [ decimal_number ]
                     | simple_name [ decimal_number : decimal_number ]
unary_expression ::= unary_operator expression
unary_operator ::= ~ | ! | & | |
binary_expression ::= expression binary_operator expression
binary_operator ::= + | - | < | > | <= | >= | == | != | & | | | ^ | && | ||
```

The following table lists the supported operators in FSM expressions.

Table 10-2: Operators in FSM Expressions Supported by Trigger

Operator	Description
Unary operators	
~	Bit-wide negation
!	Logical negation
&	Unary reduction and
	Unary reduction or
Binary operators	

Operator	Description
+	Addition
-	Subtraction
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
!=	Not equal to
&	Bit-wise and
	Bit-wise or
^	Bit-wise exclusive or
&&	Logical and
	Logical or

10.5.10 Lexical Conventions in FSM

This topic describes the lexical tokens used in source code of trigger FSMs and their conventions. The lexical tokens obey the lexical rules for Verilog language, on the while space, comment, operator, number, identifier.

- **White space**

White space contains the characters for spaces, tabs, newlines and formfeeds.

- **Comment**

Like Verilog, a one-line comment shall start with two characters “//” and end with a newline. A block comment shall start with “*” and end with “*/”, and it cannot be nested.

- **Operator**

Operators in FSMs are single- or double-character sequences, and they are used in expressions. See [Supported FSM Expressions and Operators](#) for details.

- **Number (value)**

Trigger FSMs support the following number syntax:

```

value ::= decimal_number
        | binary_number
        | octal_number
        | hex_number
decimal_number ::= unsigned_number
                | decimal_base unsigned_number
decimal_base ::= 'd | 'D
unsigned_number ::= decimal_digit { _ | decimal_digit }
decimal_digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
binary_number ::= [ size ] binary_base binary_digit { _ | binary_digit }
binary_base ::= 'b | 'B
binary_digit ::= x | X | 0 | 1
octal_number ::= [ size ] octal_base octal_number { _ | octal_number }
octal_base ::= 'o | 'O
octal_digit ::= x | X | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7
hex_number ::= [ size ] hex_base hex_digit { _ | hex_digit }
hex_base ::= 'h | 'H
hex_digit ::= x | X | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | A | b |
B | c | C | d | D | e | E | f | F
size ::= unsigned_number

```

x or X means any value is accepted as a match. For a constant value that has X, you can only use the “==” operator to a signal or to an expression.

10.5.11 Examples for FSM Programs

This topic provides three example FSM modules, which can be instantiated by vDbg to control design running.

Example 1: A State-less FSM Program

The following FSM program compares two 8-bit bus signals `A[7:0]` and `B[7:0]` as unsigned integer numbers and triggers if the value of the `A[7:0]` signal is greater than that of the `B[7:0]` signal.

```

// toggle.v
module toggle_AB(input wire clk
                 input wire [7:0] a, input wire [7:0] b);
always @(posedge clk) begin
    if (a[7:0] > b[7:0]) $trigger;
end
endmodule

```

The following vDbg Tcl commands read the `toggle.v` Verilog file, create the `t_inst` trigger instance with design signals connected to the inputs of the FSM module, `toggle_AB`, and enable the trigger instance.


```
% trigger -at -readfile toggle.v
% trigger -at -addinst t_inst -module toggle_AB -bind_ports {dut.clk}
{dut.A[7:0]} {dut.B[7:0]}
% trigger -at -enable t_inst
```

Example 2: A One-block FSM Program

The following example uses one block to describe an FSM. For clarity and flexibility, localparam symbolic constants are used to represent states of the FSM.

```
module toggle_mon(input wire clk, input wire rstn,
                 input wire [8:0] data, input wire ld);
localparam [3:0] STATE_INIT    = 4'b0000,
               STATE_LOAD     = 4'b0001,
               STATE_FETCH    = 4'b0010,
               STATE_EXECUTE  = 4'b0011;
reg [31:0] cstate;
always @(posedge clk or negedge rstn)
  if (!rstn)
    cstate <= STATE_INIT;
  else begin
    case (cstate)
      STATE_INIT: begin
        if (ld == 1'b1) cstate <= STATE_LOAD;
      end
      STATE_LOAD: begin
        if (data[5:0] > 20) cstate <= STATE_FETCH;
        else if (data[5:0] < 10) cstate <= STATE_EXECUTE;
      end
      STATE_FETCH: begin
        if (ld == 1'b0) cstate <= STATE_INIT;
      end
      STATE_EXECUTE: begin
        if (data[8:6] > 5) $trigger;
      end
      default: cstate <= STATE_INIT;
    endcase
  end
endmodule
```

Example 3: An FSM Program with A Counter

The following FSM program has two states, The FSM triggers if the `sig2` signal becomes high within 100 cycles or fewer after the `sig1` signal is high. The search for this pattern is restarted every time the `sig1` remains low for 100 cycles or more.

```

/* Trigger on sig1 high followed by sig2 high, within 100 cycles. The number of
cycles is counted by cnt1
*/
module toggle_counter(input wire clk, input wire rstn,
                     input wire sig1, input wire sig2);
reg [31:0] cstate;
reg [31:0] cnt1;

always @(posedge clk or negedge rstn)
    if (!rstn)
        cstate <= 2'b00;
    else begin
        case (cstate)
            2'b00: begin // wait until sig1 goes high
                if (sig1) begin
                    cnt1 <= 99;
                    cstate <= 2'b01;
                end
            end
            2'b01: begin
                if (sig2 && cnt1 > 10)
                    $trigger; // trigger if sig2 is high while counting
                if (cnt1 > 0) cnt1 <= cnt1 - 1;
                else cstate <= 2'b00; // start all over again
            end
            default: cstate <= 2'b00;
        endcase
    end
endmodule

```

10.6 Viewing the Waveform and Adjusting the Capture Controls

You can perform your desired step after the trigger event occurs:

- Check the signal values by using the `get_value` command.

```
get_value [<value_format>] <signal>
```

- If the physical or streaming probes are enabled, you can generate the waveform.
- Check the captured waveform that displays in the Waveform Viewer of the logic analyzer.
- Adjust the capture controls for the debug core trigger and run the design from where the running stops.

11 Managing Hardware Resources

VRM is a self-developed software system for device management of HuaPro systems. You can use VRM in the emulation mode only. `vrctl` is a tool for end users to manage VRM host information and query system status. The tool is packaged with `vDbg` in the HPE software. `vrctl` provides the following features to end users:

- Resource allocation
- Resource locking
- Multi-users at runtime
- Resource relocation

11.1 Introduction to VRM

VRM is a self-developed software system for resource allocation and management of HuaPro System. VRM can make a board as an efficient allocation unit and improve the flexibility of resource utilization. Multiple users can use the different allocation units in the system in the meantime or use the same allocation units in a controlled manner. The granularity of allocation can be board or FPGA.

The VRM software system includes `vrctl` and `vrmd`. `vrctl` together with `vDbg` runs on a runtime host and `vrmd` runs on a separate VRM host.

The following figure shows the architecture of a HuaPro system deployed with VRM.

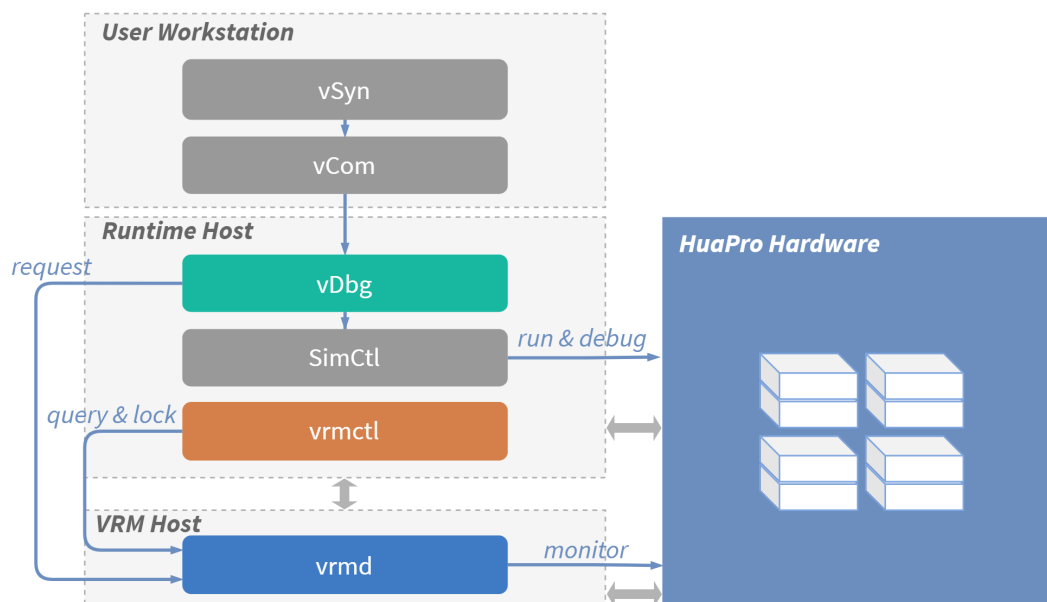


Figure 11-1: VRM Modules in HuaPro System

- **vrctl**

vrctl is a tool for end users to access VRM host information and query resources. The tool is packaged with vDbg in the HPE software. HPE is deployed on the runtime host. vrctl has the following features:

- Provides a Python interpreter `vrctl>>` for users to execute VRM commands and to check the output.
- Supports querying status of devices.
- Supports custom locking of specific resources across the cascaded devices based on the hardware configuration file.
- Returns a key after a successful request of resource locking. The key identifies the resource locking when vDbg connects to the hardware server.
- Supports dumping the system HDF that describes the actual resources and physical connections of the system. It provides a panoramic view of the system that the VRM host monitors. You can refer to the dumped system HDF to customize your hardware requirements of the DUT.

- **vrmd**

vrmd is a service process to manage devices. The host that vrmd is deployed is called the VRM host. Acted as the center of VRM, vrmd communicates with both the runtime host and each device in the system. vrmd can configure, operate, and allocate the devices as end users and administrators request. vrmd can also monitor the device status, provide usage statistics, and generate HDF based on the physical connections.

11.2 Starting vrctl

You can start vrctl in any one of the following ways:

- Execute the `vrctl` command to invoke the VRM client. When the tool is invoked, the command prompt changes to `vrctl>>`. In the command prompt, you can execute the Python commands listed in [vrctl Commands](#).

```
% vrctl
vrctl>>
```

vrctl logs you out when you use the `exit()` command line or enter Ctrl+D on the vrctl prompt.

- Specify a Python script file on the `vrctl` command line to start vrctl in the batch mode. In the script file, you can use the Python commands listed in [vrctl Commands](#).

```
vrctl <python_script>
```

vrctl logs you out when you use the `exit()` command line in the Python script file.

- If you have executed *setup.csh* or *setup.sh* that is provided in the HPE installation path, you can execute the shell commands listed in [vrctl Commands](#) in the bash or cshell prompt.

You can view all vrctl commands by executing `show_commands()`. If you want help information about a particular vrctl command, you can execute `help(<command_name>)` in the vrctl prompt after [Connecting to VRM](#). An example of the help information is as follows:

```
vrctl>> help(dump_hdf)

Help on function dump_hdf in module __main__:

dump_hdf(hdf_path: str, *args, **kw)
    Dump the description of the current system to the specified path.

    hdf_path: path to save the hdf file

    Return: True on success, otherwise return false.
```

11.3 vrctl Commands

The following table lists the vrctl Commands that you can use.

Table 11-1: A Summary of vrctl Commands

Feature	Python Command	Shell Command	Reference
VRM Host Configuration	<code>add_vrm_host</code>	–	Adding a VRM Host
	<code>set_default_vrm_host</code>	<code>vrctl_set_host</code>	Setting the Default VRM Host
	<code>show_vrm_hosts</code>	<code>vrctl_host</code>	Showing the VRM Host List
	<code>rm_vrm_host</code>	–	Removing a VRM Host
	<code>set_default_vrm_host_for_all_users</code>	–	Setting the Default VRM Host for All Users
VRM Connection	<code>connect</code>	–	Connecting to VRM
	<code>disconnect</code>	–	Disconnecting from VRM
System Information Query	<code>show_system</code>	<code>vrctl_system</code>	Querying System Resource Status

Feature	Python Command	Shell Command	Reference
Runtime Session Management	<code>dump_hdf</code>	<code>vrml_dump_hdf</code>	Dumping the System HDF
	<code>show_session</code>	–	Querying Session Information
	<code>rm_session</code>	–	Removing a Session
Resource Locking	<code>lock_resource</code>	–	Locking Hardware Resources
	<code>show_locked_resource</code>	–	Showing Locked Hardware Resources
	<code>unlock_resource</code>	–	Unlocking Hardware Resources

11.4 VRM Host Configuration

Each VRM host manages one HuaPro hardware system. When you need to use multiple hardware systems, you can use `vrmlctl` to manage the VRM host list (as shown in the following figure). VRM host configuration can facilitate the connection between the runtime host and the VRM host and the switch among different hardware systems.

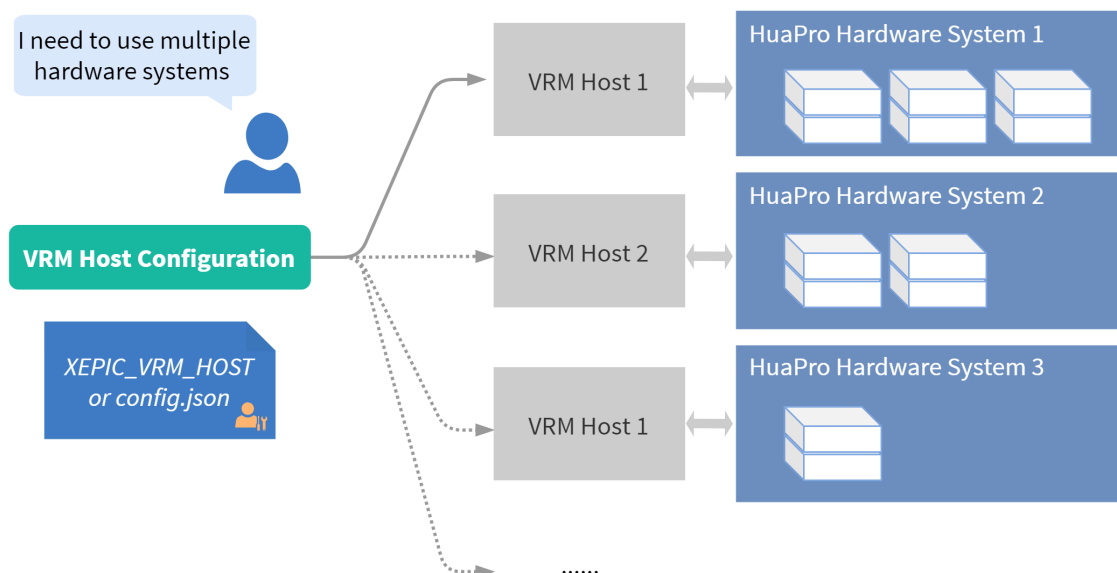


Figure 11-2: VRM Host Configuration

VRM supports the following ways to configure the default VRM host. `vrmlctl` connects to the default VRM host when you use no parameters on the `connect` command line.

- Use the `set_default_vrm_host` command in `vrmlctl` to set the default VRM host. `vrmlctl`

saves the VRM host information in `~/.vrm/config.json`.

- Root users use the `set_default_vrm_host_for_all_user` command in `vrctl` to set the default VRM host information for all users. `vrctl` saves the VRM host information in `/etc/vrm/config.json`.

11.4.1 Adding a VRM Host

You can add the VRM hosts by using the `add_vrm_host` command in `vrctl`. The first added VRM host is the default VRM host when no parameter is on the `connect` command line.

```
add_vrm_host(<vrm_ip>, <vrm_port>)
```

Example:

The following command line adds two VRM hosts to the VRM host configuration file. `vrctl` connects to the first added VRM host.

```
vrctl>> add_vrm_host("10.10.70.70", 9000)
Add vrm host[10.10.70.70:9000] successfully.
True
vrctl>> add_vrm_host("10.10.70.71", 9000)
Add vrm host[10.10.70.71:9000] successfully.
True
vrctl>> connect()
Connect to vrm host[10.10.70.70:9000] successfully.
```

The VRM host configuration file is created as follows.

```
{
  "defaultHostId": 1,
  "hosts": [
    {
      "hostname": "hvg-sw01",
      "id": 1,
      "ip": "10.10.70.70",
      "port": 9000
    },
    {
      "hostname": "hvg-sw02",
      "id": 2,
      "ip": "10.10.70.71",
      "port": 9000
    }
  ]
}
```

11.4.2 Setting the Default VRM Host

By default, `vrmmctl` uses the first VRM host that you added as the default one. You can set the default VRM host in either of the following ways:

- Use the `set_default_vrm_host` Python command in the `vrmmctl` prompt.

```
set_default_vrm_host(<vrm_ip>, [<vrm_port>])
```

- Use the `vrm_set_host` shell command.

```
vrm_set_host <vrm_ip> [<vrm_port>]
```

If not specified, the default port is 9000.

Examples:

The following command line sets the default VRM host to `10.10.70.70:9000` in the `vrmmctl` prompt.

```
vrmmctl>> set_default_vrm_host("10.10.70.70", 9000)
Set vrm host[10.10.70.70:9000] as default vrm host successfully.
```

The following command line sets the default VRM host using the shell command.

```
% vrm_set_host 10.10.70.70 9000
Vrm host list is empty.
Successfully added VRM host[10.10.70.70:9000].
Successfully set VRM host[10.10.70.70:9000] as default VRM host.
```

11.4.3 Showing the VRM Host List

Execute the `show_vrm_host` Python command or the `vrm_host` shell command without any option to show the information of the VRM hosts that you added. The VRM host list shows the following information:

- VRM host IP
- Whether the VRM host is the default
- Port number
- Hostname
- Board Index
- FPGA Status (shown in a binary number)
 - The width indicates the number of FPGAs on the board.
 - Each bit corresponds to FPGA A, B, C, and D from the highest to the lowest.

- The status `0` means the FPGA is available. `1` means in-use. `-` means disabled.

**NOTE**

When vDbg works in the non-VRM mode, do NOT rely on the reliability of the FPGA status shown by `show_vrm_host`.

In the following example, `sw02` is the default VRM host which manages two boards. On each board, there are two available FPGAs. `sw01` manages two boards.

```
vrctl>> show_vrm_host()
```

Vrm Host Ip	Is Default	Port	Hostname	Board Index	FPGA Status
10.10.70.70	Yes	9000	sw02	0	00
				1	00
10.10.70.71	No	9000	sw01	0	01
				1	-

11.4.4 Removing a VRM Host

To remove a VRM host from the list, use the `rm_vrm_host` command:

```
rm_vrm_host(<vrm_ip>, [<vrm_port>])
```

If not specified, the default port is 9000.

11.4.5 Setting the Default VRM Host for All Users

Root users can also set the default VRM host for all users by using the `set_default_vrm_host_for_all_user` command in vrctl:

```
set_default_vrm_host_for_all_user(<vrm_ip>, [<vrm_port>])
```

If not specified, the default port is 9000.

**NOTE**

`set_default_vrm_host_for_all_user` adds the VRM host information in `/etc/vrm/config.json` for all users. But `~/vrm/config.json` that users created by using the `add_vrm_host` command has higher priority.

11.5 VRM Host Connection

When the connection with VRM host is built, you can use `vrctl` to download the hardware configuration file.

11.5.1 Connecting to VRM

To connect the host where you start `vrctl` to the VRM host, use the `connect` command in `vrctl`.

```
connect([<vrm_ip>, <vrm_port>])
```

If the root user or you have set the default VRM host, you can use no parameters on the command line. The default a VRM host that you set has a higher priority.

Example:

The following command line connects the host PC/server to the VRM host `10.10.70.70:9000`.

```
vrctl>> connect("10.10.70.70", 9000)
Connect to vrm host[10.10.70.70:9000] successfully.
```

11.5.2 Disconnecting from VRM

If you want to connect to another VRM host, you need to disconnect from the current VRM host and then connect to another one. To disconnect from VRM, use the `disconnect` command without any option in `vrctl`.

Example:

```
vrctl>> disconnect()
Disconnect successfully.
```

11.6 System Information Query

`vrctl` can show real-time system resource information of the hardware that the VRM host

manages, including:

- VRM host IP and port number
- Board index and type
- Board SN number and IP
- Board usage status
 - `HEALTHY` : The board works properly.
 - `ONLINE&UNHEALTHY` : The board can connect to vrmd, but it works improperly.
 - `OFFLINE` : The board cannot connect to vrmd.
- Clock board index: the master board in a cascade system
- FPGA usage status
 - `available`
 - `locking`
 - `in-use`
- Daughter board connection
- Cable connection

For details, see [Querying System Resource Status](#).

VRM also supports dumping the system description file in the `.hdf` format. The system description file provides a full view of the hardware in the system that VRM manages. You can learn about the actual hardware configuration from the dumped system HDF. You can also refer to the dumped HDF to specify the hardware resource requirements of the DUT. For details, see [Dumping the System HDF](#).

Apart from viewing the printout in the command prompt, you can check system status from Web. For details, see [Viewing System Status](#).

11.6.1 Querying System Resource Status

To query the resource status, use either of the following ways after connecting the host PC/server to the VRM host.

- Execute the `show_system` Python command in `vrmdctl`.

```
show_system([detail = True])
```

Where `detail = True` queries information about daughter boards and cable connection.

- Execute the `vrmd_system` shell command without any option. Detailed information is printed by default.

VRM reports the resource information in the following format.

```
VRM Host:<host> Port:<port>. VRM System has <board_number> board(s).
Board <board_index>: <board_type> <total_number> FPGA(s). Clock Board: <clock_board_index>
SN:<board_SN> IP:<board_IP> Status:<board_status>
```

```
=====
|  FPGA   |  User   |  PID   | Card Connector | Design   | ElapseTime | Status   |
|-----|
|<FPGA_index>|<username>|<PID>| [<PHC_index>] | <top_name>|<hh:mm:ss>|<FPGA_status>|
=====
```

```
.....
```

In the following example, the VRM host `10.10.70.70:9000` manages two P2E devices. The master board is board 0. Harry is using board 0 FPGA C and D to run the design `prim`. The design uses the daughter board that connects to PHC 0.C.2. You can use the other FPGAs and daughter board connectors on board 0 and 1. FPGA D on Board 1 is disabled by system administrators, so `vrmtcl` does not show its information.

```
% vrmtcl
vrmtcl>> add_vrm_host("10.10.70.70")
Add vrm host[10.10.70.70:9000] successfully.
vrmtcl>>
vrmtcl>> connect()
Connect to vrm host[10.10.70.70:9000] successfully.
vrmtcl>>
vrmtcl>> show_system(detail = True)
VRM Host:10.10.70.70 Port:9000. VRM System has 2 board(s).
Board 0: P2E 4 FPGA(s). Clock Board: 0 SN:xxx IP:10.20.23.11 Status:HEALTHY
=====
|  FPGA   |  User   |  PID   | Card Connector | Design   | ElapseTime | Status   |
|-----|
|  0      |  --     |  --     |  []            |  --     |  --        | Available |
|-----|
|  1      |  --     |  --     |  []            |  --     |  --        | Available |
|-----|
|  2      |  Harry  | 268878  |  [2]           |  prim    | 00:19:16   | Locked    |
|-----|
|  3      |  Harry  | 268878  |  [2]           |  prim    | 00:19:16   | Locked    |
|-----|
Board 1: P2E 4 FPGA(s). Clock Board: 0 SN:xxx IP:10.20.23.12 Status:HEALTHY
=====
|  FPGA   |  User   |  PID   | Card Connector | Design   | ElapseTime | Status   |
|-----|
|  0      |  --     |  --     |  []            |  --     |  --        | Available |
|-----|
|  1      |  --     |  --     |  []            |  --     |  --        | Available |
|-----|
|  2      |  --     |  --     |  []            |  --     |  --        | Available |
|-----|
...
Cable Information:
PHC_CABLE_1000 (0.A.7, 0.B.7)
PHC_CABLE_1000 (0.A.15, 0.B.16)
PHC_CABLE_1000 (0.A.16, 0.D.16)
vrmtcl>>
```

11.6.2 Dumping the System HDF

To dump the HDF file, use either of the following methods:

- Use the `dump_hdf` Python command in `vrctl`.

```
dump_hdf(<hdf_path>)
```

- Use the `vr_dump_hdf` shell command.

```
vr_dump_hdf <hdf_path>
```

Example:

```
vrctl>> dump_hdf("./hw.hdf")
Download hdf successfully! Path: ./hw.hdf.
vrctl>>
```

For more information about the HDF definition, see [Hardware Description File](#).

11.6.3 Viewing System Status

VRM supports viewing real-times status of HuaPro P2E system from Web page. Visit the VRM Web page with the URL `http://<vr_host_ip>:8080`, where `<vr_host_ip>` specifies the VRM host corresponding to the system. The Web page is read-only, and no operation is supported.

Only the Chrome browser is supported. It is recommended using the Chrome version 89.0.4389.90 which is fully tested.

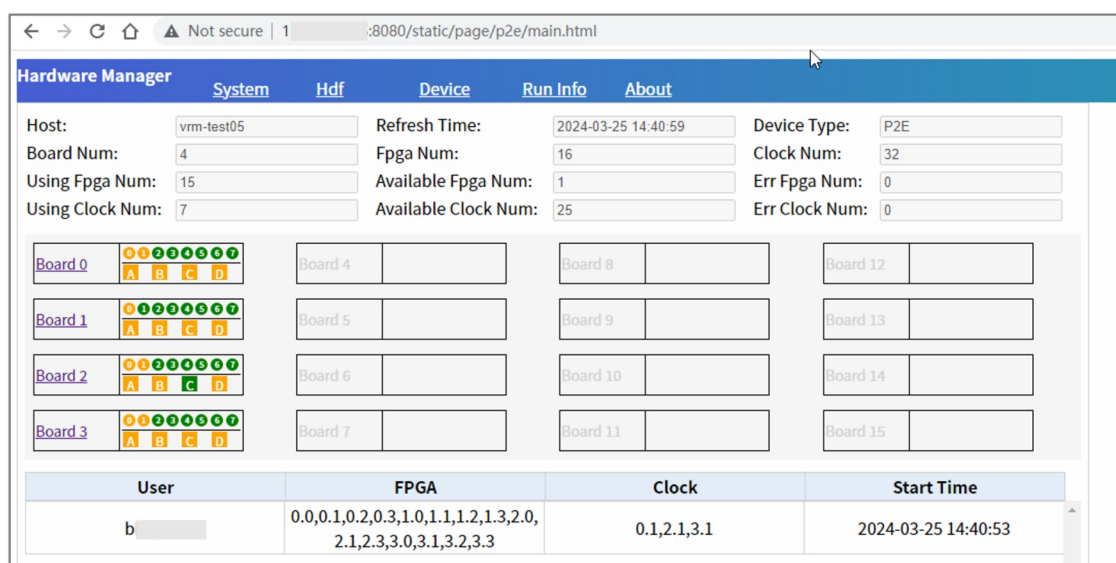


Figure 11-3: Viewing System Status From VRM Web Page

VRM Web page provides the following menus:

- **System**

Shows an overview of the system status including the occupation of FPGAs and global clocks by board and user.

The circular icons indicate the status of global clocks. GC0 is a reserved system clock. The square icons indicate the status of FPGAs. The icon changes from green to yellow when the respective global clock or FPGA is being used.

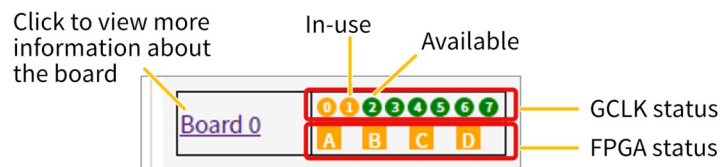


Figure 11-4: Viewing Board Status

You can click the board index to view detailed information about the board of interest.

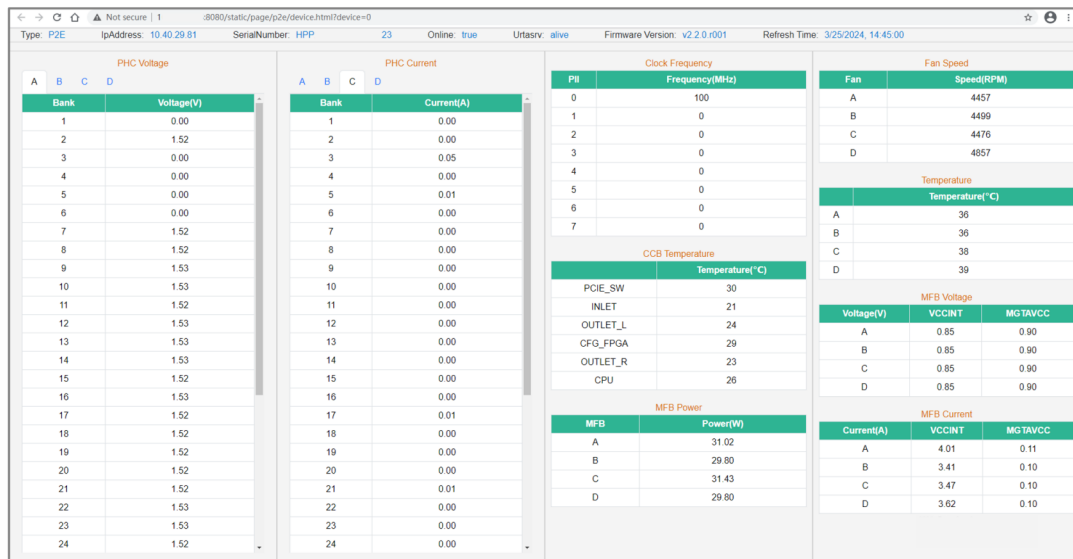


Figure 11-5: Viewing Device Status From VRM Web Page

- **Hdf**

Shows the system HDF.

- **Device**

Shows the configuration of each device in the system.

- **Run Info**

Shows the run information of vrmd.

11.7 Runtime Session Management

vrnctl supports querying and managing your vDbg sessions after you connect to a VRM host.

11.7.1 Querying Session Information

Use the `show_session` command in vrnctl to query the resource usage of the vDbg sessions connects to the VRM node.

Syntax:

```
show_session()
```

Example:

```
vrnctl>> show_session()
VRM HOST: sw-cloud1. IP: 11.11.11.11
Total Sessions : 1

Session Id : "xxxxxxxxxx"
Created by: "Harry@hvg-ow3" at "2023-05-08 15:38:00"
Peer address: "33.33.33.33:21328" Peer pid: "3713259"
Design name: "prim". Elapsed time: 00:10:00
=====
| Expected Fpga | Actual Fpga |
|-----|
| 0.[0,1] | 1.[0,1] |
|-----|
```

11.7.2 Removing a Session

Use the `rm_session` command in vrnctl to remove the session that exits unexpectedly. VRM releases the resource occupied by the abnormal session immediately. Note that you cannot remove an alive session.

```
rm_session("<session_id>")
```

You can check the session ID by using the `show_session` command.

Examples:

The following command line removes an abnormal session successfully.

```
vrnctl>> rm_session("0000000000")
Remove session successfully
```

The following command line fails to remove an alive session.

```
vrnctl>> rm_session("0000000000")
Failed to remove session. Reason: session is alive.
```

11.8 Resource Locking

VRM supports resource locking when vDbg works in the VRM mode. If the resources you want to lock are available, VRM returns a locking key to identify the successful locking operation. You can specify the locking key on the `hw_server` command at runtime to apply for the reserved resources. For more information about using the locking key, see [Using Locked Resources](#).

11.8.1 Locking Hardware Resources

To lock hardware resources, use the `lock_resource` command line in `vrnctl`:

```
lock_resource (design_path = <design_path>, timeout = <timeout>, mode = <mode>,
savekey = <save_key_switch>, path = <key_path>, location = <resource_location>,
relocation = <relocation_switch>, ecn = <number>)
```

- `design_path = <design_path>`

Specifies the path of your runtime working directory. `vrnctl` needs the hardware configuration file and the design information for allocating and locking resources. Only this option is required. If the other options are not specified, VRM locks resources as follows by default.

- The timeout is 5 minutes.
- The locking mode of the required resources is single-session.
- The locking key is printed out directly.
- The scope where VRM searches suitable resources for locking is the whole system.
- The relocation feature is disabled.

- `timeout = <timeout>`

Specifies the time limitation for the locked resources. The time unit can be `s`, `m`, `h`, and `d`. The timeout must be less than 7 days, 168 hours, 10080 minutes or 604800 seconds. If not specified, the timeout is 5 minutes by default.

- `mode = <mode>`

Specifies the locking mode of the required resources, `single_session` (default) or `multi_session`.

- `savekey = <save_key_switch>`

Chooses whether to save the locking key. Valid values are `True` and `False` (default).

- When you set `savekey = True`, VRM saves the locking key in a file.
- When you set `savekey = False` or do not set this option, VRM only prints out the locking key.
- `path = <key_path>`
Specifies the path to save the locking key file. The name of the key file is fixed to `./top_module.lkey`. If you set `savekey = True` but do not specify the path, VRM saves the locking-key file in the design path by default.
- `location = <location>`
Specifies the scope where VRM searches suitable resources for locking. For example, `[0.1-0.3, 1]` means that VRM tries to lock resources in board 0 FPGAs A-C and all FPGAs on board 1.
- `relocation = <relocation_switch>`
Selects whether to enable the relocation function. Valid values are `True` and `False` (default). By default, the relocation function is disabled.
 - When you set `relocation = True`, VRM will search for resources that are equivalent to the specified resources if the specified resources are unavailable.
 - When you set `relocation = False`, VRM will return failure if the specified resources are unavailable.
- `ecn = <number>`
Specifies the number of extra global clocks to be locked. Valid value ranges from `0` (default) to `6`. This option is only used for HuaPro P2E. By default, only the global clock that the design uses is locked. If the design uses the GC1 global clock, setting the parameter to `3` locks four clocks, GC1-GC3.

Example 1: VRM searches in board 0 FPGAs A-C and all FPGAs on board 2 and 3 for suitable resources that meet the requirements of the design. The design is in the current working directory. VRM saves the locking key in `./top_module.lkey` and will release the resources after two hours.

```
vrmsctl>> lock_resource(design_path = "./", timeout = "2h", mode =
"multi_session", savekey = True, path = "./", location = "[0.1-0.3, 1],[2, 3]")
Resources locked by "user1@10.10.80.80" at "22-11-22 16:17:18" successfully.
Locking Key: locking_key_1.
KeyFile path: ./top_module.lkey.
Expiration time: 22-11-22 18:17:18.
vrmsctl>>
```

Example 2: VRM searches in all available FPGAs for suitable resources that meet the requirements of the design. The design is in the current working directory. VRM only prints out

the locking key and will release the resources once you connect to.

```
vrctl>> lock_resource(design_path = "./", timeout = "2h", mode =
"single_session")
Resources locked by "user1@group1-01" at "2023-03-31 16:17:20" successfully!
Locking Key: 00000000-0000-0000-0000-000000000000
Expiration time: 2023-03-31 18:17:20
'00000000-0000-0000-0000-000000000000'
vrctl>>
```

11.8.2 Showing Locked Hardware Resources

You can execute the `show_locked_resource` command without any option in `vrctl` to list your locking keys.

Example:

```
vrctl>> show_locked_resource()
The resources locked by someone@somehost are as follows:
=====
| ID |      Key      | Design Name | Expiration Time | Design Path |
-----
| 0 | 00000000-0000-0000-0000-000000000000 | dut_top1 | 2023-03-31
18:17:20 | /home/user1/work/design |
-----
| 1 | 00000000-0000-0000-0000-000000000001 | dut_top2 | 2023-03-31
19:17:20 | /home/user2/work/design |
=====
Locking Key: 00000000-0000-0000-0000-000000000000
Resources:
Clock PLL1 of Board 0
FPGA [0, 1] of Board 0
Card CPAC_16G_V1 connected at 0.A.22

Locking Key: 00000000-0000-0000-0000-000000000001
Resources:
Clock PLL1 of Board 1.
FPGA [0, 1] of Board 1.
Card CPAC_16G_V1 connected at 1.A.22.

vrctl>>
```

11.8.3 Unlocking Hardware Resources

VRM releases the locked resources automatically in the following situations:

- The resources are locked in the single session mode, and you start to use the resources.
- The resources are locked in the multi session mode, and the specified timeout expires.

You can manually cancel the hardware resource locking by using the `unlock_resource` command line in `vrctl`:

```
unlock_resource (<locking_key>)
```

Example:

```
vrctl>> unlock_resource("00000000-0000-0000-0000-000000000000")
Remove the locked resources with the key ["00000000-0000-0000-0000-000000000000"] successfully.
vrctl>>
```

11.8.4 Design Relocation

When relocation is enabled and the hardware resources that are configured in `hw-config.json` is unavailable, VRM searches for an alternative that provides the same interconnection and daughter board resources. If any idle board boards in the same system meet the requirements, VRM relocates the design to the boards.

As shown in the following figure, `hw_config.json` requires resources of three FPGAs, two PHC cables, and one DDR daughter board. In the hardware system, FPGA 0.A is in-used and board 1 FPGA A is locked. Resources that are required is unavailable. VRM searches for an alternative and finally finds matched resources.

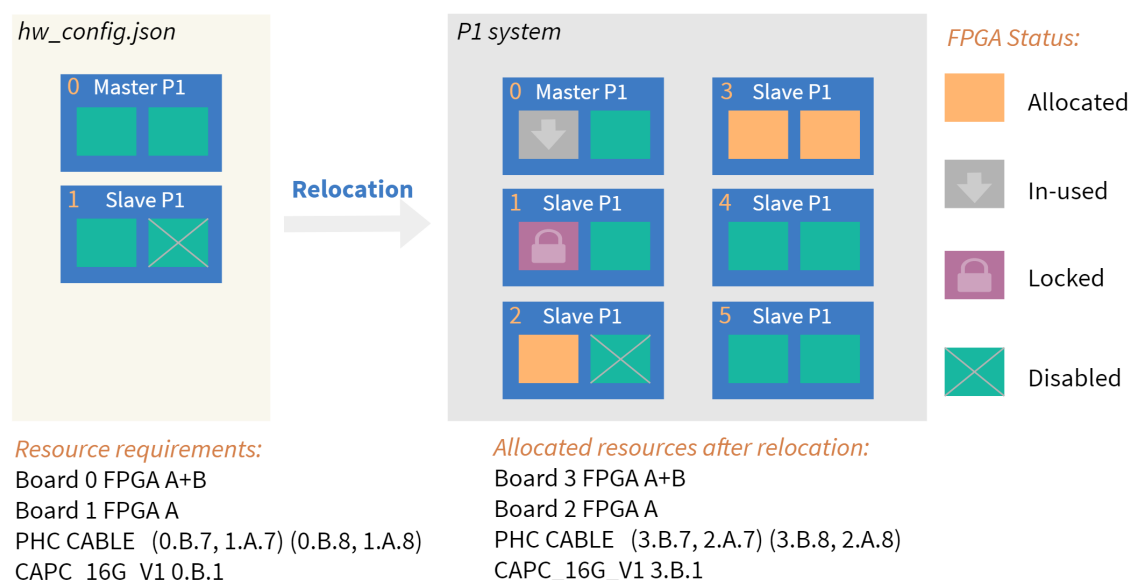


Figure 11-6: Resource Relocation Example in the P1 System

Currently, resource relocation has the following limitations:

- For HuaPro P1, the granularity of relocation is board no matter the system has only one device or cascaded devices.

- For HuaPro P2E, the granularity of relocation is FPGA when the system has only one device and is board when the system has cascaded devices.
- In a P1 cascade system, if the primary clock of a multi-board design is occupied by another design, VRM cannot relocate any resource for the design. A workaround is to change the primary clock at compile time as introduced in [Defining the Primary Clocks](#).
- In a P2E cascade system, if all global clocks are occupied, VRM cannot relocate any resource for a multi-board design.

12 Debug Station GUI

This chapter describes the runtime debugger GUI and introduces how to use the GUI.

TIP

Depending on the instrument logic for debugging that you have inserted at compile time, some operations on the GUI might be unavailable.

12.1 Starting the Runtime Debugger vDbg GUI

Start the runtime debugger vDbg GUI using the following command line. After GUI starts, you can operate on the GUI or use Tcl commands on the Console part of the GUI.

```
vdbg -gui
```

NOTE

The Runtime Debugger GUI needs to invoke open-source fonts. If you cannot start the GUI because of font configuration warnings, contact XEPIC for support.

You can quit from the GUI in one of the following three ways.

- Click **File > Exit**.
- Click the **Close** icon.
- Execute the `exit` command in the *Console* pane of the GUI.

12.2 GUI Overview

When the Debug Station GUI is started, only the *Project* pane and *Control* pane are displayed as illustrated in the following figure.

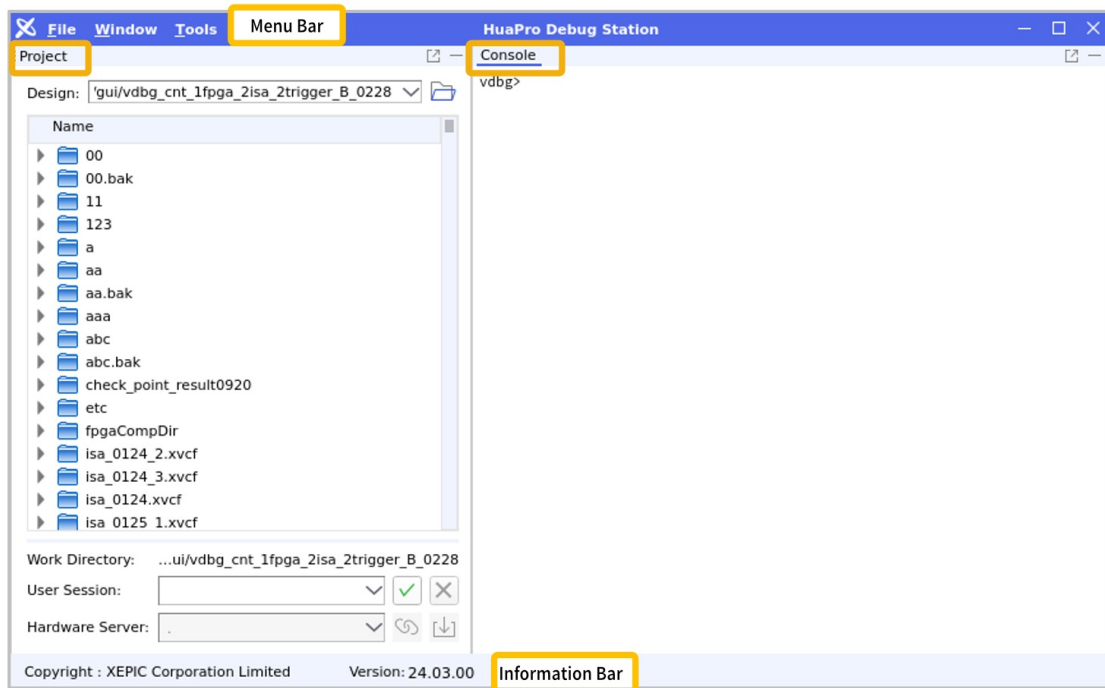


Figure 12-1: Initial Screen

GUI Component	Description
Menu bar	Closes the GUI and exits vDbg. Adjusts the GUI layout and window size.
<i>Project</i> pane	Displays the files and directories in the specified design path and downloads bitstream files.
<i>Console</i> pane	An interactive text area that collects input Tcl commands and gives the feedback information of GUI. You can recognize the input command, execute the command, and identify whether operations succeed or have errors.
Information bar	Displays the copyright and version information. For time-consuming operations, process information is displayed, too.

More panes are displayed after you successfully load a design.

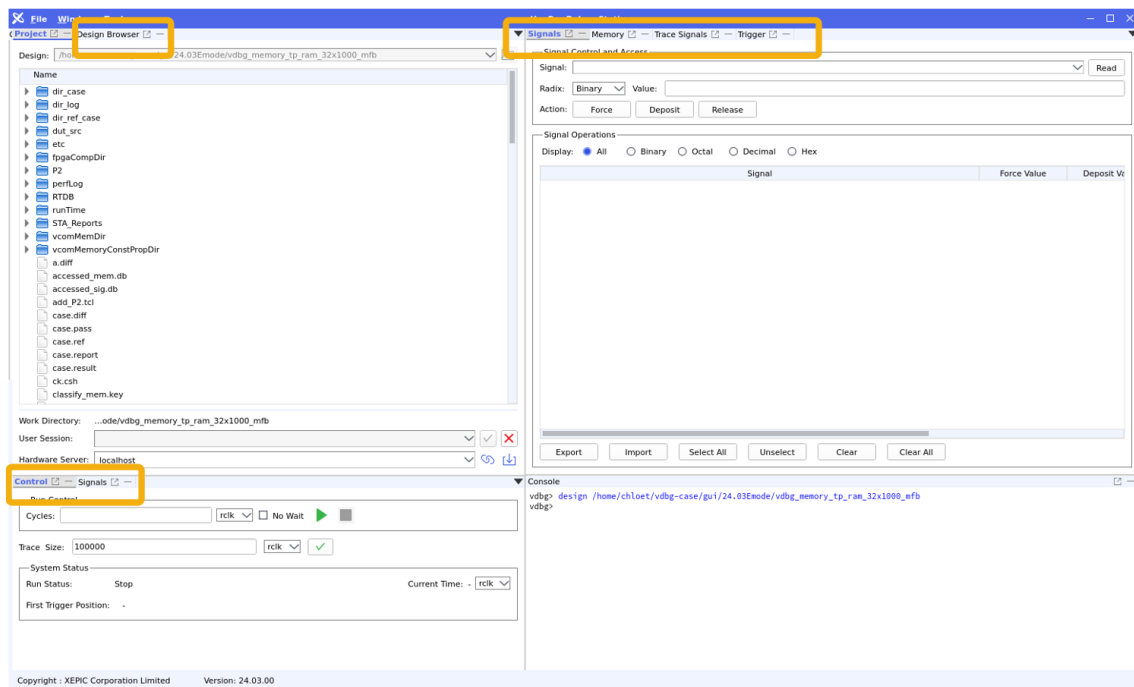


Figure 12-2: GUI Overview

Table 12-1: Component Description of Debug Station GUI

GUI Component	Description
<i>Design Browser</i> pane	Displays the hierarchy tree of a design or designs.
<i>Control</i> pane	Controls the design running and checks system status.
<i>Signals</i> pane	Forces, deposits, releases and gets signal values.
<i>Memory</i> pane	Performs memory backdoor read and write.
<i>Trace Signals</i> pane	Generates waveforms
<i>Trigger</i> pane	Sets up ATT triggers.
<i>Signals</i> pane	Displays signal names in a specified hierarchy.

12.3 Running and Debugging a Design

This section introduces how to run and debug a design from the GUI.

12.3.1 Downloading the Design to the Hardware from the GUI

This topic guides you through downloading a design in the emulation mode.

Prerequisites

Make sure that you have started GUI and waited for about three seconds.

Procedure

Perform the following steps to download a design to hardware from the *Project* pane of the GUI:

1. (Optional) By default, the design path is the working directory where you start vDbg. If required, you can click the **Folder**  icon to change the design path.

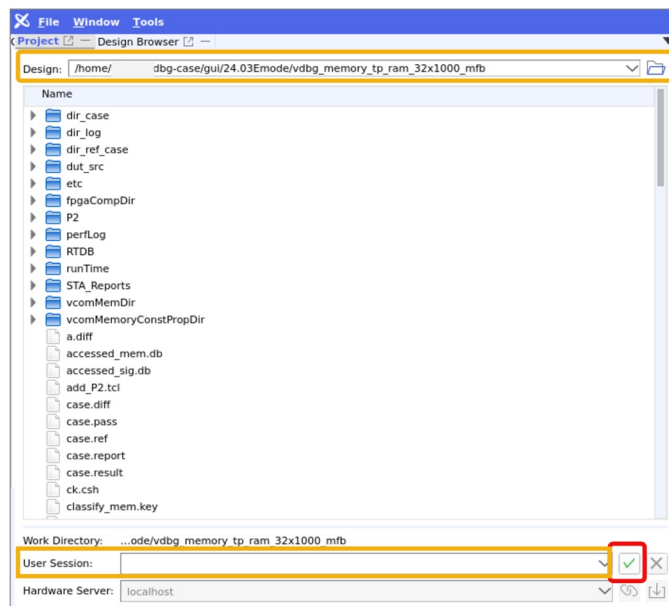


Figure 12-3: Changing the Working Directory

2. (Optional) Specify a user session in either of the following two ways:
 - Enter the name of the session directory.
 - Select an available session from the drop-down list.

By default, vDbg creates soft links to the design files and generates log files in the current working directory. If you specify a user session, the links and files are saved in the session directory named `xxx.session`.

3. In the **User Session** area, click the **Apply**  icon to load the design.

The **Apply**  icon becomes gray and the **Hardware Server** area is enabled when the design opens successfully.

4. In the **Hardware Server** area, click icon  to enable the connection between your workstation and the hardware server.

Only the localhost is supported currently.

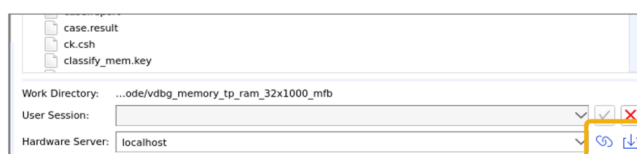
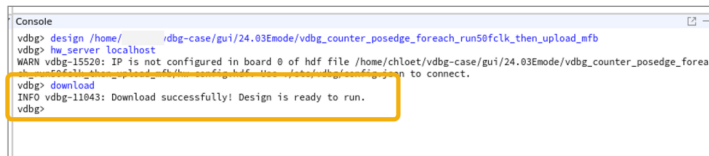


Figure 12-4: Enabling the Connection Between Workstation and Hardware Server

When the design opening succeeds, the **Control** pane and other panes appear.

5. (Optional) In the *Console* pane, execute vDbg commands to set clock frequencies and PHC bank voltages.
6. Click the **Download**  icon to start the download.



```

vdbg> design /home/.../vdbg-case/gui/24.03Emode/vdbg_counter_posedge_foreach_run50fclk_then_upload_nfb
vdbg> hw_server localhost
vdbg> vdbg-15520: IP is not configured in board 0 of hdf file /home/chloet/vdbg-case/gui/24.03Emode/vdbg_counter_posedge_forea
vdbg> download
INFO vdbg-11843: Download successfully! Design is ready to run.
vdbg>
  
```

Figure 12-5: Download Success



TIP

If the download fails, check and make sure that the system connection is built and the specified design directory has a valid compiled database.

12.3.2 Running the Design on the Hardware from the GUI

This topic guides you through running a design in the *Control* pane.

Prerequisites

Make sure you have performed the following operations before you start the procedure:

- If the design has signals that control the design running, force the signal values by referring to [Forcing Signals from the GUI](#).
- To use ATTs, create and enable trigger instances by referring to [Setting up Triggers from the GUI](#).

Procedure

Perform the following steps to run the design on the hardware from the **Control** pane:

1. In the **Run Control** area, run the design in either of the following ways:

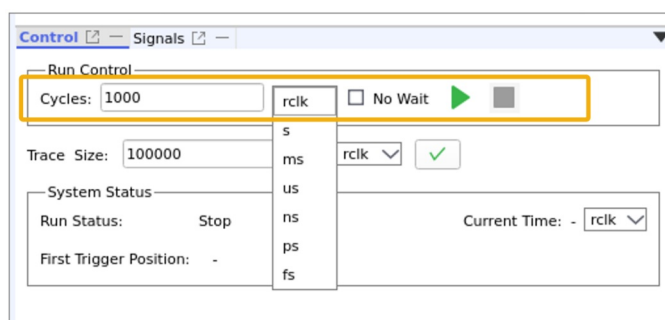


Figure 12-6: Specifying the Running Time

- Specify the number of cycles in the **Cycles** field and select the time unit from the drop-down list. Then click the **Run** ► icon to run the design.
- Select the **No Wait** checkbox and then click the **Run** ► icon to run the design in the non-blocking mode.

TIP

Keep the **Cycles** field empty when you run the design in no wait mode. Otherwise, the design cannot run.

2. (Optional) You can get the current running time of design from the **System Status** area.

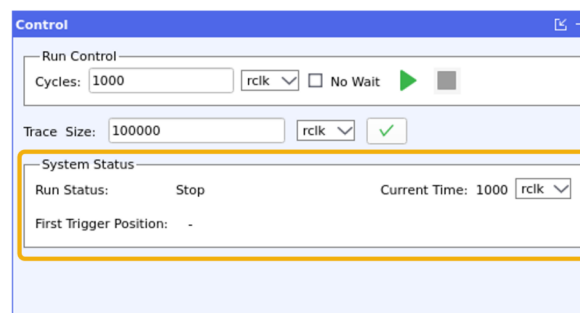


Figure 12-7: Checking Running Time and Cycle

The **System Status** area displays the current total running time of the design in the default time unit, rclk, or in your selected time unit.

3. Stop the design in one of the following three ways.
 - The design stops automatically if the running time or cycle number comes to the specified one.
 - If you arm any triggers in the design, the design stops automatically when a trigger occurs.
 - Click the **Stop** ● icon to stop the running of design manually.

12.3.3 Forcing Signals from the GUI

This topic guides you through forcing signals in the **Signals** pane.

Prerequisites

Make sure you have performed the following operations before you start the procedure:

- [Adding Nets for Signal Access](#)
- [Downloading the Design to the Hardware from the GUI](#)

Procedure

Perform the following steps to force signal values from the GUI:

1. Enter the name of the signal in the **Signal** field or select a signal from the drop-down list.

If you specify a signal name representing a bus signal without part-selection notation, vDbg forces all the bits. You can force part of bits for a bus signal by using part-selection notation as the signal name, such as `abc[7:4]`.

The drop-down list shows the read and write nets that are defined at compile time.

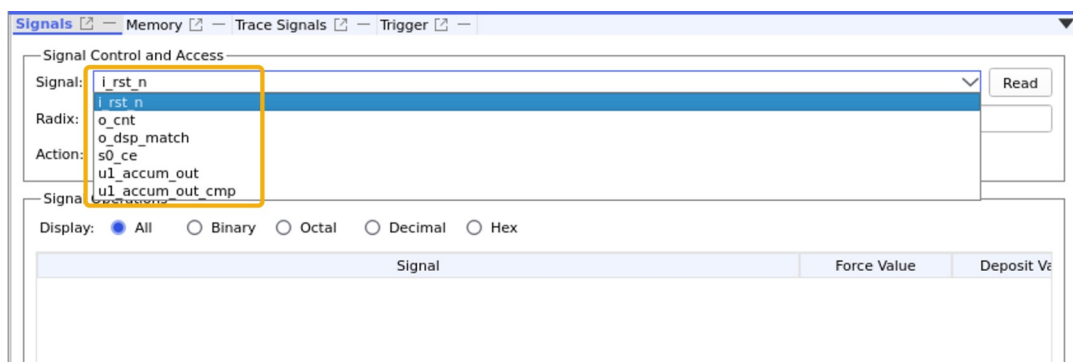


Figure 12-8: Specifying the Signal Name

2. Select the radix from the **Radix** drop-down list and enter the value in the **Value** field without prefix letter.

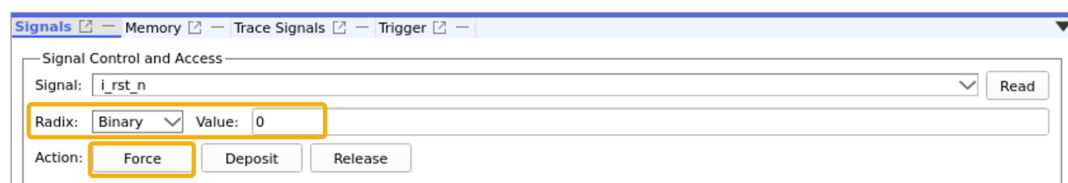


Figure 12-9: Specifying the Signal Value

3. Click **Force**.

The forced signal and corresponding value are displayed in the **Signal Operations** table. vDbg forces the signal to the value in the specified radix format until the signal is explicitly released. Signal values that are deposited can be forced.

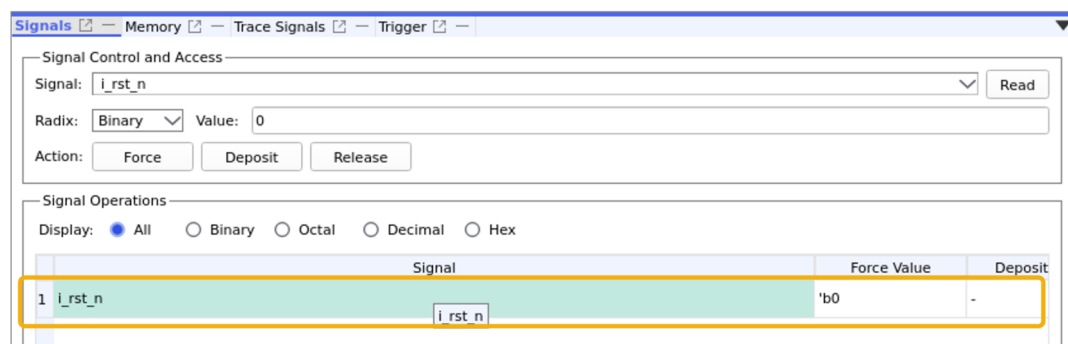


Figure 12-10: Signal Displayed in the Table

12.3.4 Depositing Signals from the GUI

This topic guides you through depositing signals in the **Signals** pane.

Prerequisites

Make sure that you have performed the following operations before you start the procedure.

- [Adding Nets for Signal Access](#)
- [Downloading the Design to the Hardware from the GUI](#)

Procedure

Perform the following steps to deposit the signal values from the GUI:

1. Enter the name of the signal in the **Signal** field or select a signal from the drop-down list.
2. Select the radix from the **Radix** drop-down list and enter the value in the **Value** field without prefix letter.
3. Click **Deposit**.

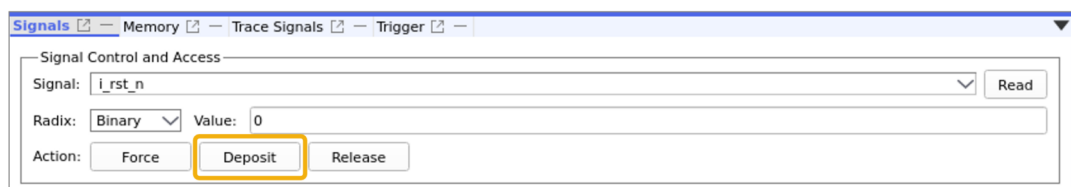


Figure 12-11: Depositing the Signal Value

The deposited signal and corresponding value displays in the **Signal Operation** area. vDbg forces the signal to the value in the specified radix format for one clock cycle and then the design can drive the signal. Signal values that are forced cannot be deposited.

12.3.5 Setting up Triggers from the GUI

This topic guides you through setting up ATTs and debug a design from the Debug Station GUI.

Prerequisites

Make sure you have performed the following operations before you start the procedure:

1. [Adding ATT Nets at Compile Time](#)
2. [Setting Up the Environment for ATT](#)
3. [Preparing ATT Trigger Modules](#)
4. [Executing Callbacks of vDbg Debugging Scripts](#)
5. [Downloading the Design to the Hardware from the GUI](#)

Procedure

Perform the following steps to create and enable triggers from the **Trigger** pane:

1. Click the  icon to select a Verilog file or files that define trigger modules and then click **Read** to load the file or files.

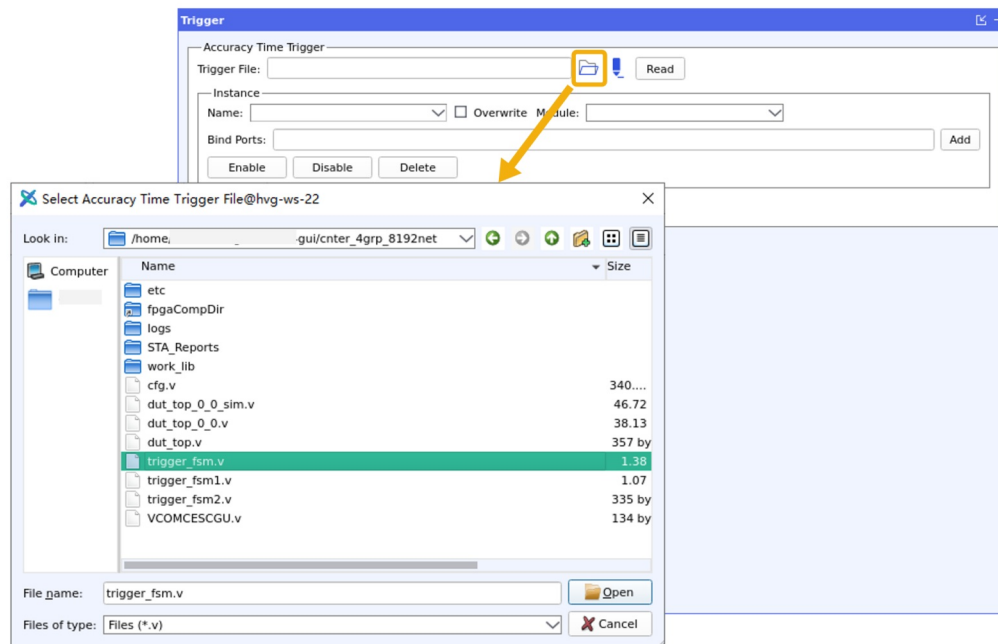


Figure 12-12: Selecting the Trigger Module

2. In the **Instance** area, instantiate trigger modules and bind ports in the following steps.

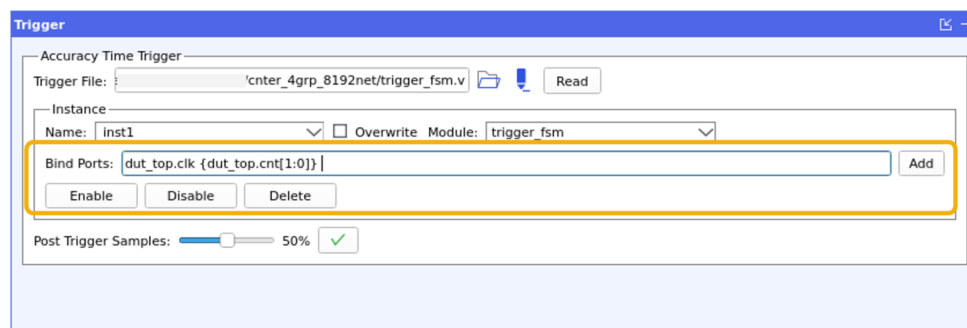


Figure 12-13: Instantiating Trigger Module Definition

- a. In the **Name** field, enter an instance name.
- b. (Optional) Select the **overwrite** checkbox to overwrite the definition of an existing instance.
- c. Select a module from the **Module** list.
- d. In the **Bind Ports** field, enter a signal or signals that are to be connected to the input ports of modules. You must specify the signal names according to the order of the ports in the module declaration.
- e. Click **Add** to create the instance.

The console displays that the instance is added successfully.

3. Click **Enable** to enable your desired trigger instances.

4. (Optional) To adjust the number of cycles that you want to run after a trigger happens, use the **Post Trigger Samples** slider and click ✓. The value is the percentage of the trace memory size.
5. Run the design and wait for trigger occurrence.

The console shows which trigger happens and the location of the trigger.

6. (Optional) After the trigger stops clocks, perform the following operations if required:
 - a. Get signal values and dump memories as you require.
 - b. Manage, such as list, delete, and overwrite, trigger instances.
 - c. Modify the trigger conditions, reload module definitions, and restart the design running.

You can use the built-in editor to modify the source code.

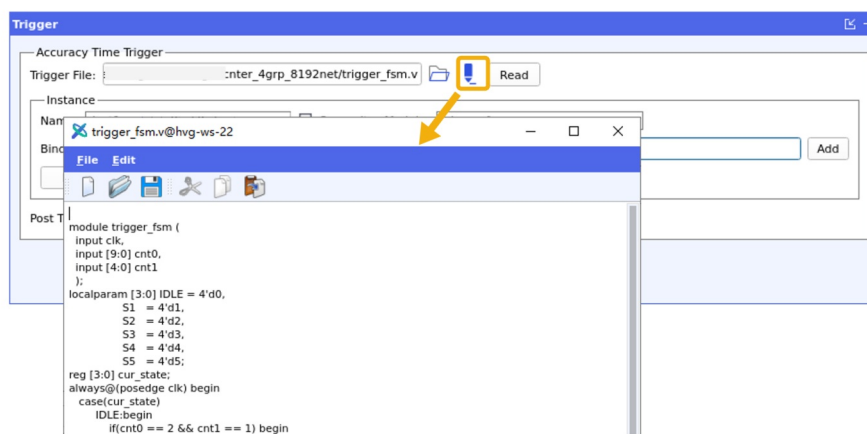


Figure 12-14: Modifying Trigger Modules

References

- [Running the Design on the Hardware from the GUI](#)
- [Getting Signal Values from the GUI](#)
- [Writing and Reading Memories from the GUI](#)

12.3.6 Releasing Signals from the GUI

This topic guides you through releasing signals in the **Signals** pane after forcing or depositing signals.

Prerequisites

Make sure that you have performed either of the following operations before you start the procedure.

- [Forcing Signals from the GUI](#)
- [Depositing Signals from the GUI](#)

Procedure

Perform the following steps to release signals from the GUI:

1. Enter the name of the signal in the **Signal** field, select a signal from the drop-down list, or select the row of the signal in the **Signal Operations** table.
2. Click **Release**.

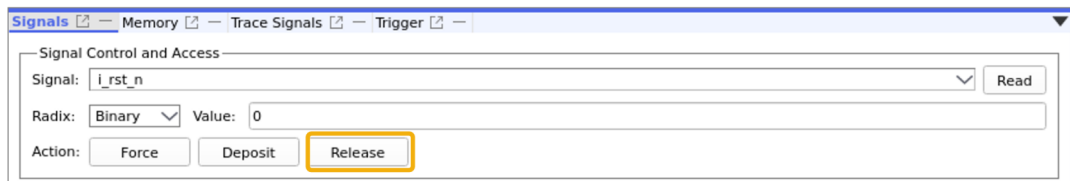


Figure 12-15: Releasing the Signal

The forced signal and corresponding value are removed from the **Signal Operations** table. The specified signal will be released in the next cycle and then the design can drive the signal.

12.3.7 Forcing and Depositing Signals through a File

In the **Signal Operations** area of the *Signals* pane, you can force and deposit the signal values in the batch mode through a text file.

Prerequisites

Make sure that you have performed either or both of the following operations before you start the procedure.

- [Forcing Signals from the GUI](#)
- [Depositing Signals from the GUI](#)

Procedure

Perform the following steps to force and deposit a list of signal values from the GUI:

1. In the **Display Objects** area, select the radix from **All** (default), **Binary**, **Octal**, **Decimal**, and **Hex** to filter the signals.

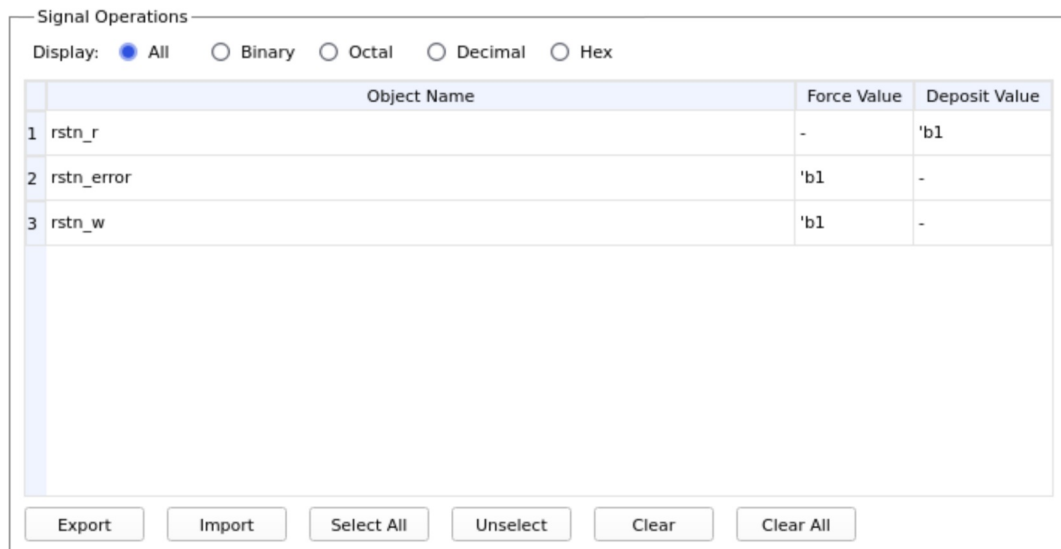


Figure 12-16: Saving the Signal Value Operations

2. (Optional) If you do not want to save certain signal operations, you can remove these signal operations from the table.

a. Select the signal operations that you do not want.

- Click a row in the table to select a signal operation.
- Press the Ctrl key and click rows to select multiple signal operations.
- Click **Select All** to select all rows in the table excluding hidden items.
- Click **Unselect** to cancel the selection.

b. Remove unwanted signal operations from the table.

- Click **Clear** to remove the selected signal operations from the table.

The cleared signal operations are not displayed in the table but the signals are still forced or deposited. To release the values, specify the signal name and click **Release**.

- Click **Clear All** to remove all the signal operations that are displayed in the table.

3. Click **Export** to export the signal operations to a text file.

Only the signal operations that are listed in the table are exported to a text file.

Here is an example of the text file.

```
Object Name  Force Value  Deposit  Value
rstn_r       -            'b1      Deposit-rstn_r-'b1
rstn_error   'b1          -        Force-rstn_error-'b1
rstn_w       'b1          -        Force-rstn_w-'b1
```


**TIP**

You can edit the saved file. Make sure that the signal names and values are valid and operations have no conflict.

- When required, click **Import** to select an existing text file that contains signal operations and apply the operations.

Signal values are forced and deposited as specified in the file. The table displays the loaded signal operations.

12.3.8 Checking System Status from the GUI

You can check the run status in the **System Status** area of the **Control** pane. The following figure displays an example of the system status.

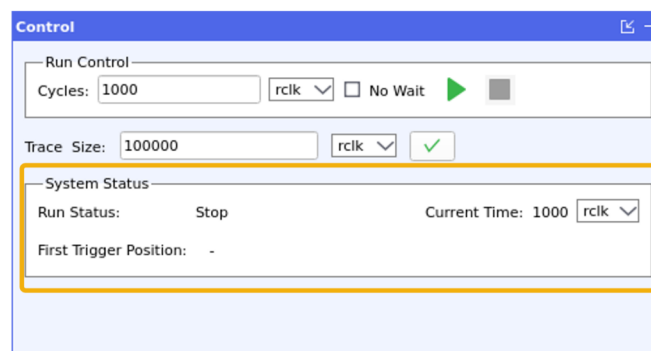


Figure 12-17: Checking System Status

Item	Description
Run Status	Displays the current run status: null, Running , or Stop .
Current Time	Displays the current total running time of the design in the default time unit, rclk. You can set the time unit as s, ms, ns, fs, or ps from the drop-down list.
First Trigger Position	Displays the position of the first trigger.

12.3.9 Entering Object Names

To quickly enter the names of signals, modules, and instances, Debug Station supports dragging and dropping of the names. You can drag signals in the following areas and drop them in the *Signals*, *Memories*, and *Trace Signals* panes:

- Debug Station

- The **Hierarchy** area in the *Design Browser* pane
- The *Name* table in the *Signals* pane

Dragging and dropping of the names in the panes of Debug Stations supports only vSyn flow and you must add the `-xndb_dbg` option on the command line to generate an XNDB file.

- Fusion Debug
 - *Hierarchy* pane
 - Source code view pane
 - Waveform pane or window

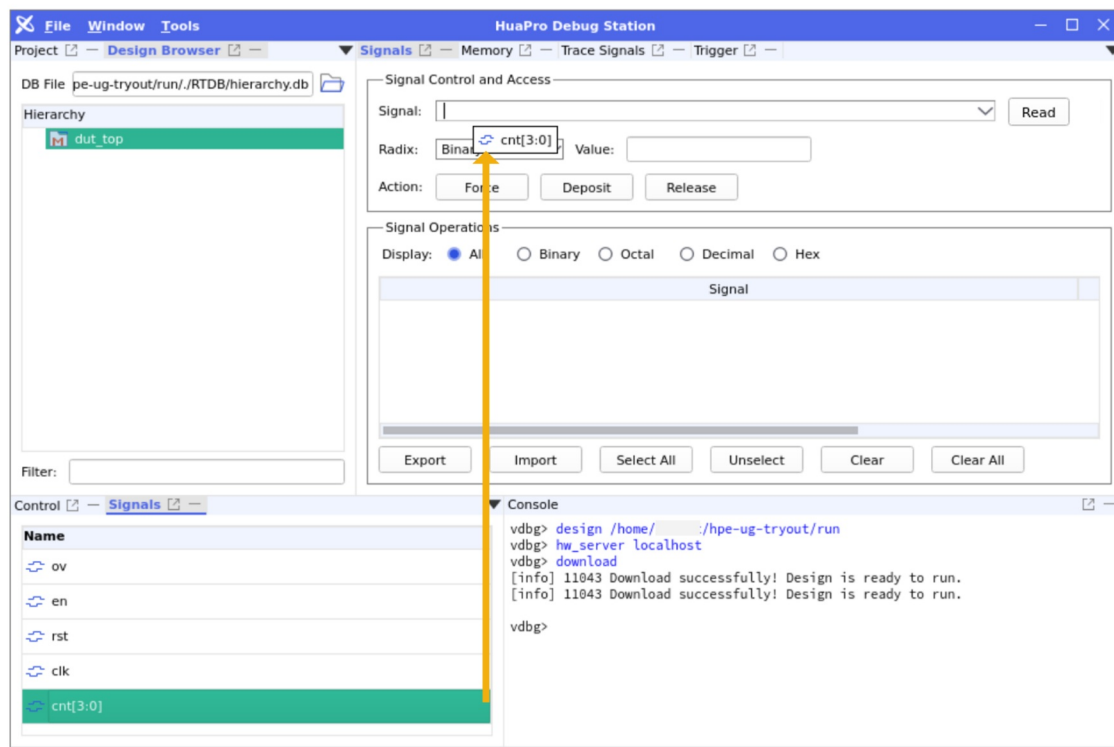


Figure 12-18: Dragging and Dropping Signal Names

12.3.10 Getting Signal Values from the GUI

This topic guides you through getting signal values in the *Signals* pane.

Prerequisites

To get values of read nets, make sure that you have performed the following steps before you start the procedure.

- [Adding Nets for Signal Access](#)
- [Downloading the Design to the Hardware from the GUI](#)

Procedure

Perform the following steps to get the signal values from the GUI:

1. In the **Signal** field, enter the signal name or select one from the drop-down list.

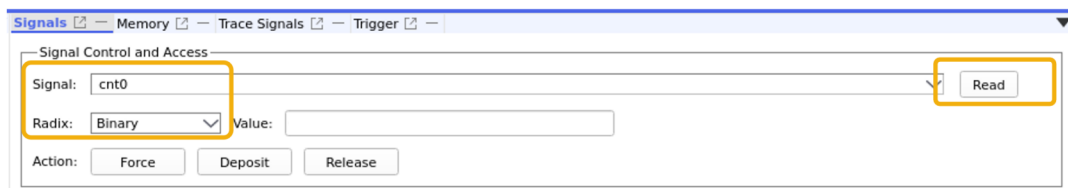


Figure 12-19: Getting the Signal Value

2. Select the radix from the **Radix** drop-down list, Binary (default), Octal, Decimal, or Hexadecimal.
3. Click **Get Value** when the design is running or stops.

The signal value is displayed in the *Value* field.

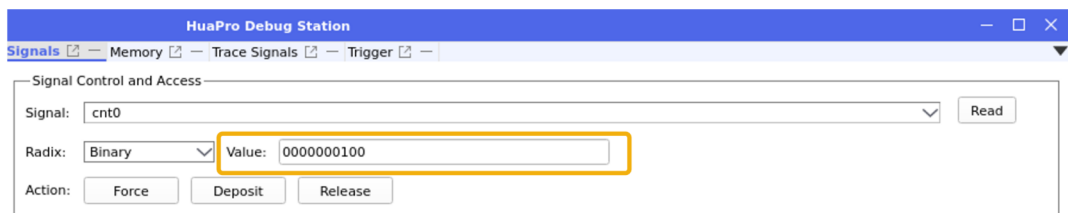


Figure 12-20: Checking the Signal Value in the Console Pane

12.3.11 Generating Waveforms from the GUI

This topic guides you through generating waveform for physical and streaming probes in the **Trace Signals** pane.

Prerequisites

Make sure that you have performed the following operations before you start the procedure.

- Physical probe tracing:
 - [Adding Nets for Physical Probes](#)
 - [Configuring the DDR4 Daughter Board](#)
 - [Downloading the Design to the Hardware from the GUI](#)
- Streaming probe tracing:
 - [Adding Nets for Streaming Probes](#)
 - [Downloading the Design to the Hardware from the GUI](#)

Procedure

Perform the following steps to generate and upload waveforms to the host from the GUI:

1. In the **Probe Type** area, select the probe type, Physical or Streaming.

Probe type options are disabled if you have not predefined trace nets at compile time.

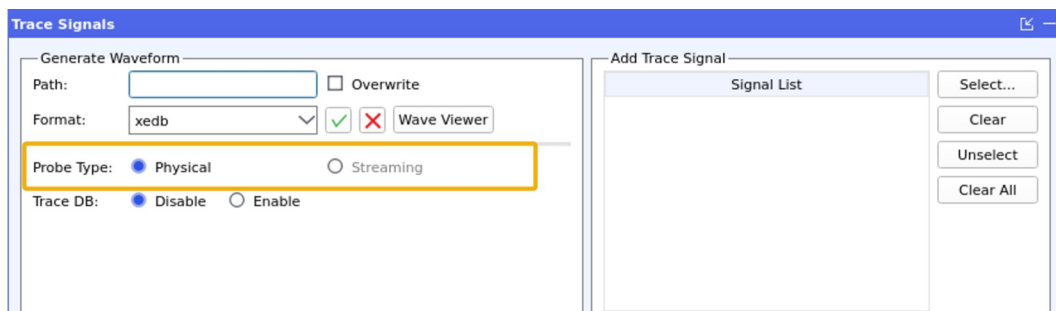


Figure 12-21: Selecting the Probe Type

2. Open a waveform database.

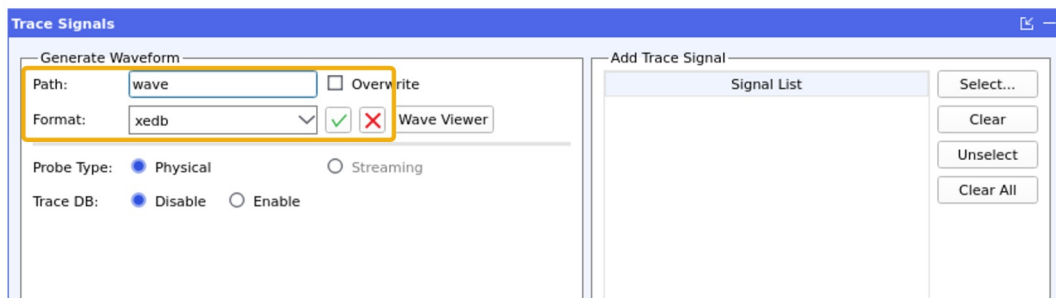


Figure 12-22: Opening a Waveform Database

- a. Enter the waveform database name in the **Waveform Name** field. Select the **Overwrite** checkbox if you want to overwrite an existing database with new trace data.

- b. Set the waveform format from the **Format** drop-down list.

Physical probe tracing supports XEDB, CWDB, and VCD formats. Streaming probe tracing supports only the XEDB format.

- c. Click the **Apply**  icon to create or open your specified waveform database.

If you do not specify a waveform database, vDbg creates one named *physicalprobe.xvcf* or *streamingprobe.xvcf* depending on your selected probe mode.

The console displays that the waveform database is successfully opened. In the **Trace DB** field, **Enable** is selected.

3. Select signals that to be added as probes.

- Select physical probes and streaming probes.
 - a. In the **Add Trace Signal** area, click **Select** to open the *Probe Signal Operation* dialog.

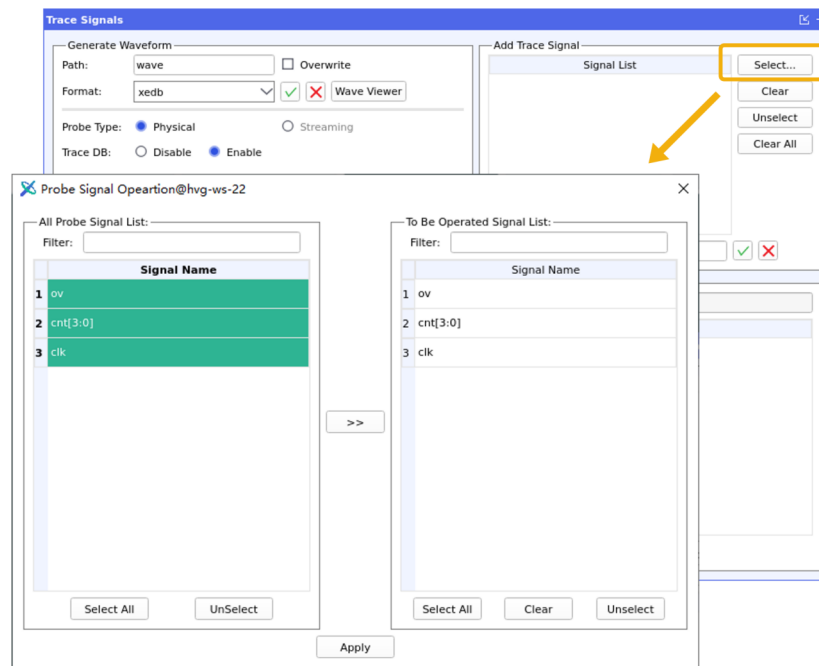


Figure 12-23: Adding Probes to Waveform Database

- b. Select probes from the **All Probe Signal List** area.

By default, all probes in the design are displayed. You can use the filter to search for desired signals.

- Press the Ctrl key and click signal in the list to select multiple signals.
- Click **Select All** to select all the signals.
- Click **Unselect** to cancel the selection.

- c. Click the >> button to add the selected signals to the **To Be Operated Signal List** area.

- d. Click **Apply** and then **Exit** to apply the selection and close the *Probe Signal Operation* dialog. To close the dialog without saving the selection, click only **Exit**.

The added probes are displayed in the signal list of the **Add Trace Signals** area.

4. (Optional) In the **Add Trace Signals** area, you can remove signals from the signal list.

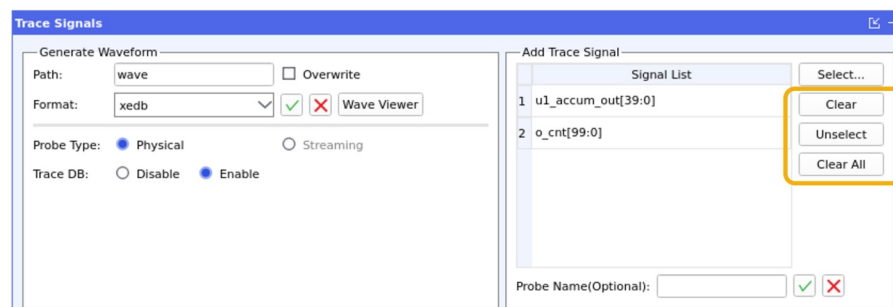


Figure 12-24: Editing the Signal List

- a. Select the probes that you do not want.

- Click a signal in the list.
 - Press the Ctrl key and click signals to select multiple signals.
 - Click **Unselect** to cancel the selection.
- b. Remove your unwanted probes from the list.
- Click **Clear** to remove the selected signals from the list.
 - Click **Clear All** to remove all the selected signals from the list.
5. Click the **Apply** ✓ icon to group a list of signals into a probe.

You can specify the probe name in the **Probe Name(Optional)** field. If not specified, the default probe name is used.

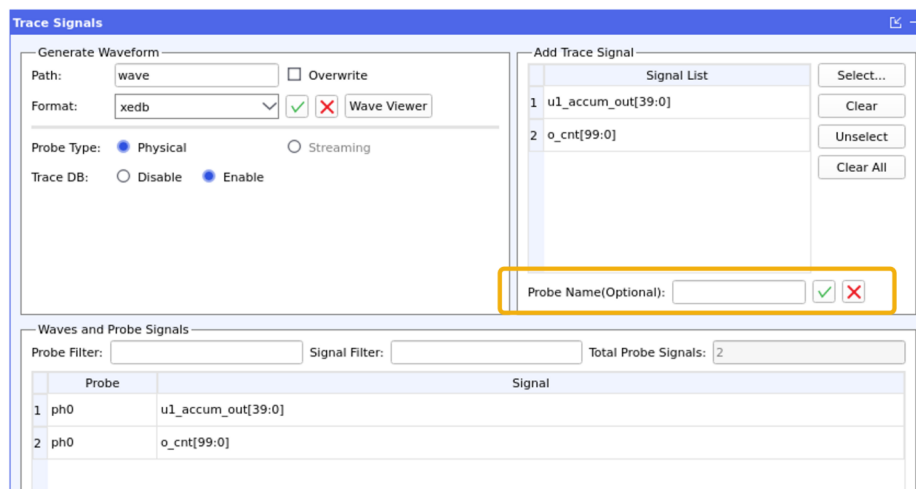


Figure 12-25: Adding Probes

The **Waves and Probe Signals** area displays the created probe.

To delete a probe, specify the probe name in the **Probe Name(Optional)** field and then click ✕.

6. In the **Control** pane, run the design by referring to [Running the Design on the Hardware from the GUI](#).

If you want to set the trace memory size for physical probes, specify the size in the **Control** pane and click the **Apply** ✓ icon. If you want to get the maximum number of trace cycles for which DDR can capture the data, see [Getting the Maximum Depth Cycle](#).

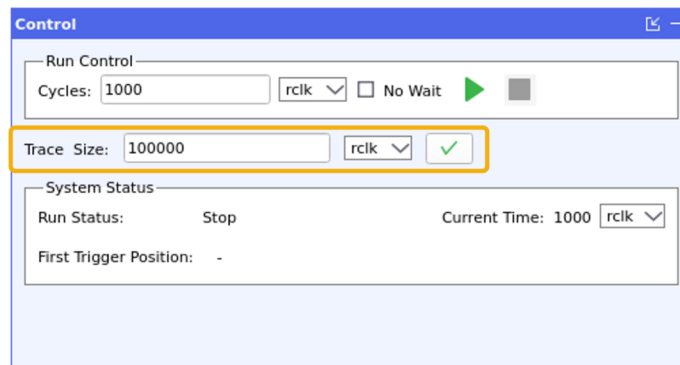


Figure 12-26: Setting the Trace Size for Physical Probes

7. In the **Waves and Probe Signals** area, click **Upload** to upload the waveform into the host.

For streaming probes, trace data is automatically uploaded to the host. For physical probes, you need to manually upload the waveform into the host. The uploading process might be time-consuming.

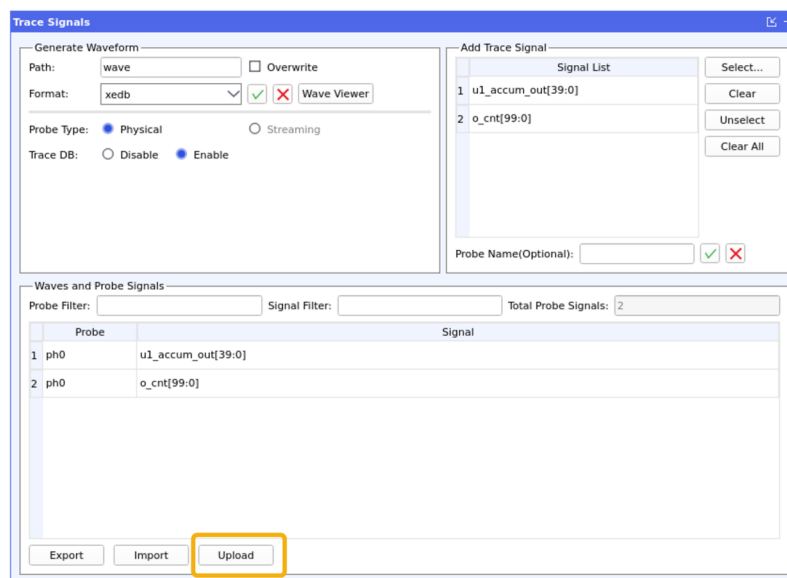


Figure 12-27: Uploading the waveform

8. In the **Generate Waveform** area, select **Disable** to disable the probe tracing in the **Trace DB** filed.

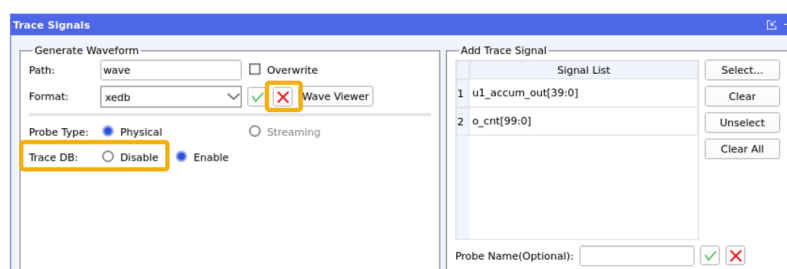


Figure 12-28: Closing the waveform

9. In the **Generate Waveform** area, click icon  to close the waveform database.

12.3.12 Adding Probes through a File

In the **Probes Display** area of the **Waveform** pane, you can save and restore waves in a batch through a text file.

Prerequisites

Make sure that you have add waves by referring [Generating Waveforms from the GUI](#).

Procedure

Perform the following steps to store and restore waves through a text file in the GUI.

1. Use the **Probe Filter** and **Signal Filter** filters to display only your desired probes.

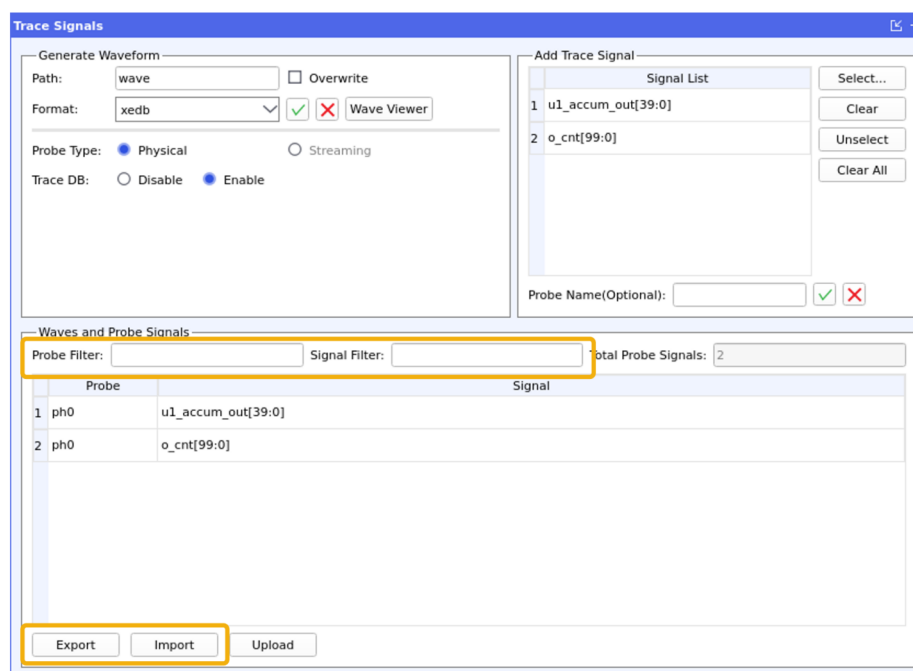


Figure 12-29: Editing the Probe List

2. Click **Export** to save the probe list to a text file.

Here is an example of the saved file.

Wave	Signal	
ph0	ov	trace_signals -add -name ph0 {ov}
ph0	cnt[3:0]	trace_signals -add -name ph0 {cnt[3:0]}
ph1	clk	trace_signals -add -name ph0 {clk}

TIP

You can edit the saved file. Make sure that the signal names are valid and operations have no conflict.

- When required, click **Import** to select an existing text file that contains wave information and add the probes to the waveform database.

Probes are added as specified in the file. The table displays the probes.

12.3.13 Viewing XEDB Waveforms from the GUI

To quickly check XEDB waveforms after probe tracing, you can set the `FUSION_DEBUG_HOME` environment variable and then invoke Fusion Debug from the GUI in either of the following ways:

- In the **Trace Signals** pane, click **Wave Viewer**. The latest waveforms are opened in xWave, and all traced signals are added.

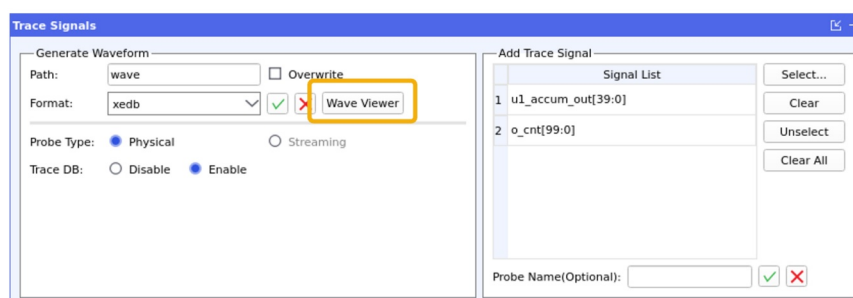


Figure 12-30: Invoking Fusion Debug from GUI

- Click **Tools > Fusion Debug**. Fusion Debug is launched with the waveform pane opened.

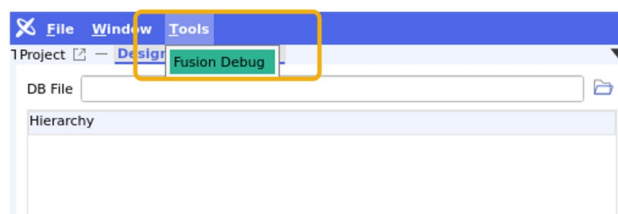


Figure 12-31: Invoking Fusion Debug from GUI

- In the **Design Browser** pane, right-click a file that contains source file lists and `fusiondebug` command options or an XEDB file and then select **Open Fusion Debug**.

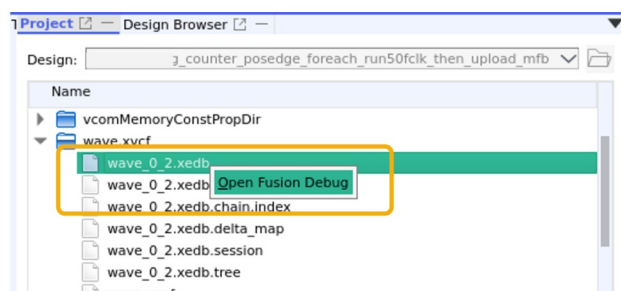


Figure 12-32: Invoking Fusion Debug from GUI

12.3.14 Writing and Reading Memories from the GUI

HPE can write the user data into the FPGA RAMs (DUT memory) and read the data from FPGA RAMs (DUT memory) to host.

Prerequisites

Make sure that you have performed the following steps before you start the procedure.

- [Adding Memories for Memory Access](#) for memory backdoor access
- [Downloading the Design to the Hardware from the GUI](#)

Write the DUT Memory

Perform the following steps to write the DUT memory in the **Memory** tab of the GUI.

1. Specify the memory instance name in full path.

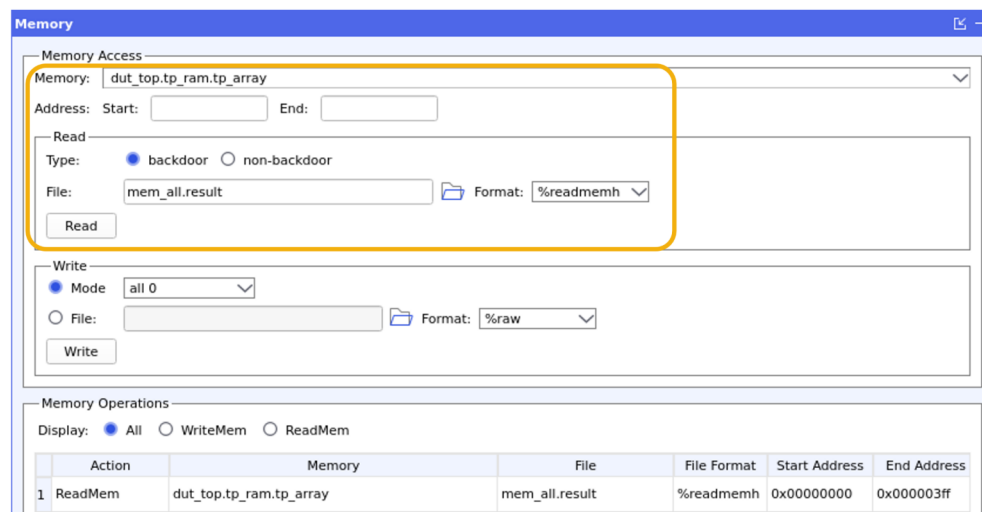


Figure 12-33: Writing the DUT Memory

You can use the `memory -list` in the *Console* pane to print out the DUT memory names.

2. Specify the start address and end address of the memory.
3. Specify the memory data in either of the following ways.
 - Click the **Folder**  icon to choose a file.
 - Select a mode. The supported write modes are 32-bit hex value (default), all0, all1, seq0, and seq1. If you select a mode, the **File** field must be empty.
4. Select the format of the file from the drop-down list. The supported file formats are `%raw`, `%readmemb`, `%readmemh`, `%b`, `%o`, and `%h`.
5. Click **Write**  to start memory write.

Read the DUT Memory

Perform the following steps to read the DUT memory in the **Memory** tab.

1. Specify the memory instance name in full path.

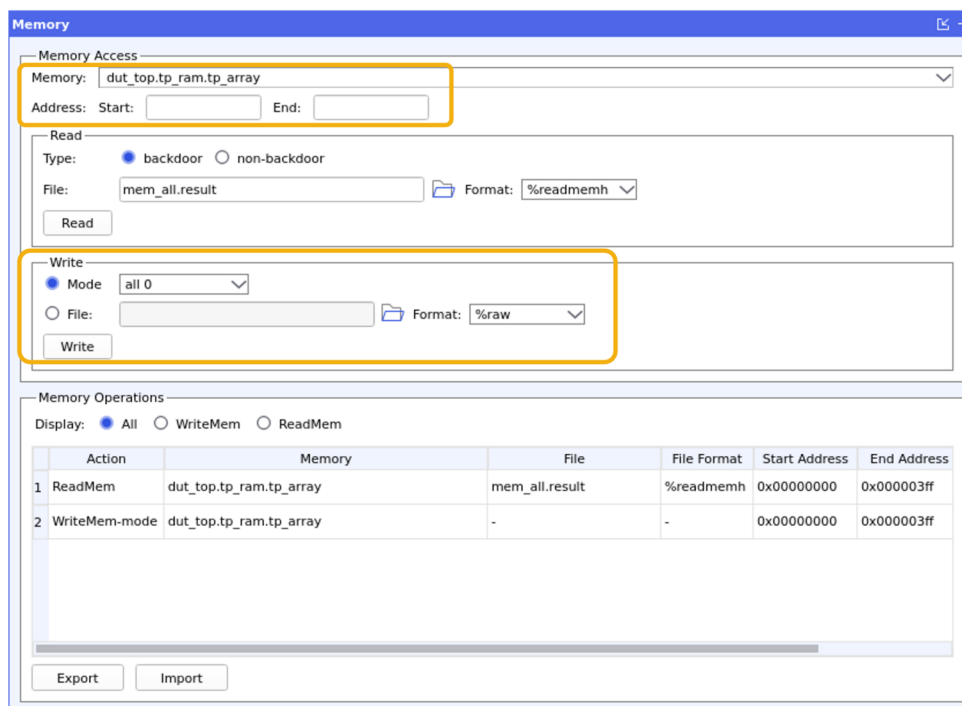


Figure 12-34: Reading the DUT Memory

2. Specify the start address and end address of the memory.
3. Select the memory read type: Memory backdoor access
4. Click the **Folder**  icon to choose the file where the data is to be saved.
5. Select the file format from the **File Format** drop-down list. The supported file format options are `%raw` , `%readmemb` , `%readmemh` , `%b` , `%o` , and `%h` .
6. Click **Read** to start memory read.

Save Memory Operations

All memory actions are listed in the **Memory Action Control** area. You can select **All** (default), **WriteMem**, or **ReadMem** to filter the memory actions.

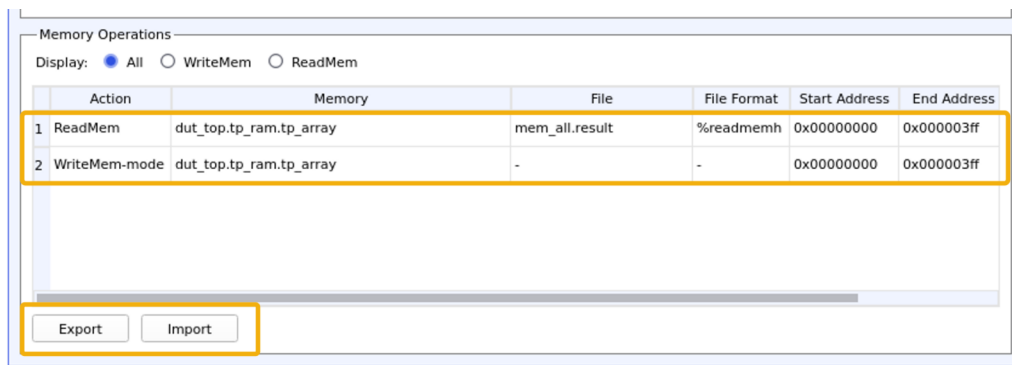


Figure 12-35: Displaying the Memory Actions

If required, you can save and display desired memory actions.

1. In the **Memory Operations** area, click **Export** to export the listed actions to a text file.
2. When required, click **Import** to display the memory actions in the **Memory Action Control** area.

The imported memory actions are only displayed in the table.

3. Select the row of your desired action listed in the table.
4. Click **Write** or **Read** to start memory write or read.

References

- [Writing User Data into DUT Memory](#)
- [Reading Data from DUT Memory](#)

12.4 Navigating the Graphical User Interface

Debug Station provides a flexible and customizable GUI. This chapter introduces how to customize the GUI layout.

12.4.1 Undocking and Re-docking the Panes

Each pane is dockable and docked by default. A dockable pane can attach above, below, left, right or over another dockable pane.

1. Undock a pane in either of the following ways. The pane becomes a stand-alone window after undocking.
 - Click the **Undock** icon in the upper right corner of the pane.
 - Drag the title bar of the pane and then release the title bar immediately.

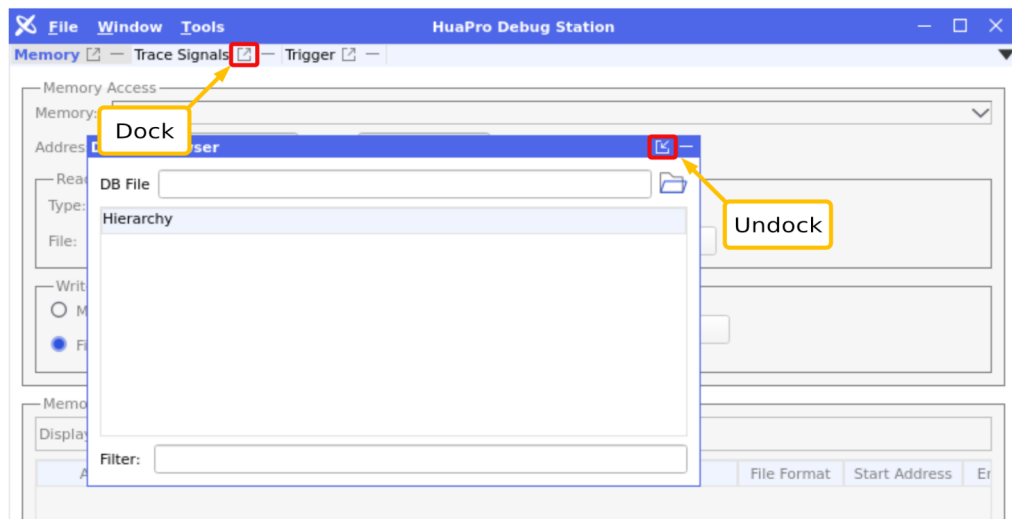


Figure 12-36: Dock and Undock Icons

2. Dock a window in either of the following ways.

- Click the **Dock** icon. The window is docked to the last docking position.
- Keep dragging the window until a docking position indicator displays. Drag the window to one of the five indicated areas as you desire and then release the mouse. The window is docked to the new position. If you dock a window over another dockable pane, a tab is created.

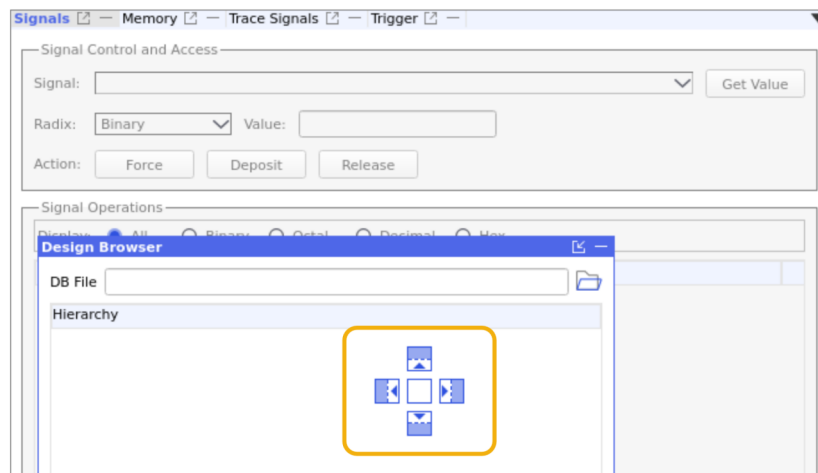


Figure 12-37: Docking Position Indicator

12.4.2 Adjusting the GUI Layout

You can adjust the size of the undocked window or docked pane to meet your requirements. After adjusting the GUI layout, you can save and restore the current layout in the **Window** menu in the following steps:

1. Click **Window > Save Layout** to save the current layout to a JSON file, for example *layout1.json*.

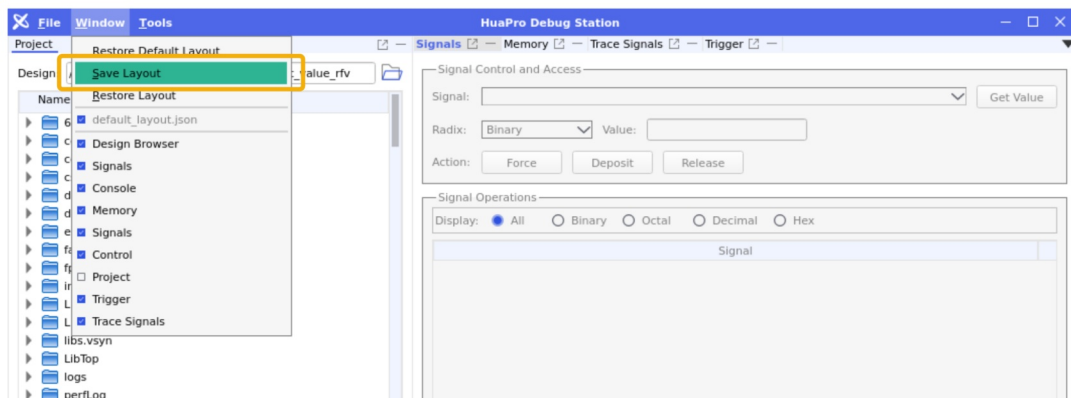


Figure 12-38: Saving the Layout

2. After you change the layout or when you start the GUI next time, click **Window > Restore Layout** to restore the layout using the file that you have saved previously.

The layout is restored and your restored layout filename is displayed in the **Window** menu.

3. (Optional) You can also click **Restore Default Layout** to display the default layout. The default layout file is *default_layout.json*.

12.4.3 Hiding and Unhiding a Pane

Perform the following steps to hide and unhide a pane.

1. Hide a pane in one of the following two ways.
 - Click the **Hide** icon on the title bar.



Figure 12-39: Clicking the Hide Icon

- Right-click the title area of any pane. Clear the checkbox of the pane that you want to hide.

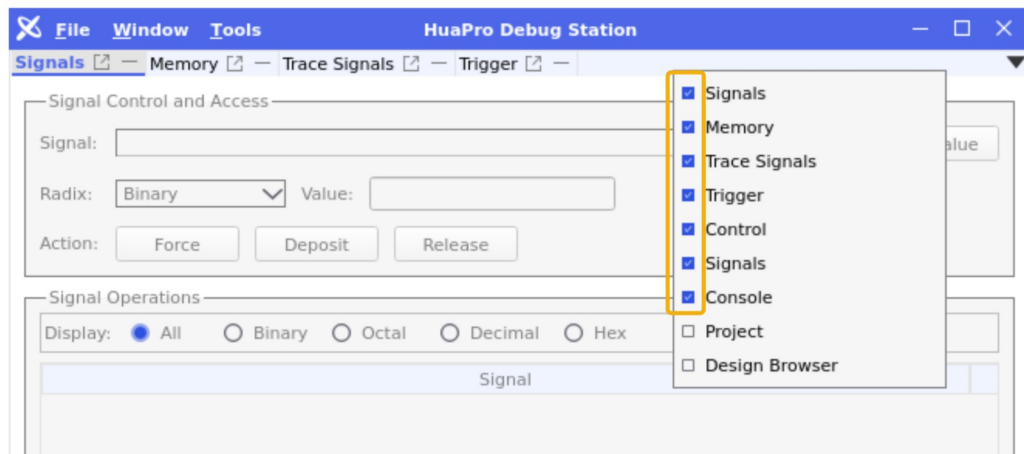


Figure 12-40: Hiding Panes from the Title Area of Panes

- In the **Window** menu, clear the checkbox of the pane that you want to hide.

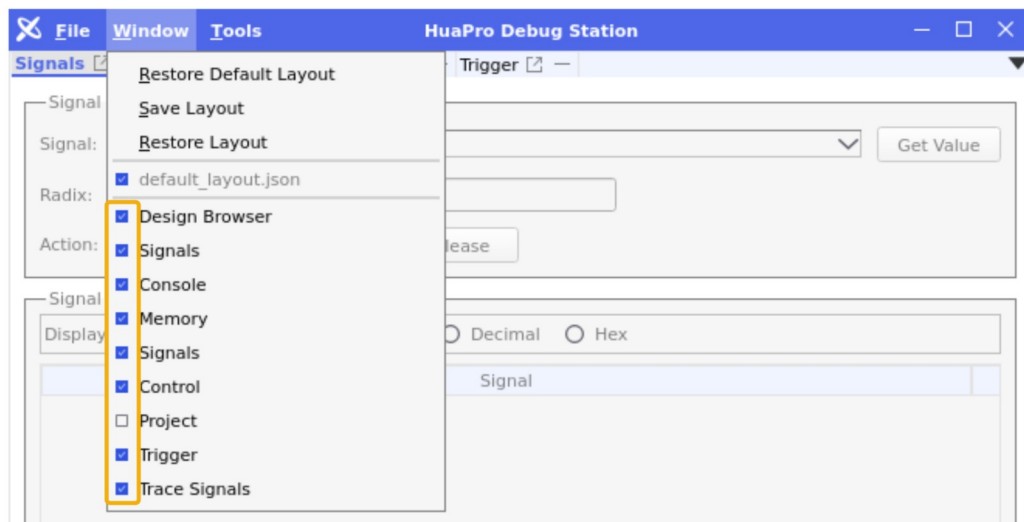


Figure 12-41: Hiding Panes from the Window Menu

2. Unhide a pane by selecting the checkbox of the pane that you want to display.

13 HPE Command Reference

This chapter describes the commands used to verify designs using HPE. Commands for HPE are categorized based on the executables in HPE package:

- [vsyn](#)
- [vcom](#)
- [vdbg](#)
- [vrmctl](#)

13.1 vsyn

Use the `vsyn` command line to start vSyn.

Syntax

```
vsyn <source-files> [<options>]
```

Where,

- `<source-files>` specifies the source files which can be one or more Verilog or SystemVerilog files.

Two or more files can be separated by space in the command line. Files in a command line are processed in a left-to-right order. You can also specify a file list or file lists to load the source code. Files written in a file list are processed from the first line to the last line.

- `<options>` specifies options that control how vSyn works. See the following topics for more information about options. The options are listed alphabetically.

13.1.1 -append-log

Appends messages to the log file specified by the `-l/-log` option. In the elaboration step of two-step synthesis or the incremental synthesis, if this option is not used for an existing log file, the new messages will overwrite the old one.

Example

```
% vsyn dut.v -o vsyn.g.v -database libs.vsyn  
% vsyn dut.v -o vsyn.g.v -database libs.vsyn -append-log
```

13.1.2 -area-report <file>

Customizes the file name of the area report which reports the FPGA resources that the

synthesized design utilizes. `<file>` specifies the file name and path of the output area report.

Example

```
% vsyn -area-report ./report/area.log dut.v
```

13.1.3 -attribute <file>

Annotates attributes from a JSON file. `<file>` specifies a JSON file which includes attributes.

13.1.4 -basic-gate-only

Uses only Verilog built-in primitives for combinational gates.

13.1.5 -bit-enable-for-vcom

Uses the bit enable model for vCom.

13.1.6 -block-ram <n>

Specifies the array size threshold for using BRAM. Default value is 256. Valid value of `<n>` is any non-negative integer or `-1`. Setting the value to `-1` disables the block RAM synthesis. Higher number means more LUTs. Lower number means more BRAMs. This is a hidden option.

13.1.7 -c|-parse-only

Controls vSyn to only parse the design. vSyn checks the syntax errors when analyzing input files and saves the generated intermediate files in a design library.

13.1.8 -D|-define <macro>[=<value>]

Defines text macros for the synthesis. `-D <macro>[=<value>]` or `-define <macro>[=<value>]` specifies the name and value of a macro. You can specify multiple macros with repetition of `-D` or `-define`.

- `<macro>` specifies the name of the macro.
- `<value>` specifies the value of the macro. If not specified, the value of macro is `1` by default. If you specify formal arguments, you must enclose the list of formal argument names in parentheses and separated by commas.

13.1.9 -database <dir>

Specifies a directory to save and restore the intermediate data in incremental synthesis.

`<dir>` specifies the directory.

Example

```
% vsyn dut.v -o vsyn.g.v -database libs.vsyn
% vsyn dut.v -o vsyn.g.v -database libs.vsyn
```

13.1.10 -defparam <name>=<value>

Specifies parameter values. You can specify multiple parameter values with repetition of `-defparam`.

13.1.11 -dsp <n>

Specifies the gate-count threshold for inferring DSP blocks from operators. Default value is 500. Valid value of `<n>` is any non-negative integer or `-1`. Setting the value to `-1` forbids inferring DSP blocks. Higher number means more LUTs. Lower number means more DSPs. This is a hidden option.

13.1.12 -dup-free-map

Enables LUT remapping enhancement. This feature is in trial period currently.

13.1.13 -error-limit <num>

Limits the number of printed error messages. `<num>` specifies the number of errors that will be printed. Setting the number to zero disables the limit.

13.1.14 -f|-file <filename>

Specifies a file list or file lists that contain source files and options. The relative paths in the file list are relative to the directory where you run vSyn.

13.1.15 -F <filename>

Specifies a file list or file lists that contain source files and options. The relative paths in the file list are relative to the directory where the file list is located.

13.1.16 -formality <file>

Generates Formality scripts in a file automatically. This option is only for testing purposes.

13.1.17 -h|-help

Prints the options and displays the descriptions.

13.1.18 -hcm <N>

Specifies the size threshold for automatic mark of HCMs. Valid value of the threshold is `-1` or any non-negative integer smaller than 37. The default threshold is 30, which means the HCM size threshold is 1Gb. Setting the threshold to `-1` disables automatic mark of HCM.

13.1.19 -help-hidden

Prints the hidden options and displays the descriptions.

13.1.20 -I|-incdir <dir>

Specifies the searching path for ``include` directives. You can specify multiple directories in the file list or with repetition of `-I` or `-incdir`.

Example

```
% vsyn -incdir dut_src -incdir dur_src_m top.v -o netlist.v
```

13.1.21 -ignore-redefinition

Uses the first definition if a module has duplicated definition.

13.1.22 -l|-log <file>

Saves vSyn logs into a specified file. By default, the log file is *vsyn.log*.

13.1.23 -libext <ext>

Searches definitions of unknown module in the library files with the specified extension.

`<ext>` specifies the file extension for searching unknown modules. The `-libext` option must be used together with the `-y` option.

Example

Take the following structure of the source files for example:

```
- top.v
- lib
- full_adder.v
```

```
- half_adder.v
```

The following command line searches for unknown modules in the specified path with the specified extension .v:

```
% vsyn top.v -y lib/ -libext .v -o top.g.v
```

13.1.24 -lut-ram <n>

Specifies the array size threshold for using LUTM. If array size is below the threshold, only registers are used. Default value is 256. Valid value of <n> is any non-negative integer or -1. Setting the value to -1 disables the LUTM synthesis.

13.1.25 -nolog

Synthesizes the design without generating the log file.

13.1.26 -o|-netlist <file>

Specifies the name of the output netlist file. If this option is not specified, the source file is synthesized without any output netlist.

13.1.27 -q|-quiet

Enables the quiet mode to suppress non-essential messages.

13.1.28 -routability

Synthesizes the design with routability optimization enabled.

13.1.29 -separate-unit

Treats each file in the command line as a separate compilation unit. By default, this option is not enabled and all files in the command line are treated as a single compilation unit.

13.1.30 -std <year>

Specifies the SystemVerilog and Verilog standard to use. Valid values: 1995, 2001, 2005, 2009, 2012, and 2017 (default). Note that this option is under test.

13.1.31 -support-two-clocks-for-bram

Supports two clocks for synthesis from asynchronous read and write ports to synchronous ports. Note that the functionality of memory is not guaranteed to be correct.

13.1.32 -t|-top <name>

Specifies the top-level modules of the design. You can specify multiple top-level modules with repetition of `-t` or `-top`. If the top-level module is not specified, vSyn analyzes the design and selects the top-level modules by default.

13.1.33 -u

Converts all identifiers in the source design files and in the generated netlist file to uppercase. You can convert all Verilog and SystemVerilog identifiers to uppercase to make sure that the source description does not rely on the case sensitivity.

13.1.34 -undefine-synthesis-macro

Undefines the `SYNTHESIS` macro.

13.1.35 -undefined-module-wrapper <file>

Generates undefined module wrapper file. This option is only for testing purposes.

13.1.36 -undriven-nets <value>

Specifies the driver for undriven nets during synthesis. Valid values are `0`, `1`, `x`, and `z`. If the option is not specified, vSyn leaves the undriven nets dangling. This is a hidden option.

13.1.37 -v <libfile>

Searches definitions of unknown modules in the specified library files. `<libfile>` specifies library files. You can specify multiple files in the file list or with repetition of `-v`. Each library file can have the definition of multiple modules.

Example

Take the following structure of the source files for example:

```
- top.v
- lib
  - full_adder.v
  - half_adder.sv
```

The following command line searches for unknown modules in the specified file:

```
% vsyn top.v -v lib/full_adder.v -o top.g.v
```

13.1.38 -version

Prints the version information of vSyn.

13.1.39 -visibility <value>

Specifies the level of RTL signal preservation. Valid values: `module`, `port`, and `state` (default).

- `module` keeps all the signals in the module from being optimized. Set the visibility to `module` if you want to debug the internal signals at runtime.
- `state` preserves all ports and the sequential logic.
- `port` prohibits visibility of all internal signals and keeps only the ports from being optimized.

Example

Take the following `top.v` for example.

```
module test(input clk, a, b);
    reg c,d;
    wire e;

    always @(posedge clk) c = a;
    always @(posedge clk) d = b;

    assign e = c & d;

endmodule
```

- The following command line preserves all signals.

```
% vsyn -visibility module top.v
```

- Either of the following two command lines preserves the ports and the internal signals, `c` and `d`. `c` and `d` are preserved because they are flip-flops. `e` is not preserved because its value can be calculated from `c` and `d`.

```
% vsyn -visibility state top.v
```

```
% vsyn top.v
```

- The following command line preserves none of the three internal signals. Only the ports, `clk`, `a`, and `b`, are preserved.

```
% vsyn -visibility port top.v
```

13.1.40 -work <lib>

Specifies a design library to store intermediate files. The default is *work*. Do not create the default *work* directory using Linux commands. You must use the `vsyn` command to create the *work* directory.

Example

```
% vsyn -work tmp top.v -o netlist.v -top top
```

13.1.41 -xndb_dbg <file>

Generates a netlist to preserve all RTL signal names.

:::tip The synthesis time might increase when you add the `-xndb_dbg` option for netlist generation.

13.1.42 -xpm-script <file>

Creates Vivado synthesis scripts for XPM cells in a file. This option is only for testing purposes.

13.1.43 -y <libdir>

Searches definitions of unknown modules in the specified library directories. You can specify multiple files in the file list or with repetition of `-y`. The names of the files in the library directories must be the same as the names of the corresponding instantiations in the module description. Each file in the specified library can only have the definition of one module.

You can use the `-y` option together with the `-libext` option.

Example

```
% vsyn top.v -y lib/ -libext .v -o top.g.v
```

```
% vsyn top.v -y /usr/lib/verilog -y /usr/lib/cmos
```

13.2 vcom

Use the `vcom` command line to start vCom.

Syntax

```
vcom [<compilation_script>] [-version]
```

Where,

- `<compilation_script>` specifies the compilation script file.
- `-version` prints out the version of vCom.

If none of the above options is specified, vCom is started in the interactive prompt.

For the description of the vCom commands that you can use to compile designs, see the following topics.

13.2.1 annotate_xdc_clock

Annotates a clock when you specify the clock as the start point or end point of the timing path.

Syntax

```
annotate_xdc_clock -clock <clock_name> [-wire <object> | -port <object>]
```

Where,

- `-clock <clock_name>` specifies a user-defined clock domain name. This is a required option.
- `-wire <object>` specifies a set of nets that serve as the clock of sequential cells.
- `-port <object>` specifies a set of clock pins (either input or output pins) or primary clock ports of the design.

NOTE

`-wire` and `-port` are mutually exclusive options.

Example

```
% annotate_xdc_clock -clock clk1 -wire clk1
% annotate_xdc_clock -clock clk2 -wire clk2
% set_false_path -from clk1 -to clk2
```



```
% annotate_xdc_clock -clock clk1 -port {fd_0.C fd_2.C}
% annotate_xdc_clock -clock clk2 -port {fd_3.C fd_4.C}
% set_false_path -from clk1 -to clk2
```

```
% set_false_path -from {fd_0 fd_2} -to {fd_3 fd_4}
```

13.2.2 bind_ports

Binds the netlist macro instance to the debug logic instance.

Syntax

```
bind_ports -netlist_macro_ports {<nm_port_name>} -uni_debug_ports {<channel_id>
<port_name>}
```

Where,

- <nm_port_name> specifies the ports of the netlist macros to be bound to AXI4 interface.
- <channel_id> specifies the location of DDR4 daughter board that is to be used for DDR4 memory backdoor. The location must be in the format of <board_index>.<fpga_index>.<channel_index>.
 - <board_index> specifies the board index.
 - <fpga_index> specifies the FPGA index.
 - <channel_index> specifies the channel index.
- <port_name> specifies the AXI4 interface signals.

Example

```
% bind_ports -netlist_macro_ports {ddr4.chip0.c0_ddr4_ui_clk} -uni_debug_ports
{0.A.0 dut_axi_s0_clk}
```

13.2.3 break_all_loops

Breaks or keeps the loops in a specific instance scope or scopes.

Syntax

```
break_all_loops -add|-keep {<hier_inst_path> ...}
```

Where,

- <hier_inst_path> specifies the full name of a hierarchy instance in the netlist. To specify more than one instance on the command line, separate the parameters with a space or

spaces.

- `-add` specifies that the loops in the scopes are broken by inserting breakers.
- `-keep` specifies that the loops in the scopes are kept.

Example

```
break_all_loops -add {top.inst1 top.inst2}
break_all_loops -keep {top.\block[1].inst top.\block[0].inst }
```

13.2.4 cable_connection

Defines an interface for terminal assignment.

Syntax

```
cable_connection -def {<interface> <type> <location>}
```

Where,

- `<interface>` specifies the interface name. The interface name can be any given identifier consisting of alphanumeric characters and underscores. The first character of the interface name cannot be a digit.
- `<type>` specifies the type of the physical connector in the system.

Valid values are as follows.

- `PHC` specifies PHC connectors on HuaPro P1 and P2E.
- `ORG` specifies GPIO connector on HuaPro P1.
- `<location>` specifies the location of physical connectors.

For PHC connectors, the location syntax is `<board_id>.<FPGA_id>.<connector_id>`. For GPIO connectors, the location syntax is `<board_id>.<FPGA_id>`.

- `<board_id>` specifies the hardware where the connector is located. You can specify the board ID from zero to the integer that less than the maximum number of boards in the system.
- `<FPGA_id>` specifies the FPGA where the connector is located. Valid values are A and B in HuaPro P1 and A-D in HuaPro P2E.
- `<connector_id>` specifies the bank of the PHC connector that you use for cable connection. Valid values are 1-24.

13.2.5 clock_localization

Controls whether to enable clock localization for optimization.

Syntax

```
clock_localization -enable|-disable
```

Where,

- `-enable` enables the clock localization.
- `-disable` disables the clock localization.

13.2.6 clock_port

Excludes some ports from the clock region to enhance the clock region detection.

Syntax

```
clock_port -exclude {<port_path> ...}
```

Where `<port_path>` specifies the port that is to be excluded from the clock region.

13.2.7 clock_recognition

Controls whether to enable clock recognition for optimization.

Syntax

```
clock_recognition -enable|-disable
```

Where,

- `-enable` enables the clock recognition optimization.
- `-disable` disables the clock recognition optimization.

13.2.8 const_prop_option

Enables or disables the constant propagation optimization on LUTs, DSPs, and memories.

Syntax

```
const_prop_option -enable|-disable <method>
```

Where `<method>` can be one of the followings or multiple methods separated by space:

- `all` specifies all supported optimization methods.
- `LUT_reduction` specifies the constant propagation optimization on LUTs.
- `DSP_const_prop` specifies the constant propagation optimization on DSPs.

- `MEM_const_prop` specifies the constant propagation optimization on memories.

13.2.9 control_set_optimization

Controls whether to enable control set optimization and sets the loader number threshold.

```
control_set_optimization -enable|-disable
control_set_optimization -global {loader_num <N>}
control_set_optimization -instance {<instance_path_name> loader_num <N>}
```

Where,

- `<N>` specifies the loader number threshold. The control sets whose loader number is less than or equal to the threshold are to be optimized.
- `<instance_path_name>` specifies the full path name of an instance. The loader number threshold for the instance has higher priority than that for the global.

13.2.10 create_clock

Declares the input clocks in a user design and configure the frequencies of these clocks.

Syntax

```
create_clock -sig_name <signal_name> -frequency <number><unit>
```

Where,

- `-sig_name <signal_name>` specifies the source clock in the user design that is to be mapped to the FPGA resources.
- `-frequency <number><unit>` specifies the clock frequency and unit. Supported frequency units are KHz, MHz, and GHz. The units are case-insensitive. Be aware that your specified frequency is a string with no separating space between the number and the unit.

Example

The following command lines define two user clocks `clk0` and `clk1` in the emulation mode:

```
% create_clock -sig_name clk0 -frequency 300MHz
% create_clock -sig_name clk1 -frequency 60MHz
```

13.2.11 create_generated_nmclock

Defines the clocks for netlist macros. The command must be executed before user-defined partitioning constraints.

Syntax

```
create_generated_nmclock -name <clock_name> -port <nm_clock_port> [-insert_bufg]
```

Where,

- `-name <clock_name>` specifies a user-defined clock name, which is used in XDC. This is a required option.
- `-port <nm_clock_port>` specifies a clock port of the netlist macro. This is a required option. For multiple clock ports, specify the ports on separate command lines.
- `-insert_bufg` inserts a BUFG primitive to the specified clock port to improve runtime performance.

Example

```
% create_generated_nmclock -name aclk -port top.u_pcie_wrapper.aclk
% create_generated_nmclock -name bclk -port top.u_ddr_wrapper.bclk
```

13.2.12 delay_cell

Breaks a loop or loops in the DUT before loop detection.

Syntax

```
delay_cell -add {<sink_port_path> ...}
```

Where `<sink_port_path>` specifies the path of the synchronous ports. vCom adds an FD cell as a driver of target leaf instances to break the loop.

13.2.13 delay_bin

Defines a delay bin in reference to a signal (reference signal) and label the relation.

Syntax

```
delay_bin -def {<bin> <delay> <reference_signal>}
```

Where,

- `<bin>` specifies the name of the delay bin. The delay bin name cannot be one of the names of existing terminals and also cannot contain spaces. The specified name can be used as the constraint argument in the `terminal_delay` command line.

- `<delay>` is a non-negative integer that specifies the number of primary clock cycles for the delay.
- `<reference_signal>` specifies the reference signal. The reference signal must be a primary IO signals or a pre-defined delay bin and cannot be an existing bus signal.

If any conflicts, vCom obeys the rules as described below:

- Rule 1: If duplicate definitions of delay bin are present in the `delay_bin` command lines, the last command line is effective.
- Rule 2: New definition of delay bin has no effect on previous reference of the delay bin.

Example

Example 1: In the following example, the actual compile result is that `binA` means 3-cycles delay with regard to `sig1`.

```
% delay_bin -def {binA 5 sig0}
% delay_bin -def {binA 3 sig1}
```

Example 2: In the following example, the actual compilation result is that `binA` means 3-cycles delay with regard to `sig1`. Even though `binA` is redefined, `binB` still means 8-cycles delay with regard to `sig0`.

```
% delay_bin -def {binA 5 sig0}
% delay_bin -def {binB 3 binA}
% delay_bin -def {binA 3 sig1}
```

13.2.14 design_edit

Loads and checks the compilation constraint commands.

Syntax

```
design_edit
```

13.2.15 design_edit_option

Controls whether vCom removes loadless memories and how to tie all driverless logic.

Syntax

```
design_edit_option -add {removeLoadless <N>} |
                  -add tieDriverlessLow|tieDriverlessHigh
```

Where

- `-add {removeLoadless <N>}` specifies the effort level. Valid values of `<N>` are `0` (default) and `1`.
 - `0` removes loadless gates.
 - `1` removes all loadless logic, including gates, memories, and registers.
- `-add tieDriverlessLow` ties driverless nets to GND, and `-add tieDriverlessHigh` ties driverless nets to VCC.

Customizes the pre-design-edit area report, *design_original_report* and post-design-edit area report, *de_area_report*.

Syntax

```
design_edit_option -add {area_report [layer:<level>] [util:<value>]
[res_type:all|fpga|<res_type>]}
```

Where,

- `layer:<level>` specifies the level of design hierarchy. Default value is `1`. Setting the level to zero disables the generation of area reports.
- `util:<value>` specifies the percentage value of resource utilization. Default value is `0.1`.
- `res_type:all|fpga|<res_type>` specifies the resource types. Default value is `all`. The parameter values are case-insensitive. Valid values are listed as follows:
 - `all` reports the resources of all types, including FPGA resources (FF, LUT, LUTM, DSP, BRAM, URAM) and debug logic resources.
 - `fpga` reports only FPGA resources.
 - `<res_type>` specifies a resource type or multiple types separated by the separator bar (|) without any space, for example, `FF|LUT|dsp`. Currently, this option is not fully tested.

13.2.16 design_generation

Performs one-step design generation. The command must be executed after the `design_edit` command.

Syntax

```
design_generation
```

13.2.17 design_load

Loads netlist files from libraries for vCom.

Syntax

```
design_load [-library {<library_list>}] -top <top_name>
```

Where,

- `-library <library_list>` specifies a library or libraries to search for the source files.
- `-top <top_name>` specifies the name of the top module. This is a required option.



NOTE

- If `design_read` has generated the default library, *work*, vCom loads the default library automatically even if you do not specify the `-library` option.
- The libraries to be loaded must be created on the `design_read` command line.
- Netlist macros must be defined before you load the design. For more information about the `netlistmacro` command, see [Defining Netlist Macros](#).

Example

Either one of the following group of command lines loads netlist files from the default *work* library:

```
% design_read -netlist ./rev_1/dut_top.vm
% design_load -top dut_top
```

```
% design_read -library work -netlist ./rev_1/dut_top.vm
% design_load -library work -top dut_top
```

The following command lines load netlist files from a specific library:

```
% design_read -library lib0 -netlist ./rev_1/dut_top.vm
% design_load -library lib0 -top dut_top
```

The following command lines load netlist files from specific libraries:

```
% design_read -library lib0 -netlist ./rev_1/dut_top1.vm
% design_read -library lib1 -netlist ./rev_1/dut_top2.vm
% design_load -library {lib0 lib1} -top dut_top
```

The following command lines specify the netlist macro and load the design:

```
% design_read -netlist {./rev_1/dut_top.vm ./dut_src/fifo_bb.v}
% netlistmacro -add {work fifo} -resource {LUT:8000 BRAM:18}
% design_load -top dut_top
```


13.2.18 design_read

Reads netlist files and generates libraries for different netlist files. The command must be executed before you load the design.

Syntax

```
design_read [-work <library_name>] -netlist {<file_list>}
```

Where,

- `-work <library_name>` specifies a library name that vCom will create. If you do not specify the `-work` option, vCom generates a default library named *work*.
- `-netlist {<file_list>}` specifies the netlist files and netlist macro files to be read. You can specify multiple files within the curly braces. This is a required option.

Example

The following command line uses the default *work* library and reads two netlist files from the current directory:

```
% design_read -netlist {designL1.g.v designL2.g.v}
```

The following command line uses the *lib0* library and reads one netlist file from the indicated path:

```
% design_read -work lib0 -netlist ./rev_1/dut_top.vm
```

The following command line compiles a netlist file and a netlist macro file to the default *work* library:

```
% design_read -netlist {./rev_1/dut_top.vm ./dut_src/fifo_bb.v}
```

13.2.19 direct_phc

Enables or disables direct PHC for the PHC connection. By default, direct PHC is disabled. The command must be executed before partition constraints.

Syntax

```
direct_phc -enable|-disable
```

Where,

- `-enable` enables direct PHC.

- `-disable` disables direct PHC.

13.2.20 display

Shows all steps in the vCom flow chart with the step ID and name of each step.

Syntax

```
display
```

13.2.21 dump_clock_region_ports

Dumps the ports that belong to the clock region into the *Clock_Region_Ports.log* file.

Syntax

```
dump_clock_region_ports
```

13.2.22 dynamic_trigger

Controls whether the ATT trigger feature is enabled or not.

Syntax

```
dynamic_trigger -enable|-disable
```

Where,

- `-enable` enables the ATT triggers.
- `-disable` disables the ATT triggers.

13.2.23 elim_common_logic

Sets the effort level for common logic elimination.

Syntax

```
elim_common_logic -enable -times <value>
```

Where `<value>` specifies the number of times common logic elimination runs, which is also the effort level. The fewer times you set, the more common logic may remain. This option must be used with `-enable`.

13.2.24 emulator_spec

Specifies an HDF for vCom. You must describe the hardware for compilation by providing vCom with an HDF. The HDF defines the hardware board, cable, and interconnection needs.

Syntax

```
emulator_spec -add|-reconstruct_interconnection {file <file_name>}
```

Where,

- `-add` loads the specified HDF to describe the hardware configuration.
- `-reconstruct_interconnection` loads the specified HDF and enables auto cabling to reconstruct the interconnection.
- `<file_name>` specifies an HDF.



NOTE

- The `-add` option conflicts with the `-reconstruct_interconnection` option. Use only one of the options in compilation.
- The content of the HDF must be in the valid format that is defined in [Hardware Description File](#). Otherwise, error message prints.

Example

The following command line loads an HDF *hw.2fpga.hdf* from the current path:

```
% emulator_spec -add {file hw.2fpga.hdf}
```

The following command line loads an HDF *project1.hdf* from the specified path:

```
% emulator_spec -add {file /usr/local/project1.hdf}
```

13.2.25 emulator_util

Adds the constraints of resource utilization for specific FPGAs. If the command lines conflict, the last command line is effective.

Syntax

```
emulator_util -add {default|<location> [<min>] <max>}
emulator_util -add {default|<location> <resource_type>:<max> ...}
emulator_util -available_fpga_num <part_num>
```

Where,

- `default` specifies all FPGAs that are enabled in the hardware description file. The constraints are set for all enabled FPGAs.
- `<location>` specifies the FPGA ID in the format of `<board_index>.<fpga_index>`. The board index ranges from zero to any integer that is less than the maximum number of the boards in the system. For example, `0.A` means the FPGA A on the board 0.
- `<min>` specifies the minimum percentage of utilization. If you do not specify the minimum value, vCom takes `0` as the minimum.
- `<max>` specifies the maximum percentage of utilization. The percent symbol can be omitted or not.
- `<resource_type>:<max>` specifies the maximum percentage of utilization for specific resource type. If not specified, vCom sets the maximum percentage of utilization for the resource type to 60% by default. If you want to specify more than one type of resource, separate the `<resource_type>:<max>` parameters with a space. The supported resource types are FF, LUT, LUTM, DSP, BRAM, and URAM.
- `<part_num>` specifies the number of parts that you expect. The actual partition results might differ from the expected.

Example

Example 1: The following command lines use the combination of options. The `0.A`, `0.B`, and `1.A` FPGAs are used and other FPGAs are not used. The utilization of all resource type of FPGA is limited to between 40% and 60%.

```
% emulator_util -add {default 0 0}
% emulator_util -add {0.A 40 60}
% emulator_util -add {0.B 40 60}
% emulator_util -add {1.A 40 60}
```

Example 2: In the following example, the first command line sets the maximum resource utilization of all FPGAs to 0%. The second command line sets the maximum resource utilization of FPGA `1.A` to 50%. The combination of the two command lines specifies that only FPGA `1.A` is to be used.

```
% emulator_util -add {default 0}
% emulator_util -add {1.A 50}
```

Example 3: In the following example, the command line sets the maximum utilization for each type of resource in FPGA `0.A`.

```
% emulator_util -add {0.A FF:10 BRAM:25 LUTM:25 LUT:20 DSP:30 URAM:20}
```

Example 4: In the following example, either of the command lines set the maximum utilization for FF and BRAM resource in FPGA 0.A .

```
% emulator_util -add {0.A FF:10% BRAM:20%}
% emulator_util -add {0.A FF:10 BRAM:20}
```

13.2.26 fc_load

Loads the compilation result, and then you can run to the other step in the vCom flow chart.

Syntax

```
fc_load {<file_path>}/result_pool_<index>
```

Where <file_path> specifies the path of *result_pool_**.

13.2.27 fc_save

Saves the compilation result after you run to a step in the vCom flow chart. The results will be saved as *result_pool_** under the specified file path.

Syntax

```
fc_save {<file_path>}
```

Where <file_path> specifies a file path to save the partition result.

13.2.28 global_net

Sets the specific nets that need to be connected to the top level module as global nets, for example netlist macro ports that connect to non-existing DUT top ports directly.

Syntax

```
global_net -port {<nm_port_name> <port_name>}
```

Where,

- <nm_port_name> specifies the net name or the port name of the netlist macro.
- <port_name> specifies the primary port of the design top to be created by this command.

Example

```
% global_net -port nm_inst.data_in data_in_nm
% global_net -port {nm_inst.data_out[31:0]} data_out_nm
```

13.2.29 indivisible_group

Sets user-defined indivisibility rules by group.

Syntax

```
indivisible_group -add {<group_name>}
```

Where `<group_name>` specifies the group that you do not want to be partitioned. The group must be already defined in the `par_group` command line.

13.2.30 indivisible_module

Sets user-defined indivisibility rules by module.

Syntax

```
indivisible_module -add {<module_name>}
```

Where `<module_name>` specifies the module for which the instantiation you do not want to partition.

13.2.31 indivisible_util

Sets user-defined indivisibility rules by utilization.

Syntax

```
indivisible_util -add {<n>}
```

Where `<n>` specifies the maximum FPGA resource utilization of one module. If the resource utilization of the module is less than the specific percentage of utilization `<n>` of one FPGA, the module will not be partitioned.

13.2.32 instance_stub

Removes an instance to reduce design size or estimate the possible improvement of timing.

Syntax

```
instance_stub -add {<instance_name>} [-keep_input] [-keep_output]
```

Where,

- `-add {<instance_name>}` specifies the full path name of an instance.

- `-keep_input` retains the inputs and makes vCom treat them as read nets. If not specified, the loadless inputs will be removed from the netlist.
- `-keep_output` retains the outputs and makes vCom treat them as write nets. If not specified, vCom ties driverless outputs to GND or VCC as specified by the `design_edit_option -add tieDriverlessHigh|tieDriverlessLow` command.

13.2.33 interconn_constraint

Adds or removes constraints for interconnection. You must execute the `interconn_dump` command to make the modification of constraints take effect.

Syntax

```
interconn_constraint -add|-remove -disabled_connector {<location> <location>
...}
interconn_constraint -add|-remove -disabled_interconnection {<location>:
<location> ...}
interconn_constraint -add|-remove -fixed_interconnection {<cable_type>:
<location>:<location> ...}
interconn_constraint -display
```

Where,

- `-add` adds user-defined constraints.
- `-remove` removes the user-defined constraints that are added.
- `-display` displays all the user-defined constraints for auto cabling.
- `-disabled_connector` disables a connector or several connectors. To specify more than one connector on the command line, separate the location parameters with a space.
- `-disabled_interconnection` disables the connection between the specific connector pair. A pair of connectors is in the format of `<location>:<location>`. To specify more than one pair of connectors on the command line, separate the connector pair parameters with a space.
- `-fixed_interconnection` fixes the connection between the specific connector pair and specifies the cable type.
- `<location>` specifies the ID of the external PHC connector. Each external PHC connector in the system has a unique ID, which can be described as `<board_index>.<fpga_index>.<connector_index>`.
- `<cable_type>` specifies the cable type. Auto cabling supports PHC interconnection cable and PHC interconnection daughter boards. Mini-SAS cables are not supported yet. The cable type must match the connector type. For example, `PHC_CABLE_1000:0.A.MA1` is invalid because the `0.A.MA1` connector is a Mini-SAS HD port.

 NOTE

When you load an HDF with `emulator_spec -add`, you cannot use the `interconn_constraint` command to disable the connections or connectors defined in the HDF.

13.2.34 interconn_dump

Enables auto cabling. The command must be executed after the `design_edit` command.

Syntax

```
interconn_dump -file <file>
```

Where `<file>` specifies the file name and extension of the dumped interconnection configuration file.

13.2.35 lite_phc

Enables or disables lite PHC for the PHC connection. By default, lite PHC is enabled. The command must be executed before partition constraints.

Syntax

```
lite_phc -enable|-disable
```

Where,

- `-enable` enables lite PHC.
- `-disable` disables lite PHC.

13.2.36 local_fpga_hold_fix

Enables vCom to detect the short FD-related timing paths and insert data-path delays to the timing paths and customizes the optimization.

```
local_fpga_hold_fix -enable|-disable  
local_fpga_hold_fix [-lut_levels <value>] [-direct_only] [-hierarchies]
```

Where,

- `-lut_levels <value>` sets the threshold of LUT level. The default LUT level is 3, which means LUT1 chains are inserted to timing paths with less than 3 logic levels until the level number is not less than 3.
- `-direct_only` controls vCom to insert LUT1 chains to only the timing paths with directed connected FDs, that is, timing paths with zero combinational logic level.
- `-hierarchies` controls vCom to process only the timing path end points within the provided hierarchies.

13.2.37 logic_replication

Controls whether to enable auto logic replication for optimization and guides the auto logic replication.

Syntax

```
logic_replication -enable [-slack_threshold <value>] [-delay_threshold <value>]  
logic_replication -disable
```

Where,

- `-enable` turns on the automatic logic replication. When you enable it without any threshold settings, vCom replicates logic for the timing paths that have two or more hops, which means that vCom handles the timing paths that cross the partition boundaries twice or more times.
- `-slack_threshold <value>` specifies the slack threshold. Valid values are any non-negative integers. vCom performs logic replication on the timing paths that have lower slack than the specified threshold. Slack thresholds have higher priority than delay thresholds. Note that this option is under test currently.
- `-delay_threshold <value>` specifies the delay threshold. Valid values are any non-negative integers. vCom performs logic replication on the timing paths that have higher delay than the specified threshold.
- `-disable` disables the automatic logic replication feature.

13.2.38 memory_access

Specifies the DUT memories that you want to backdoor read and write at runtime.

Syntax

```
memory_access -add <memory>
```

Where `<memory>` specifies the memories that you can access at runtime.

13.2.39 memory_options

Sets the memory balance strategy. Note that the option is case-sensitive.

```
memory_options -add {bram_balance SMART|ADAPTIVE}
```

Where,

- `SMART` : Balances the usage of memory resources as much as possible.
- `ADAPTIVE` : Converts as little BRAM resources as possible to make the resource utilization lower than upper limit.

Specifies memories as HCMs, which are mapped to external DDR.

```
memory_options -add {use_xdram <memory_instance_path>}
```

Where `<memory_instance_path>` specifies the path of the memory.

13.2.40 module_stub

Removes all the instances of a module to reduce design size or estimate the possible improvement of timing.

Syntax

```
module_stub -add {<module_name>} [-keep_input] [-keep_output]
```

Where,

- `-add {<module_name>}` specifies a module name.
- `-keep_input` retains the inputs and makes vCom treat them as read nets. If not specified, the loadless inputs will be removed from the netlist.
- `-keep_output` retains the outputs and makes vCom treat them as write nets. If not specified, vCom ties driverless outputs to GND or VCC as specified by the `design_edit_option -add tieDriverlessHigh|tieDriverlessLow` command.

13.2.41 netlistmacro

Specifies the FPGA resource requirements of netlistmacros, and sets the default voltage of driverless ports.

Syntax

```
netlistmacro -add {<library> <module>} -resource {<resource_type>:<number> ...}
[-set_driverless high|low]
netlistmacro -instance <instance_path> -package <package_path> [-location
<fpga_id>] [-channel <channel_index>]
```

Where,

- `-add {<library> <module>}` specifies the library and module that the netlist macro belongs to. This is a required option for the standard netlist macro flow.
- `-resource {<resource_type>:<number> ...}` specifies the resources that the netlist macro occupies. This is a required option for the standard netlist macro flow. You need to specify the value for at least one of the following resource types. If more than one type of resource need to be specified, separate the `<resource_type>:<number>` parameters with a space. The supported resource types are as follows:
 - LUT
 - LUTM (LUTs used as distributed RAM)
 - DSP
 - FF
 - BRAM
- `-set_driverless high` specifies that VCC is connected to the driverless ports of the netlist macro. This is optional for the standard netlist macro flow. `-set_driverless low` specifies GND. If this option is not specified, the default setting is low (GND).
- `-instance <instance_path>` specifies the instance name of the netlist macro. This is a required option.
- `-package <package_path>` specifies the netlist macro package that corresponds to the daughter board you want to use. This is a required option.
- `-location <fpga_id>` specifies the FPGA that the netlist macro instance corresponds to in the format `<board_index>.<fpga_index>`. This option is only required for DDR4 daughter boards.
- `-channel <channel_index>` specifies the channel that the backdoor interfaces of the netlist macro instance connects to. Valid channel indexes are 0-3. This option is only used for DDR4 daughter boards. If this option is not specified, DDR4 backdoor is disabled.

Example

The following example sets the modules `register`, `counter`, and `memory` as netlist macros and constrain the FPGA resource utilization for each module.

```
% netlistmacro -add {work register} -resource {LUT:100 }
% netlistmacro -add {work counter} -resource {LUT:400 BRAM:1}
% netlistmacro -add {work memory} -resource {LUT:400 BRAM:6}
```

The following example defines two netlist macros and loads source files of each.

```
netlistmacro -instance pcie3_ddr4.chip0_wrapper -package
${HPE_HOME}/netlist_macro_packages/xepic_ddr4_e1 -location 0.A -channel 0
netlistmacro -instance pcie3_ddr4.ep_wrapper -package
${HPE_HOME}/netlist_macro_packages/xepic_pcie_ep
```

13.2.42 netlistmacro_param

Specifies the module or instance name and parameter values of netlist macros.

Syntax

```
netlistmacro_param -module|-instance <name> -attribute <parameter> <value>
```

Where,

- `-module <name>` specifies the module name of the netlist macro.
- `-instance <name>` specifies the instance name of the netlist macro.
- `-attribute <parameter> <value>` defines a parameter. `<parameter>` specifies a parameter name. `<value>` specifies the value of the parameter.

Use curly braces around the value in the following cases:

- The value contains blank spaces.
- The value contains environment variables, and you do not want to perform substitutions on the characters between the braces.

TIP

If you want to use an environment variable to specify the file path of the netlist macro, such as `$env(netlistmacro_src_dir)`, the environment variable needs to be defined before you run the vCom compilation.

To specify multiple values for a parameter, combine all values in one string enclosed by double quotation marks and separate the values with a comma.

Example

Example 1: The following command lines in a compilation script file substitutes the `xdc_path` variable with the working directory and specify the netlist files and constraint files for the `BOX` module:

```
% set xdc_path [pwd]
% puts "xdc_path: ${xdc_path}"
% netlistmacro_param -module BOX -attribute NL_FILE_PATH ./box.vm
% netlistmacro_param -module BOX -attribute XDC_DIR_PATH ${xdc_path}/box
```

Example 2: The following command lines in a compilation script file perform no substitution on the environment variable, `XDC_PATH`, and specify the netlist files and constraint files for the module instantiation `i_box`:

```
% netlistmacro_param -instance i_box -attribute NL_FILE_PATH "./i_box1.vm,
./i_box2.vm"
% netlistmacro_param -instance i_box -attribute XDC_DIR_PATH
${$env(XDC_PATH)}/box}
```

13.2.43 nmport

Removes the holdfix to shorten the timing path. vCom inserts holdfix to the input ports of netlist macros. Note that the debugging of the input port will become not supported any longer.

Syntax

```
nmport -dnot_touch {<input_ports>}
```

Where `<input_ports>` specifies an input port or input ports of netlist macros.

Example

```
% nmport -dont_touch {dut_top.simple1.cnt_in}
```

13.2.44 par_assign

Restricts the placement of a specific group that is already defined.

Syntax

```
par_assign -add {<group_name> <hardware_location>}
```

Where,

- `<group_name>` specifies a group name. The group must be already defined on the `par_group` command line.

- `<hardware_location>` specifies where to assign the group. The syntax of the location is `<board_index>.<FPGA_id>`. You can specify a set of locations and connect them with plus signs (+), for example, `0.A`, `0.A+1.A`, `0`, `0+1`, and `0+1.A+2`.

13.2.45 par_group

Defines a partition group and associates the group with specific netlist hierarchies.

Syntax

```
par_group -instance {<group_name> <instance>}
par_group -exclude {<group_name> <instance>}
par_group -net {<group_name> <net>}
```

Where,

- `-instance` indicates that the specified instance is to be added to the partition group.
- `-exclude` indicates that the specified instance is to be removed from the partition group.
- `-net` indicates that the leaf instances that drive the specified net is to be added to the partition group.
- `<group_name>` defines a partition group that is composed of certain instances. If the group name exists, vCom adds the specified instance to the group.
- `<instance>` specifies an instance to be added to the partition group.
- `<net>` specifies a net to be added to the partition group.

13.2.46 par_opt_priority

Sets the partition strategy.

Syntax

```
par_opt_priority -set <optimization_type>
```

Where `<optimization_type>` can be `min_cut` and `timing_driven`.

13.2.47 partition_option

Customizes the partition area report, *partition_report*.

Syntax

```
partition_option -add {area_report [layer:<level>] [util:<value>]
[res_type:all|fpga|<res_type>]}
```

Where,

- `layer:<level>` specifies the level of design hierarchy. Setting the level to zero disables the generation of area report.
- `util:<value>` specifies the percentage value of resource utilization.
- `res_type:all|fpga|<res_type>` specifies the resource types. The parameter values are case-insensitive. Valid values are listed as follows:
 - `all` reports the resources of all types, including FPGA resources (FF, LUT, LUTM, DSP, BRAM, URAM) and debug logic resources.
 - `fpga` reports only FPGA resources.
 - `<res_type>` specifies a resource type or multiple types separated by the separator bar (|) without any space, for example, `FF|LUT|dsp`. Currently, this option is not fully tested.

13.2.48 prepartition_option

Sets the Area Threshold L. The module whose FPGA resource utilization is smaller than the Area Threshold L is considered as tiny modules. Tiny modules are dissolved in partition.

Syntax

```
prepartition_option -add {dissolve_util <n>}
```

Where `<n>` specifies the area threshold. By default, the area threshold is 0 and tiny module dissolving is disabled.

13.2.49 primary_clock

Defines the source global clock that the design uses.

Syntax

```
primary_clock -set <clock_name>
```

Where `<clock_name>` can be one of the global clocks GC1, GC2, GC3, ... GC8. Global clock GC0 is reserved and cannot be specified on this command.

13.2.50 read_net

Specifies the nets that you want to read at runtime.

Syntax

```
read_net -add <net>
```

Where `<net>` specifies the nets that you can read with the runtime command lines.

13.2.51 remap_optimization

Enable or disables automatic LUT remapping optimization. Customizes the proportion of LUT2, LUT3, LUT4, LUT5, and LUT6.

Syntax

```
remap_optimization -enable|-disable
remap_optimization -global
    {lut_cost <lut2_cost> <lut3_cost> <lut4_cost> <lut5_cost> <lut6_cost>} |
    {lut_cost [LUT5 <lut5_cost>] [LUT6 <lut6_cost>]} |
    {lut_size <n>}
remap_optimization -instance
    {<inst> lut_cost <lut2_cost> <lut3_cost> <lut4_cost> <lut5_cost> <lut6_cost>}
|
    {<inst> lut_cost [LUT5 <lut5_cost>] [LUT6 <lut6_cost>]} |
    {<inst> lut_size <n>}
remap_optimization -dissolve_util {<N>}
remap_optimization -effort {default|high}
```

Where,

- `-enable` enables the automatic LUT remapping optimization.
- `-disable` disables the automatic LUT remapping optimization.
- `-global` configures LUT remapping for all instances in the design.
- `-instance` configures LUT remapping for a specific instance. The constraint added to instances has higher priority than global settings.
- `<inst>` specifies the hierarchy name of an instance.
- `<lut2_cost>`, `<lut3_cost>`, `<lut4_cost>`, `<lut5_cost>`, and `<lut6_cost>` specify the maximum cost of LUT2, LUT3, LUT4, LUT5, and LUT6, respectively.
- `<n>` specifies the largest LUT that can be used. Valid values of `<n>` are 4, 5, and 6. For example, setting the value to 5 disables the mapping of LUT6.
- `<N>` specifies the area threshold. The modules whose FPGA resource utilization are smaller than the specified value get dissolved. Setting the area threshold to 0 disables the module dissolving in LUT remapping. The maximum area threshold you can set is one tenth of the minimum FPGA resource utilization limitation set by the `emulator_util` command.

- `-effort {default|high}` specifies the optimization level of LUT remapping, `default` or `high`. When `high` is specified, the proportion of LUT5s and LUT6s is further decreased. You can use this option tentatively to get better optimization results.

13.2.52 routing_localization

Controls whether to enable routing localization for optimization.

Syntax

```
routing_localization -enable|-disable
```

Where,

- `-enable` enables routing localization.
- `-disable` disables routing localization.

13.2.53 run_to

Runs to any step in the vCom flow chart.

Syntax

```
run_to <step_id>|<step_name>
```

Where,

- `<step_id>` specifies the step ID.
- `<step_name>` specifies the short name of a step.

You can use either `<step_id>` or `<step_name>` to uniquely specify a step of flow chart. For example, `run_to NSplit` is equivalent to `run_to 2.2.2`.

13.2.54 set_false_path

Sets false paths. The command line must be executed before the `design_edit` command.

Syntax

```
set_false_path -from <object> -to <object> -through <object>
```

Where,

- `-from <object>` specifies start points of timing paths. A start point can be an annotated clock, a sequential cell (i.e. clock object), or a clock pin of a cell.

- `-to <object>` specifies end points of timing paths. An end point can be an annotated clock, a sequential cell, or a pin of a cell.
- `-through <object>` specifies the cells or pins that the timing paths pass through.

NOTE

- Currently, you cannot use the `-from`, `-to`, and `-through` options simultaneously on one command line. You can use any two of these options or use one option individually.
- The `-from` and `-to` options can accept multiple clocks or multiple pins and cells, but the two object types cannot be specified for one option in combination.

13.2.55 set_max_delay

Sets the maximum delay for timing paths.

Syntax

```
set_max_delay [-from <object>] [-to <object>] [-through <object>] <delay_value>
```

Where,

- `-from <object>` specifies the starting point. The starting point can be one of the XDC declared clocks, registers, top level input or bidirectional ports, netlistmacro outputs, sequential cell clock pins.
- `-to <object>` specifies the end point. The end point can be one of the XDC declared clocks, registers, top level output or bidirectional ports, netlistmacro inputs.
- `-through <object>` specifies leaf instance ports and hierarchical instance ports.
- `<delay_value>` specifies the value of the desired maximum delay in nanoseconds for paths between start and end points. This is a required option.

NOTE

At least one of the `-from`, `-to`, and `-through` options must be used on the `set_max_delay` command line.

13.2.56 set_phc_speed

Sets the TDM bit rates of the PHC cables.

Syntax

```
set_phc_speed -add {<cable_type> <bit_rate>}
```

Where,

- **<cable_type>** specifies the type of PHC interconnection cable. Valid values are list as follows.
 - **PHC_CABLE_500** (0.5m)
 - **PHC_CABLE_1000** (1.0m)
 - **PHC_CABLE_2000** (2.0m)
- **<bit_rate>** specifies the value of the bit rate. Valid values for HuaPro P1 systems are 1000, 1200, and 1400. Valid values for HuaPro P2E systems are 1000, 1200, 1400, and 1600.

Example

```
% set_phc_speed -add {PHC_CABLE_500 1600}  
% set_phc_speed -add {PHC_CABLE_1000 1400}  
% set_phc_speed -add {PHC_CABLE_2000 1000}
```

13.2.57 streaming_trace_net

Specifies the nets that you want to trace in the streaming probing mode.

Syntax

```
streaming_trace_net -add <net>
```

Where **<net>** specifies the nets to be added as probes.

13.2.58 synchronous_reset

Controls whether vCom performs the optimization on all asynchronously reset registers.

Syntax

```
synchronous_reset -enable|-disable
```

Where,

- **-enable** enables vCom to handle all asynchronously reset registers.
- **-disable** disables the synchronous reset feature.

13.2.59 terminal_assign

Assign a terminal to a specific pin of the interface that is already defined for a specific FPGA on

the `cable_connection` command line.

Syntax

```
terminal_assign -add {<interface> <pin> <signal>} [-weak_drive <value>] [-target_active_edge <value>]
```

Where,

- `<interface>` indicates an interface name that is already specified in the `cable_connection` command line.
- `<pin>` is the index of a valid pin on the connector of the interface.
- `<signal>` is a valid I/O signal in the design. Only port signals in the top module are supported. Signals such as `dut_top.cnt.clk_out_cnt0` and `dut_top.simple1.clk_in` are not supported.
- `-weak_drive <value>` specifies I/O constraints for the input signal or bidirectional/tristate primary IO signals that does not have weak driver. Valid values are listed as follows:
 - `pullup` : Applies a weak logic high level.
 - `pulldown` : Applies a weak logic low level.
 - `retain_state` : Applies a keeper.
 - `target_drive` (default): Applies nothing.
- `-target_active_edge <value>` specifies the clock edge which is used for timing analysis. If the target device works only on the positive edge or negative edge of the fastest user clock, you can set this option to relax the timing requirement. Valid values are listed as follows:
 - `pe` : Uses the positive edge of the fastest user clock.
 - `ne` : Uses the negative edge of the fastest user clock.
 - `be` (default): Uses both the positive edge and negative edge of the root clock.

NOTE

You cannot only use clock signals as input terminals.

Example

Example 1: The following command lines assign the terminal `data_input[5]` to a specific pin of a PHC connector.

```
% cable_connection -def {cable1 PHC 1.A.3}
% terminal_assign -add {cable1 A0 data_input[5]}
```

Example 2: The following command lines assign the terminal `clk` to a specific FPGA pin.

```
% cable_connection -def {a1 ORG 1.A}
% terminal_assign -add {a1 BE27 clk}
```

Example 3: The following command lines assign the terminals `odd`, `odd_in`, `even`, and `even_in` to specific pins of two PHC connectors.

```
% cable_connection -def {PHC_0_A_cable PHC 1.A.3}
% cable_connection -def {PHC_0_B_cable PHC 1.B.20}
% terminal_assign -add {PHC_0_A_cable A0 odd}
% terminal_assign -add {PHC_0_A_cable A1 even_in}
% terminal_assign -add {PHC_0_B_cable D2 odd_in}
% terminal_assign -add {PHC_0_B_cable D3 even}
```

13.2.60 terminal_delay

Inserts a delay bin to a target signal.

Syntax

```
terminal_delay -add {<target_signal> <bin>}
```

Where,

- `<target_signal>` specifies the target signal. The target signal must be a valid I/O signal.
- `<bin>` specifies the delay bin to be inserted. The delay bin must be already defined in the `delay_bin` command line.

If any conflicts, vCom obeys the rules as described below:

- Rule 1: If duplicate insertions of delay bin are present in the `terminal_delay` command lines, the last command line is effective.
- Rule 2: If the delay bin insertion generates a loop, the insertion is forbidden and an error message prints.

Example

Example 1: In the following example, the actual compile result is that the delay bin `binB` is inserted to `tarSig`.

```
% terminal_delay -add {tarSig binA}
% terminal_delay -add {tarSig binB}
```

Example 2: The following example shows two cases of forbidden delay bin insertion because of the loop generation:

```
% delay_bin -def {binA 5 sig0}
% terminal_delay -add {sig0 binA}
```

```
% delay_bin -def {binA 5 sig0}
% delay_bin -def {binB 5 sig1}
% delay_bin -def {binC 5 sig2}
% terminal_delay -add {sig0 binB}
% terminal_delay -add {sig1 binC}
% terminal_delay -add {sig2 binA}
```

13.2.61 terminal_osync

Adds output signals to a sync-group.

Syntax

```
terminal_osync -add {<signal> <group>}
```

Where,

- **<signal>** specifies an output signal to be added to a sync-group. The signal must be a valid primary output signal.
- **<group>** specifies the group name for the synchronized output signals.

If any conflicts, vCom obeys the rules as shown below:

- **Rule 1:** If you insert a delay bin to a target signal with regard to a reference signal, you cannot add both of the two signals to a sync-group.
- **Rule 2:** If you add the reference signal to a sync-group, vCom inserts the delay to the target signal by adding the delay that is inserted because of the sync-group and the delay that is defined by the delay bin.
- **Rule 3:** If you add the target signal to a sync-group, vCom inserts the delay by calculating the larger delay between the delay that is inserted because of the sync-group and the delay that is defined by the delay bin.
- **Rule 4:** Complicated cases with a conflict are not supported yet.

Example for Rule 1:

In the following example, an error returns because of the conflict between `ref` and `tgt`.

```
% delay_bin -def {binA 5 ref}
% terminal_delay -add {tgt binA}
% terminal_osync -add {ref group}
% terminal_osync -add {tgt group}
% terminal_osync -add {sig group}
```

Example for Rule 2:

The following command lines insert the delay bin `binA` (one-clock cycle) to `tgt` in reference to `ref`, and add `ref` and `sig` to the sync-group `group`.

```
% delay_bin -def {binA 1 ref}
% terminal_delay -add {tgt binA}
% terminal_osync -add {ref group}
% terminal_osync -add {sig group}
```

The following figure shows that if `sig` lags behind `ref` one clock cycle and `ref` lags behind `tgt` one clock cycle, the actual inserted delay of `tgt` is two-clock cycles. In the sync-group, `ref` and `sig` appear at the target system at the same time.

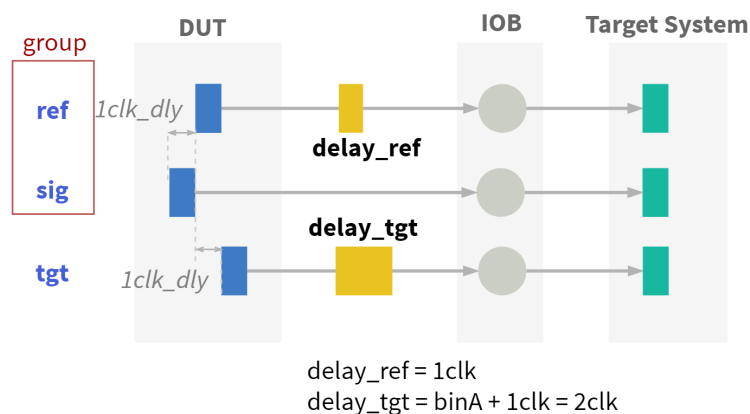


Figure 13-1: Adding the Reference Signal to a Sync-group

Example for Rule 3:

The following command lines insert the delay bin `binA` (one-clock cycle) to `tgt` in reference to `ref`, and add `tgt` and `sig` to the sync-group `group`.

```
% delay_bin -def {binA 1 ref}
% terminal_delay -add {tgt binA}
% terminal_osync -add {tgt group}
% terminal_osync -add {sig group}
```

The following figure shows that if `sig` lags behind `ref` one clock cycle and `ref` lags behind

`tgt` one clock cycle, the actual inserted delay of `tgt` is two clock cycles. In the sync-group, `sig` and `tgt` appear at the target system at the same time.

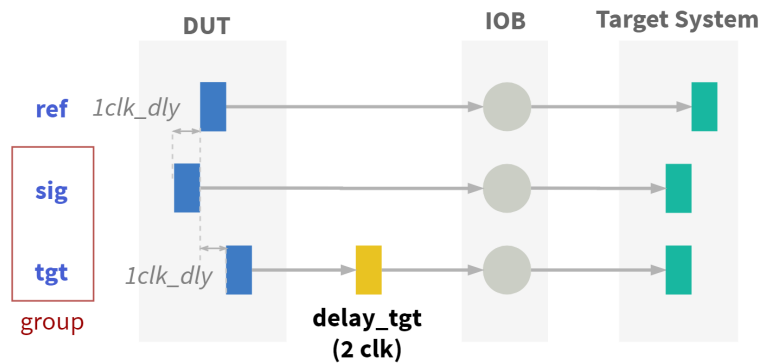


Figure 13-2: Adding the Reference Signal to a Sync-group

Example for Rule 4

In the following complicated example, according to the constraints, `sig1` and `sig5` are synchronized and both of them should be at least 5 cycles slower than `sig2`. On the other hand, `sig2` and `sig3` are synchronized and both of them should be at least 10 cycles slower than `sig5`. This causes a conflict. Therefore, an error message prints for this example.

```
% delay_bin -def {bin1 5 sig1}
% delay_bin -def {bin2 5 sig2}
% delay_bin -def {bin4 5 sig4}
% delay_bin -def {bin5 5 sig5}
% terminal_delay -add {sig0 bin1}
% terminal_delay -add {sig1 bin2}
% terminal_delay -add {sig3 bin4}
% terminal_delay -add {sig4 bin5}
% terminal_osync -add {sig1 group0}
% terminal_osync -add {sig5 group0}
% terminal_osync -add {sig2 group1}
% terminal_osync -add {sig3 group1}
```

13.2.62 terminal_keep

Keeps certain terminals from being optimized.

Syntax

```
terminal_keep -add {<signal_list>}
```

Where `<signal_list>` specifies the user IO signals in the design. You need to separate the signals with a space.

 NOTE

You cannot assign the IO signals that are specified in the `terminal_keep` command line. Otherwise, vCom exits with an error message.

Example

```
% terminal_keep -add {c0_sysm_clk_p}  
% terminal_keep -add {c0_sysm_clk_n}  
% terminal_keep -add {c0_ddr4_act_n}
```

13.2.63 trace_net

Specifies the nets that you want to trace in the physical probing mode. vDbg can only generate the waveforms for those defined probes at runtime.

Syntax

```
trace_net -add <net>  
trace_net -add <inst_path> [-depth <n>] [-third_party_netlist]  
trace_net -port <inst_path> [-depth <n>]
```

Where,

- `-add <net>` specifies the nets to be added as physical probes.
- `-add <inst_path>` specifies the instance in which the nets are to be added as probes.
- `-depth <n>` indicates the nets or ports in the specified instance and its sub-instance with the `<n>` depth level. The default depth-level is `0`.
- `-third_party_netlist` filters out ports that might cause PnR failure or incorrect functionality of design. This option can only be used for the netlist generated by the third-party synthesizer.
- `-port <inst_path>` specifies the instance in which the ports are to be added as probes.

13.2.64 trigger_net

Specifies the nets that you want to work with the trigger feature at runtime.

Syntax

```
trigger_net -add <nets> -clk <clks> [-inst_num <instance_num>]
```

Where,

- `-add <net>` specifies the nets that can trigger the design stop at runtime. This is a required option. If the `dynamic_trigger -enable` is executed, `-add <nets>` specifies the nets that are to be used as ports of ATT instances at runtime. For multiple nets, separate them with spaces.
- `-clk <clks>` specifies all possible clocks that might be used for the specified trigger nets. This is a required option and used for ATT triggers only.
- `-inst_num <instance_num>` specifies the maximum number of the trigger instances for the FPGA where the specified trigger nets are placed. The valid range of instance number is 1-8, including 1 and 8. This option is used for ATT triggers only.

13.2.65 trigger_net_group

Groups the nets that will be triggered in one FPGA.

Syntax

```
trigger_net_group -name <group> -add <nets> -clk <clocks> [-inst_num  
<instance_num>]
```

Where,

- `-name <group>` specifies a group name. This is a required option.
- `-add <nets>` specifies the nets that are placed in one FPGA. This is a required option.
- `-clk <clocks>` specifies all possible clocks that might be used for the specified trigger nets. This is a required option and used for ATT triggers only.
- `-inst_num <instance_num>` specifies the maximum number of the trigger instances for the FPGA where the specified trigger net group is placed. The valid range of instance number is 1-8, including 1 and 8. This option is used for only ATT triggers.

13.2.66 write_net

Specifies the nets that you want to write, force, or release at runtime.

Syntax

```
write_net -add <net>
```

Where `<net>` specifies the nets that you can write with runtime command lines.

13.3 vdbg

Use the `vdbg` command line to start vDbg.

Syntax

```
vdbg [<run_script>] [-gui] [-version]
```

Where,

- `<run_script>` specifies the run script file.
- `-gui` starts vDbg in the GUI mode.
- `-version` prints out the version of vDbg.

If neither of the above options is specified, vDbg is started in the interactive prompt.

For the description of the vDbg commands that you can use to run and debug designs, see the following topics.

13.3.1 after_blocking

Selectively blocks out command execution for a period of time when the design runs in the non-blocking mode.

Syntax

```
after_blocking [<time_value>] [<time_unit>]
```

Where,

- `<time_value>` specifies the number of seconds or milliseconds to wait before timing out. The maximum time value must be less than 2^{48} .
- `<time_unit>` specifies the time unit for the number. Valid time units are `ms` and `s`. The default time unit is `s` if you only specify the time value. If no timeout is specified, the command execution is blocked out indefinitely.

13.3.2 deposit

Sets a signal or a signal group to a specified value for one clock cycle.

Syntax

```
deposit <signal> <value>  
deposit -g|-group <signal_group> <value>
```

Where,

- `<signal>` specifies the signal name.

- `<value>` specifies the value of a signal or a signal group.
- `-g <signal_group>` or `-group <signal_group>` specifies the signal group name.

13.3.3 design

Opens or closes a design.

Syntax

```
design <path> [{-session <session_name> [-overwrite]]}  
design -close
```

Where,

- `<path>` specifies the path of the design files. This is a required option when you open a design.
- `<session_name>` specifies the debug session to be created in the current working directory. vDbg saves the design file links and log files in the session directory named `xxx.session`. If not specified, the debug session is the current runtime working directory where you start vDbg.
- `-overwrite` is used to overwrite the session directory when the specified session name exists. If a directory named `abc.session` already exists without the `-overwrite` option specified, the existing session directory will be renamed to `abc.session.bak`.
- `-close` is used to close the design. This is a required option when you close a design. If you specify a session name when opening the design, use `design -close` will switch the current working path to the father directory of session and close the design.

13.3.4 download

Downloads the bitstream files into the connected FPGA or FPGAs.

Syntax

```
download
```

13.3.5 dump_mem

Dumps the memory data without compile-time presetting. LUTRAM and BRAM are supported. URAM is not supported.

Syntax

```
dump_mem [-format <value_format>] -name <memory_name> -file <file_name> [-start
<mem_addr>] [-end <mem_addr>]
```

Where,

- `<value_format>` specifies the value format. Valid formats are `%b`, `%o`, `%h`, and `%x`. If not specified, the value format is `%b` by default.
- `<memory_name>` specifies the full name of the memory.
- `-format <file_format>` specifies the format of the file in which the data is saved. Values of `<file_format>` are `%raw`, `%readmemb`, `%readmemh`, `%b`, `%o`, and `%h`.

You must specify the `%raw`, `%readmemb`, and `%readmemh` file format options on the `dump_mem` command line. You can optionally specify the `%b`, `%o`, and `%h` file format because vDbg can identify the format from the content.

- `-name <mem_name>` specifies the full name of the user memory where the data is to be saved.
- `-file <filename>` specifies the file where the data is to be saved. If the memory address is optimized, vDbg dumps only the data in the valid addresses of the memory into the specified file.
- `-start <start_addr>` specifies the start address of the accessible memory. The start address must be in hex format, such as `0xa1f`. If you do not specify the start address, vDbg uses the default start address of memory (Instance Start).

The start address and the end address must not cross the board of the memory. The start address must be less than or equal to the end address.

- `-end <end_addr>` specifies the end address of the accessible memory. Same as the start address, the end address must be in hex format. If you do not specify the end address, vDbg uses the default end address of memory (Instance End).

13.3.6 exit

Exits vDbg session.

13.3.7 force

Sets a given value to a signal or a signal group.

Syntax

```
force <signal> <value>
force -g|-group <signal_group> <value>
```

Where,

- `<signal>` specifies the signal to be forced.

You can also force part of bits for a bus signal by using part-selection notation as the signal name, such as `abc[7:4]`. If you specify a bus signal without part-select notation, vDbg forces all the bits.

- `-g <signal_group>` or `-group <signal_group>` specifies the signal group to be forced.
- `<value>` specifies the value of a signal or a signal group.
 - When the signal is single-bit, the `<value>` argument must be either `0` or `1`.
 - When the signal contains several bits, the `<value>` argument can be specified in binary, octal, decimal, or hexadecimal. For example, `'b011`, `'o1234`, `'d088`, and `'hfcfd`. The value must be prefixed by an appropriate letter. A number without the radix prefix is assumed to be decimal. For example, `88` is the same as `'d88`. The value must be adjusted according to the signal length by intercepting the value from lower bits or filling `0` in higher bits.

13.3.8 get_clock

Gets the current frequencies of global clocks. The command must be executed after the `download` command.

Syntax

```
get_clock -clock [<clock_id>]
```

Where `<clock_id>` specifies the index of the global clock that the design uses or not. If the clock index is not specified, you get the current frequency of the global clock that the design uses.

13.3.9 get_div

Shows the current divider count number that the design uses.

Syntax

```
get_div
```

13.3.10 get_max_depth_cycles

Gets the maximum number of trace cycles for which DDR can capture the data.

Syntax

```
get_max_depth_cycles
```

13.3.11 get_phc_vol

Get the current PHC bank voltage.

Syntax

```
get_phc_vol -id <board_index>.<FPGA_id> -bank <PHC_id>
```

Where,

- `-id <board_index>.<FPGA_id>` specifies the FPGA ID.
 - `<board_index>` specifies the board in which the PHC connector is located.
 - `<FPGA_id>` specifies the FPGA in which the PHC connector is located.
- `-bank <PHC_id>` specifies a PHC connector index or PHC connector indexes. For multiple PHC connector indexes, use a comma to separate them, for example, `-bank 1,2,3,4`.

13.3.12 get_time

Gets the run time of design since the beginning of the `run` execution.

Syntax

```
get_time [rclk|<timeunit>]
```

Where,

- `rclk` gets the current cycle number. It can be omitted.
- `<timeunit>` specifies the time unit for the run time. Valid values for the time unit are `fs`, `ps`, `ns`, `us`, `ms`, and `s`.

13.3.13 get_value

Gets the current values of a signal or a signal group.

Syntax

```
get_value [<value_format>] <signal>  
get_value [<value_format>] -g|-group <signal_group>
```

Where,

- `<value_format>` specifies the format of the signal value. You can also specify the value format after the signal name. Valid formats are as follows. If the value format is not specified, binary format is used.
 - `%b` : binary format (default).
 - `%o` : octal format.
 - `%d` : decimal format. `%d` is not supported for signal groups.
 - `%h` : hexadecimal format.
- `<signal>` specifies the full path name using the period hierarchy separator.

13.3.14 help

Gets a list of all runtime commands and prints out detailed reference information.

Syntax

```
help [<command>]
```

Where `<command>` specifies a command name. If no command name is specified, a list of all runtime commands is printed.

13.3.15 hw_server

Connects vDbg to the hardware server where SimCtl runs or releases the connection. The command must be executed after you open a design.

Syntax

```
hw_server <hostname> [-key <key_value>|-keyfile <file_path>]
hw_server <hostname> -location <mapping_of_locations>
hw_server -list
hw_server -release
```

Where,

- `<hostname>` specifies the hostname for SimCtl. This is a required option for building connection. For now, you can start the debug processor on the same host as vDbg. The `<hostname>` can be specified as a period character (.), `localhost` , or `127.0.0.1` .
- `-key <key_value>` specifies the value of a locking key to use the resources that you have locked. This option is only used in the vrm mode.
- `-keyfile <file_path>` specifies the locking key file to use the resources that you have locked. This option is only used in the vrm mode.

- `-location <mapping_of_locations>` specifies the mapping of locations for single-FPGA relocation in the format `[<origin_fpga_id>:]<target_fpga_id>`.
 - `<original_fpga_id>` specifies the ID of the FPGA that the design uses in the format of `<board_index>.<fpga_index>`.
 - The colon separates the original FPGA from the target FPGA. The colon together with the original FPGA ID can be omitted on the command line.
 - `<target_fpga_id>` specifies the ID of the FPGA that is available for relocation.
- `-list` prints out the available hostnames when you open the design. Currently, only `localhost` is listed.
- `-release` is used to release the hardware resource.

13.3.16 memory

The `memory` command can do the following tasks.

- Writing the user data in a specified file into the DUT memory or DDR4 memory.
- Saving the data in the DUT memory or DDR memory into a specified file
- Printing out all or a collection of the DUT memories.

Only one `memory` command can be executed at a time. LUTRAM and BRAM are supported. URAM is not supported.

13.3.16.1 -write

Writes the user data in a specified file into the DUT memory or the DDR4 memory.

Syntax

DUT memory write:

```
memory -write [-format <file_format>] -name <mem_name> {-file <filename> | -mode
<mode_type>} [-start <start_addr>] [-end <end_addr>]
```

DDR4 memory write:

```
memory -write -fpga <fpga_id> -channel <channel_index> {-file
<ddr4_mem_filename> | -mode all0|all1} [-format <ddr4_mem_file_format>] [-start
<start_addr>] [-end <end_addr>]
```

Where,

- `-format <file_format>` specifies the format of the file in the user memory. You must specify the file formats on the `memory -write` command line. Values of `<file_format>` are `%read`, `%raw`, `%readmemb`, `%readmemh`, `%b`, `%o`, and `%h`.
- `-name <mem_name>` specifies the full name of the user memory that contains the data to be written into the DUT memory.
- `-file <filename>` specifies the file that contains the data to be written into the DUT memory.
 - The way of writing data into DUT memory must be specified as the pattern way or file way. In other words, one of the `-file <filename>` and `-mode <mode_type>` options must be used with the `memory -write` command. Meanwhile, `-file <filename>` and `-mode <mode_type>` cannot be specified at the same time.
 - If the memory address is optimized, only the data in the valid addresses of the memory is written into the DUT memories.
- `-mode <mode_type>` specifies the write mode of DUT memory. The supported write modes are as follows:
 - (Default) 32-bit hex value in the `0x<hex_value>` format, such as `0xaa1` and `0x12f`.
 - `all0` : Set all memory data with `0`.
 - `all1` : Set all memory data with `1`.
 - `seq0` : Set all memory data with a sequence from `0`.
 - `seq1` : Set all memory data with a sequence from `1`.
- `-start <start_addr>` specifies the start address of the accessible memory. The start address must be in hex format, such as `0xa1f`. If you do not specify the start address, vDbg uses the default start address of memory (Instance Start). The start address and the end address must not cross the board of the memory. The start address must be less than or equal to the end address.

TIP

When using a file in `%readmemh` format for DDR4 memory write, you can specify the start address either in the file or with the `-start` option. If neither the file nor the command line contains the start address, vDbg uses the 0 address by default. If both the file and command line contain the start address, the address specified in the file takes effect.

- `-end <end_addr>` specifies the end address of the accessible memory. Same as the start address, the end address must be in hex format. If you do not specify the end address, vDbg uses the default end address of memory (Instance End).

 TIP

When using DDR4 memory write, the `-end` option is optional. If the `-end` option is not specified, vDbg calculates the end address according to the data size in the specified file.

- `-fpga <fpga_id>` specifies the FPGA where the DDR4 memory locates. The FPGA ID must be in the format of `<board_index>.<fpga_index>`.
 - `<board_index>` specifies the board index.
 - `<fpga_index>` specifies the FPGA index. Valid values for P2E devices are A-D or 0-3.
- `-channel <channel_index>` specifies the index of channel that is configured by the `bind_ports` command in the standard netlist macro flow or by the `netlistmacro` command in the simplified netlist macro flow.
- `-file <ddr4_mem_filename>` specifies the text file that contains the data to be written into the DDR4 memory.
- `-mode all0|all1` specifies the write mode of DDR4 memory. The supported write modes are as follows:
 - `all0` : Set all memory data with `0` .
 - `all1` : Set all memory data with `1` .
- `-format <ddr4_mem_file_format>` specifies the format of the file. The valid value of `<ddr4_mem_file_format>` is `%fread` currently. This option is only required when the content is the `$fread` binary data.

Example

Example 1: The following command line writes the memory data from a file named *check_point.reference* into the DUT memory in the readmemh format:

```
% memory -write -name dut_top.tp_ram.tp_array -file check_point.reference -
format %readmemh -start 0 -end 0x7ff
```

Example 2: The following command line writes the memory data from a file named *check_point.reference* into the DDR4 memory that connects to Channel 0 of FPGA 0.A:

```
% memory -write -fpga 0.A -channel 0 -file check_point.reference
```

13.3.16.2 -read

Saves the data in the DUT memory or DDR memory into a specified file.

Syntax

```
memory -read {-format <file_format>} {-name <mem_name>} {-file <filename>} [-start <start_addr>] [-end <end_addr>]
```

```
memory -read -fpga <fpga_id> -channel <channel_index> -file <ddr4_mem_filename> -start <start_addr> -end <end_addr> [-format <ddr4_mem_file_format>] [-append]
```

Where,

- `-format <file_format>` specifies the format of the file in which the data is saved. Values of `<file_format>` are `%fread`, `%raw`, `%readmemb`, `%readmemh`, `%b`, `%o`, and `%h`.
- `-name <mem_name>` specifies the full name of the user memory where the data is to be saved.
- `-file <filename>` specifies the file where the data is to be saved. If the memory address is optimized, vDbg dumps only the data in the valid addresses of the memory into the specified file.
- `-start <start_addr>` specifies the start address of the accessible memory. The start address must be in hex format, such as `0xa1f`. If you do not specify the start address, vDbg uses the default start address of memory (Instance Start).

The start address and the end address must not cross the board of the memory. The start address must be less than or equal to the end address.

- `-end <end_addr>` specifies the end address of the accessible memory. Same with the start address, the end address must be in hex format. If you do not specify the end address, vDbg uses the default end address of memory (Instance End).
- `-fpga <fpga_id>` specifies the FPGA where the DDR4 memory locates. The FPGA ID must be in the format of `<board_index>.<fpga_index>`.
 - `<board_index>` specifies the board index.
 - `<fpga_index>` specifies the FPGA index.
- `-channel <channel_index>` specifies the index of channel that is configured by the `bind_ports` command in the standard netlist macro flow or by the `netlistmacro` command in the simplified netlist macro flow.
- `-file <ddr_mem_filename>` specifies the text file to store the data read from the DDR4 memory.
- `-format <ddr4_mem_file_format>` specifies the format of the file. The valid value of `<ddr4_mem_file_format>` is `%fread` currently. This option is only required when the content is the `$fread` binary data.

- `-append` appends the dumped data to the end of the content in the memory file. If this option is not set, the new data overwrites the old data.

Example

Example 1: The following command line exports the memory data to a file named *check_point.result* in the readmemh format:

```
% memory -read {-name dut_top.tp_ram.tp_array} {-file check_point.result} {-format %readmemh} -start 0 -end 0x7ff
```

Example 2: The following command line stores the memory data in a file named *check_point.result* from the address `0` to `0x7ff` :

```
% memory -read -fpga 0.A -channel 0 -file check_point.result -start 0 -end 0x7ff
```

13.3.16.3 -list

Prints out all or a collection of the DUT memories.

Syntax

```
memory -list [-pattern <pattern_exp>]
```

Where,

- `-list` prints out names of specific DUT memories.
- `-pattern <pattern_exp>` lists the DUT memories that match the specified pattern expression. If the pattern is not specified, all DUT memories are listed.

Example

Example 1: The following command lines list all DUT memory names:

```
vdbg> design .
vdbg> memory -list

dut_top.sp_ram_78x2K_1.sp_ram.sp_array
dut_top.sp_ram_78x2K_2.sp_ram.sp_array
dut_top.sp_ram_137x32K.sp_ram.sp_array
dut_top.sp_ram_156x8K_1.sp_ram.sp_array
dut_top.sp_ram_156x8K_2.sp_ram.sp_array
dut_top.sp_ram_156x8K_3.sp_ram.sp_array
dut_top.sp_ram_156x8K_4.sp_ram.sp_array
dut_top.sp_ram_137x128K_1.sp_ram.sp_array
dut_top.sp_ram_137x128K_2.sp_ram.sp_array
```

Example 2: The following command lines list DUT memory names that match the specific pattern expression:

```
vdbg> design .
vdbg> memory -list -pattern (.*).sp_ram.sp_array

dut_top.sp_ram_78x2K_1.sp_ram.sp_array
dut_top.sp_ram_156x8K_1.sp_ram.sp_array
dut_top.sp_ram_137x128K_1.sp_ram.sp_array
```

13.3.17 set_clock

Sets the frequencies of the global clocks and GT clocks. The command must be executed after the `hw_server` command and before `download`.

Syntax

```
set_clock -clock [<clock_id>] <number_of_frequency> <frequency_unit> | -outside
set_clock -gtbank -id <fpga_id> -bank <band_id_or_interface_name> -frequency
<freq_setting>
```

Where,

- `-clock` indicates that the command line is to set the frequencies of global clocks.
- `<clock_id>` specifies the global clock. You can select GC1-GC8 for P1 devices and GC1-GC7 for P2E devices. If the clock ID is not specified, the command sets the frequency of the global clock that the design uses.
- `<number_of_frequency>` specifies the value of the clock frequency. The valid frequency range is from 0.16 MHz to 350 MHz.
- `<frequency_unit>` specifies the frequency unit of clocks. The frequency unit is case-insensitive. Supported frequency units are Hz, KHz, MHz, and GHz.
- `-outside` indicates that the source of the global clock is an external clock. This option is only supported for HuaPro P1. The clock ID must be one of GC1 - GC6, which corresponds to CLKIN 1-6 external clock-in port. GC7 and GC8 are not supported. The `-outside` and `<number_of_frequency> <frequency_unit>` options are mutually exclusive. Use either of them for one global clock.
- `-gtbank` indicates that the command line is to set the frequencies of GT clocks.
- `-id <fpga_id>` specifies the FPGA ID in the format `<board_index>.<fpga_index>`.
- `-bank <band_id_or_interface_name>` specifies the bank ID or interface name. Refer to the tables in [Setting the Frequencies of GT Clocks](#) for supported banks in HuaPro P1 and P2E. You can specify bank IDs, interface names, or both and separate them with commas.

- `-frequency <freq_setting>` specifies the frequency setting in the format `<number_of_frequency><frequency_unit>`. Valid frequency ranges from 60 MHz to 350MHz. Setting the frequency to zero means to disable the GT clock.

13.3.18 set_div

Sets the divider count number as required. The command must be executed after bitstream download finishes.

Syntax

```
set_div <div_number>
```

Where `<div_number>` specifies the number of frequency divider. The number can be any integer that is equal to or more than the divider count number that vCom sets.

13.3.19 set_phc_vol

Sets the PHC bank voltage. The command must be executed before downloading the bitstream files. Otherwise, the voltage settings are ignored.

Syntax

```
set_phc_vol -id <board_index>.<FPGA_id> -bank <PHC_id> -voltage <voltage_number>
```

Where,

- `-id <board_index>.<FPGA_id>` specifies the FPGA ID.
 - `<board_index>` specifies the board in which the PHC connector is located.
 - `<FPGA_id>` specifies the FPGA in which the PHC connector is located.
- `-bank <PHC_id>` specifies a PHC connector index or indexes. For multiple PHC connector indexes, use a comma to separate them, for example, `-bank 1,2,3,4`.
- `-voltage <voltage_number>` specifies the PHC voltage. The default unit is V. If you do not specify the voltages of PHC connectors, vDbg sets the voltages to 0 by default.

Example

Example 1: The following command line sets the voltage of Board 0 FPGA 0 PHC connector 16 to 1.0V:

```
% set_phc_vol -id 0.0 -bank 16 -voltage 1.0
```

Example 1: The following command line sets the voltage of Board 0 FPGA 0 PHC connectors 15,

16, and 18 to 1.8V:

```
% set_phc_vol -id 0.0 -bank 15,16,18 -voltage 1.8
```

13.3.20 set_post_trigger_samples

Sets post-trigger samples. The command must be used before the `tracedb -enable` command.

Syntax

```
set_post_trigger_samples <cycle_num>|<percentage_value>
```

Where,

- `<cycle_num>` specifies the number of cycles with the `rclk` unit or not, such as `100`, `100rclk`, and `100 rclk`. The clocks continue run the specified number of cycles after a trigger happen. The number must be greater than zero. For physical probes, the number must be smaller than the maximum tracing cycle.
- `<percentage_value>` specifies the percentage of the maximum trace size, such as `10%`, and `50%`.

13.3.21 set_time_unit

Sets the default time unit.

Syntax

```
set_time_unit [<value>]
```

Where `<value>` specifies the time unit. Valid time units are `fs`, `ps`, `ns`, `us`, `ms`, and `s`. If the `<value>` option is not specified, `vDbg` prints the current default time unit that you set.

13.3.22 set_trace_size

Sets the trace memory size for the physical probe tracing. The command must be used before the `tracedb -enable` command.

Syntax

```
set_trace_size [<size>]
```

Where `<size>` specifies the amount of traced data to be uploaded into the trace file. If not

specified, the current value of the `set_trace_size` parameter returns. The minimum trace size is 100 rclk.

The `<size>` can be specified in one of the following format:

- `<number>rclk` : Number of probe samples.
- `<number><time_unit>` : Simulation time. Valid values of the time unit: fs, ps, ns, us, ms, and s.
- `<number>%` : Percentage value of the maximum possible trace memory size.
- `<number>` : A number without units. The unit is the same as the current setting of the `set_time_unit` parameter.

Example

Example 1: The following command line sets the trace memory size to 3000 rclk:

```
% set_trace_size 3000rclk
```

Example 2: The following command line sets the trace memory size to 1600 nanoseconds:

```
% set_trace_size 1600ns
```

Example 3: The following command line gets the current value of the `set_trace_size` parameter:

```
% set_trace_size  
1600 ns
```

Example 4: The following command line sets the trace memory size to the maximum possible value.

```
% set_trace_size 100%
```

Example 5: The following command line sets the trace memory size to half of the maximum possible value.

```
% set_trace_size 50%
```

13.3.23 set_vrm

Switches to the vrm mode, and controls whether the relocation is enabled or not.

Syntax

```
set_vrm -on [-relocation|-relocationoff]
```

Where,

- `-on` enables the vrm mode. This is a required option.
- `-relocation` enables the relocation function explicitly.
- `-relocationoff` disables the relocation function.

13.3.24 signal_group

Defines a signal group and adds signals to the group.

Syntax

```
signal_group -add <signal_group> <signals>
```

Where,

- `-add <signal_group>` specifies the signal group name. The specified signal group name must not exist.
- `<signals>` specifies the names of signals or a part of the signal bits separated by a space.

Example

Example 1: The following command line defines the signal group `s0` and includes three signals into the group.

```
% signal_group -add s0 {a[1:0]} {b[2]} {c}
```

Example 2: The following command line defines the signal group `s1` and includes three parts of the signal `out` into the group. The specified signal parts have interval overlaps of `out[14:10]` and have different bit widths.

```
% signal_group -add s1 {out[10:0]} {out[20:12]} {out[14:10]}
```

13.3.25 stop

Stops user clocks manually.

Syntax

```
stop
```

13.3.26 release

Releases the signal or the signal group that is previously forced by using the `force` command.

Syntax

```
release <signal>
release -all
release -g|-group <signal_group>
```

Where,

- `<signal>` specifies the signal to be released.
- `-all` releases all the forced signal values.
- `-g <signal_group>` or `-group <signal_group>` specifies the signal group name.

13.3.27 run

Starts the user clocks. You can control the running time and use the `stop` command to stop the user clocks. If you use the `run` command without any option and do not use the `stop` command, user clocks will stop at 2^{48} rclk.

Syntax

```
run [<number_of_cycle> rclk | <time_value> [<time_unit>] | -nowait]
```

Where,

- `<number_of_cycle>` specifies the number of clock cycles.
- `<time_value>` specifies the length of time to run the design. You can specify the run time by `<time_value>` and `<time_unit>`.
- `<time_unit>` specifies the time unit for the number. Valid values for the time unit are `fs`, `ps`, `ns`, `us`, `ms`, and `s`. If you do not specify the time unit, vDbg use the time unit that is set by the `set_time_unit` command.
- `-nowait` runs the design in the non-blocking mode (no wait mode).

Example

Example 1: The following command line runs the design for 1000 clock cycles

```
% run 1000rclk
```

Example 2: The following command line runs the design for 1000 ns:

```
% run 1000ns
```

Example 3: The following command line runs the design for 1000 of current runtime time unit:

```
% run 1000
```

13.3.28 tracedb

The `tracedb` command can be used to perform the following operations

- Enabling or disabling probe tracing.
- Opening or closing the waveform database.
- Uploading capture data.

Syntax

```
tracedb
    -enable|-disable
    -open <db_name> [-vcd|-xedb|-cwdb] [-overwrite]
    -upload [-start <begin_cycle>] [-end <end_cycle>]
    -close

tracedb -stream
    -enable|-disable
    -open <db_name> [-vcd|-xedb|-cwdb] [-overwrite]
    -flush
    -close
```

Where,

- `-stream` switches to the streaming probe mode.
- `-enable` enables the data capture.
- `-disable` disables the data capture.
- `-open <db_name>` specifies the name of the waveform database to be created.
- `-vcd|-xedb|-cwdb` specifies the format of the waveform to be generated. In the streaming probe mode, only `-xedb` can be used. The supported waveform formats are listed as follows.
 - `-vcd` specifies the VCD waveform file format. This option is not supported for streaming probe mode.
 - `-xedb` specifies the XEDB waveform file format.
 - `-cwdb` specifies the CWDB waveform file format. This option is not supported for streaming probe mode.
- `-overwrite` overwrites the existing waveform database.

The `tracedb` command supports only one active waveform at a time.

In physical and streaming probe mode, if the specified waveform file already exists, and you specify the `-overwrite` option, the existing waveform file is backed up, and a new file is created. If you do not specify the `-overwrite` option for an existing waveform file, an error message prints.

- `-upload` uploads the trace data to the host PC. This option is used for physical probe mode only.
- `-start <begin_cycle>` specifies the start cycle of the time window of interest. If not specified, the start cycle is the first cycle in the RAW data. This option is used for offline debugging only.
- `-end <end_cycle>` specifies the end cycle of the time window of interest. If not specified, the end cycle is the last cycle in the RAW data. This option is used for offline debugging only.
- `-flush` uploads the trace data from the trace buffer to the waveform file. This option is used for streaming probe mode only.
- `-close` closes the current waveform database.

13.3.29 trace_signals

Adds and manages trace signals for waveform generation.

Syntax

```
trace_signals
    -add {[-name <probe_name>] <signals>} [-errorok]
    -list_signals [<instance>|<pattern>]
    -save <tcl_file>
    -import <tcl_file>
    -delete <probe_names>

trace_signals -stream
    -add {[-name <probe_name>] <signals>} [-errorok]
    -list [<probe_names>]
    -save <tcl_file>
    -import <tcl_file>
    -delete <probe_names>

trace_signals -ff
    -enable|-disable
    -add [-waveform_file <probe_name>] <signals> [-errorok]
    -list [<probe_names>]
    -save <text_file>
    -import <text_file>
    -delete <probe_names>
```

```
trace_signals -add {<signals_and_instances>} [-depth <value>]
trace_signals -list_scopes
```

Where,

- `-add` adds a signal or signals to a probe. In the physical and streaming probe modes, all specified objects must be predefined at compile time by using the `trace_net` command. In the GRP probe mode, the signals to be added to probes must be in the visible scope.
- `-name <probe_name>` specifies a user-defined name for the probe to be created. If you do not specify the probe name, vDbg uses a default probe name. This option is used for physical and streaming probes.
- `<signals>` specifies a signal name or multiple signal names separated by space. All signals that are specified on one command line are added into the same probe. The signals that are added in probes are called *probe signals*. Only predefined trace nets can be added to probes. You can use `-add *` to specify all available trace nets that are predefined at compile time to physical probes in a batch. Streaming and GRP probe modes do not support `-add *` currently.
- `-errorok` ignores the errors about the missing nets. This option is used for physical, streaming, and GRP probes.
- `-delete <probe_names>` deletes a probe or multiple probes that have been created. Use a space to separate the probe names. If you do not specify the probe name, vDbg deletes all the probes. Wildcard characters can be used in the `<probe_names>` argument. The asterisk (*) matches any number of characters. This option is used for physical, streaming, and GRP probes.
- `-list <probe_names>` shows the probe signals that exist in the specified probe or in multiple probes. If you do not specify the `<probe_names>` argument with the `-list` option, all the nets within all existing probes are shown. This option is used for physical, streaming, and GRP probes.
- `-list_signals [<instance>|<pattern>]` shows the trace nets that can be added to physical probes.
 - `-list_signals <instance>` shows the trace nets in the specified instance.
 - `-list_signals <pattern>` shows the trace nets that match the specified name pattern. The asterisk (*) in the name pattern matches any number of characters. For example, `-list_signals top.a.b*` shows the trace nets whose name starts with `top.a.b`.
 - `-list_signals` or `-list_signals *` shows all the defined trace nets.
- `-save <tcl_file>` creates a Tcl script that can be executed to re-create the current `tracedb` and `trace_signals` commands. This option is used for physical and streaming

probes.

- `-import <tcl_file>` imports the Tcl script from the file that is saved using the `-save` option. This option is used for physical and streaming probes.
- `-stream` switches to the streaming probe mode.
- `-ff` switches to the GRP probe mode.
- `-enable` enables the data capture. This option is only for GRP probes.
- `-disable` disables the data capture. This option is only for GRP probes.
- `-waveform_file <probe_name>` specifies a user-defined name for the GRP probe and the waveform file at the same time. All objects that are specified on one command line are added into the same probe and waveform file. If you do not specify the name, vDbg uses a default name, *flopflip<n>*. This option is only for GRP probes.
- `-save <text_file>` creates a text file that can be loaded to re-add the GRP probes. This option is only for GRP probes.
- `-import <text_file>` adds the trace signals listed in the text file to the corresponding probes as specified. This option is only for GRP probes.

13.3.30 trigger

Creates, enables, and manages trigger instances as well as hides undesired trigger messages.

Syntax

```
trigger -at
  -readfile <files> |
  -addinst <inst_name> [-overwrite] -module <module_name> -bind_ports {<net0>}
  {<net1>} ... |
  -enable <instance_names>|* |
  -disable <instance_names>|* |
  -listmodule [<module_names>|*] |
  -listinst [<instance_names>|*] |
  -delete <instance_name>|* |
  -silent|-verbose
```

Where,

- `-at` selects the ATT trigger mode. The `-at` option must follow the `trigger` command.
- `-readfile <files>`

Loads the Verilog files that contain definitions of FSM modules. You can specify multiple files and separate them with spaces. Both absolute and relative paths are supported. vDbg uses the path you specify on the `design` command as the root path for the relative paths.

- `-addinst <inst_name> [-overwrite] -module <module_name> -bind_ports {<net0>} {<net1>} ...`

Creates a trigger instance and binds ports.

- `-addinst <inst_name>` specifies an instance name.
- `-overwrite` overwrites the definition of an existing instance. It is an optional parameter.
- `-module <module_name>` specifies the name of the module to be instantiated.
- `-bind_ports {<net0>} {<net1>} ...` specifies a signal or multiple signals to be connected to the input ports of modules. Multiple signals are separated by space. Note that you must specify the nets according to the order of the ports in module declaration.

- `-enable <instance_names>|*`

Enables a trigger instance or multiple instances. `<instance_names>` specifies an instance name or multiple instance names which are separated with spaces. If you specify `-enable *`, all trigger instances are enabled.

- `-disable <instance_names>|*`

Disables a trigger instance or multiple instances. `<instance_names>` specifies an instance name or multiple instance names which are separated with spaces. If you specify `-disable *`, all trigger instances are disabled.

- `-listmodule [<module_names>|*]`

Lists trigger modules. `<module_names>` specifies a module name or multiple module names which are separated with spaces. If you do not specify any module name or specify `-listmodule *`, all trigger modules are listed.

- `-listinst [<instance_names>|*]`

Lists trigger instances. `<instance_names>` specifies an instance name or multiple instance names which are separated with spaces. If you do not specify any instance name or specify `-listinst *`, all trigger instances are listed.

- `-delete <instance_name>|*`

Deletes a trigger instance or multiple instances. `<instance_names>` specifies an instance name or multiple instance names which are separated with spaces. If you specify `-delete *`, all trigger instances are deleted.

- `-silent|-verbose`

Controls the detail level of messages about triggers.

- `-silent` removes the messages of trigger occurrence from the vDbg terminal. The command must be used before running a design.

- `-verbose` shows the messages of trigger occurrence in the vDbg terminal.

13.4 vrmctl

This section describes the vrmctl commands that you can use to query and lock hardware resources in the VRM mode.

13.4.1 add_vrm_host

Adds VRM hosts to the VRM host list.

Syntax

```
add_vrm_host(<vrm_ip>, <vrm_port>)
```

Where,

- `<vrm_ip>` specifies the IP address of the VRM host.
- `<vrm_port>` specifies the port of the VRM host.

13.4.2 connect

Connects the host where you start vrmctl to the VRM host. If no parameter is specified on the `connect` command line, the default VRM host configured by `set_default_vrm_host` and `set_default_vrm_host_for_all_user` are used. If the default VRM host is not set, the first VRM host added by `add_vrm_host` is used.

Syntax

```
connect([<vrm_ip>, <vrm_port>])
```

Where,

- `<vrm_ip>` specifies the IP address of a VRM host.
- `<vrm_port>` specifies the port exported by the vrmd service.

13.4.3 disconnect

Disconnects the host PC/server from the VRM host.

Syntax

```
disconnect()
```

13.4.4 dump_hdf

Dumps the HDF file of the hardware system that the connected VRM host controls.

Syntax

```
dump_hdf(<hdf_path>)
```

Where `<hdf_path>` specifies the file name and file path to save the HDF.

13.4.5 lock_resource

Locks hardware resources.

Syntax

```
lock_resource (design_path = <design_path>, timeout = <timeout>, mode = <mode>,  
savekey = <save_key_switch>, path = <key_path>, location = <resource_location>,  
relocation = <relocation_switch>, ecn = <number>)
```

Where,

- `design_path = <design_path>`

Specifies the path of your runtime working directory. `vrnctl` needs the hardware configuration file and the design information for allocating and locking resources. Only this option is required. If the other options are not specified, VRM locks resources as follows by default.

- The timeout is 5 minutes.
- The locking mode of the required resources is single-session.
- The locking key is printed out directly.
- The scope where VRM searches suitable resources for locking is the whole system.
- The relocation feature is disabled.

- `timeout = <timeout>`

Specifies the time limitation for the locked resources. The time unit can be `s`, `m`, `h`, and `d`. The timeout must be less than 7 days, 168 hours, 10080 minutes or 604800 seconds. If not specified, the timeout is 5 minutes by default.

- `mode = <mode>`

Specifies the locking mode of the required resources, `single_session` (default) or `multi_session`.

- `savekey = <save_key_switch>`

Chooses whether to save the locking key. Valid values are `True` and `False` (default).

- When you set `savekey = True`, VRM saves the locking key in a file.
- When you set `savekey = False` or do not set this option, VRM only prints out the locking key.

- `path = <key_path>`

Specifies the path to save the locking key file. The name of the key file is fixed to `.top_module.lkey`. If you set `savekey = True` but do not specify the path, VRM saves the locking-key file in the design path by default.

- `location = <location>`

Specifies the scope where VRM searches suitable resources for locking. For example, `[0.1-0.3, 1]` means that VRM tries to lock resources in board 0 FPGAs A-C and all FPGAs on board 1.

- `relocation = <relocation_switch>`

Selects whether to enable the relocation function. Valid values are `True` and `False` (default). By default, the relocation function is disabled.

- When you set `relocation = True`, VRM will search for resources that are equivalent to the specified resources if the specified resources are unavailable.
- When you set `relocation = False`, VRM will return failure if the specified resources are unavailable.

- `ecn = <number>`

Specifies the number of extra global clocks to be locked. Valid value ranges from `0` (default) to `6`. { } This option is only used for HuaPro P2E. { } By default, only the global clock that the design uses is locked. Setting the parameter to `6` locks all global clocks, GC1-GC7. For example, if the design uses the GC1 global clock, setting the parameter to `3` locks four clocks, GC1-GC4.

13.4.6 rm_session

Removes the session that exits unexpectedly.

Syntax

```
rm_session("<session_id>")
```

Where `<session_id>` specifies the session ID that is displayed using `show_session` command.

13.4.7 rm_vrm_host

Removes VRM hosts from the VRM host list.

Syntax

```
rm_vrm_host(<vrm_ip>, [<vrm_port>])
```

Where,

- `<vrm_ip>` specifies an existing IP address in the VRM host list.
- `<vrm_port>` specifies the port of the VRM host. If not specified, the default port is 9000.

13.4.8 set_default_vrm_host

Set the default VRM host.

Syntax

```
set_default_vrm_host(<vrm_ip>, [<vrm_port>])
```

Where,

- `<vrm_ip>` specifies the IP address of the VRM host.
- `<vrm_port>` specifies the port of the VRM host. If not specified, the default port is 9000.

13.4.9 set_default_vrm_host_for_all_user

Sets the default VRM host for all users. The command is only for root users.

Syntax

```
set_default_vrm_host_for_all_user(<vrm_ip>, [<vrm_port>])
```

Where,

- `<vrm_ip>` specifies the IP address of the VRM host.
- `<vrm_port>` specifies the port of VRM. If not specified, the default port is 9000.

13.4.10 show_locked_resource

Lists your locking keys and corresponding design name, expiration time, and design path.

Syntax

```
show_locked_resource()
```

13.4.11 show_system

Shows the resource status. The command must be used after the host PC/server is connected to the VRM host.

Syntax

```
show_system([detail = True])
```

Where `detail = True` queries information about daughter boards and cable connection.

13.4.12 show_session

Shows the resource usage of the vDbg sessions connects to the VRM node.

Syntax

```
show_session()
```

13.4.13 show_vrm_hosts

Shows the information of the VRM hosts that you added.

Syntax

```
show_vrm_host()
```

13.4.14 unlock_resource

Cancels the hardware resource locking manually

Syntax

```
unlock_resource (<locking_key>)
```

Where `<locking_key>` specifies the key that uniquely identify the locked resources.

14 Using the RLM License

XEPIC uses Reprise License Manager (RLM) to control and manage customer licenses. This topic describes starting the RLM license server and also provides useful tips for troubleshooting possible licensing problems.



TIP

The license server is typically started and maintained by the system administrator (IT).

14.1 License Components and Files

X-EPIC software installers include the following components that licensing requires:

- **rlm server:** a binary file named *rlm*
- **xepic server:** a binary file named *xepic*
- **RLM utilities:** *rlmdown*, *rlmstat*, and *rlmutil*

A license server consists of one *rlm* process and at least one *xepic* process. The license server must keep running when you use any licensed software products. *rlmdown* can shut down license servers. *rlmstat* can obtain status from license servers. *rlmutil* is an all-in-one utility.

In addition to the required licensing components, you need a license file which describes your rights to the products to start a license server. To obtain a license, contact your XEPIC Sales Representative and provide Sales with the MAC address of the server. Each license is bound with one MAC address.

The license file contains the information that configures the license servers and describes all the licenses granted to you from the independent software vendor (ISV). The license file has the following three types of lines:

- **HOST Line:** Specifies the machine where the *rlm* server runs.
- **ISV Line:** Specifies information about the *xepic* server.
- **LICENSE Lines:** Specify the usage rights to an X-EPIC software product or products.

The format of a license file is shown as follows:

```
#
#-----
# Date: mm/dd/yyyy hh:mm
# Licensed to customer
#-----
# Copyright (c) 2020-2022 by XEPIC Technology Co., Ltd.
# ALL RIGHTS RESERVED http://www.x-epic.com
#-----
```

```
# Use of the license keys provided herewith is governed by your x-epic End User
software License Agreement, including any applicable supplements and addenda
#
# If you feel that keys are missing(or incorrect), please contact your x-epic
account manager
#
# NOTICE: DO NOT DELETE/MODIFY HOSTID OR ANY OTHER KEYS IN THIS FILE , ONLY
HOSTNAME AND PORT CAN BE MODIFIED
#-----
#

HOST <hostname> <hostid> <port-number>
ISV <isvname>
LICENSE <isv> <product> <version> <exp-date> <count> hostid=<hostid-string>
      start=<start-date> _ck=<license-line-checksum> sig=<license-key>
```

More information about the license file is listed as follows:

Line	Character	Description
HOST	<hostname>	Specifies the standard hostname of the rlm server. This name is not an input to the license key signature algorithm, so you can change the name at any time without affecting your licenses.
	<hostid>	Specifies the unique identifier for a computer or server.
	<port-number>	Specifies the RLM port, a port that RLM attempts to use for communication. This port number is not an input to the license key signature algorithm, so you can change the port to any available port.
ISV	<isvname>	Specifies the xepic server.
LICENSE	<isv>	Specifies the name of the ISV that grants the rights. ISV is fixed to <code>xepic</code> .
	<product>	Specifies the product feature for which license rights are granted.
	<version>	Specifies the latest product version that the license supports, in the form of <code>N.M</code> .
	<exp-date>	Specifies the license expiration date, in the form of <code>DD-MM-YYYY</code> .
	<count>	Specifies the number of licenses that is granted. <code>0</code> or <code>uncounted</code> means an uncounted, node-locked license.

Line	Character	Description
	hostid= <hostid- string>	Specifies the host on which the licenses can be used.
	<start-date>	Specifies the license start date, in the form of DD-MM-YYYY .
	_ck=<license- line- checksum>	Specifies the SHA-256 checksum of the license line.
	sig=<license- key>	Specifies the digital signature of all the license data.

14.2 Starting the License Server

Before starting a license server, make sure that you have obtained a license file and installed an X-EPIC software product.

Perform the following steps to start a license server.

1. Place the license file in the license server machine.

In the initial license file, the RLM port is 5053 and the hostname is localhost . Replace <hostname> and <port-number> with actual values if necessary.

IMPORTANT

Except for <hostname> and <port-number> on the HOST line, other characters are fixed by the license file. Do NOT change the fixed characters. Otherwise, the license file will become invalid.

2. Execute the `rlm` command as follows:

```
<installation_dir>/xepic_license/rlm [-c <license_file>]
```

Where,

- <installation_dir> is the directory where you install X-EPIC software.
- -c <license_file> specifies the path name of the license file or a directory containing license files.

To better specify which license file to use, you can use the `-c` option. If the `-c` option is not used, `rlm` checks the path that is specified by the `RLM_LICENSE` environment variable. If neither the `-c` option nor `RLM_LICENSE` is set, `rlm` checks any `.lic` files in both the `rlm` binary directory, `<installation_dir>/xepic_license`, and the current working directory.

TIP

By default, `rlm` runs in the foreground and occupies the terminal. To run `rlm` in the background, you can add the ampersand symbol (`&`) at the end of the `rlm` command line.

- Set the `RLM_LICENSE` environment variable to enable X-EPIC software to find the license file or license server in either of the following format:
 - `<license_file_pathname>`
 - `<port>@<host>`, where `<port>` is the RLM port in the license file, and `<host>` is the hostname in the license file. You can use the `<port>@<host>` format when you start the license server on a different machine from X-EPIC software.

14.3 Maintaining the License Server

After starting a license server, you can manage the license server as described below.

- Check status of the license server in either of the following ways:
 - Execute the `rlmstat -a` command to print all status from the `rlm` server and the `xepic` server.
 - In your web browser, enter `http://<rlm_server_host_name>:<web_server_port>` and then click **status** in the left command list to get status information.
- Use the `rlmdown` utility to shut down the `rlm` server or the `xepic` server.

Syntax:

```
rlmdown [-i <isv>] [-c <license_file>]
```

Where,

- `-i <isv>` specifies the server. To shut down only the `xepic` server, you can specify `<isv>` as `xepic`. To shut down the `rlm` server itself, you can specify `<isv>` as `RLM`.
- `-c <license_file>` specifies the path of a license file or a directory containing license files. `rlmdown` shuts down all license servers that are specified in the license file or files.

If you do not specify any options on the `rlmdown` command line, `rlmdown` shuts down the first license server that is specified in the `RLM_LICENSE` environment variable.

- Run a single license server for multiple license files.

You might need to use multiple X-EPIC software products and get more than one license file. To manage the RLM license server system efficiently, you can run a license server as follows:

1. Execute `rlmdown -i RLM` to shut down the license server on the license server machine.
2. Place all license files (named *something.lic*) in a directory and use the same RLM port in the files.
3. Restart the license server and specify the directory in the `-c <license_file>` option.
`rlm` processes all license files in the directory and uses a single RLM port and a single Web Server port.

- Update the license when the license server is running.

If you add new, additional license files, you must put the new license files in the same directory as the old one then do an `rlmreread` on the license server.

Syntax:

```
rlmutil rlmreread xepic
```

14.4 Ports Used in the RLM Licensing System

Port occupation is a common type of blocking issues when you start multiple license servers or restart a license server. RLM uses three ports to communicate between the license server and X-EPIC software. The ports are described and designated to help you configure RLM ports.

The screenshot shows the 'Web Server Port' interface for 'Reprise License Server Administration'. The status for 'rlm' on 'SFG-Infra0' is shown as 'port 5153'. The RLM Port is 5153. The RLM software version is v13.0 (build 2), RLM comm version is v1.2, debug log file is _stdout_, and license file is license%*.lic. The rlm Statistics table shows start time, messages, and connections. The ISV Servers table shows the status of the xepic server.

rlm Statistics		Since Start	Since Midnight	Recent
Start time	01/05 09:33:26	01/05 09:33:27	01/05 09:33:27	
Messages	0 (0sec)	0 (0sec)	0 (0sec)	
Connections	0 (0sec)	0 (0sec)	0 (0sec)	

ISV Servers									
Name	port	Running	Restarts	Server Status	License Usage	Debug Log	REREAD	TRANSFER	SHUTDOWN
xepic	56913	Yes	0	xepic	xepic	xepic	xepic	xepic	xepic

Figure 14-1: Ports shown on the RLM Web Server

- **RLM port:** X-EPIC Software use the RLM port to find the rlm server. The RLM port is configured on the HOST line in license files. The default RLM port is 5053.
- **ISV port:** After X-EPIC software finds the rlm server, software uses the ISV port to communicate back and forth. ISV port is assigned with a random port by default when a license server starts. The ISV port can be configured on the ISV line in license files. For example, `ISV xepic port=5678`.
- **Web Server port:** The Web Server port defaults to 5054, and can only be configured when you start a license server.

14.5 Troubleshooting Licensing Problems

If you encounter some licensing issues, refer to the following instructions for troubleshooting. For details about using RLM, refer to [RLM Blog](https://www.reprisesoftware.com/blog) (<https://www.reprisesoftware.com/blog>).

14.5.1 RLM Port Occupation

Problem

License server startup fails with an error message like the following:

```
Port 5053 in use, waiting...
```

Resolution

- Port 5053 might be occupied by an existing process, possibly by an RLM-licensed product. Check if the existing process is needed. If not, shut down the process. If the existing process is needed, you can edit the license file to specify another RLM port on the HOST line and then restart the license server.

NOTE

Do NOT use port 5054 for the rlm server in the license file. Port 5054 is used for the RLM Web Server by default.

- When you restart a license server, port 5053 might be occupied by the previous rlm process. In this case, you can use the `rlmdown` utility with the following command line to shut down the rlm server and then restart the server. Note that `RLM` must be in capital letters.

```
rlmdown -i RLM
```

14.5.2 Web Server Port Occupation

Problem

License server startup fails with an error message like the following:

```
Cannot bind Web Server port xxxx, exiting
```

Resolution

The Web Server port is 5054 by default. If the Web Server port is occupied by a process and the process cannot be shut down, you can use the `-ws` option on the `rlm` command line as follows to configure the Web Server port:

```
rlm -ws <port> <other_options>
```

Where `<port>` specifies the Web Server port. `<other_options>` specifies other options of the `rlm` command.



TIP

Use a port for the Web Server that is different from the RLM port.

14.5.3 RLM Lock File Problems

Problem

Starting an additional license server fails with an error message like the following:

```
(xepic) Cannot set server lock - exiting
(xepic) This usually means another copy of "xepic" is running
(rlm)
(rlm) xepic:0 initialization error: 8, not restarting
(rlm)
(rlm) xepic - lockfile problems
```

Resolution

Delete the lock file, `/var/tmp/.rlmlockxepic`. Then restart the license server.

14.5.4 Cannot Connect to Server

Problem

When you invoke an X-EPIC software, an error message is reported as follows:

```
Cannot connect to server (comm: -4)No route to host (errno: 113)
```

Resolution

- The license server machine is down. Power on the machine and start the license server.
- The license server is not running. Start the license server.
- The hostname or port in the license file is incorrect. Contact the system administrator for the correct hostname and port.
- The `RLM_LICENSE` environment variable is set to an incorrect path of license files or license servers. Contact the system administrator for the correct hostname and port.
- A Firewall is blocking access to the rlm server or the xepic server. For more information about the resolution, see [Firewall Issues](#).

14.5.5 Firewall Issues

Problem

A firewall does not permit X-EPIC software to access the RLM port or ISV port.

Resolution

Perform the following steps to use RLM across a firewall.

1. On the license server machine, open the license file.
2. On the ISV line, add `<isv-port>` as follows, and then save the file.

```
ISV xepic port=<isv-port>
```

3. Open the port through your firewall.
4. Restart the license server to make the port change take effect.

Another method is stopping the firewall with the `systemctl stop firewall` command line, but the method is not recommended.

14.5.6 Cannot Recognize ISV Name

Problem

- License server startup fails with an error message like the following:

```
Cannot find the ISV server binary
```

- X-EPIC software startup fails with an error message like the following:

```
rlm does not know this ISV name (-175)
```

Resolution

- The ISV line in the license file is modified. Change the ISV line back to `ISV xepic`.
- RLM cannot find the `xepic` binary file. Use either of the following methods:
 - Add the `<installation_path>/xepic_license` directory into your `PATH` environment variable.
 - Make sure that the `xepic` binary file is located in the same directory as the `rlm` binary file.

14.5.7 No ISV Servers

Problem

License server startup fails with an error message like the following:

```
(No ISV servers to start)
```

Resolution

Check the license file and make sure that the ISV line is `ISV xepic`.

14.5.8 Cannot Find rlm

Problem

An error message indicates that the `rlm` command is not found when you start the license server.

Resolution

Execute the `rlm` command in the `rlm` binary directory, `<installation_dir>/xepic_license`. You also can add the `rlm` path into your `PATH` environment variable.

14.5.9 Hostname Translation Failure

Problem

An error message indicates that the hostname cannot be translated when you start the license server or software products.

Resolution

Add the `rlm` license server hostname, which is specified in the license file or `RLM_LICENSE`, with the corresponding IP address to the `/etc/hosts` directory.

14.5.10 UDP Port Occupation

Problem

Starting a license server succeeds with a warning message like the following:

```
... error binding UDP port 5053, port in use
This is probably due to another copy of RLM running
While not fatal, this instance of RLM won't respond
to broadcast requests.
```

Resolution

UDP port occupation does not block the startup of license servers. To ignore the warning message, you can use the `-noudp` option on the `rlm` command line.

Appendix A - File Templates and Examples

This topic provides templates and examples of files that you need to prepare for HPE inputs, including - [HDF](#)

- [Compilation Script File](#)
- [Run Script File](#)

A.1 HDF

A template of HDFs are shown as follows. You can take the template as reference when you prepare the HDF that the design needs.

```
## Specify HDF version
VERSION += <hdf_version>

## Specify device type
DEVICE += <hardware_type>

## Add board. index and type are required. SN and CLK are optional.
BOARD += {"index": <board_index>, "type": <board_type>, "SN": <SN_number>,
"CLK": <clk_board_index>}

## Remove FPGA. board and fpga are required.
FPGA -= {"board": <board_index>, "fpga": <fpga_index>}

## Add connection. type and connection are required.
CABLE += {"type": <cable_type>, "connection": <connector_id_pair>}

## Add daughter board. type and connection are required.
CARD += {"type": <card_type>, "connector": <connector_id>,"protocol":
<connection_mode>,"channel": <connection_parameter>, "peer": <card_host_ip>}
```

For a large 4-FPGA design which runs on two HuaPro P1 devices, XEPIC provides a PHC connection solution for better runtime performance. The HDF that defines the recommended hardware topology is shown as follows. Contact XEPIC supports if you want to know more about the solution.

```
## Specify HDF version
VERSION += 1.0

## Specify device type
DEVICE += P1

## Add board 0 and 1
BOARD += {"index": 0, "type": "TWO_CHIP_BOARD", "CLK": 0}
```



```

BOARD += {"index": 1, "type": "TWO_CHIP_BOARD", "CLK": 0}

## PHC connection for 16G DDR daughter boards
CARD += {"type": "CAPC_16G_V1", "connector": "0.A.22"}
CARD += {"type": "CAPC_16G_V1", "connector": "0.B.1"}
CARD += {"type": "CAPC_16G_V1", "connector": "1.A.22"}
CARD += {"type": "CAPC_16G_V1", "connector": "1.B.1"}

## PHC connection through interconnection daughter boards
CABLE += {"type": "INTERCONN_CARD_H", "connection": "(0.A.9, 0.B.16)"}
CABLE += {"type": "INTERCONN_CARD_H", "connection": "(0.A.10, 0.B.15)"}
CABLE += {"type": "INTERCONN_CARD_H", "connection": "(0.A.11, 0.B.14)"}
CABLE += {"type": "INTERCONN_CARD_H", "connection": "(0.A.12, 0.B.13)"}
CABLE += {"type": "INTERCONN_CARD_H", "connection": "(0.A.1, 0.B.24)"}

CABLE += {"type": "INTERCONN_CARD_H", "connection": "(1.A.9, 1.B.16)"}
CABLE += {"type": "INTERCONN_CARD_H", "connection": "(1.A.10, 1.B.15)"}
CABLE += {"type": "INTERCONN_CARD_H", "connection": "(1.A.11, 1.B.14)"}
CABLE += {"type": "INTERCONN_CARD_H", "connection": "(1.A.12, 1.B.13)"}
CABLE += {"type": "INTERCONN_CARD_H", "connection": "(1.A.1, 1.B.24)"}

## PHC connection through 0.5 m interconnection cables
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.16, 1.A.16)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.21, 1.A.21)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.B.8, 1.B.8)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.B.9, 1.B.9)"}

## PHC connection through 1.0 m interconnection cables
CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.A.2, 1.B.22)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.A.8, 1.B.23)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(1.A.2, 0.B.22)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(1.A.8, 0.B.23)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.A.4, 1.A.4)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.A.3, 1.B.21)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(1.A.3, 0.B.21)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.B.4, 1.B.4)"}

```

Here is an example of HDF for HuaPro P2E.

```

## Specify HDF version
VERSION += 2.0

## Specify device type
DEVICE += P2E

## Add board 0 and 1
BOARD += {"index": 0, "type": "FOUR_CHIP_BOARD", "SN": "RRD000000000000",
"CLK": 0}
BOARD += {"index": 1, "type": "FOUR_CHIP_BOARD", "SN": "RRD000000000000",
"CLK": 0}

```

```
## PHC connection
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.2, 0.B.2)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.9, 0.B.9)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.33, 0.B.33)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.34, 0.B.34)"}

CABLE += {"type": "PHC_CABLE_500", "connection": "(0.C.2, 0.D.2)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.C.9, 0.D.9)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.C.33, 0.D.33)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.C.34, 0.D.34)"}

CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.10, 0.D.10)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.16, 0.D.16)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.21, 0.D.21)"}

CABLE += {"type": "INTERCONN_CARD_V", "connection": "(0.A.17, 0.D.20)"}
CABLE += {"type": "INTERCONN_CARD_V", "connection": "(0.A.18, 0.D.19)"}
CABLE += {"type": "INTERCONN_CARD_V", "connection": "(0.A.19, 0.D.18)"}
CABLE += {"type": "INTERCONN_CARD_V", "connection": "(0.A.20, 0.D.17)"}

CABLE += {"type": "PHC_CABLE_500", "connection": "(0.B.10, 0.C.10)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.B.16, 0.C.16)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.B.21, 0.C.21)"}

CABLE += {"type": "INTERCONN_CARD_V", "connection": "(0.B.17, 0.C.20)"}
CABLE += {"type": "INTERCONN_CARD_V", "connection": "(0.B.18, 0.C.19)"}
CABLE += {"type": "INTERCONN_CARD_V", "connection": "(0.B.19, 0.C.18)"}
CABLE += {"type": "INTERCONN_CARD_V", "connection": "(0.B.20, 0.C.17)"}

CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.11, 0.C.11)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.A.15, 0.C.15)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.A.32, 0.C.32)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.A.35, 0.C.35)"}

CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.B.11, 0.D.11)"}
CABLE += {"type": "PHC_CABLE_500", "connection": "(0.B.15, 0.D.15)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.B.32, 0.D.32)"}
CABLE += {"type": "PHC_CABLE_1000", "connection": "(0.B.35, 0.D.35)"}

```

A.2 Compilation Script File

Here is a template of the compilation script file for a basic debug flow. You can copy, modify, and use the script as you require. Note that this template does not include all the compile-time commands. See [List of vCom Tcl Commands](#) for a list of all compile-time commands.

```
## Set Auto Loop Breaking
set env(AUTO_LOOP_BREAK) 1

```

```

# set env(LOOP_DETECT_REMOVE_VCOMBREAKER) 1
# set env(DISABLE_LOOP_DETECT) 1

##### Read Design #####
design_read [-work <lib_name>] -netlist {<fileA> <fileB> \
                                     <fileC> <fileD> }
design_read [-work <lib_name>] -netlist {<fileE>}
design_read [-work <lib_name>] -netlist {<fileF>}

##### Specify Netlist Macro Constraints in Simplified Flow #####
netlistmacro -instance <instance_path> -package <package_path>

##### Specify Netlist Macro Constraints in Standard Flow #####
# design_read [-work <lib_name>] -netlist {<netlistmacro_file>}
# netlistmacro -add {<lib> <module>} -resource {LUT:<x> LUTM:<x> DSP:<x> FF:<x>
BRAM:<x>} [-set_driverless high|low]
# netlistmacro_param -module <module_name> -attribute <parameter> <value>
# netlistmacro_param -instance <instance_name> -attribute <parameter> <value>

##### Load Design #####
design_load [-library {<lib_list>} -top <top_name>

##### Specify HDF #####
emulator_spec -add {file <file_name>.hdf}
# emulator_spec -add {file <file_name>.json}
# emulator_spec -reconstruct_interconnection {file <file_name>.json}

##### Specify Loop Breaking #####
# break_all_loops -add {<hier_inst_path> ...}
# delay_cell -add {<sink_port_path> ...}

##### Define Clocks #####
create_clock -sig_name <sig_clk> -frequency <freq_num>KHz/MHz/GHz
# primary_clock -set <clock_name>
# create_generated_nmclock -name <clock_name> -port <nm_clock_port> [-
insert_bufg]

# dump_clock_region_ports
## Convert gated-clock logic paths into data paths
# clock_port -exclude {<port_path> ...}

##### Specify Exceptional Constraints #####
# set_false_path -from <object> -to <object> -through <object>
# annotate_xdc_clock -clock <clock_name> [-wire <object> | -port <object>]

##### Specify IO Constraints #####
cable_connection -def {<interface_name> <PHC|ORG> <board_id>.<FPGA_id>.<
connector_id>
terminal_assign [-disable_ff_insert] -add {<interface> <pin> <signal>}
}
terminal_keep -add {<signal_list>

```

```

delay_bin -def {<bin> <delay> <reference_signal>}
terminal_delay -add {<target_signal> <bin>}
terminal_osync -add {<signal> <group>}

## Remove the FD that to be inserted to the input ports of neltist macros
# nmport -dnot_touch {<input_ports_of_nm_instance>}

##### Specify Debug Features #####
read_net -add <net>
write_net -add <net>
memory_access -add <memory>
trace_net -add <net>
trace_net [-depth <n>] -add <instance>
streaming_trace_net -add <net>

## For ATTs
# dynamic_trigger -enable
# trigger_net -add <nets> -clk <clks> [-inst_num <instance_num>]
# trigger_net_group -name <group> -add <nets> -clk <clks> [-inst_num
<instance_num>]

##### Specify Utilization Constraints #####
emulator_util -add {<location> [<min>] <max>}
emulator_util -add {default 0}
emulator_util -add {<board_index>.<FPGA_id> LUT:x% LUTM:x% DSP:x% FF:x% BRAM:x%
# emulator_spec -reconstruct_interconnection {file <file_name>}

##### Specify PHC Connection #####
# direct_phc -enable
# set_phc_speed -add <1000|1200|1400|1600>

##### Specify Optimizatin Strategies #####
# routing_localization -enable
# clock_localization -disable
clock_recognition -enable
elim_common_logic -enable -time 3

# logic_replication -enable
# logic_replication -enable [-slack_threshold <value>] [-delay_threshold
<value>]

##### Specify Partition Strategies #####
# par_opt_priority -set timing_driven
# indivisible_util -add {0.1}
# indivisible_module -add {<module_name>}
# indivisible_group -add {<group_name>}

par_group -instance {<group_name> top.moduleB}
par_group -exclude {<group_name> top.moduleB.moduleC}

par_group -net {<group_name> top.netA top.netB}

```

```
par_assign -add {<group_name> <board_index>.<FPGA_id>}

##### Design Edit #####
# design_edit_option -add {removeLoadless <N>}
# design_edit_option -add tieDriverlessLow|tieDriverlessHigh
design_edit

## Enable auto cabling
interconn_dump -file <filename>.json
# run_to CLocal
# interconn_dump -file <filename>.json

##### Design Generation #####
design_generation
```

A.3 Run Script File

Here is a template of *run.tcl* for a basic memory-access and physical-probe flow in the non-VRM mode. You can copy, modify, and use the script as you require.

Note that this template does not include all the runtime commands. See [List of vDbg Tcl Commands](#) for a summary of runtime commands.

```
#Programming the FPGAs
puts "Start setting"
design <path> [{-session <session_name> [-overwrite]]}
hw_server localhost
puts "Done"
puts "Downloading the bitstream files to FPGAs ..."
download
puts "Done"

#Enabling physical probes
puts "Adding physical probes"
set_trace_size <size>
tracedb -enable
tracedb [-open] <db_name> -vcd
trace_signals -add [{-name <probe_name>] <signals>}
puts "Done"

#Running the design
puts "Running the design ..."
force <signal> <value>
run [<number_of_run>] [<time_unit>]
puts "Done"

#Reading DUT memory
puts "Reading DUT memory ..."
memory -read {-format <file_format>} {-name <mem_name>} {-file <filename>} [-
start <start_addr>] [-end <end_addr>]
puts "Done. Check the memory data."

#Generating the waveform
puts "Uploading waveform ..."
tracedb -upload
tracedb -close
puts "Done. Check the waveform file."
exit
```

Appendix B - Interface Signals for DDR4 Memory Backdoor

The following table lists all interface signals that are required for DDR4 memory backdoor. All the listed signals must be bound for each channel to be used. The asterisk * in the signal names corresponds to the channel index, 0, 1, 2, or 3.

Table B-1: Interface Signals for DDR4 Memory Backdoor

Name	Width	I/O	Description
User Interface			
dut_ddr*_calib_complete	1	O	Active-high. Indicates that the initialization and calibration are complete, and the system is ready for write and read transactions.
AXI4 Interface			
dut_axi_s*_clk	1	I	Global clock.
dut_axi_s*_araddr	64	I	The address of the first transfer in a read transaction.
dut_axi_s*_arburst	2	I	Burst type of read transaction.
dut_axi_s*_arcache	4	I	The way how a read passes through a system.
dut_axi_s*_arid	4	I	ID tag for a read transaction.
dut_axi_s*_arlen	8	I	The number of transfers in a read transaction.
dut_axi_s*_arlock	1	I	Information about the atomic characteristics of a read transaction.
dut_axi_s*_arprot	3	I	Protection characteristics for a read transaction.
dut_axi_s*_arqos	4	I	Quality of Service for a read transaction.
dut_axi_s*_arready	1	O	Indicates whether a read transaction can be initiated to slave.
dut_axi_s*_arregion	4	I	Region for a read transaction.
dut_axi_s*_arsize	3	I	The number of bytes for each transfer in a read transaction.

Name	Width	I/O	Description
dut_axi_s*_arvalid	1	I	Indicates whether the information from the master on the read address channel is valid.
dut_axi_s*_awaddr	64	I	The address of the first transfer in a write transaction.
dut_axi_s*_awburst	2	I	Burst type of write transaction.
dut_axi_s*_awcache	4	I	The way that a write passes through a system.
dut_axi_s*_awid	4	I	ID tag for a write transaction.
dut_axi_s*_awlen	8	I	The number of transfers in a write transaction.
dut_axi_s*_awlock	1	I	Information about atomic transaction.
dut_axi_s*_awprot	3	I	Protection characteristics for a write.
dut_axi_s*_awqos	4	I	Quality of Service for a write.
dut_axi_s*_awready	1	O	Indicates whether a write transaction can be initiated to slave.
dut_axi_s*_awregion	4	I	Region for a write.
dut_axi_s*_awsize	3	I	The number of bytes in each transfer in a write.
dut_axi_s*_awvalid	1	I	Indicates whether the information from master on the write address channel is valid.
dut_axi_s*_bid	4	O	ID tag for a read data and response.
dut_axi_s*_bready	1	I	Indicates whether a read data transfer can be initiated to master.
dut_axi_s*_bresp	2	O	Read response information for the read data transfer.
dut_axi_s*_bvalid	1	O	Indicates whether the information from slave on the read data channel is valid.
dut_axi_s*_rdata	256	O	Read data for the read data transfer.
dut_axi_s*_rid	4	O	ID tag for a read data and response.

Name	Width	I/O	Description
dut_axi_s*_rlast	1	O	Indicates whether the read data transfer is the last one in a read transaction.
dut_axi_s*_rready	1	I	Indicates whether a read data transfer can be initiated to master.
dut_axi_s*_rresp	2	O	Read response information for the read data transfer.
dut_axi_s*_rvalid	1	O	Indicates whether the information from slave on the read data channel is valid.
dut_axi_s*_wdata	256	I	Write data for the write data transfer.
dut_axi_s*_wlast	1	I	Indicates whether the write data transfer is the last one in a write transaction.
dut_axi_s*_wready	1	O	Indicates whether a write data transfer can be initiated to slave.
dut_axi_s*_wstrb	32	I	Write strobe information for the write data transfer.
dut_axi_s*_wvalid	1	I	Indicates whether the information from master on the write data channel is valid.